



Technical Paper
PLCopen Technical Committee 2 – Task Force
Function Blocks for Motion Control:
Part 4 – Coordinated Motion

PLCopen Working Document
Version 2.0 ‘Release for Comments’

DISCLAIMER OF WARRANTIES

THIS DOCUMENT IS PROVIDED ON AN “AS IS” BASIS AND MAY BE SUBJECT TO FUTURE ADDITIONS, MODIFICATIONS, OR CORRECTIONS. PLCOPEN HEREBY DISCLAIMS ALL WARRANTIES OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING ANY WARRANTY OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, FOR THIS DOCUMENT. IN NO EVENT WILL PLCOPEN BE RESPONSIBLE FOR ANY LOSS OR DAMAGE ARISING OUT OR RESULTING FROM ANY DEFECT, ERROR OR OMISSION IN THIS DOCUMENT OR FROM ANYONE’S USE OF OR RELIANCE ON THIS DOCUMENT.

Copyright © 2002 - 2025 by PLCopen. All rights reserved.

Date: November, 2025

Function Blocks for Motion Control

The following paper is a document under construction within the PLCopen Task Force Motion Control, Technical Document Version 2.0.

It summarizes the results of the PLCopen Task Force Motion Control, containing contributions of all its members. The present specification was written thanks to the following members of the Task Force:

Hilmar Panzer	CODESYS (formerly 3S – Smart Software Solutions)
Georg Seidel	CODESYS (formerly 3S – Smart Software Solutions)
Christian Müller	ABB
Klaus Bernzen	Beckhoff
Josef Papenfort	Beckhoff
Wilfried Plaß	Beckhoff
Wolfgang Czech	Bosch Rexroth
Friedrich Forthuber	B & R
Martin Schrott	B & R
Ed Baker	Control Techniques
Roland Schaumburg	Danfoss
Ryszard Bochniak	Eckelmann
Colin Leitner	Festo
Djafar Hadiouche	GE Fanuc
David Tobon	Huawei
Jürgen Hipp	ISG
Joachim Mayer	ISG
Harald Buchgeher	Keba
Joachim Strobel	Kuka Robotics
Lars Perner	Lenze
Candido Ferrio	Omron
Josep Lario	Omron
Yoshikazu Tachibana	Omron
Christian Ruf	Parker Hannifin
Klas Hellmann	Phoenix Contact
Tommaso Fochi	Rockwell
Markus Müller	SEW Eurodrive
Gregor Winterer	SEW Eurodrive
Willi Gagsteiger	Siemens Automation & Drives
Hans Peter Otto	Siemens Automation & Drives
Markus Leopold	Siemens Automation & Drives
Jürgen Fieß	Schneider Electric Motion Deutschland (formerly Berger Lahr)
Wolfgang Fien	Schneider Electric Motion Deutschland (formerly Berger Lahr)
Istvan Ulvros	TetraPak
Valentin Etourneau	Yaskawa
Tang Ming Yu	PLCopen China
Eelco van der Wal	PLCopen
Roeland Hagesteijn	PLCopen

Change Status List:

Version number	Date	Change comment
V 1.0	December 3, 2008	Official release
V2.0WD0.1	March 22, 2018	First doc for Update. Merge of Feedback items and Corrigendum
V2.0 WD0.11	March 26, 2021	Added comment to CmdAcpt
	March 31, 2023	Markus Leopold added the text to the selected FBs. Basis for V 0.8. Not published
	April 7, 2023	The lay-out has been harmonized and minor errors corrected.
		In the working doc of Part 4 (V 2.0) a CommandAccepted was introduced with similar behavior. It was decided to accept this proposal here too. (May 23, 2023). Extended.
	June 23, 2023	- Changes added as result of the F2F meeting in Verl (May 23) - Comments Ch. 14.1 – 14.7 included as well as 14.8 – 14.14 Graphical representation of FBs added
	August 15, 2023	- V0.8d as result of the discussion on the inputs per FB via webmeetings - All delayed FBs for Trigger “removed” and moved to doc Postponed Functionalities FBs that were identified as “non-supported” moved to Postponed Functionalities
	Dec. 2023 / Jan 2024	- Previous changes accepted for clarity - Not supported and delayed FBs removed. - FBs grouped in logical order. Replacing Mandatory (M) and Optional (O) with Basic (B) and Extended (E) (in line with Part 4)
	Feb. 2024	Separated in 4 parts: this document, FBs for Part 4, changes to Part 1, and application examples. This is the main working doc
	March 2024	- Many FBs rewritten in standard format and examples added - Not all links are redefined. To be done - Still some FBs that should be removed
V2.0 WD 02	April 24, 2024	As result of the webmeeting this week
V2.0 WD 03	July 23, 2024	Introduction merged with remote systems definition (SRCI)
V2.0 WD 04	July 25, 2024	FBs merged from “PLCopen Robotics Profile for Part 4 V04 comparison decisions - Review_LP - result of biweekly 2024-07-02.docx”
V2.0 WD05	July 25, 2024	With inclusion of the new FBs based on SRCI via doc “PLCopen Robotics Profile V09g_GS_2024-04-15
V2.0 WD07	August 8, 2024	Included description for World coordinate system (WCS)
V2.0 WD08	August 27, 2024	Removed old comment
V2.0 WD09	September 11, 2024	Put chapters in the right order. Corrected order of outputs for all FBs
V2.0 WD09	February 27, 2025	Replaced MC_ReadDHParameters, moved chapters and amended example figures
V2.0	November 18, 2025	Released for Comments

Table of Contents

.....	1
1 GENERAL	9
1.1 OBJECTIVES	9
1.2 INTRODUCTION	10
1.3 OVERVIEW OF THE DEFINED FUNCTION BLOCKS.....	11
1.3.1 Length of FB names and ways to shorten them.....	12
1.4 GLOSSARY	13
2 CHANGES AND ADDITIONS TO PREVIOUS VERSIONS AND PART 1.....	15
2.1 MAJOR CHANGES COMPARED TO VERSION 1.0.....	15
2.2 DIFFERENCES TO PART 1 VERSION 2.0.....	16
2.2.1 Input Execution Mode.....	16
2.2.2 Output CommandAccepted	16
2.2.3 Output CommandID.....	18
3 PRINCIPLES OF COORDINATED MOTION	19
3.1 COORDINATE SYSTEM AND KINEMATIC TRANSFORMATION	19
3.1.1 Kinematic transformation	22
3.2 HOW DO COMMANDS BEHAVE IN DYNAMIC COORDINATE SYSTEMS.....	23
3.3 MOVEMENTS	24
3.4 BLENDING AND BUFFERING OF MOVEMENTS	25
4 DEFINITION OF POSITION	26
4.1 CONFIGURATIONS AND TURN INFORMATION.....	26
4.2 EXAMPLE FOR MC_POS_REF	27
4.3 RECOMMENDATION FOR MC_CART_REF	28
5 MODEL	29
5.1 STATE DIAGRAM	30
5.2 RELATIONSHIP SINGLE AXIS AND GROUPED AXES STATE DIAGRAMS.....	31
6 AXES GROUPING.....	33
6.1 CREATING AND USING AN AXES GROUP	34
7 ROBOT DYNAMICS AND PAYLOAD.....	36
7.1 MC_RIGID_BODY_DYNAMIC_REF.....	37
8 TOOL MANAGEMENT.....	38
8.1 MC_TOOL_REF	39
8.2 EXAMPLE PROCEDURE FOR A MULTI-GRIPPER-APPLICATION.....	40
9 FUNCTION BLOCKS FOR COORDINATED MOTION	41
9.1 MC_ADDAXISToGROUP	42
9.2 MC_REMOVEAXISFROMGROUP	43
9.3 MC_UNGROUPALLAXES	44
9.4 MC_GROUPREADCONFIGURATION	45
9.5 MC_READAXISGROUPINFO.....	47
9.6 MC_GROUPENABLE.....	48
9.7 MC_GROUPDISABLE.....	49
9.8 MC_GROUPHOME.....	50
9.9 MC_GROUPPOWER	51
9.10 TRANSFORMATION FBS.....	52
9.10.1 MC_SetKinTransform (ACS to MCS)	52
9.10.2 MC_SetCartesianTransform (WCS to PCS and WCS to MCS)	54
9.10.3 MC_SetCoordinateTransform (WCS to PCS and WCS to MCS).....	56
9.10.4 MC_ReadKinTransform (ACS to MCS).....	57
9.10.5 MC_ReadCartesianTransform (WCS to PCS and WCS to MCS).....	58

9.10.6	MC_ReadCoordinateTransform (MCS to PCS).....	59
9.11	MC_READDHPARAMETERS.....	60
9.12	MC_READJOINTINFO	62
9.13	MC_GROUPSETPOSITION.....	63
9.14	MC_GROUPREADPOSITION.....	64
9.15	MC_GROUPREADVELOCITY	65
9.16	MC_GROUPREADACCELERATION.....	66
9.17	MC_GROUPREADMOTIONSTATE.....	67
9.18	MC_GROUPREADCOMMANDINFO	68
9.19	MC_GROUPREADPARAMETER.....	69
9.19.1	Table of Parameters.....	70
9.20	MC_GROUPWRITEPARAMATER.....	71
9.21	MC_GROUPWRITEREFERENCEDYNAMICS.....	72
9.22	MC_GROUPREADREFERENCEDYNAMICS	73
9.23	MC_GROUPWRITEDEFAULTDYNAMICS	74
9.24	MC_GROUPREADDEFAULTDYNAMICS.....	75
9.25	MC_GROUPSTOP	76
9.26	MC_GROUPHALT.....	80
9.27	MC_GROUPINTERRUPT	82
9.28	MC_GROUPCONTINUE	83
9.29	MC_GROUPREADSTATUS	84
9.30	MC_GROUPREADERROR	85
9.31	MC_GROUPRESET	86
9.32	MC_GROUPREADSWLIMITS	87
9.33	MC_GROUPWRITESWLIMITS.....	88
9.34	MC_MOVELINEARABSOLUTE.....	89
9.34.1	Example 1	90
9.34.2	Example 2	92
9.35	MC_MOVELINEARRELATIVE.....	94
9.35.1	Example 1	96
9.35.2	Example 2	98
9.36	MC_MOVECIRCULARABSOLUTE	100
9.36.1	Example 1	103
9.36.2	Example 2	104
9.36.3	Example 3	105
9.37	MC_MOVECIRCULARRELATIVE	109
9.37.1	Example 1	113
9.37.2	Example 2	114
9.38	MC_MOVEDIRECTABSOLUTE.....	116
9.39	MC_MOVEDIRECTRELATIVE.....	118
9.39.1	Example 1	119
9.40	MC_GROUPWAITTIME	120
9.41	MC_PATHSELECT	122
9.42	MC_MOVEPATH	123
9.43	MC_GROUPSETOVERRIDE.....	124
9.44	MC_GROUPJOG.....	126
9.45	MC_GROUPJOGVECTOR	128
9.46	MC_GROUPWRITEJOGGINGDYAMICS.....	129
9.47	MC_GROUPREADJOGGINGDYNAMICS.....	130
9.48	MC_READRIGIDBODYDYNAMIC	131
9.49	MC_WRITERIGIDBODYDYNAMIC.....	132
9.50	MC_WRITETOOLDATA.....	133
9.51	MC_READTOOLDATA	134
9.52	MC_SELECTTOOL	135
9.53	MC_READTOOL	136
9.54	MC_WRITEPAYLOADDATA	137
9.55	MC_READPAYLOADDATA.....	138
9.56	MC_SELECTPAYLOAD	139
9.57	MC_READPAYLOAD	140

9.58	MC_TRANSFORMPOSITION	141
10	AXES GROUP SYNCHRONIZED MOTION AND TRACKING	142
10.1	SYNCHRONIZATION	143
10.1.1	Synchronization of single axis to an axes group	144
10.1.2	Synchronization of an axes group to a single axis	145
10.2	TRACKING	146
10.3	MC_SYNCAXISToGROUP	149
10.4	MC_SYNCGROUPToAXIS	150
10.5	MC_SETDYNCOORDTRANSFORM	152
10.6	MC_TRACKCONVEYORBELT	154
10.7	MC_TRACKROTARYTABLE	158
11	TRANSITION, BLENDING AND BUFFERING OF MOVEMENTS - TERMINOLOGICAL DEFINITIONS	159
11.1	INPUT PARAMETER FOR BLENDING	160
11.2	BUFFER MODES	161
11.2.1	BufferMode “mcAborting”	162
11.2.2	BufferMode “mcBuffered”	162
11.2.3	Transition Velocity behavior	162
11.3	TRANSITIONMODE	163
11.3.1	TransitionMode “TMNone” (insert no transition curve)	163
11.3.2	TransitionMode “TMStartVelocity” (Transition with given maximum velocity)	163
11.3.3	TransitionMode “TMConstantVelocity” (Transition with given constant velocity)	164
11.3.4	TransitionMode “TMCornerDistance” (Transition with given corner distance)	165
11.3.5	TransitionMode “TMMaxCornerDeviation” (Transition with given maximum corner deviation)	166
11.4	MC_COMMAND_ID	167
11.5	MC_CIRC_PATHCHOICE	168
11.6	MC_ORIENTATION_MODE	169
11.7	MC_GROUP_SWLIMITS	172
	Example 1:	172
	Example 2:	172
11.8	TRANSITION REFERENCE POINT	173
11.9	TOOL COORDINATE SYSTEM (TCS) AND BUFFERED MOVEMENTS	174
APPENDIX 1	COMPLIANCE PROCEDURE AND COMPLIANCE LIST	175
APPENDIX 1.1	STATEMENT OF VENDOR	176
APPENDIX 1.1.	SUPPORTED DATA TYPES	177
APPENDIX 1.2.	SUPPORTED BUFFER MODES	177
APPENDIX 1.3.	SUPPORTED TRANSITION MODES	178
APPENDIX 1.4.	SHORT OVERVIEW OF THE FUNCTION BLOCKS	179
APPENDIX 1.5.	THE PLCOPEN MOTION CONTROL LOGO AND ITS USAGE	215
APPENDIX 2	RELATION OF PLCOPEN COORDINATED MOTION TO SRCI	216
LINK TO LAYERS IN SRCI		216

Table of Figures

Figure 1.	DIFFERENT USE-CASES FOR THE 'COMMANDACCEPTED'-SIGNAL.....	17
Figure 2.	OVERVIEW OF THE COORDINATE SYSTEMS AND TRANSFORMATIONS	19
Figure 3.	TRANSFORMING COORDINATE SYSTEMS	20
Figure 4.	EXAMPLE FOR SPECIFYING POINT P IN PCS, MCS, WCS OR ACS ASSUMING A SCARA ROBOT WITH TWO ROTARY AXES.....	21
Figure 5.	EXAMPLE FOR REACHING THE SAME POSITION IN SPACE WITH A) A CARTESIAN HANDLING (2 LINEAR AXES) AND B) A SCARA (2 ROTARY AXES) WITH TWO POSSIBLE CONFIGURATIONS (ELBOW DOWN AND ELBOW UP). (NOTE: THE ORIENTATION IS FIXED IN BOTH EXAMPLES).....	22
Figure 6.	DIFFERENT TYPES OF MOVEMENTS.....	24
Figure 7.	TRAJECTORIES AND PROCESS OF VELOCITY IN PRINCIPLE OF TWO CONSECUTIVE MOTION COMMANDS IN THREE MODES.....	25
Figure 8.	THE STATE DIAGRAM	30
Figure 9.	RELATIONSHIP SINGLE AXIS AND GROUPED AXES STATE DIAGRAMS.....	31
Figure 10.	OVERVIEW AXES GROUP	33
Figure 11.	TYPICAL TIMING DIAGRAM FOR SETTING THE TRANSFORMATION	53
Figure 12.	MC_GROUPSTOP TIMING DIAGRAM.....	77
Figure 13.	BEHAVIOR OF MC_GROUPSTOP IN COMBINATION WITH MC_MOVELINEARRELATIVE	77
Figure 14.	EXAMPLE OF MC_GROUPSTOP IN COMBINATION WITH TWO MC_MOVELINEAR_ABSOLUTE.....	79
Figure 15.	BEHAVIOR OF MC_GROUPHALT IN COMBINATION WITH MC_MOVECIRCULARABSOLUTE	81
Figure 16.	"MOVELINEARABSOLUTE" EXAMPLE	90
Figure 17.	EXAMPLE MC_MOVELINEARABSOLUTE.....	92
Figure 18.	EXAMPLE MC_MOVELINEARRELATIVE.....	96
Figure 19.	SECOND EXAMPLE WITH MC_MOVELINEARRELATIVE AND BLENDING	98
Figure 20.	"MOVECIRCULARABSOLUTE" EXAMPLE 1 – "CIRCMode" = MCBORDER	103
Figure 21.	"MOVECIRCULARABSOLUTE" EXAMPLE – "CIRCMode" = MCCENTER.....	105
Figure 22.	"MOVECIRCULARABSOLUTE" EXAMPLE – "CIRCMode" = MCCENTER - "TOLERANCE"	106
Figure 23.	EXAMPLE MC_MOVECIRCULARABSOLUTE	107
Figure 24.	VISUALISATION OF MC_MOVECIRCULARABSOLUTE SEQUENCE.....	108
Figure 25.	"MOVECIRCULARRELATIVE" EXAMPLE – "CIRCMode" = MCBORDER	113
Figure 26.	"MOVECIRCULARRELATIVE" EXAMPLE – "CIRCMode" = 1	114
Figure 27.	"MOVECIRCULARRELATIVE" EXAMPLE – "CIRCMode" = MCCENTER - "TOLERANCE"	115
Figure 28.	EXAMPLE 1 MC_MOVEDIRECTRELATIVE	119
Figure 29.	GRAPHICAL EXPLANATION OF MC_GROUPSETOVERRIDE.....	125
Figure 30.	GRAPHICAL EXPLANATION OF COORDINATION.....	142
Figure 31.	GRAPHICAL EXPLANATION OF SYNCHRONIZATION	143
Figure 32.	SYNCHRONIZATION OF SINGLE AXIS TO AN AXES GROUP	144
Figure 33.	SYNCHRONIZATION OF AN AXES GROUP TO A SINGLE AXIS	145
Figure 34.	TRACKING AND SYNCHRONIZATION	146
Figure 35.	EXAMPLE TIMING DIAGRAM TRACKING	148
Figure 36.	EXAMPLE MC_SYNCGROUPTOAXIS	151
Figure 37.	EXAMPLE MC_TRACKCONVEYORBELT	156
Figure 38.	TIMING DIAGRAM FOR EXAMPLE OF MC_TRACKCONVEYORBELT	157
Figure 39.	ORIENTATION MODES.....	169
Figure 40.	THE PLCOPEN MOTION CONTROL LOGO.....	215

Table of Tables

Figure 41.	OVERVIEW OF THE DEFINED FUNCTION BLOCKS.....	11
Figure 42.	LIST OF RULES TO SHORTEN NAMES	12
Figure 43.	CHANGES AND ADDITIONS TO PART 1 VERSION 2.0	16
Figure 44.	OVERVIEW OF THE INFLUENCE OF ACTIVE GROUP COMMANDS ON A SINGLE AXIS STATE	32
Figure 45.	OVERVIEW OF BUFFER MODES	161
Figure 46.	OVERVIEW OF TRANSITION VELOCITY MODES	161
Figure 47.	OVERVIEW OF AVAILABLE TRANSITION MODES	163
Figure 48.	SUPPORTED DATATYPES.....	177
Figure 49.	SUPPORTED DERIVED DATATYPES.....	177
Figure 50.	OVERVIEW OF BUFFER MODES	177
Figure 51.	OVERVIEW OF AVAILABLE TRANSITION MODES	178
Figure 52.	SHORT OVERVIEW OF THE FUNCTION BLOCKS	181

1 General

In 2008 PLCopen released Part 4 of the Motion Control suite of specifications, dealing with coordinated motion, for example robotics. Basically, with this specification one can control a robot in the PLC or one can see it as an interface to a robotic controller, which will do the calculations and movements.

Especially for this last option we received feedback from robotics companies with specific requirements, which resulted in a Corrigendum and the basis for Version 2 of this specification.

Also, with the release of the Standard Robot Command Interface, SRCI, within Profinet International, PI, based on this document, a merger of both specifications was proposed. This document reflects this merger. As such, the original PLCopen document is extended with new function blocks, as well as information on position, trajectories, changes in transition and buffer modes.

Profinet International, PI, released version 1.3 of the Standard Robot Command Interface (SRCI) in November 2022, supported by many robot vendors. This specification is based on PLCopen Part 4 and provides an interface to remote robot controllers. For this, Part 4 was extended with the knowledge from the community of robot vendors, as many of them were involved.

The SRCI specification adds configuration functionalities and defines an interface to the different robot vendors. The essence of PLCopen is to incorporate these missing functionalities in the PLCopen Motion Control standard while maintaining the same Look & Feel by providing backward compatibility. Also, these functionalities need to be mapped to the defined interface and network. As such it is not restricted to Profinet as network.

1.1 Objectives

The objective of this document is to define a set of extensions to “Part 1 - PLCopen Function Blocks for Motion Control”, focused to the coordinated multi-axes motion in 3D space, to serve most user’s application needs in this area. A key addition in this part is the kinematic model, linking the different axes to a mechanical construct and as such defining the movement of the relevant endpoint.

Implementations can include architectures with integrated solutions or remote connections to an intelligent controller like a dedicated robot-controller, which includes the kinematic model and specifics for the application like welding or moving parts. The goal is to provide a programming interface to the user that hides as much as possible these different implementations, making the development of the application software as independent as possible from the underlying architecture.

1.2 Introduction

The PLCopen Motion Control Function Blocks are specified at such a level that the user quickly recognizes the functionality and that the effect of the activation and of sequencing with other function blocks is clear. Path oriented movements are programmed either with specific robot-oriented programming languages, or “G-code” (for instance cf. DIN 66025) as used in the CNC world. But without a doubt, the movements which can be described in these languages are applicable to a broader area of use. This PLCopen initiative transforms the functionalities known in the CNC and Robotic world to the PLC world. With this, an additional part is added to the range of PLCopen Motion Control specifications. The relationship with the other PLCopen parts is shown below.

Part 1 deals with single-axis motion control and master / slave motion control, a type of coordinated motion control where the master axis position is used to generate one or more slave axis position commands. For multi-dimensional movements, one goes beyond this point via a grouping of a set of axes, without a master axis. This is done via the definition of a set of function blocks with related coordinated motion functionality as well as a higher-level state diagram, linking the single axis state diagrams in the group. In this way better trajectory planning is possible. Also, the current master/slave axes can have the problem that if an error occurs, the other axes have no knowledge about this and continue their movement. By combining axes in a group, one knows upfront which axes are involved and has the basis for better error behavior.

From a PLCopen point of view, the axes group is programmed from the PLC. This document is focused on the control of kinematics, as shown in the picture below.



For an overview of the changes between version 1.0 and version 2.0 of this document, see 2 Changes and additions to previous versions and Part 1

1.3 Overview of the defined Function Blocks

Table below gives an overview of the defined function blocks, divided into administrative and motion related sets. Function blocks that were modified from the previous version are marked with (C), while newly introduced function blocks are marked with (N).

<i>Administrative</i>	
MC_AddAxisToGroup	MC_GroupReadStatus
MC_RemoveAxisFromGroup	MC_GroupReadError
MC_UngroupAllAxes	MC_GroupReset
MC_GroupReadConfiguration	(N) MC_GroupReadSWLimits
(N) MC_ReadAxisGroupInfo	MC_PathSelect
MC_GroupEnable	MC_GroupSetOverride
MC_GroupDisable	(N) MC_GroupReadJoggingDynamics
MC_SetKinTransform	(N) MC_ReadRigidBodyDynamics
MC_SetCartesianTransform	(N) MC_ReadToolData
MC_SetCoordinateTransform	(N) MC_SelectTool
MC_ReadKinTransform	(N) MC_ReadTool
MC_ReadCartesianTransform	(N) MC_ReadPayloadData
MC_ReadCoordinateTransform	(N) MC_SelectPayload
(N) MC_ReadDHPParameters	(N) MC_ReadPayload
MC_GroupSetPosition	(N) MC_GroupPower
(C) MC_GroupReadPosition	(N) MC_GroupWriteParameter
(C) MC_GroupReadVelocity	(N) MC_GroupWriteReferenceDynamics
(C) MC_GroupReadAcceleration	(N) MC_GroupWriteDefaultDynamics
(N) MC_GroupReadMotionState	(N) MC_GroupWriteSWLimits
(N) MC_GroupReadCommandInfo	(N) MC_GroupWriteJoggingDynamics
(N) MC_GroupReadParameter	(N) MC_WriteRigidBodyDynamic
(N) MC_GroupReadReferenceDynamics	(N) MC_WriteToolData
(N) MC_GroupReadDefaultDynamics	(N) MC_WritePayloadData
(N) MC_ReadJointInfo	(N) MC_TransformPosition
<i>Motion</i>	
<i>Coordinated</i>	<i>Synchronized /Tracking</i>
MC_GroupHome	MC_SyncAxisToGroup
MC_GroupStop	MC_SyncGroupToAxis
MC_GroupHalt	MC_TrackConveyorBelt
MC_GroupInterrupt	MC_TrackRotaryTable
MC_GroupContinue	MC_SetDynCoordTransform
MC_MoveLinearAbsolute	(N) MC_GroupWaitTime
MC_MoveLinearRelative	MC_MovePath
MC_MoveCircularAbsolute	(N) MC_GroupJog
MC_MoveCircularRelative	(N) MC_GroupJogVector
MC_MoveDirectAbsolute	
MC_MoveDirectRelative	

Table 1: Overview of the defined Function Blocks

1.3.1 Length of function block names and ways to shorten them

There are systems that only support a limited number of significant characters in the name. For these rules for shorter names are provided here. These names are still seen as compliant, although these names have to be mentioned in the certification document.

Group	Grp
Remove	Rem
Cartesian	Cart
Command	Cmd
Coordinate	Coord
Transformation	Trans
Kinematic	Kin
Dynamic	Dyn
Synchronized	Sync
Configuration	Cfg
Position	Pos
Velocity	Vel
Acceleration	Acc
Linear	Lin
Circular	Circ
Absolute	Abs
Direct	Dir
Relative	Rel
Actual	Act
Conveyor	Conv
Jogging	Jog
Write	Wrt
Read	Rd

Table 2: List of rules to shorten names

1.4 Glossary

Name/ Acronym	Explanation
ACS	Axes Coordinate System: The system of coordinates related to physical motors and the single movements caused by single drives.
Blending	A way that consecutive function blocks cooperate in the transition from the first to the next.
Contour curve	Inserted curve that modifies the original path. It is the resulting curve after blending.
Coordinate system	The reference system in which a coordinate or path is described.
Corner deviation	The shortest distance between the programmed corner point and the contour curve.
Corner distance	Distance of the start point of the contour curve to the programmed target point.
Direction	The orientational components of a vector in space. (Note: this is different from the MC Direction input as used in part 1).
Drive	A unit controlling a motor via the current and timing in its coils.
Flange	The mechanical interface on the robot arm where the tool is mounted; typically defined by a standardized geometry and coordinate system for consistent tool attachment.
Group-FB	The set of function blocks that can work on a group of axes.
MCS	Machine Coordinate System - the system of coordinates that is related to an axes group. A Cartesian coordinate system with the origin in a fixed position relative to the axes group (the origin is defined during the machine setup, typically relative to a commonly used WCS). (Note: with Cartesian build machines, MCS is a Cartesian Coordinate system and may be identical to ACS, or mapped via a trivial transformation). The coordinate system from the physical axes ACS is linked to the MCS via a kinematic transformation (forward and backward conversion). The MCS represents a space with up to 6 dimensions. By default, the MCS is identical to the WCS.
Motor	An actuator focused to a movement, converting electrical energy in a force or torque.
Orientation	The rotational components of a vector in space.
Path	Set of continuous positions and orientation information in multi-dimensional space Geometrical description of a space curve that the TCP of an axes group moves along.
PathData	Description of a path which can include additional information like velocity and acceleration.
PCS	The coordinate system of the product can be called PCS – Product Coordinate System (or “Program Coordinate System” in CNC world, or Programmers Coordinate System). The PCS is based on the WCS typically by shifting and maybe rotating the PCS relative to the WCS. The Zero point of the PCS is related to the product and can be changed during runtime by the program. The real work piece can have a rotation or shift to the WCS or even might be moving relative to the WCS. By specifying a trajectory in PCS one is able to describe the trajectory independent from the machine situation. To map these two coordinate systems (WCS to PCS and vice versa), a cartesian or cylindrical transformation is normally done. By default, the PCS is identical to the WCS. To interact with multiple objects in an automation solution, systems may support multiple PCS.
Position	Position means a point in space which is described by different coordinates. Depending on the used system and transformation, it can consist of up to 6 dimensions (coordinates) meaning 3 Cartesian coordinates in space and 3 coordinates for the orientation. In ACS there can be even more than 6 coordinates. If the same position is described in different coordinate systems, the values of the coordinates are different.
Pose (not used)	Position and orientation (DIN EN ISO 8373). Position is used instead in this document.
RC	Robot Controller
Scara	A special kinematic for robot or handling applications.
Speed	Speed is the absolute value of the velocity without direction.
SRCI	Standard Robot Command Interface as provided by Profinet International (PI)
Synchronization	Combines an axis or axes group (as slave) with an axis as master in order that the slave executes its path with synchronization to the progress of the master, meaning linked to a one-dimension source for synchronization.
TCP	Tool Center point, the point in the machine that is commanded to move, typically the center or the head of the tool. In the center of the TCP, the TCS is located. The TCS can be described in different coordinate systems.

TCS	The coordinate system of the tool can be called TCS (Tool Coordinate System).
Tool	The end-effector or device mounted on the robot that interacts with the workpiece or environment, such as a gripper, welding torch, or suction cup.
Tracking	It is characterized by an axes group that follows with its movement the movement of another axis or axes group.
Trajectory	Time dependent description of the path the TCP of an axes group moves along. Additionally, to the geometrical description of the space curve, time dependent state variables like velocity, acceleration, jerk, forces etc. are specified.
Velocity	For a group of axes this means: - in ACS the velocities of the different axes; - in WCS, MCS and PCS it provides the velocity of the TCP.
WCS	World Coordinate System - the system of coordinates that is related to the origin of the application. A cartesian coordinate system with the origin in a fixed position of the machine, typically used to describe and program the movement of multiple axes groups within a common coordinate system. The WCS represents a space with up to 6 dimensions. By default, the MCS and PCS of an axes group are identical to the WCS If systems do not support a WCS, coordinate transformations of PCS are directly related to the MCS

2 Changes and additions to previous versions and Part 1

2.1 Major changes compared to Version 1.0

- The general outputs CommandAccepted and CommandID have been added, which mainly address the needs of distributed systems for communication handshakes (see 2.2).
- A vendor specific position type MC_POS_REF has been introduced instead of the ARRAY of REAL (see 4).
- The enum type MC_COORD_SYSTEM has been introduced and the already defined coordinate systems have been extended by WCS, FCS and TCS (see 3.1).
- Dynamic values (Velocity, Acceleration, Deceleration, and Jerk) can now optionally be passed as a percentage of the reference dynamics. (see new parameter mcDynamicsMode of type MC_DYNAMICS_MODE of axes group).
- New approach for buffering and blending commands (see 11):

This change mainly was caused to support the way industrial robot controllers do blending, mainly based on the end point of a movement, not the start point.

Therefore the parameter mcTransitionReferencePoint of type MC_TRANSITION_REFERENCE of an axes group has been introduced. By this parameter one can decide if inputs of a movement FB, referring to buffering and blending, refer to the starting point or to the end point of the movement.

Further, the input BufferMode (MC_BUFFER_MODE) only specifies if a command is buffered or aborting.

The set of inputs TransitionVelocity (MC_TRANSITION_VELOCITY), TransitionMode (MC_TRANSITION_MODE) and TransitionParameter (MC_TRANSITION_PARAMETER) specify the blending of movements.
- Support for jogging has been added (see 9.44 ff).
- Support for tool handling and payload has been added (see 9.50 ff).
- The following new function blocks have been added:
 - MC_ReadAxisGroupInfo
 - MC_GroupPower
 - MC_ReadDHPParameters
 - MC_ReadJointInfo
 - MC_GroupReadMotionState
 - MC_GroupReadCommandInfo
 - MC_GroupReadParameter
 - MC_GroupWriteParameter
 - MC_GroupWriteReferenceDynamics
 - MC_GroupReadReferenceDynamics
 - MC_GroupWriteDefaultDynamics
 - MC_GroupReadDefaultDynamics
 - MC_GroupReadSWLimits
 - MC_GroupWriteSWLimits
 - MC_GroupWaitTime
 - MC_GroupJog
 - MC_GroupJogVector
 - MC_GroupWriteJoggingDynamics
 - MC_GroupReadJoggingDynamics
 - MC_GroupReadRigidBodyDynamic
 - MC_GroupWriteRigidBodyDynamic
 - MC_WriteToolData
 - MC_ReadToolData
 - MC_SelectTool
 - MC_WritePayloadData
 - MC_ReadPayloadData
 - MC_TransformPosition

2.2 Differences to Part 1 Version 2.0

Feedback led to some general changes and additions which not only affect Part 4, but also would be applicable to Part 1. Those changes and additions might be integrated into future versions of Part 1.

<i>Change, Addition</i>	<i>Description</i>
Output CommandAccepted (BOOL) added	See Fout! Verwijzingsbron niet gevonden. Output CommandAccepted for a more elaborate explanation.
Output CommandID (MC_COMMAND_ID) added	See 2.2.3 Output CommandID for a more elaborate explanation.
Parameter mcDynamicsMode (MC_DYNAMICS_MODE) of axes group	By this parameter of the axes group one can select if dynamic values of FBs are given absolute or by a percentage value (see 9.19.1).
Behavior of Buffer Modes	The behavior of Buffer Modes has changed and is described in detail in 11.
Naming rules of ENUM members	The pre-fix “mc”, “mcTM”, “mcTV” may be omitted, if ENUM values have to be qualified by the ENUM Type Name. The usage of the pre-fix “mc”, or lack of it, should be consistent.

Table 3: Changes and additions to Part 1 Version 2.0

2.2.1 Input Execution Mode

The input MC_EXECUTION_MODE is an ENUM providing information on the behavior of administrative function blocks.

The modes are:

- mcImmediately - the functionality is immediately valid and may influence the on-going motion but not the state
- mcDelayed - the functionality is valid when the ongoing motion command is done.
- mcQueued - the new functionality becomes valid when all previous commands are done. This applies to buffered motion commands and queued administrative commands.

If a previous command is aborted, all following commands are aborted.

If a previous command reports an error, all following commands report an error..

2.2.2 Output CommandAccepted

Depending on the target system, there might be a non-deterministic time for communication from the command interface to the motion controller. During this time the communication channel is busy and cannot be used for other commands. All the existing outputs (Busy, Active, Done, Error, CommandAborted, c.f. function blocks for motion control - Chapter 2.4.1) will not return the required information. This leads to the introduction of an optional output signal ‘CommandAccepted’. This output clearly signals that communication is done. Depending on the target system the PLC programmer has to consider this information. If not, it can happen that communication is corrupted, and commands are not executed correctly.

‘CommandAccepted’ is set when the command has been added to the command buffer. This can be delayed if the command buffer is full or the buffering needs time to complete, e.g. in remote connections. When ‘CommandAccepted’ is TRUE, the function block can be safely reused to add another command to the buffer, by executing again.

‘CommandID’ is valid while ‘CommandAccepted’ is TRUE

If the Busy output goes to FALSE, also CommandAccepted goes to FALSE (falling edge). CommandAccepted also gets FALSE with a new trigger of Execute.

The signal sequences as shown hereunder show different use-cases for the ‘CommandAccepted’-Signal:

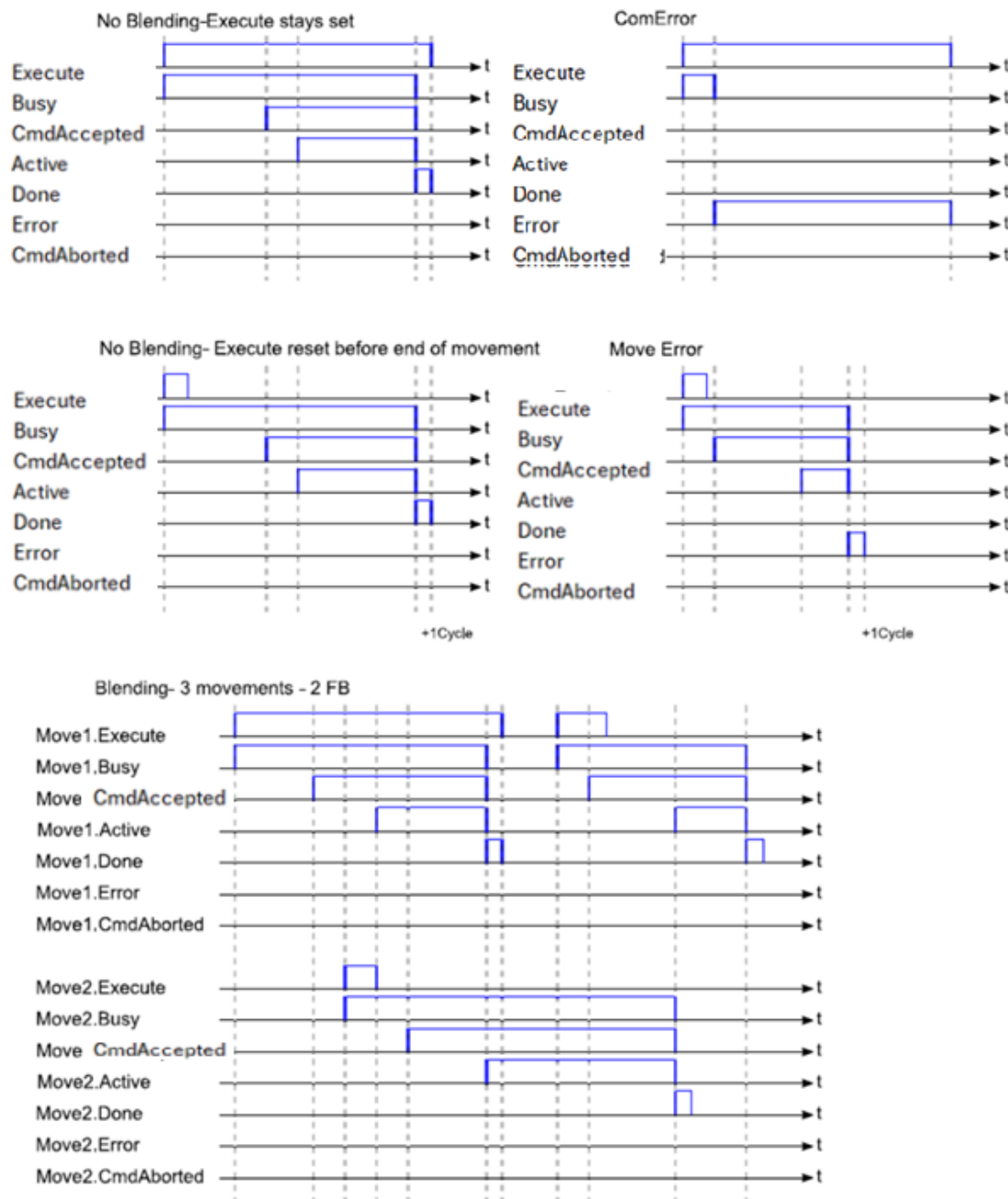


Figure 1: Different use-cases for the 'CommandAccepted'-Signal

Once set, the CommandAccepted output is reset together with the Busy output. In Figure 2 the outputs CommandAccepted and CommandAborted have been abbreviated to CmdAccepted and CmdAborted respectively.

2.2.3 Output CommandID

Some systems may support identification of a buffered command with a specific CommandID. This output is generated by the system with a unique code for every triggering of the Execute input of the FB. This output is only valid while the 'CommandAccepted' output is TRUE.

MC_COMMAND_ID is a vendor specific data type, that uniquely identifies a buffered motion or queued administrative command.

It is used for the extended output CommandID of these buffered and queued commands.

The CommandID of the active motion command can be read by MC_GroupReadMotionState.

The CommandID can be used for the input CommandID of the function block MC_GroupReadCommandInfo, to get additional status information.

The CommandID = 0 on a buffered or queued function block means that the command hasn't been accepted yet.

Example implementations:

(a)

MC_COMMAND_ID: UINT;

MC_COMMAND_ID is increased with every new command, sent to the axes group.

MC_COMMAND_ID might be reset under certain conditions, e.g. GroupDisabled to prevent overflow

(b)

MC_COMMAND

3 Principles of coordinated motion

3.1 Coordinate system and kinematic transformation

The essence of a trajectory is the coordinated motion of two or more axes from a starting point to a target point via a defined path with a specified path velocity. As path one can think of a straight line, a circular movement, or via a spline function. The definition of a path– or any position information - in space requires a coordinate system. Within this specification several types of coordinate systems are defined:

MC_COORD_SYSTEM	DESCRIPTION
ACS	Axis related
MCS	Axes group related
WCS	Fixed reference coordinate system
PCS	Product or Workpiece related PCS1 to PCSn if multiple PCS' are supported
FCS	Flange of the robot
TCS	Tool of the robot (TCP)

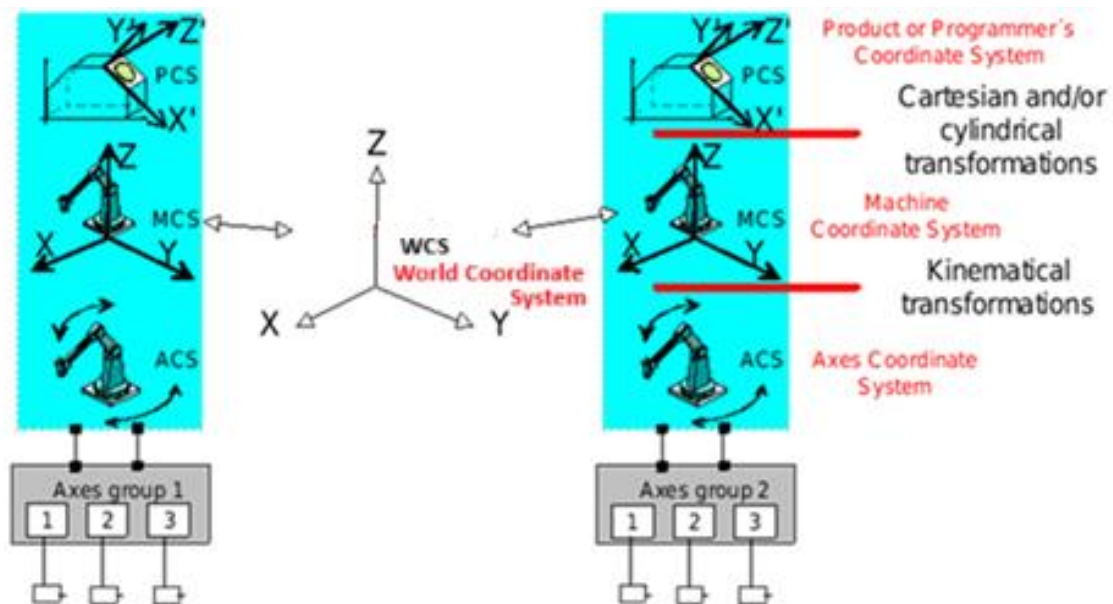


Figure 2: Overview of the coordinate systems and transformations

For a more detailed description of the different coordinate systems, see 1.4 Glossary.

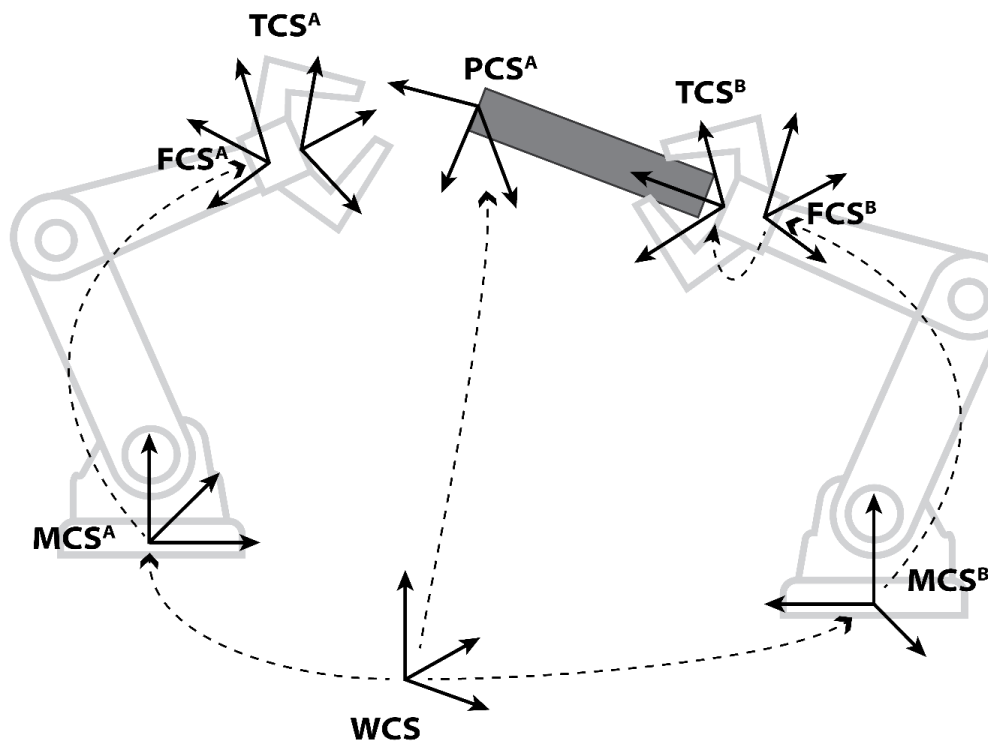


Figure 3: Transforming Coordinate Systems

To specify a point or orientation in space a position always must be related to a coordinate system. By means of transformations this position can be transformed to other coordinate systems. Within this specification, function blocks are defined for these transformations, hiding the complexity of these transformations. All multi axes motion commands are related to only one of the coordinate systems at the same time.

Example

The example below demonstrates how a point P, which is situated on a 2D workpiece (red trapezoid), can be described equivalent in PCS (blue), MCS (black), WCS (magenta) and ACS (green). Point P could be specified by referring to PCS resulting in the position $P_{PCS} = (x_{PCS}, y_{PCS})$. Given the shift and orientation of PCS relative to WCS, point P equivalently could be specified by $P_{WCS} = (x_{WCS}, y_{WCS})$. The same is valid for the MCS. Assuming a SCARA robot with two rotary axes point P also could be described by the angles of the axes $P_{ACS} = (\phi_1, \phi_2)$.

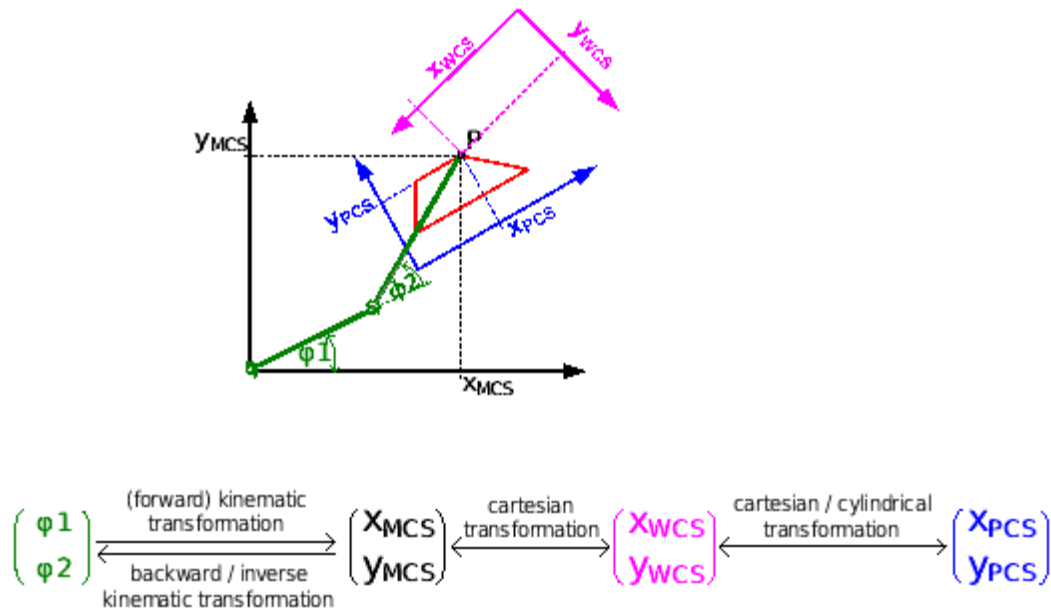


Figure 4: Example for specifying point P in PCS, MCS, WCS or ACS assuming a SCARA robot with two rotary axes

3.1.1 Kinematic transformation

Axes are connected via mechanical links providing movements of the 'Tool Center Point', TCP in space. TCP is a distinguished point of the machine, sometimes also called 'Point of Interest', POI, or 'effector'. The physical assembly of the axes and therefore the position of the TCP in MCS is described by a so-called kinematic transformation. The kinematic transformation connects ACS to MCS (forward conversion). By applying the kinematic transformation on a position related to ACS, this position can be transformed into a position in MCS. The other way round, applying the inverse kinematic transformation, a position related to MCS can be transformed into a position in ACS (backward conversion).

With simple cartesian machine constructions, in which axes are directly oriented in X-, Y-, and Z-directions of MCS, the kinematic transformation can easily be specified. One just has to define which axis is in the X-direction, which in Y, and which in the Z-direction. In the simplest case ACS is identical to MCS and one needn't distinguish between both. But in practice there are many non-cartesian structures, like SCARA robots or Tripods, where the kinematic transformation is more complex.

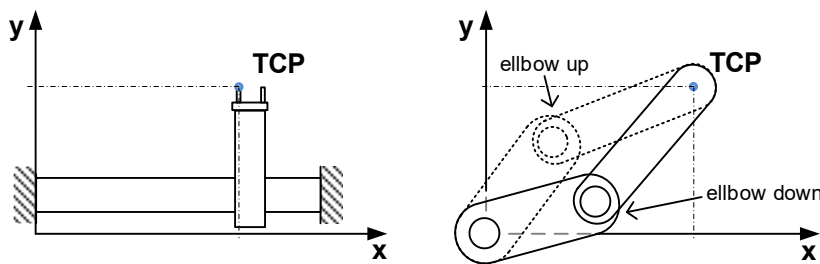


Figure 5: Example for reaching the same position in space with
a) a cartesian handling (2 linear axes) and
b) a SCARA (2 rotary axes) with two possible configurations (elbow down and elbow up).
(Note: the orientation is fixed in both examples)

The example above demonstrates how a position in space could be reached by a cartesian handling or a SCARA. Whereas the positions of the linear axes are identical to the coordinates of the position in MCS, the positions of the axes of the SCARA are not that easy to calculate. Additionally, there are two possible solutions of the backward kinematic transformation, different configurations of the machine: elbow down and elbow up.

3.2 How do commands behave in dynamic coordinate systems

If the TCP should follow a moving target, this can be achieved by a dynamic coordinate transformation, leading to a PCS which is moving in relation to the WCS.

The activation of a dynamic transformation is done by activating MC_SetDynCoordTransform , MC_TrackConveyorBelt, or MC_TrackRotaryTable.

If there is a dynamic transformation active, the axis may follow the dynamic transformation or stay in the static MCS, WCS or ACS. The following example shows the behavior. The example describes a robot fetching a screw from a fixed position and mounting it on a product that is moving on a belt. The example uses the MCS as static coordinate system, for WCS this is valid respectively.

Step	Move command	Axes (group) behavior	Application example
1	Activating Transformation ACS to MCS	Group is staying still (not moving)	Initialization MCS is static
2	Motion command in MCS	Group moves to the commanded position in MCS and stays in MCS afterwards (not moving)	Moving to standby position and waiting for products
3	Motion command in MCS	Group moves to the commanded position in MCS and stays in MCS (not moving)	Moving to a fixed box of screws
4	Motion command in MCS	Picking command	Picking up a screw
5	Activating a dynamic PCS	PCS is active and moves synchronized with the belt	PCS is ready for use
6	Motion command in dynamic PCS	Group moves to commanded position in PCS and is moving together with the dynamic PCS	Placing the screw and following the product on the belt
7	Screwing command	Group is still following the product on the belt	Screw is being screwed into the product
8	Motion command in MCS	Group moves to commanded position in MCS at the fixed screw box, picking up another screw	Moving to the fixed box of screws and waiting for the next product on the belt
9	Motion command in dynamic PCS	Group moves to commanded position in PCS and is moving together with the dynamic PCS	Placing the screw to the new product and following the product on the belt

Rule: An axes group stays in the coordinate system which is specified with the last motion command. If this is a PCS with dynamic transformation, it will follow the PCS (keeping the same position in this PCS).

3.3 Movements

Applying a movement on a machine via a function block causes the TCP to move towards the new commanded position. The kind of function block applied specifies the path via which the new target position is reached. (Note: the coordinate system in which the new commanded position is specified does not have an influence on the path.)

Basically, there are two types of movements which must be distinguished:

Point - to - Point movements, PTP (also referred to as Joint Interpolated Movements):

With this type of movement, the essence is to reach the commanded position as fast as possible. This can be achieved by moving each axis on the shortest way from its starting position to its target position. Usually, this kind of movement is the fastest way to reach a new commanded position, because at any time at least one axis moving at its dynamic limit. The path and the path velocity of the TCP are not important. They are determined by the process of the positions of the axes and the kinematic transformation of the machine. Therefore, this kind of movement is applicable for handlings and whenever the path of the TCP is not crucial. It is recommended that all axes arrive at the commanded position at the same point in time (synchronized).

The applicable function blocks as specified herein are:

- MC_MoveDirectAbsolute
- MC_MoveDirectRelative

Cartesian Path movements, CP (also referred to as Continuous Path movements):

CP movements cause the TCP to move along a defined path in Cartesian space. A path can be (a set of) a straight line, a circular movement, or a spline function. The path via which the new commanded position is reached is important. For example, this is essential if a workpiece is being processed. Further, the path velocity of the TCP can be controlled directly. Contrary to joint interpolated movements the process of the position of each axis is determined by the desired path and the inverse kinematic transformation.

The applicable function blocks as specified herein are:

- MC_MoveLinearAbsolute
- MC_MoveLinearRelative
- MC_MoveCircularAbsolute
- MC_MoveCircularRelative
- MC_MovePath

The figure below illustrates the differences between different types of movement by means of a theoretical machine.

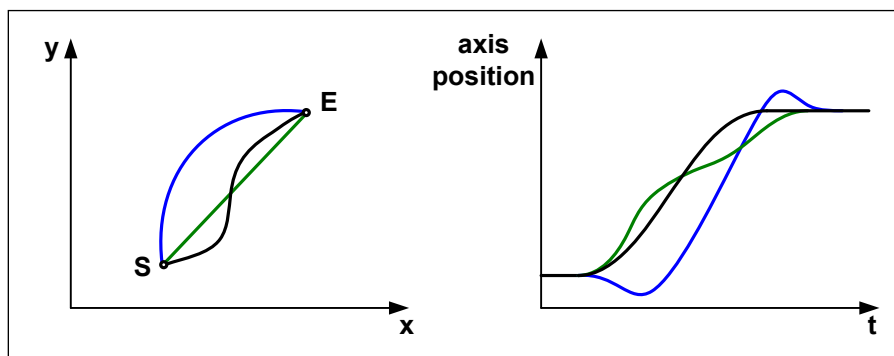


Figure 6: Different types of movements

MC_MoveDirect (black), MC_MoveLinear (green) and MC_MoveCircular (blue)
and typical positions of one of the axes of the machine participating in the movement

3.4 Blending and Buffering of movements

A fundamental part of interpolated motion control is blending of (buffered) consecutive motion commands on an axes group. Without blending the TCP of an axes group moves towards the commanded position, decelerates and comes to standstill exactly at the commanded position. The following buffered motion command doesn't become active until now. Obviously, the axes group has to accelerate again. In many applications a different behaviour of the TCP is desired, and one wants to concatenate movements without stopping.

Reasons for this are:

- Reduction of the process cycle time (e.g. pick and place)
- Generate a smoother movement in order to reduce the mechanical stress
- Some applications demand a constant Velocity of the TCP (e.g. applying glue, painting, welding, etc.)

All this can be achieved by different types of blending. Common to all types of blending is a modification of the original path, resulting in a smooth trajectory without corners.

Blending of motion commands in interpolated motion control differs from blending of motion commands on single axes. With single axes the commanded position is always reached. Just the velocity at the time when the commanded position is reached (or passed) can be changed according to the input parameter BufferMode.

With interpolated motion control several types of blending can be thought of, depending on the application and process. Therefore, new types of blending must be introduced for interpolated motion control.

The input parameter for blending might vary due to the kind of interpolation method applied. So, this input is vendor specific.

The type of inserted curve that modifies the original path (the 'contour curve') is not part of this specification and can be defined by the vendor specific input parameter for blending.

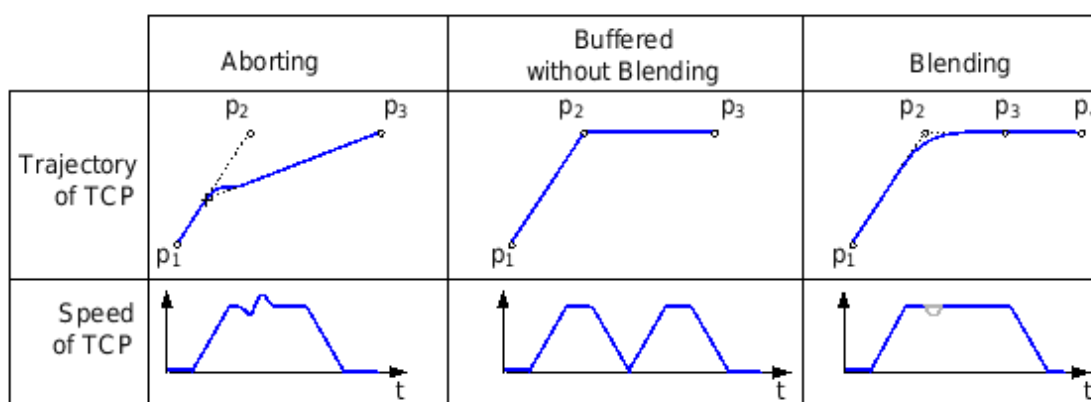


Figure 7: Trajectories and process of Velocity in principle of two consecutive motion commands in three modes

For details see 11 - Transition, blending and buffering of movements.

4 Definition of Position

The position of an axes group is used in many function blocks, for example as the target position of a linear movement. A vendor specific data type MC_POS_REF is introduced to represent the position of an axes group.

Note: In version 1.0 of Part 4, the position was represented as an array of REAL, which was interpreted as ACS positions or cartesian positions.

For compatibility, existing implementations may simply create this type as an alias to ARRAY[1..N] OF REAL.

4.1 Configurations and turn information

To fully specify the position of an axes group in a cartesian coordinate system (like for example the MCS), some kinematic models require additional information such as the configuration (also sometimes called mode) and turn information for rotary joints with a working range of more than 360°.

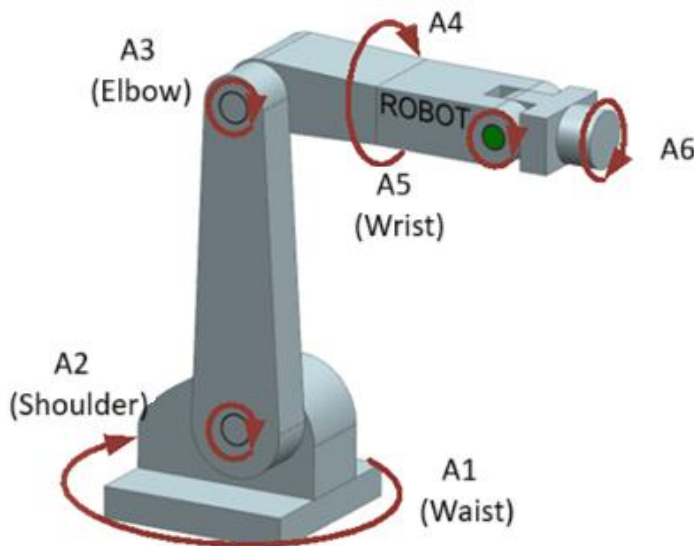
An example for the configuration would be the “elbow mode” for a scara robot. A scara with two rotary joints can reach each position in the X/Y plane in exactly two ways: with the elbow joint pointing to the right or to the left.

It is vendor specific whether this additional information is part of the position type MC_POS_REF. These three options have been identified for configuration and turn information. Which option is used is vendor specific.

- Part of MC_POS_REF. In this case a MC_POS_REF fully defines the position of the axes group, no matter whether it is represented in the ACS or in a cartesian coordinate system.
- Additional inputs to the movement FBs. If the coordinate system of the target position or distance is cartesian, these inputs are required to make the position unique for some kinds of axes groups.
- Configuration using additional administrative FBs. A separate FB can be used to define the configuration and turn information that is used for the next movement that is commanded in a cartesian coordinate system.

The configuration is represented by the vendor specific data type MC_CONFIG_DATA. An example definition:

```
TYPE MC_CONFIG_DATA : STRUCT
  ConfigValid: BOOL // 0=no; 1=yes
  shoulder : BOOL; // 0= back; 1 = front if ConfigValid = 1
  elbow : BOOL; // 0 = up; 1 = down if ConfigValid = 1
  wrist : BOOL; // 0 = non-flip; 1 = flip if ConfigValid = 1
END_STRUCT
END_TYPE
```



The turn information is represented by the vendor specific data type MC_TURN_INFO. An example definition follows on the next page.

```
TYPE MC_TURN_INFO : STRUCT
    // Turn information for each rotary joint. (If joint i is linear, then
    // aTurns[i] is ignored.)
    // The value of aTurns[i] has the following meaning for rotary joints:
    //   n = 0: free mode, the period of the axis is determined
    //           by the system
    //   n < 0: an axis position in the range ]-180°, 180°] + n*360° is used
    //   n > 0: an axis position in the range ]-180°, 180°] + (n-1)*360° is used

    aTurns : ARRAY[1..N] OF SINT;
END_STRUCT
END_TYPE
```

The behavior of linear and circular movements with mis-matching MC_CONFIG_DATA and MC_TURN_INFO is vendor specific.

The way the MC_TURN_INFO is used in motion commands is vendor specific.

4.2 Example for MC_POS_REF

We provide an example for the implementation of MC_POS_REF for systems that bundle the configuration and turn information into MC_POS_REF.

```
// This is the example for the position of an axes group, MC_POS_REF,
// MC_CART_REF, MC_CONFIG_DATA, MC_TURN_INFO and MC_AXES_POS_REF.
// Either a cartesian position or an axis position.
TYPE MC_POS_REF :
STRUCT
    c : MC_CART_POS_REF;
    a : MC_AXES_POS_REF;
END_STRUCT
END_TYPE
// This structure describes a cartesian position of an axes group
// with all required additional information to make it unique.
TYPE MC_CART_POS_REF :
STRUCT
    // The position and orientation of the TCP
    tcp : MC_CART_REF;
    // The configuration information of the axes group
    cfg : MC_CONFIG_DATA;
    // The turn info of rotary axes with a range > 360°
    turnInfo : MC_TURN_INFO;
    // the position of the auxiliary axes in the system are added here
    auxiliaryAxes : ARRAY [1..N] OF REAL
END_STRUCT
END_TYPE
TYPE MC_AXES_POS_REF :
STRUCT
    a1 : REAL; // Position of the Axis1
    a2 : REAL;
    a3 : REAL;
    a4 : REAL;
    a5 : REAL;
    a6 : REAL;
END_STRUCT
END_TYPE
```

4.3 Recommendation for MC_CART_REF

The following recommendation of MC_CART_REF is compatible to SRCI and RpyOrientationType from OPC 10000-210:2023-01 where RX, RY, RZ are mapped to A, B, C respectively.

TYPE MC_CART_REF : STRUCT

X : REAL;

Y : REAL;

Z : REAL;

RX : REAL;

RY : REAL;

RZ : REAL;

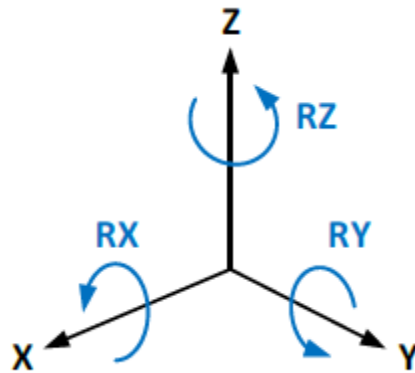
END_STRUCT

END_TYPE

Translation and Orientation

The axes X, Y, Z are arranged according to an orthogonal right-hand coordinate system.

The orientation consists of three elements: RX, RY, RZ. The direction of rotation is arranged according to ISO 9787:2013(E).

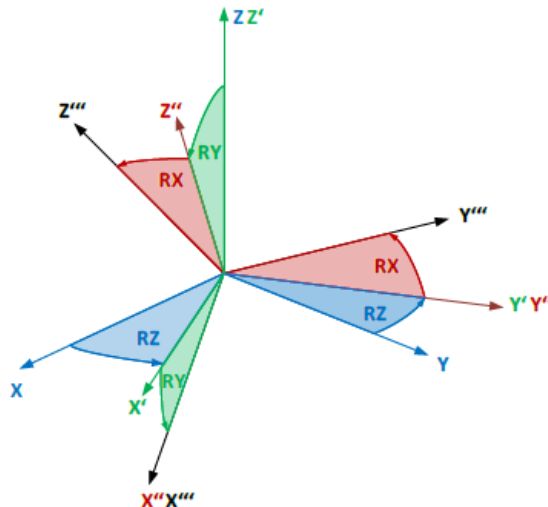


Right-hand coordinate system according to ISO 9787:2013(E).

The rotations are defined as extrinsic x-y-z rotations, which is identical to intrinsic z-y'-x'' rotations.

This are then performed in the following intrinsic sequence:

1. Yaw at Z axis (RZ)
2. Pitch at the (already turned) Y axis (RY)
3. Roll at the (already twice-turned) X axis (RX)



Rotation of a frame

5 Model

This chapter defines the models used to describe and structure coordinated motion functionality. These models provide a conceptual framework for understanding the behavior of motion systems at both the single-axis and multi-axes (grouped axes) level.

The chapter is organized into the following sub-sections:

5.1 – State Diagram

5.2 – Relationship between Single-Axis and Grouped Axes State Diagrams

5.3 – Input Execution Modes

Together, these models support a consistent and standardized understanding of motion control, enabling better interoperability and clarity across different implementations.

5.1 State diagram

The state-diagram of the group describes the commanded state of the group of axes. It is on top of the state diagram per axis (like defined in Parts 1 and 2). While axes are in a group state, the single axis state diagram is also active per axis. Therefore, interdependencies between the 2 types of state-diagrams exist.

GroupDisabled is the initial state at power up where a group can be created. Issuing MC_GroupEnable leaves this state. The next state is GroupStandby. In this state the group is enabled, and no function block has control on one of the axes in the group. In this state the group can additionally be altered and homed if needed (State GroupHoming).

In the state GroupHoming a homing sequence can be defined for a group of axes. This can be applicable due to the mechanical constraints of multiple motors (for example in a mechanical construct looking like the letter “I“ with 2 motor mechanically coupled via one band or belt moving over the form of the letter I, need to be homed differently.).

If a function block has control over (one of the axes of) the group, the state changes to GroupMoving.

GroupStopping is a special state that deals with the MC_GroupStop command, which automatically transfers to the state GroupStandby as soon as “Done” is SET and “Execute” is FALSE in MC_GroupStop.

In case an error arises (in one of the axes) the state changes to GroupErrorStop, which can only be left via issuing MC_ResetGroup.

Explanations:

- Group motion commands will usually lead to a SynchronizedMotion state in the single axis state diagram.
- A GroupErrorStop will not lead to ErrorStops of the grouped axes as the error may only affect the group. In case of a single axis ErrorStop the Group will also change to GroupErrorStop as the single error affects the group.

The state diagram reflects the state of the group and the issued function blocks.

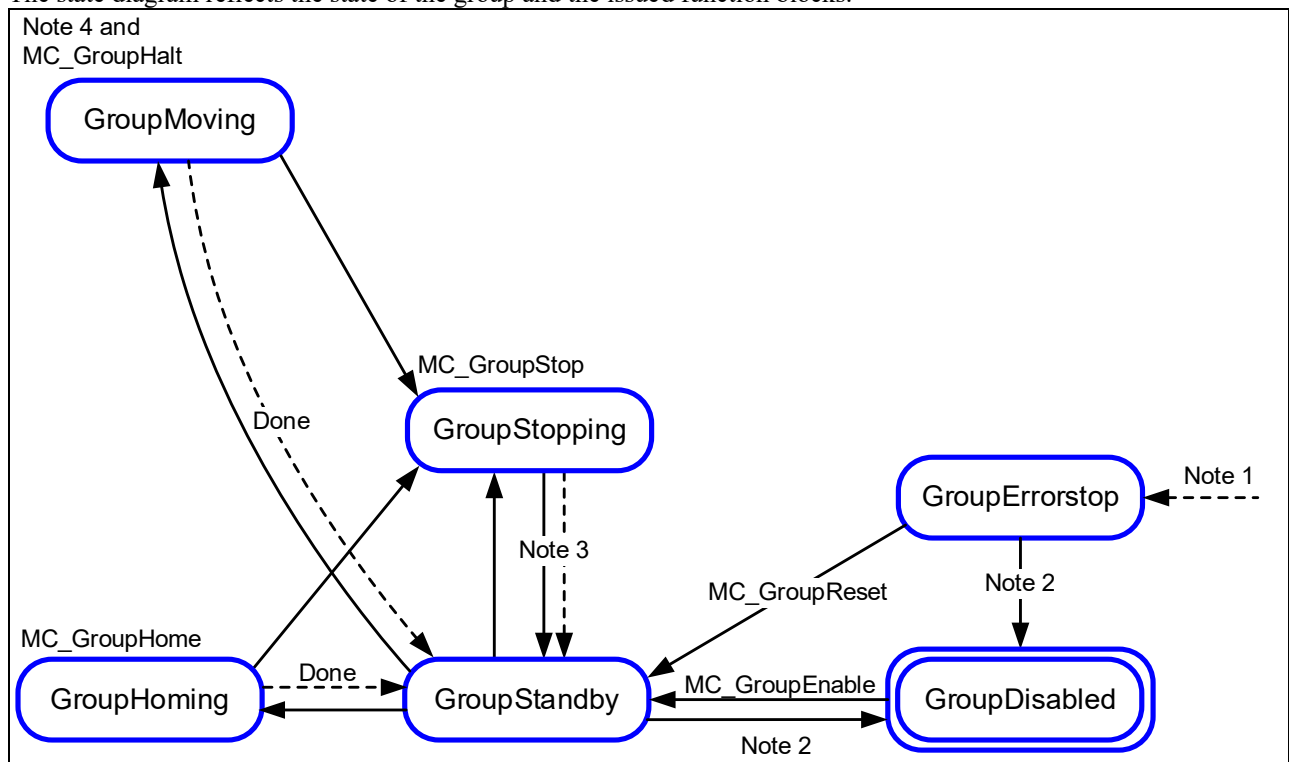


Figure 8: The State Diagram

Note to transitions: Continuous lines are commanded transitions; dotted lines are automatic transitions.

Note 1: From any state, if AXES_GROUP_REF is enabled. An error in the axes group occurred.

Note 2: MC_GroupDisable, MC_UngroupAllAxes or MC_RemoveAxisFromGroup, if the last axis is removed from the group.

Note 3: MC_GroupStop.DONE AND NOT MC_GroupStop.EXECUTE.

Note 4: Applicable for all non-administrative (moving) function blocks.

5.2 Relationship single axis and grouped axes state diagrams

Example of the relationship between 3 single axes combined in an axes group.

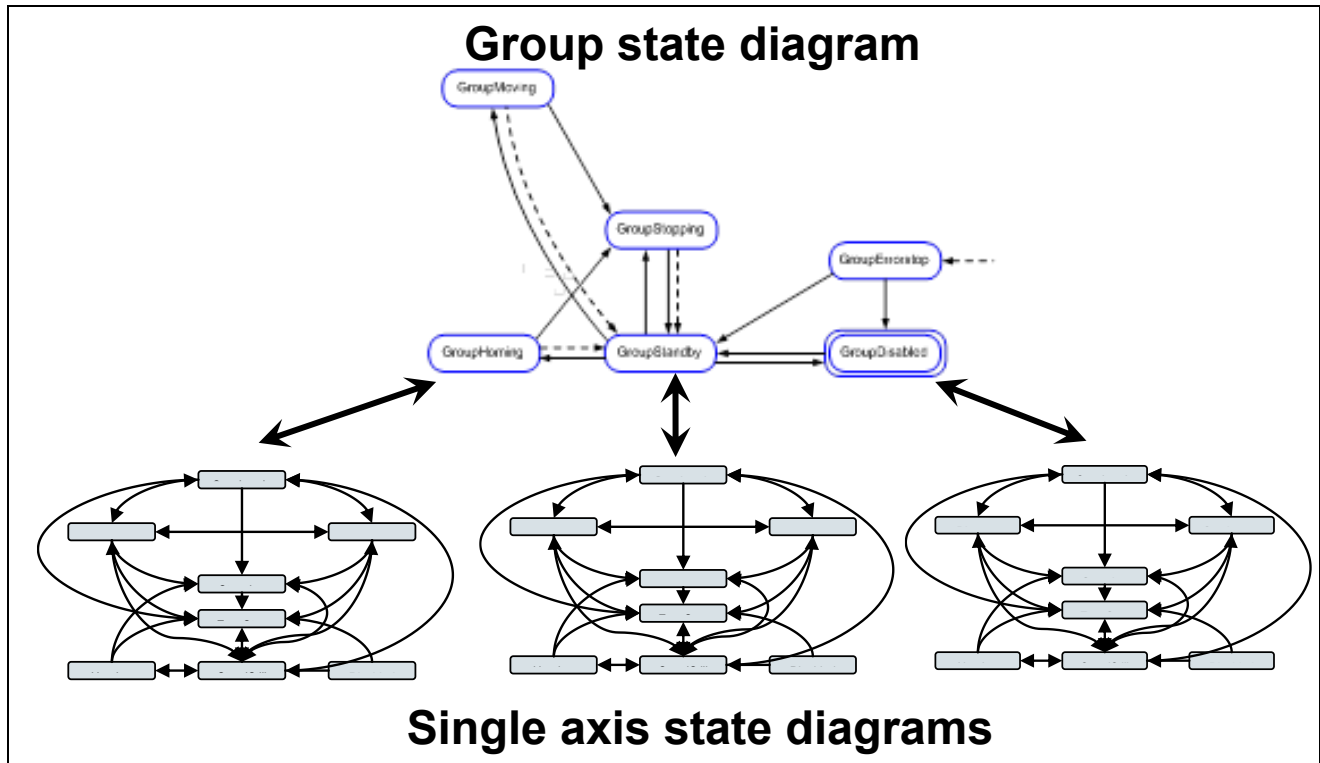


Figure 9: Relationship Single Axis and Grouped Axes State Diagrams

When a number of axes are grouped, and a single axis command, like MC_MoveAbsolute, is issued to an axis in this group, there are basically 3 options:

1. Not allowed. Issuing a single axis command is not accepted and not performed: it signals this by setting the error output of the applicable (issued) single axis function block. There is no change to the group, and as such continues their movements.
2. Aborting the current group command(s), as well as following group commands, and continuing with the single axis command only. The remaining axes of the group move to the state StandStill (via an implicit MC_Halt per axis). The original trajectory will not be finalized. However, the Axes Group stays operational, and the next group command is possible without further action.
3. Superimpose the single axis commands to the group commands.

This specification does not restrict any of these options. This means that different implementations of this behavior will exist, and the vendor of the system must specify what their system does support.

General rules for the interaction between a single axis towards its groups (for all 3 options above):

Those rules are only applicable if the axes group is not in GroupDisabled or GroupErrorStop.:

- If at least one axis in the group is moved by a command, then the group is in the state GroupMoving.
- If all axes are in StandStill, the group can be in the state GroupStandby, GroupDisabled or GroupErrorStop.
- If one axis in a group is in ErrorStop, the whole group is in GroupErrorStop.
- If a single axis MC_Home is issued the group is in state GroupMoving.
- If a single axis MC_Stop is issued the group is in state GroupMoving.
- If supported by the system, it is allowed to disable a single axis of the axes group without influencing the axes group state. This can be useful to save energy or to apply a mechanical brake for a single axis not involved in the on-going motion.

General rules for the interaction between a group and the single axis in it (for all 3 options above):

- If the group is commanded by a group moving command, all the single axes in the group are in the state SynchronizedMotion
- If the group is in the state GroupStandby, the states of the single axes do not have to be all in StandStill
- If the group is in the state GroupErrorStop the state of the single axis is not affected
- If a group gets an error and moves to the state GroupErrorStop, one still can move one or more single axes in the group by issuing single axis commands. The individual axes in the group are not directly affected by the GroupError.

Command	Group State	Axis state
MC_MoveLinearXxx MC_MoveCircularXxx MC_MoveDirectXxx MC_MovePath MC_GroupHalt MC_TrackConveyorBelt MC_TrackRotaryTable MC_SetDynCoordTransform	GroupMoving	SynchronizedMotion
MC_GroupStop	GroupStopping	SynchronizedMotion
MC_GroupReset	GroupErrorStop	MC_GroupReset acts (also) as MC_Reset on each axis in the group (and acknowledge all errors)
MC_GroupHome	GroupHoming	Homing for each axis in the group which is performing homing

Table 4: Overview of the influence of active group commands on a single axis state

Explanation: A stopping group leaves the single axis in synchronized motion as none of the single axis performs a single axis stop.

Notes on Error Behavior of the group and single axis

If an axis, which is member of a group, reports an axis error and this axis is not needed for the commanded movement by means of position is constant or position doesn't care it is possible to avoid a group error stop or to resume from a group error stop in a user-friendly yet vendor specific way:

A) Remove not needed axes from the group before commanding the movement. Axis errors of removed axes do not influence the group and a GroupErrorStop is avoided. It is also possible to remove not needed axes from the group in state GroupErrorStop allowing a change to state GroupStandby.

B) One may implement the feature to mask axes preventing them from reporting errors to the group. Axis errors of masked axes are not visible from the group and a GroupErrorStop could be avoided. It is also possible to mask not needed axes in state GroupErrorStop allowing a change to state GroupStandby.

Example: imagine a robot with a 3 axes tool. The tool axes are for tool on/off (drawing, gluing, etc.), turning (cutting) and moving up-down (drilling). If the turning axis reports an axis error during drawing and turning it is not needed, there is no reason for GroupErrorStop.

6 Axes Grouping

Within this specification for interpolation, the related axes are grouped in an “Axes Group”, and can be accessed via the type AXES_GROUP_REF. The relationship between the different axis levels and groups is shown hereunder.

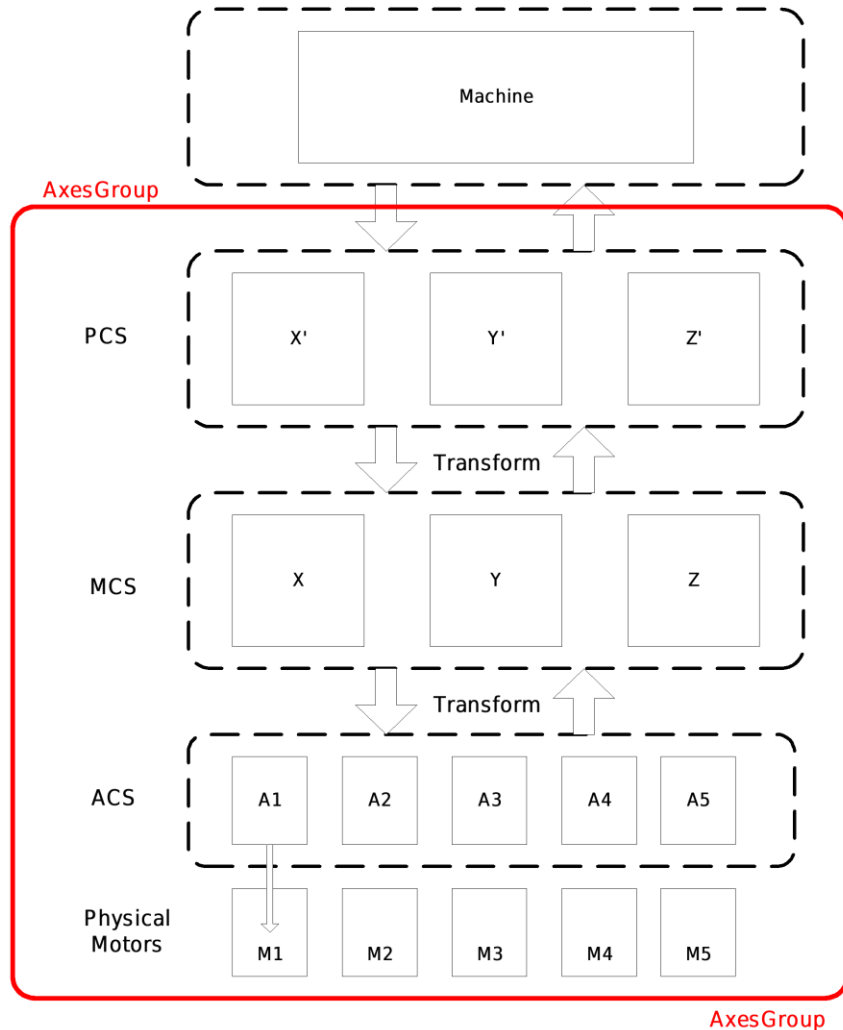


Figure 10: Overview Axes Group

The Axes Group shown in red above provides the interface to the user of the group of axes. To access the relevant coordinate system, the relevant function blocks have an input CoordSystem

An existing Axes Group can be changed. This can be applicable with a tool change which includes an additional motor. This changes both the number of axes as well as the kinematic model. The change can be made in the states GroupDisabled and GroupStandby and adds the axis to the group and changes the link to the applicable kinematic model. The same procedure is valid for changing the tool back, although the MC_RemoveAxisFromGroup is used. Having an axis in the group that is not linked to the kinematics model (yet) is allowed - however the use case is not defined.

Of course, it is possible to use a graphical software tool to generate the steps above, even in a different sense like starting from the kinematics model and generating the software steps to create the axes group with the connections to the transformations like described above.

6.1 Creating and using an Axes Group

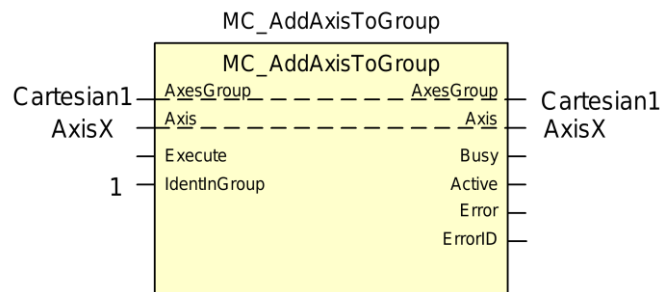
To create a group, one can go through the following steps:

1. If necessary, move the first axis (or all axes) to the relevant position(s) via single axis commands
2. Give the group a name (create a variable of type AXES_GROUP_REF). Group now in state GroupDisabled.
3. Add the first axis to this group via MC_AddAxisToGroup
4. Repeat this till all axes are defined in the group in the right order
5. Link the kinematic model to the group via MC_SetKinTransform even if it is per default a Cartesian system
6. Link the next level(s) of transformation(s) to the group via MC_SetCartesianTransform and /or MC_SetCoordinateTransform
7. Enable the group (via MC_GroupEnable) to use it.

Example

If not done per axis yet, one switches on the power and performs a homing sequence per relevant axis with the single axis function blocks. These relevant axes are now in the single axis state StandStill. (Note: Possibly the homing must (additionally) be done in the group itself, due to special constructional constraints).

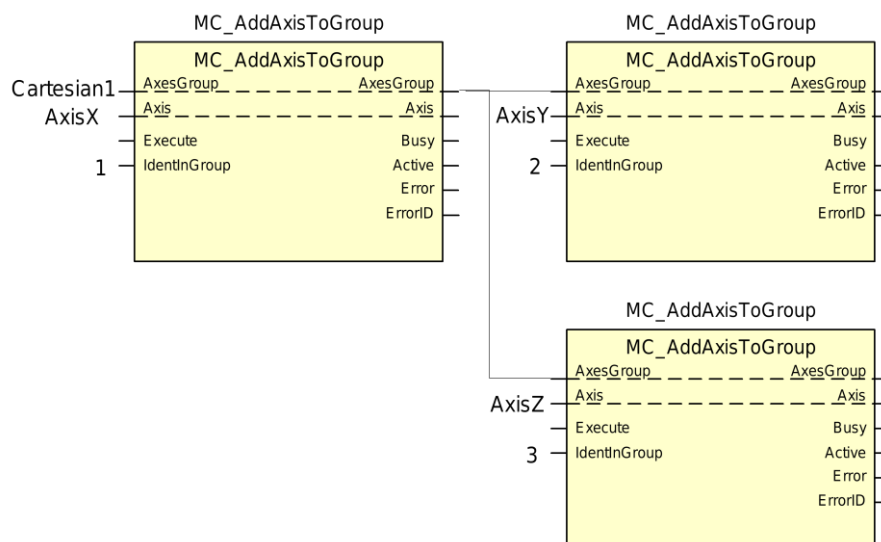
One creates a group by adding the first axis to it and giving it a name.



For instance: MC_AddAxisToGroup with a group named Cartesian1 for AXES_GROUP_REF. The used Axis is referenced via AXIS_REF, like AxisX. The input IdentInGroup gives the reference in the group, like 1.

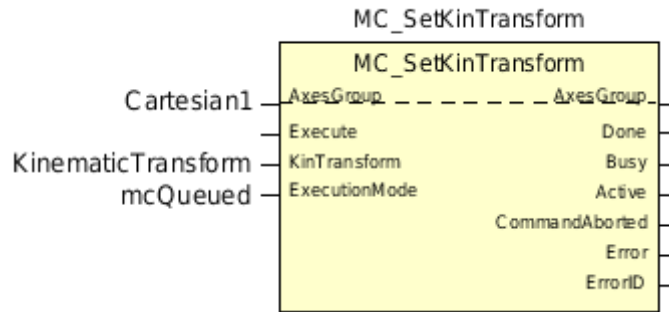
Now a group has one Axis, AxisX, coupled in position 1. The state is still GroupDisabled.

Now we add a second axis, called AxisY, in position 2, and a third axis, AxisZ, in position 3.



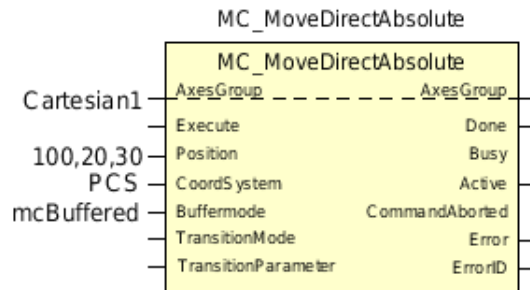
The group now consists of 3 axes, with all axes in the state GroupDisabled.

The next step is linking the transformations to this group. For instance, for the kinematic transformation we use MC_SetKinTransform.



The other relevant function blocks for the transformations are MC_SetCartesianTransform and MC_SetCoordinateTransform.

With issuing MC_GroupEnable we transfer the state of the group to GroupStandby, enabling it to accept movements. A movement changes the state to GroupMoving. To tell the relevant movement command in which coordinate system the coordinates are applicable, the input CoordSystem is used.



After “Done” is set, the state changes back to GroupStandby.

With MC_UngroupAllAxes you ungroup all axes at once, getting them all to the state StandStill for each axis.

7 Robot Dynamics and Payload

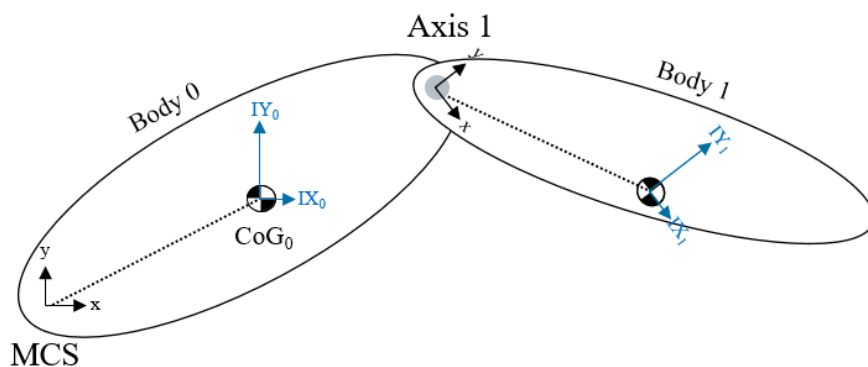
To calculate forces and torques that occur during movement of an axes group, or even when stationary, all mechanical quantities of all (moving) parts of an axis group must be known. This comprises all links, the tool and the payload of the axes group. For this a data structure MC_RIGID_BODY_DYNAMIC_REF is defined, which specifies all mechanical quantities of one rigid body.

MC_RIGID_BODY_DYNAMIC_REF can have 10 values for each rigid body: mass, center of mass (3 values), 6 moments of inertia (can be 3 orientation & 3 inertias). In addition, friction can be a part (or part of the joint definition). (Note: these 10 values seem to be sufficient for other rigid body dynamics, like delta robots)
In many cases a separate software tool is used to generate this description of the robot itself.

Typical usage of this is shown in the function blocks MC_ReadRigidBodyDynamic, MC_WriteRigidBodyDynamics, MC_ReadPayloadData and MC_WritePayloadData.

7.1 MC_RIGID_BODY_DYNAMIC_REF

MC_RIGID_BODY_DYNAMIC_REF is a data structure that holds the center of gravity, moments of inertia, and mass of a link (rigid body) with respect to the base coordinate system or the axis (joint) where the link is attached to.



```

TYPE MC_RIGID_BODY_DYNAMIC_REF :
STRUCT
    // The center of gravity and the orientation of the
    // principal axes (see IX/IY/IZ). Expressed relative
    // to the base coordinate system.
    CenterOfGravity : MC_CART_REF;
    // The mass in [kg].
    Mass : REAL;
    // The moment of inertia I of the load around the
    // X-axis of frame CenterOfGravity in [kg*m²]
    IX : REAL;
    // The moment of inertia I of the load around the
    // Y-axis of frame CenterOfGravity in [kg*m²]
    IY : REAL;
    // The moment of inertia I of the load around the
    // Z-axis of frame CenterOfGravity in [kg*m²]
    IZ : REAL;
END_STRUCT
END_TYPE

```

8 Tool management

There are several function blocks defined to support working with different tools. The tool transformation is a specific form of coordinate transformation from FCS to TCS.

The function blocks for Tool Management all use the datatype MC_TOOL_REF as defined in 8.1 MC_TOOL_REF.

8.1 MC_TOOL_REF

The datatype MC_TOOL_REF is a vendor specific data type to describe the tool. It includes at least the offset and orientation of the tool coordinate system (TCS) to the robot flange.

The displacement values in MC_TOOL_REF are like the ones in MC_COORD_REF and vendors should use the same definition and naming here. Vendor specific extension might apply to MC_TOOL_REF (e.g. gripper opening and closing times, dynamic parameters, max. payload) which are not applicable to MC_COORD_REF. Therefore, two data types are defined.

In this example, MC_TOOL_REF contains additional information about the center of mass and inertia of the tool.

```
TYPE MC_TOOL_REF :  
STRUCT  
    // The kinematic parameters of the tool: the TCP  
    // expressed in the flange coordinate system.  
    ToolFrame : MC_COORD_REF;  
    // Optional: the rigidbodydynamic of the tool (excluding the payload)  
    RigidBodyDynamic : MC_RIGID_BODY_DYNAMIC_REF;  
    <additionalData> : VENDOR_SPECIFIC;  
END_STRUCT  
END_TYPE
```

If OOP is supported, MC_TOOL_REF could also extend MC_COORD_REF.

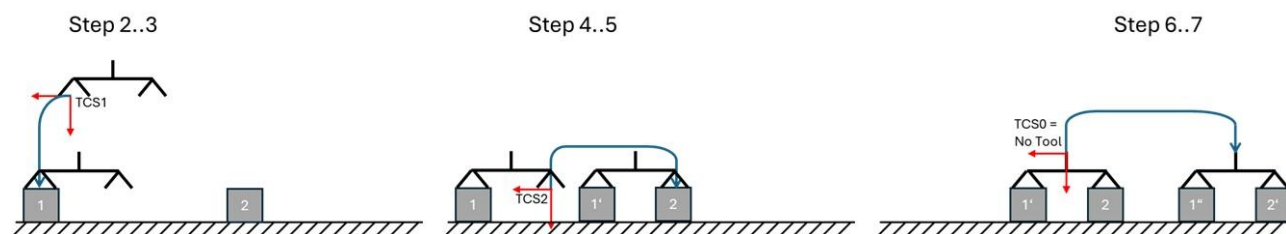
8.2 Example procedure for a multi-gripper-application

Below shows an example of changing a gripper thus changing the TCP via MC_SelectTool.

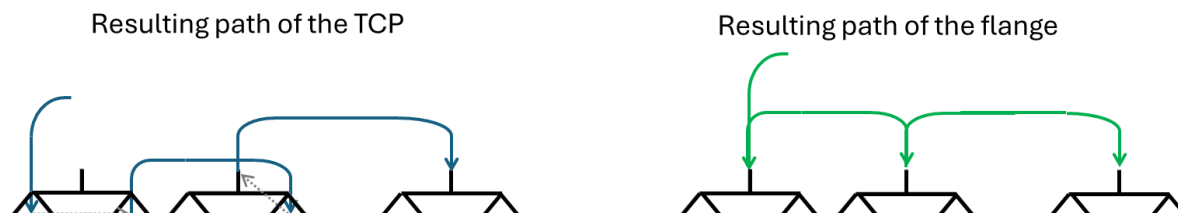
Step	Description	Used function blocks
1	Define the offsets of both grippers using tool 1 and tool 2	MC_WriteToolData
2	Select gripper 1	MC_SelectTool
3	Execute pick movement to object 1	MC_MoveLinearAbsolute
4	Activate gripper 2	MC_SelectTool
5	Execute pick movement to object 2	MC_MoveLinearAbsolute
6	Activate NoTool if necessary	MC_SelectTool
7	Execute place movement to the deposit position	MC_MoveLinearAbsolute
8 → 2	Repeat motion sequence	

Note: If a motion is already commanded, MC_SelectTool will be buffered within the motion queue and only acts on following movements that are executed afterwards.

The above example results in the following motion profiles.



In the resulting path the TCP position will be discontinuous while changing the tool (dashed gray lines) while the position of the flange and therefore the positions of the ACS axes are continuous.,



9 Function Blocks for Coordinated Motion

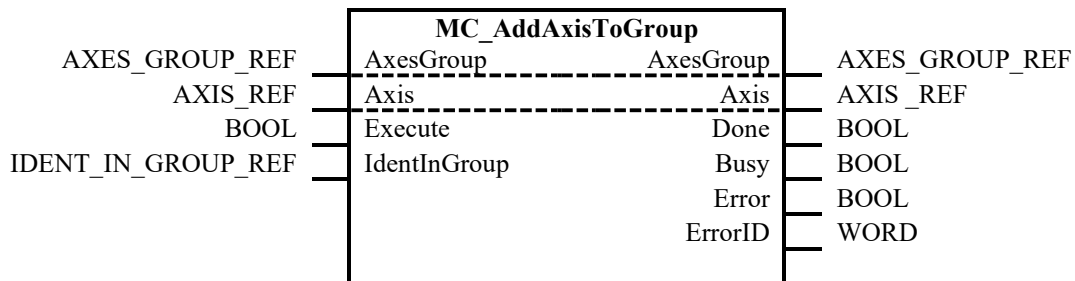
This chapter defines the set of function blocks that support coordinated motion control within the PLCopen framework. These function blocks are designed to enable the structured programming of multi-axis motion applications based on the principles defined in earlier PLCopen specifications.

The function blocks are organized into two main categories:

- **Administrative Function Blocks:** These provide the necessary tools to configure, manage, and monitor coordinated motion groups. They include functionalities such as group creation, axis assignment, synchronization control, and status handling.
- **Motion Function Blocks:** These define the actual motion commands used for coordinated movement of multiple axes. They include commands for path planning and motion execution.

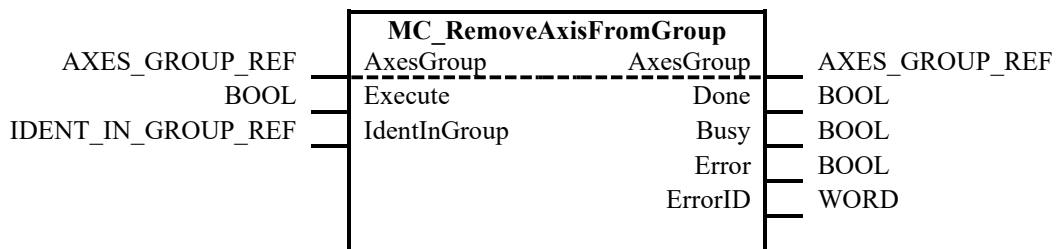
9.1 MC_AddAxisToGroup

FB-Name		MC_AddAxisToGroup		
This function block adds one axis to a group in a structure axes group. This is an administrative FB, since no movement is generated. The command cannot be buffered.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
B	Axis	AXIS_REF	Reference to the axis to be added	
VAR INPUT				
B	Execute	BOOL	Start the grouping process at the rising edge	
E	IdentInGroup	IDENT_IN_GROUP_REF	Identifies the order in the group of the added axis. Done via a REF in order to give the different axes a name in the order, which can be coupled to the names in the kinematic model (like “foot”, “shoulder”)	
VAR OUTPUT				
B	Done	BOOL	AXES_GROUP_REF is valid and the axis added	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Note: Each IdentInGroup can be used only once, otherwise it leads to an error				



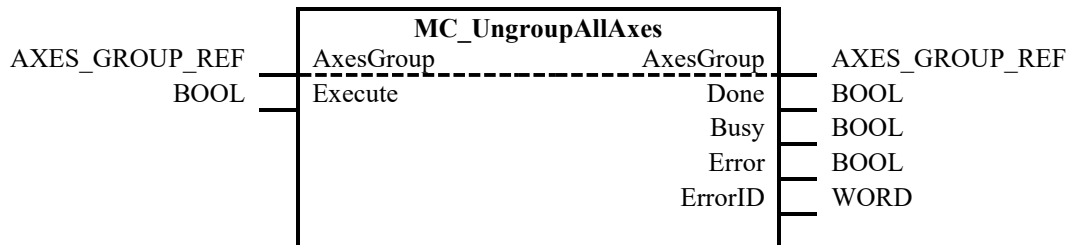
9.2 MC_RemoveAxisFromGroup

FB-Name		MC_RemoveAxisFromGroup		
This function block removes one axis from the group AxesGroup. This is an administrative FB, since no movement is generated. The command cannot be buffered. If there is no axis left in the group, the state changes to GroupDisabled.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the axis removal process at the rising edge	
E	IdentInGroup	IDENT_IN_GROUP_REF	Identifies the axis in the group	
VAR OUTPUT				
B	Done	BOOL	AXES_GROUP_REF is valid and the axis removed	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes: If issued on a group that is not in GroupDisabled, GroupStandby or GroupErrorStop, it generates an error and the FB is not executed.				



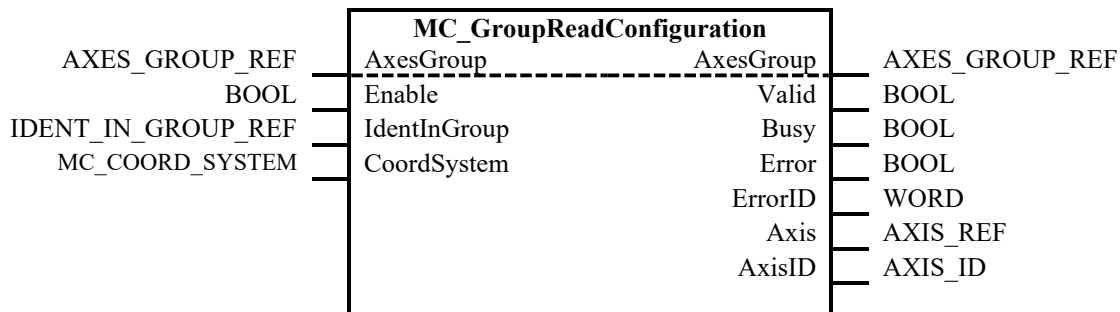
9.3 MC_UngroupAllAxes

FB-Name		MC_UngroupAllAxes		
This function block removes all axes from the group AxesGroup. This is an administrative FB, since no movement is generated. The command cannot be buffered. After finalization the state is changed to GroupDisabled				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the process at the rising edge	
VAR OUTPUT				
B	Done	BOOL	All axes are removed	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes: If issued on a group that is not in GroupDisabled, GroupStandby or GroupErrorStop, it generates an error and the FB is not executed.				



9.4 MC_GroupReadConfiguration

FB-Name		MC_GroupReadConfiguration		
This function block gets the axis reference according to the given group identifier in order to read the current configuration of an axes group. For CoordSystem-Input “ACS” you get a conventional axis reference, but for other values of the CoordSystem-Input you get the axis reference of a virtual axis according to the transformation that is active. This is an administrative FB, since no movement is generated.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to the group of axes	
VAR_INPUT				
B	Enable	BOOL	Gets the axis reference according to the given group identifier while enabled	
B	IdentInGroup	IDENT_IN_GROUP_REF	Identifies the axis in the group	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system.	
VAR_OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	Axis	AXIS_REF	Reference to the selected axis	
E	AxisID	AXIS_ID	Alternative reference to the axis, uniquely identifying the AXES	
Notes: An AXIS_REF as output-only is not supported by all systems. For this reason, there is an alternative AXIS_ID specified.				



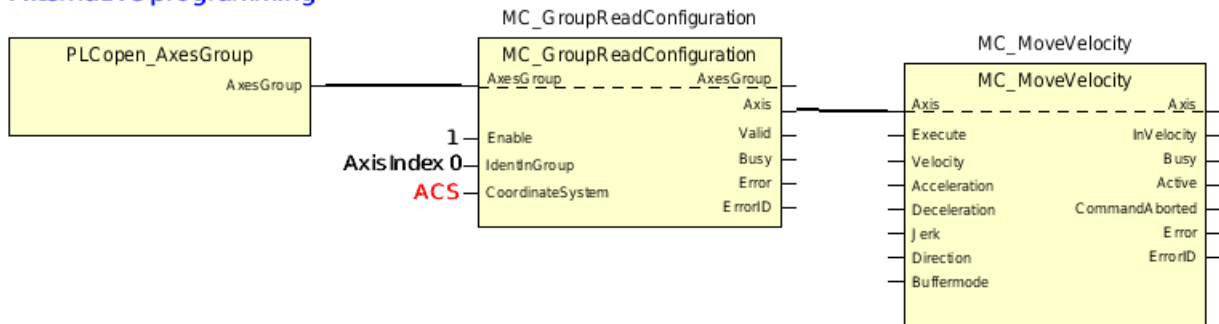
Examples on how to use this function block:

1. Conventional single axis motion → a single axis is moved

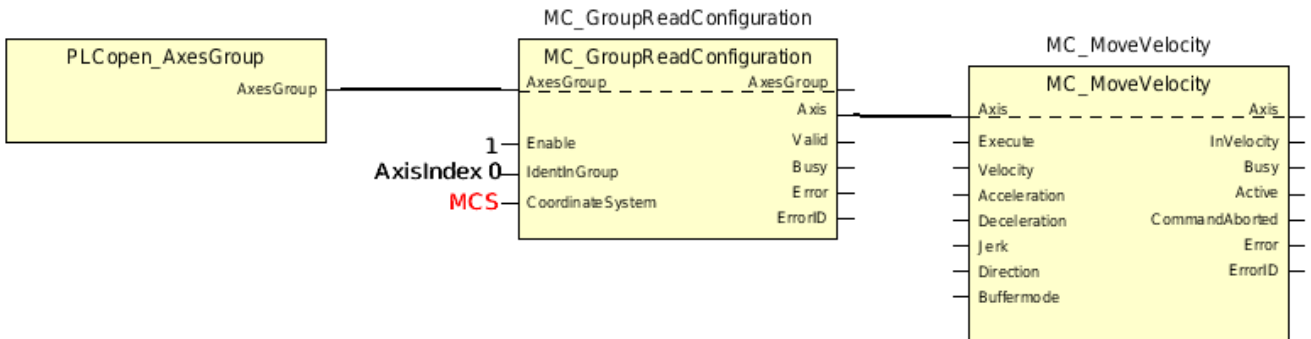
Conventional programming



Alternative programming

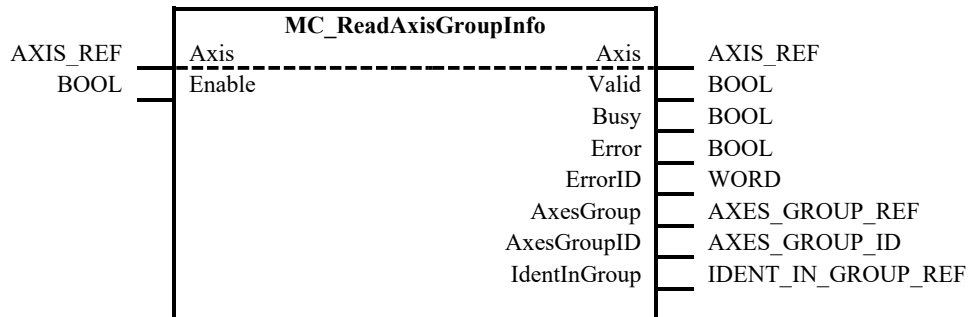


2. Virtual single axis motion → a path axis is moved



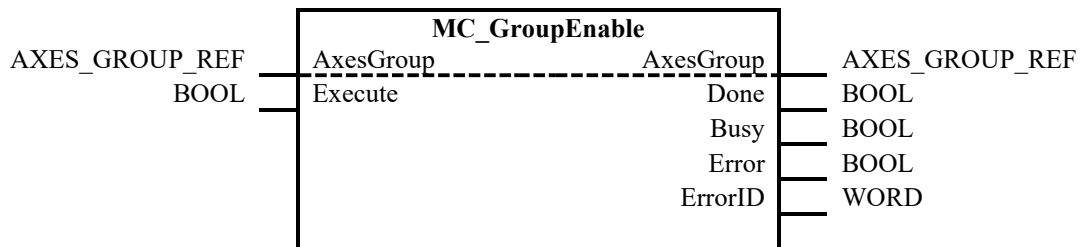
9.5 MC_ReadAxisGroupInfo

FB-Name		MC_ReadAxisGroupInfo		
This function block returns the axes group of an axis, as well as its position in it.				
VAR IN OUT				
B	Axis	AXIS_REF	Reference to the axis	
VAR INPUT				
B	Enable	BOOL	Gets the axes group info while enabled	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
E	AxesGroup	AXES_GROUP_REF	Reference to the axes group	
E	AxesGroupID	AXES_GROUP_ID	Alternative reference to the axes group, uniquely identifying the AXES_GROUP	
B	IdentInGroup	IDENT_IN_GROUP_REF	Identifies the axis in the group	
Notes: An AXES_GROUP_REF as output-only is not supported by all systems. For this reason, there is an alternative AXES_GROUP_ID specified.				



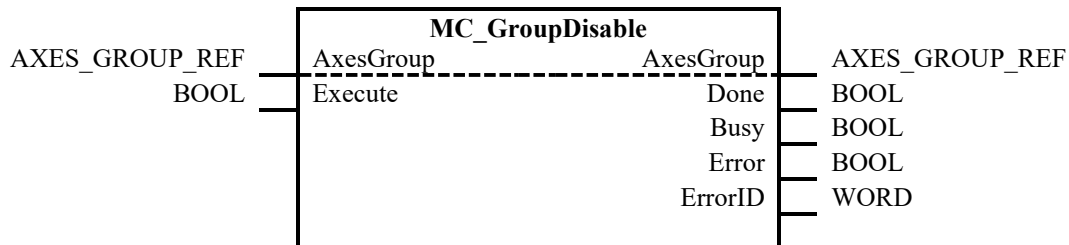
9.6 MC_GroupEnable

FB-Name		MC_GroupEnable		
This function block changes the state for a group from GroupDisabled to GroupStandby. This is an administrative FB, since no movement is generated. See Chapter 3 Principles of coordinated motion				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR_INPUT				
B	Execute	BOOL	Start the change of state at the rising edge	
VAR_OUTPUT				
B	Done	BOOL	AxesGroup in state GroupStandby	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Note: The command does not influence the power state of any of the single axes in the group				



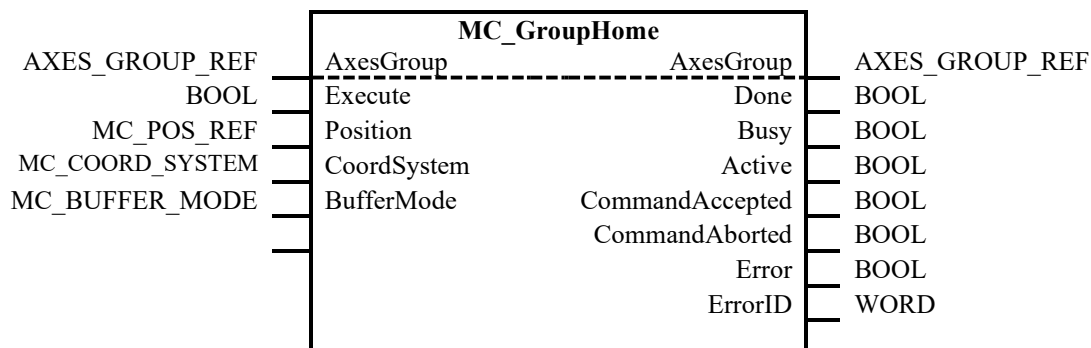
9.7 MC_GroupDisable

FB-Name		MC_GroupDisable		
This function block changes the state for a group to GroupDisabled, although it is an administrative FB, since no movement is generated. If the axes are not standing still while issuing this command, it is up to the application to take the necessary precautions.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the change of state at the rising edge	
VAR OUTPUT				
B	Done	BOOL	AxesGroup in state GroupDisabled	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Note: The command does not influence the power state of any of the single axes in the group				



9.8 MC_GroupHome

FB-Name		MC_GroupHome		
This function block commands the AxesGroup to perform the «search home» sequence. The details of this sequence are vendor specific. The “Position” input is used to set the absolute position when reference signal is detected. This FB completes at “GroupStandby”.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the homing sequence at the rising edge	
B	Position	MC_POS_REF	Absolute position when reference signal is detected.	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system of the position input.	
E	BufferMode	MC_BUFFER_MODE	Buffer Modes “mcAborting” and “mcBuffered” are used. All other modes are not allowed and return an error.	
VAR OUTPUT				
B	Done	BOOL	Homing sequence ended successfully	
E	Busy	BOOL	The FB is not finished	
E	Active	BOOL	Indicates that the FB is processed	
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Note: Blending is not supported by MC_GroupHome				



9.9 MC_GroupPower

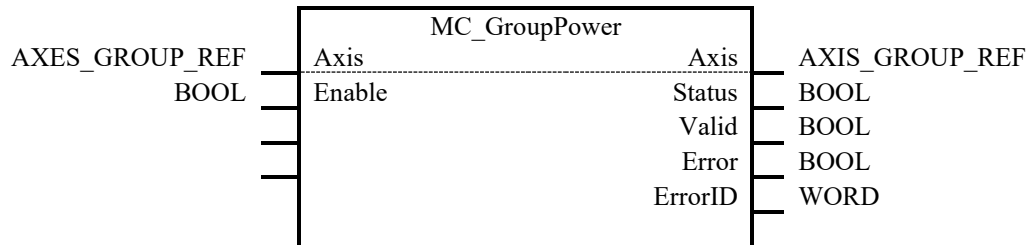
FB-Name		MC_GroupPower		
This function block controls the power stage (On or Off) of the axes group.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR_INPUT				
B	Enable	BOOL	If 'Enable' is true, power is being enabled.	
DVAR_OUTPUT				
B	Status	BOOL	Effective state of the power stage	
E	Valid	BOOL	If true, a valid set of outputs is available at the FB	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	

Notes:

MC_GroupPower does not influence the group state diagram when 'Enable' = TRUE

If power fails (also during operation) it will generate a transition to the 'GroupErrorStop' state.

Either MC_Power or MC_GroupPower is allowed per axis at the same time. Adding an axis to a powered axes group, enables power for that added axis.

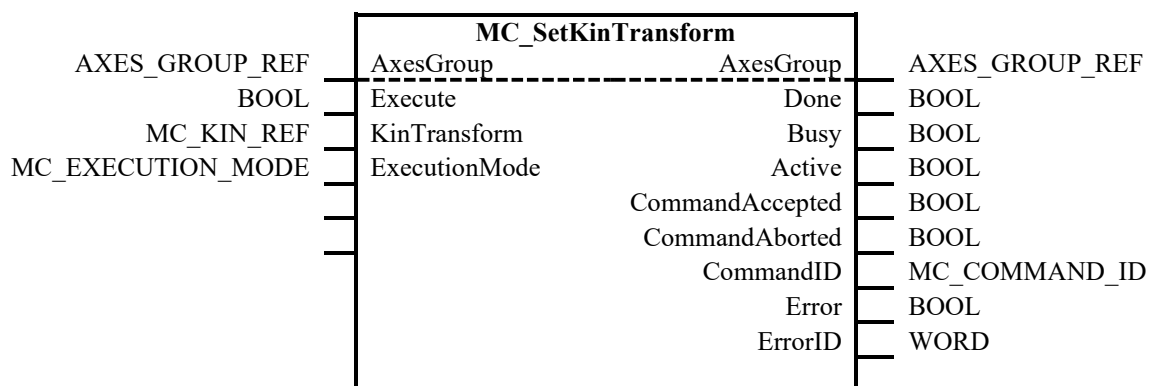


9.10 Transformation FBs

This chapter provides an overview of the transformation function blocks. Although they are administrative FBs, the transformation FBs can be buffered.

9.10.1 MC_SetKinTransform (ACS to MCS)

FB-Name		MC_SetKinTransform	
This function block sets a kinematic transformation between the ACS and MCS based on the predefined kinematic model.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT			
B	Execute	BOOL	Sets the kinematic model transformation on rising edge
E	KinTransform	MC_KIN_REF	Reference to a Kinematic Model. Vendor specific datatype.
E	ExecutionMode	MC_EXECUTION_MODE	Describes when the command is executed and the new transformation becomes valid. (See 2.2.1 Input Execution Mode)
VAR OUTPUT			
B	Done	BOOL	Transformation is set successfully
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB is processed
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	Command is aborted by another command
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes:			
- A kinematic transformation is a representation of machine construction. For kinematically simple machine constructions, like a three axes Cartesian robot, a kinematic transformation may not be necessary. - The input KinTransform refers to a kinematic model including the parameters. The details of the kinematic model and of the parameters are outside the scope of PLCopen. - The system may support a neutral KinTransform. When activating the neutral transformation, the axes are referenced in the ACS system again.			



Timing Diagram:

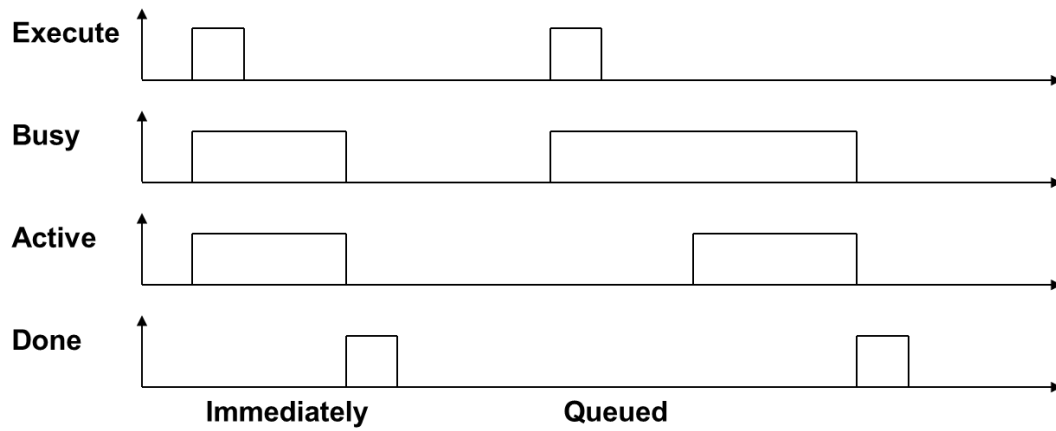
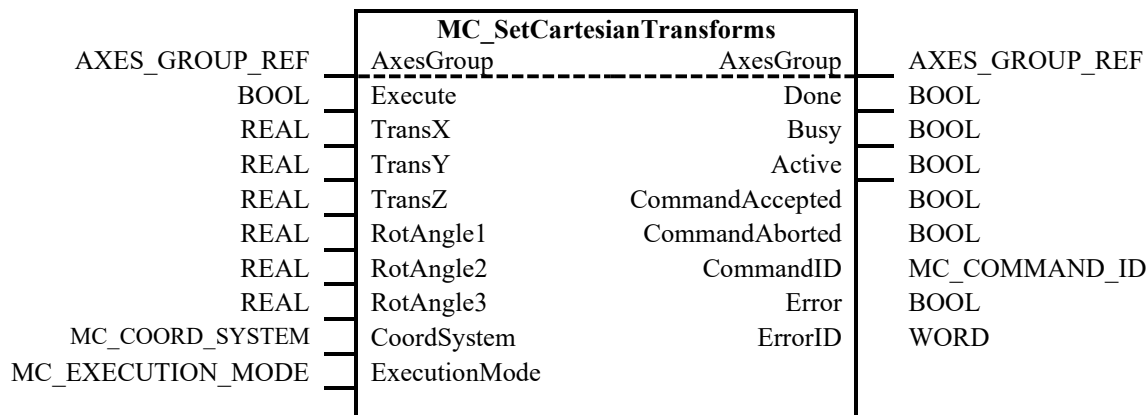


Figure 11: Typical timing diagram for setting the transformation

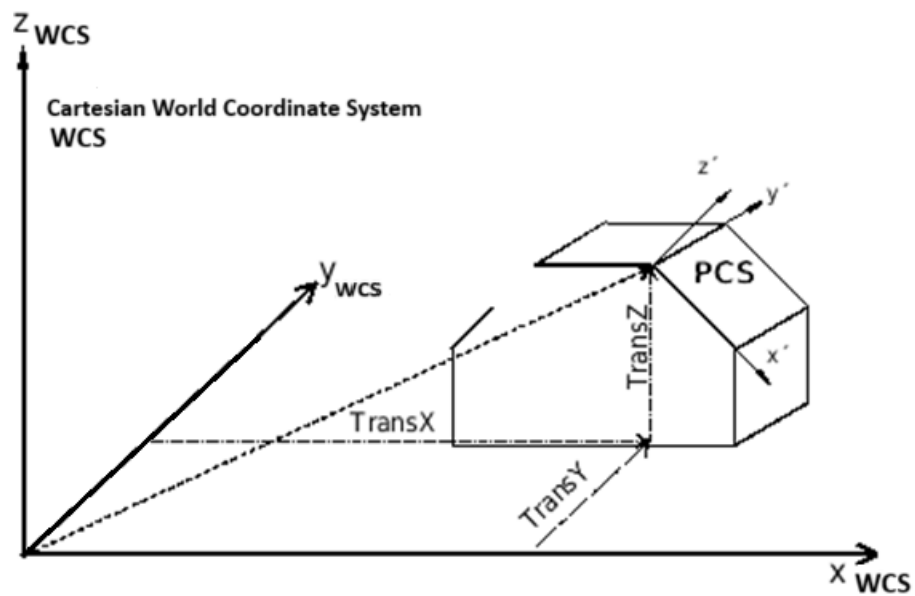
9.10.2 MC_SetCartesianTransform (WCS to PCS and WCS to MCS)

FB-Name		MC_SetCartesianTransform	
This function block sets a Cartesian transformation between the WCS and PCS or WCS and MCS.			
VAR IN OUT			
B	AxesGroup	AXES GROUP REF	Reference to a group of axes
VAR INPUT			
B	Execute	BOOL	Sets the cartesian transformation on rising edge
B	TransX	REAL	X-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	TransY	REAL	Y-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	TransZ	REAL	Z-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle1	REAL	Rotation angle component from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle2	REAL	Rotation angle component from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle3	REAL	Rotation angle component from the origin of the WCS to the selected CoordSystem (MCS or PCS)
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system, which should be set: MCS or PCS
E	ExecutionMode	MC_EXECUTION_MODE	Describes when the command is executed and the new transformation becomes valid. (See 2.2.1 Input Execution Mode).
VAR OUTPUT			
B	Done	BOOL	Transformation is set successfully
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB is processed
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	Command is aborted by another command
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes:			
- The interpretation and order of the vector components (specifically the rotation components) are vendor specific - Resetting of PCS to the base can be done by a execution of this FB with {TransX, TransY, TransZ, RotAngle1, RotAngle2, RotAngle3 }={0, 0, 0, 0, 0, 0} as translation and rotation input values. - More than one cartesian transformation can be applicable at the same time on the same group of axes.			



The timing diagram is equal to that of MC_SetKinTransform

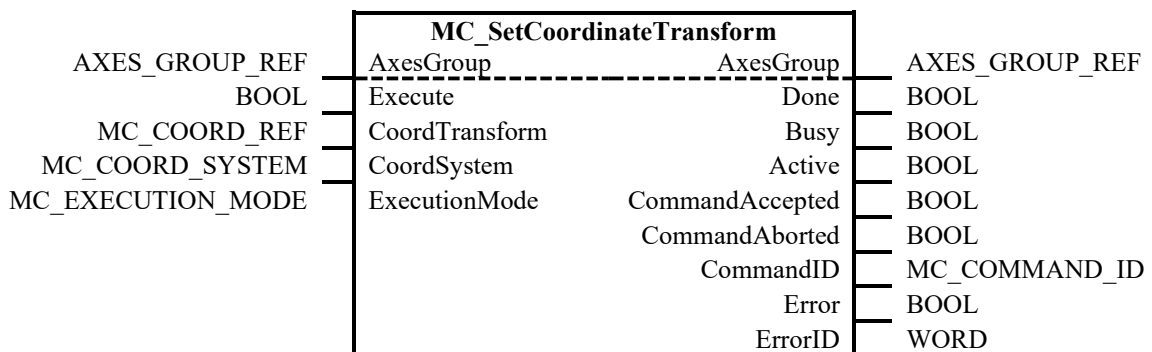
Definition of the translation:



For an example of the order of the rotation, see Ch. 3.4 Blending and Buffering of movements

9.10.3 MC_SetCoordinateTransform (WCS to PCS and WCS to MCS)

FB-Name		MC_SetCoordinateTransform		
This function block sets a coordinate transformation between the WCS and PCS or WCS and MCS. It can be used for generic transformations e.g. non-cartesian transformations like cylindrical.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Sets the coordinate transformation on rising edge	
E	CoordTransform	MC_COORD_REF	Reference to a Coordinate Transformation. Vendor specific datatype.	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system, which should be set: MCS or PCS	
E	ExecutionMode	MC_EXECUTION_MODE	Defines the chronological sequence of the FB. mcImmediately is the default behaviour. Both mcImmediately and mcQueued are supported.	
VAR OUTPUT				
B	Done	BOOL	Transformation is set successfully	
E	Busy	BOOL	The FB is not finished	
E	Active	BOOL	Indicates that the FB is processed	
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed	
E	CommandAborted	BOOL	Command is aborted by another command	
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes:				
- CoordTransform refers to a coordinate transformation including the parameters. The details of the transformation and of the parameters are outside the scope of PLCopen. - Resetting of PCS to the base can be done by a execution of this FB with a neutral transformation. - When PCS is dynamic (in the sense that the PCS is moving relative to MCS), one should use MC_SetDynCoordTransform. - This FB does not start a movement (administrative FB).				



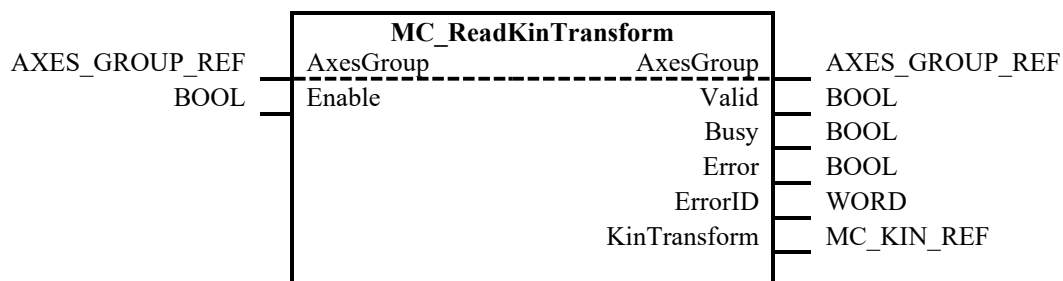
Example of MC_COORD_REF

As an example of MC_COORD_REF, one can use a structure of the 6 inputs X, Y, Z, and rotations as defined in MC_SetCartesianTransforms.

The timing diagram is equal to that of MC_SetKinTransform

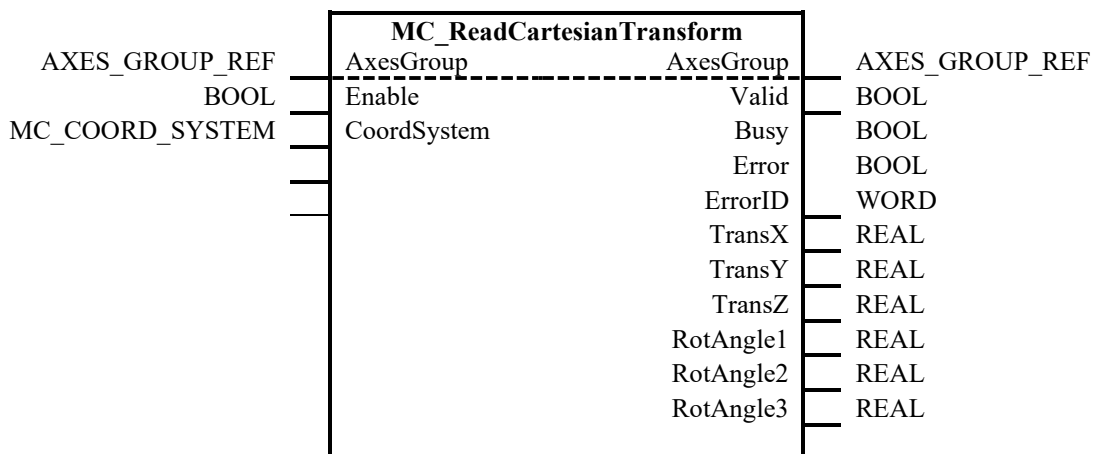
9.10.4 MC_ReadKinTransform (ACS to MCS)

FB-Name		MC_ReadKinTransform		
This function block reads the kinematic transformation that is active between the ACS and MCS.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR_INPUT				
B	Enable	BOOL	Get the actual kinematic transformation reference of the axes group continuously while enabled	
VAR_OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	KinTransform	MC_KIN_REF	Reference to a Kinematic Model. Vendor specific datatype.	
Notes:				



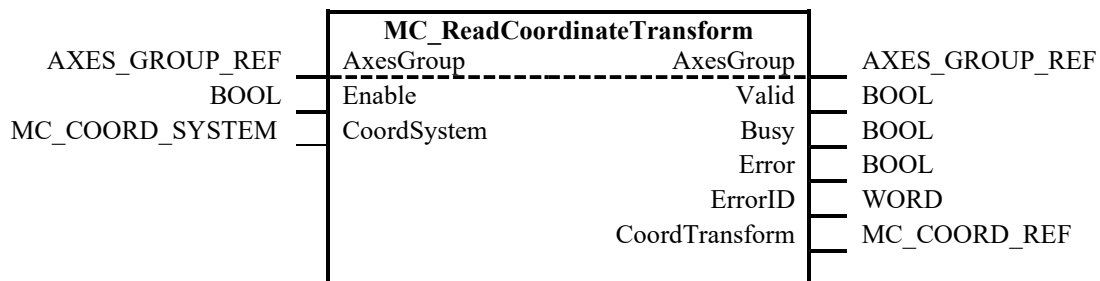
9.10.5 MC_ReadCartesianTransform (WCS to PCS and WCS to MCS)

FB-Name		MC_ReadCartesianTransform	
This function block reads the parameter of the cartesian transformation that is active between the WCS and PCS or WCS and MCS respectively.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT			
B	Enable	BOOL	Get the cartesian transformation parameter of the axes group continuously while enabled
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system, which should be read: MCS or PCS
VAR OUTPUT			
B	Valid	BOOL	True if valid outputs are available
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
B	TransX	REAL	X-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	TransY	REAL	Y-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	TransZ	REAL	Z-component of Translation Vector from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle1	REAL	Rotation angle 1 from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle2	REAL	Rotation angle 2 from the origin of the WCS to the selected CoordSystem (MCS or PCS)
B	RotAngle3	REAL	Rotation angle 3 from the origin of the WCS to the selected CoordSystem (MCS or PCS)
Notes: -			



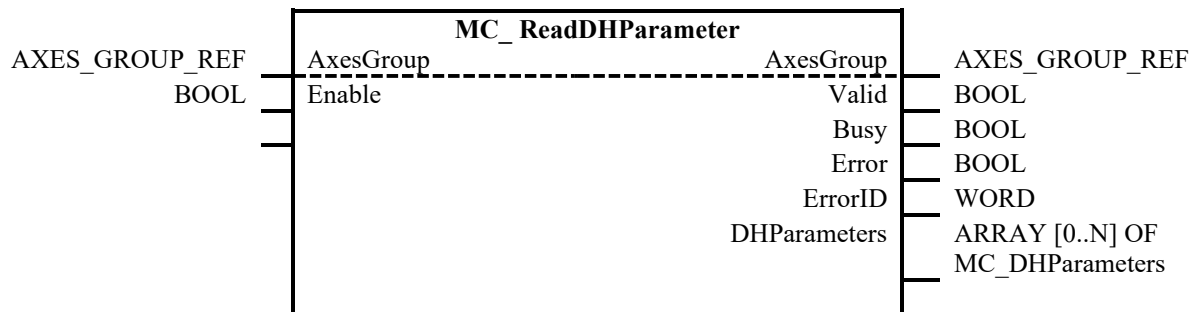
9.10.6 MC_ReadCoordinateTransform (MCS to PCS)

FB-Name		MC_ReadCoordinateTransform		
This function block reads the parameter of the coordinate transformation that is active between the WCS and PCS or WCS and MCS respectively				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Enable	BOOL	Get the actual coordinate transformation reference of the axes group continuously while enabled	
B	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system, which should be read: MCS or PCS	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	CoordTransform	MC_COORD_REF	Reference to a Coordinate Transformation from the origin of the WCS to the selected CoordSystem (MCS or PCS). Vendor specific datatype.	
Notes: -				



9.11 MC_ReadDHPParameters

FB-Name		MC_ReadDHPParameters	
This function block reads Denavit-Hartenberg (DH) parameters of the kinematic			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR INPUT			
B	Enable	BOOL	Reads DHPParameters if TRUE
VAR OUTPUT			
B	Valid	BOOL	The command has been completed successfully.
E	Busy	BOOL	The FB is being processed
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
B	DHPParameters	ARRAY [0..N] OF MC_DHPParameter	DH parameters for each link. DHPParameters[0] refers to the base link.
Note 1: This FB is only applicable to serial kinematics			



DH parameters may be used to calculate the forward kinematics to convert joint positions to a cartesian position with orientation.

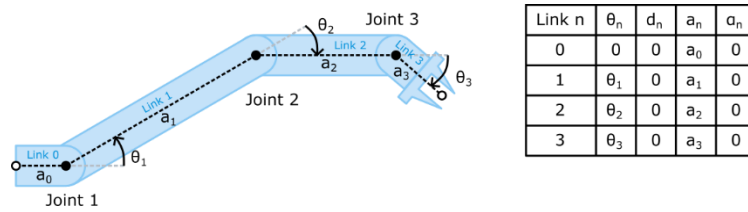
They consist of four parameters:

- θ_n (Theta)
- d_n
- a_n
- α_n (Alpha)

Every parameter is returned for each link attached to the kinematic chain, with link 0 being the base of the kinematic.

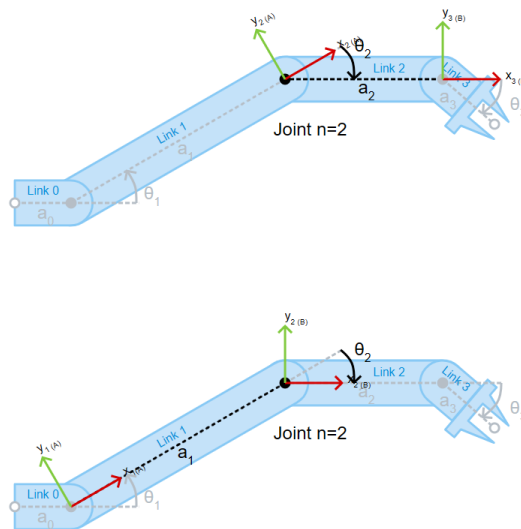
Structure Name		MC_DHPParameters	
B	Theta	REAL	In rotary units
B	D	REAL	In linear units
B	A	REAL	In linear units
B	Alpha	REAL	In rotary units

The following illustration shows an example of a kinematic with 4 links and three joints:



Example DH parameters

Note: DH-parameters are commonly used to derive DH transformations. The classic (distal) transformation for joint n uses parameters θ_n , d_n , a_n and α_n . The modified (proximal) transformation for joint n uses parameters a_{n-1} , α_{n-1} , θ_n and d_n .



Example of both variants of DH-transformation for joint 2. Classic (distal) on top, modified (proximal) at bottom.

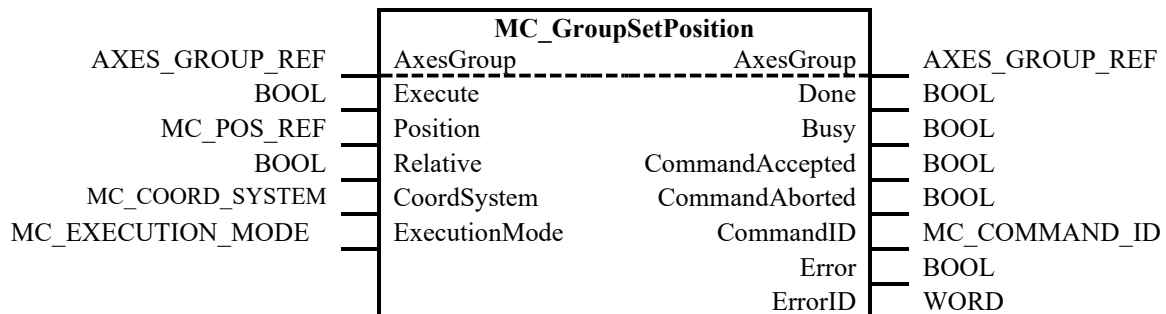
9.12 MC_ReadJointInfo

FB-Name		MC_ReadJointInfo		
This function block reads joint information of the kinematic				
VAR IN OUT				
	B	AxesGroup	AXES GROUP REF	
VAR INPUT				
	B	Enable	BOOL	Reads DHParameters if TRUE
VAR OUTPUT				
	B	Valid	BOOL	The command has been completed successfully.
	E	Busy	BOOL	The FB is being processed
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	B	JointInfo	ARRAY [0..N] OF JointInfo	Joint information on each joint.
Note 1: This FB is only applicable to serial kinematics				

Structure Name		MC_JointInfo	
B	ZeroPosition	REAL	
B	DirectionClockwise	BOOL	TRUE if the point moves clockwise around Z with increasing position (left-handed rule)

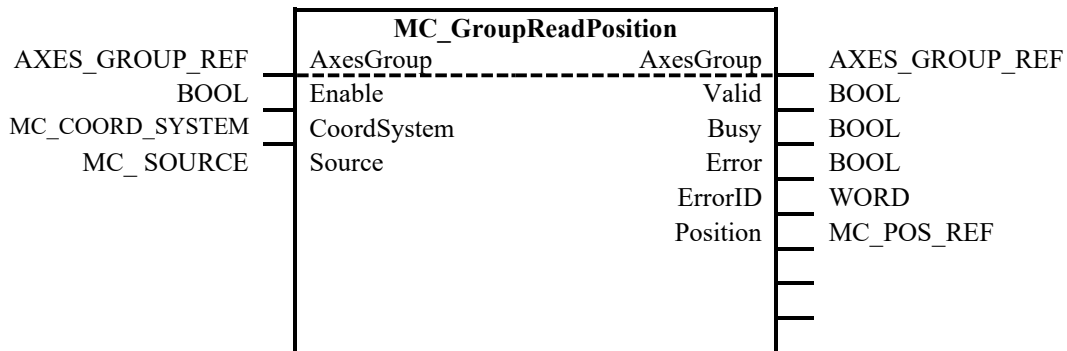
9.13 MC_GroupSetPosition

FB-Name		MC_GroupSetPosition	
This function block sets the position of all axes in a group without influencing the movement of the axes. The new coordinates are described in an array. With the coordinate system input the according coordinate system is selected. It can be seen as a way of referencing or a transformation. MC_GroupSetPosition shifts the position of the addressed coordinate system and affects the higher-level coordinate systems (e.g. if ACS is selected, MCS and PCS are affected).			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axis
VAR INPUT			
B	Execute	BOOL	Start the action at rising edge
B	Position	MC_POS_REF	Position of the axes group after execution. If Relative = TRUE, position is used as distance.
E	Relative	BOOL	Mode of position inputs - RELATIVE =True, ABSOLUTE = False (Default)
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system of the position input.
E	ExecutionMode	MC_EXECUTION_MODE	Defines the chronological sequence of the FB. mcImmediately is the default behaviour. Both mcImmediately and mcQueued are supported.
VAR OUTPUT			
B	Done	BOOL	'Position' has new value
E	Busy	BOOL	The FB is not finished
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	Command is aborted by another command
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Note:			



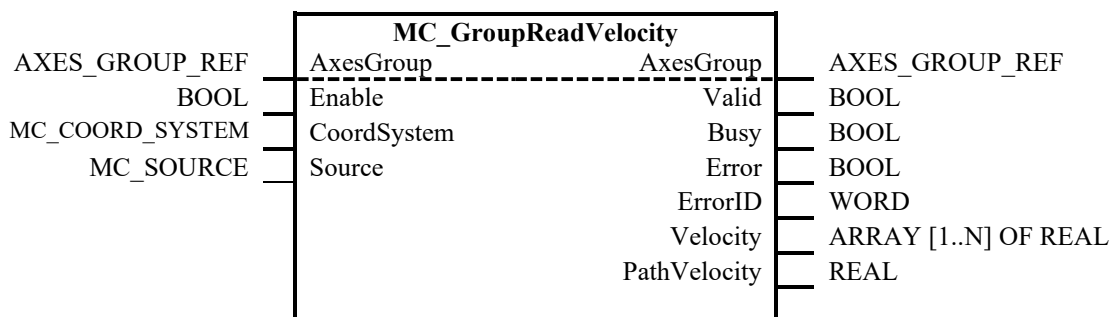
9.14 MC_GroupReadPosition

FB-Name		MC_GroupReadPosition		
This function block returns the position in the selected coordinate system of an axes group. This is an administrative FB, since no movement is generated. The source input allows to return the position as commanded to the axes or to be derived from the actual values of the axes in the group.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR_INPUT				
B	Enable	BOOL	Get the actual position in the selected coordinate system of the axes group continuously while enabled	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.	
E	Source	MC_SOURCE	Defines the source of the relevant data: mcCommandedValue; mcSetValue, mcActualValue.	
VAR_OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	Position	MC_POS_REF	Current position of the group.	
Notes: This FB replaces the FB MC_GroupReadActualPosition.				



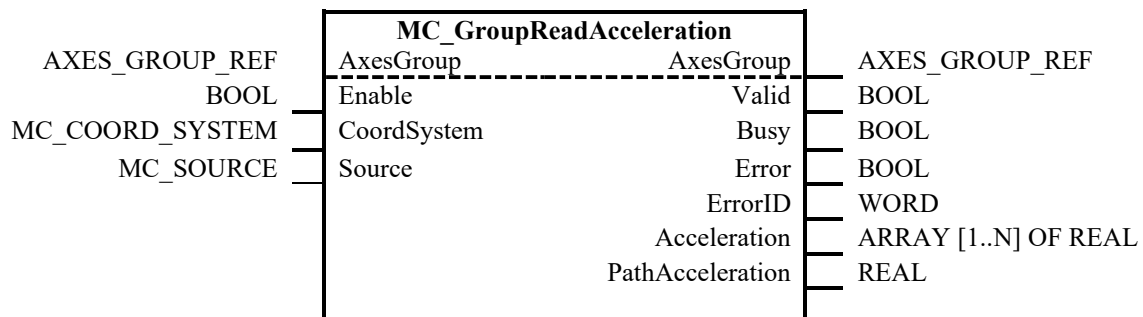
9.15 MC_GroupReadVelocity

FB-Name		MC_GroupReadVelocity	
This function block returns the velocity in the selected coordinate system of an axes group. This is an administrative FB, since no movement is generated. The source input allows to return the velocity as commanded to the axes or to be derived from the actual values of the axes in the group.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT			
B	Enable	BOOL	Get the actual velocity in the selected coordinate system of the axes group continuously while enabled
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.
E	Source	MC_SOURCE	Defines the source of the relevant data: mcCommandedValue; mcSetValue, mcActualValue.
VAR OUTPUT			
B	Valid	BOOL	True if valid outputs are available
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
B	Velocity	ARRAY [1..N] OF REAL	Current velocity of the group: - in ACS the velocities of the different axes - in MCS and PCS it provides the velocity of the TCP
E	PathVelocity	REAL	Current path velocity (speed, combined result) of the TCP.
Notes: This FB is the replacement for the FB MC_GroupReadActualVelocity			



9.16 MC_GroupReadAcceleration

FB-Name		MC_GroupReadAcceleration		
This function block returns the acceleration in the selected coordinate system of an axes group. This is an administrative FB, since no movement is generated. The source input allows to return the acceleration as commanded to the axes or to be derived from the actual values of the axes in the group.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Enable	BOOL	Get the actual acceleration in the selected coordinate system of the axes group continuously while enabled	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.	
E	Source	MC_SOURCE	Defines the source of the relevant data: mcCommandedValue; mcSetValue, mcActualValue.	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	Acceleration	ARRAY [1..N] OF REAL	Current acceleration of the group: - in ACS the acceleration of the different axes - in MCS and PCS it provides the acceleration of the TCP	
E	Path Acceleration	REAL	Current combined path acceleration of the TCP.	
Notes: This FB replaces the FB MC_GroupReadActualAcceleration				



9.17 MC_GroupReadMotionState

FB-Name		MC_GroupReadMotionState		
This function block returns the motion state of an axes group. This is an administrative FB, since no movement is generated.				
The motion state is always relative to the coordinate system of the active movement, or, if there is no active movement, to that of the last active one. If there has been no active movement since the axes group has been enabled or reset, the MCS is used. We call this coordinate system the active coordinate system in the following.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Enable	BOOL	Get the motion state of the axes group continuously while enabled	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available. Valid is FALSE in states GroupError and GroupDisabled.	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	Tracking	BOOL	Indicates whether the active coordinate system is a dynamic coordinate system, such as a conveyor belt or a rotary table.	
B	InSync	BOOL	If Tracking is TRUE, then InSync indicates that the axes group is following the dynamic coordinate system without lag. (For example, when tracking is started, there might be a ramp-in phase until the axes group is synchronized to a belt.) Always TRUE if Tracking is FALSE.	
B	InPosition	BOOL	The axes group is at the commanded position. (The axes group may still be moving, if the active coordinate system is dynamic.)	
B	Standstill	BOOL	The velocity of the axes group is '0' relative to the active coordinate system.	
B	ConstantVelocity	BOOL	The axes group is moving with constant velocity relative to the active coordinate system. Velocity may be '0'.	
B	Accelerating	BOOL	The axis group is accelerating relative to the active coordinate system.	
B	Decelerating	BOOL	The axis group is decelerating relative to the active coordinate system.	
E	ActiveCommand ID	MC_COMMAND_ID	The CommandID of the active movement. An ID of 0 means that no coordinated movement is active. See 11.4 MC_COMMAND_ID	
Notes:				
- The outputs reflect the commanded positions and velocities of the group.				
- The outputs Standstill, ConstantVelocity, Accelerating and Decelerating reflect the movement along the path.				
- The behavior for single axis movement and MC_MoveDirectAbsolute and MC_MoveDirectRelative is vendor specific.				

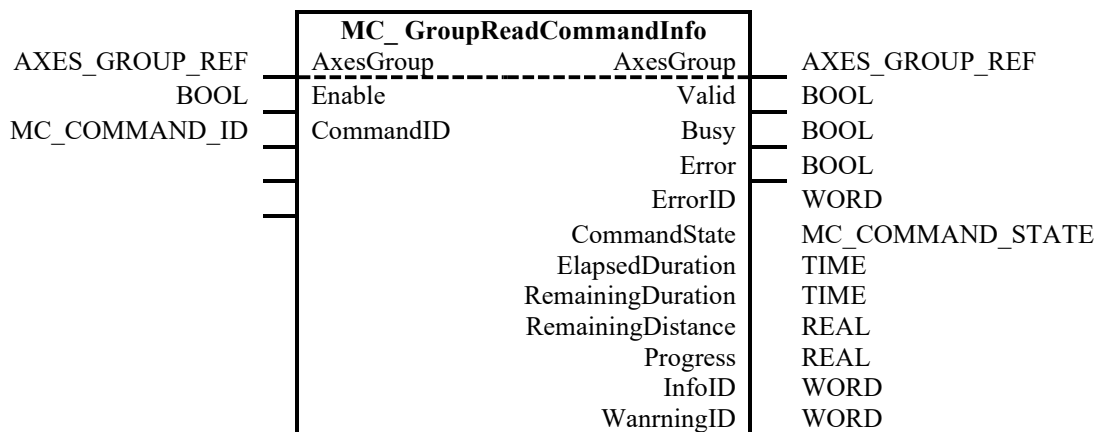
9.18 MC_GroupReadCommandInfo

FB-Name		MC_GroupReadCommandInfo		
This function block reads information about a command. It can be used to query information about movement commands and MC_GroupWaitTime.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL	Reads the command info if TRUE	
B	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
VAR OUTPUT				
B	Valid	BOOL	The command info has been read	
E	Busy	BOOL	The FB is being processed	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
B	CommandState	MC_COMMAND_STATE	The state of the command (accepted, active)	
E	ElapsedDuration	TIME	The elapsed duration of the command. Only set for state mcActive, 0 otherwise.	
E	RemainingDuration	TIME	The remaining duration until the command is done. 0 if the state is not mcActive or still unknown.	
E	RemainingDistance	REAL	The remaining distance of the command. Only set for state mcActive, 0 otherwise.	
E	Progress	REAL	Value between 0 and 1 indicating the progress of the command. Only set for state mcActive, 0 otherwise.	
E	InfoID	WORD	The ID of the last informational message for the given command. 0 if no informational message is present.	
E	WarningID	WORD	The ID of the last warning message for the given command. 0 if no warning message is present.	

MC_COMMAND_STATE

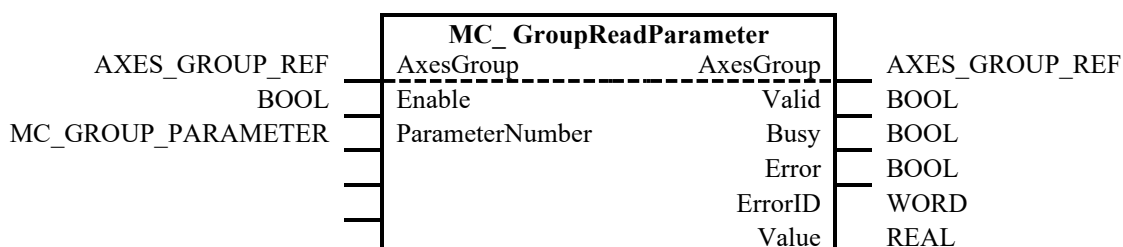
The enumeration MC_COMMAND_STATE defines the possible states of a command.

No.	MC_COMMAND_STATE	Description
1	mcAccepted	The command has been accepted, but it is not active yet.
2	mcActive	The command is active, meaning that it has control over the axes group.



9.19 MC_GroupReadParameter

FB-Name		MC_GroupReadParameter		
This function block returns the value of a parameter of the axes group. The returned value is converted to Real if necessary. If not possible, the vendor has to supply a vendor specific FB to read the parameter.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL	Reads parameter continuously if TRUE	
B	ParameterNumber	MC_GROUP_PARAMETER	Number of the parameter. See table of parameters.	
VAR OUTPUT				
B	Valid	BOOL	A valid output is available at the FB	
E	Busy	BOOL	The FB is being processed	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
B	Value	REAL	Value of the specified parameter	



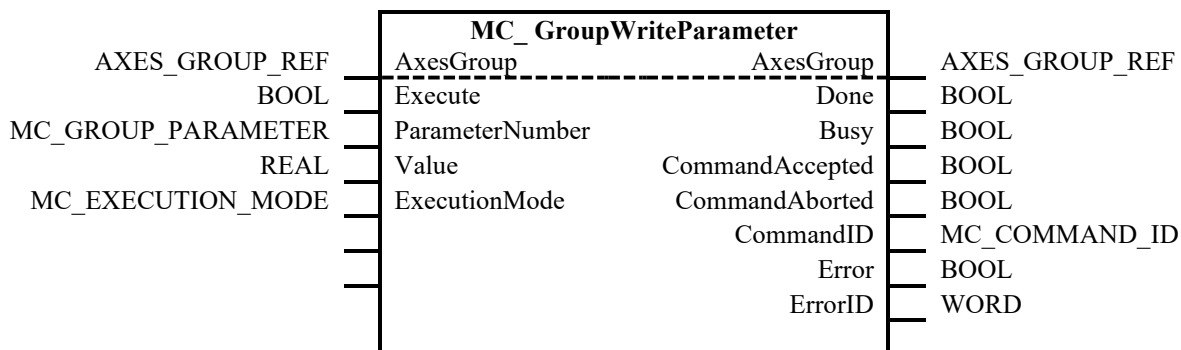
9.19.1 Table of Parameters

The enumeration `MC_GROUP_PARAMETER` defines the parameters that can be used with `MC_GroupReadParameter` and `MC_GroupWriteParameter`. Vendors can extend the list of supported parameters, but these are not allowed to start with “mc”.

Parameter name	Datatype	Description	Read	Write
mcDynamicsMode	MC_DYNAMICS_MODE	mcAbsolute: The dynamics of movements are expected as absolute values. The Velocity is expected in [u/s], the Acceleration and Deceleration are expected in [u/s ²], and the Jerk is expected in [u/s ³].		
mcTransitionReferencePoint	MC_TRANSITION_REFERENCE_POINT	mcPercentage: The dynamics of movements are expected as a percentage between 0 and 100 of the reference dynamics. See also MC_GroupReadReferenceDynamics and MC_GroupWriteReferenceDynamics. The reference point for the transition between two coordinated movement function blocks (like MC_MoveLinearAbsolute). This impacts the TransitionVelocity, TransitionMode and TransitionParameter inputs of the MC_MoveXXX-inputs. A change in the transition reference is only allowed, if the axes group is in GroupDisabled or GroupStandby and no movements are already buffered. See Ch. 11.5	B	E

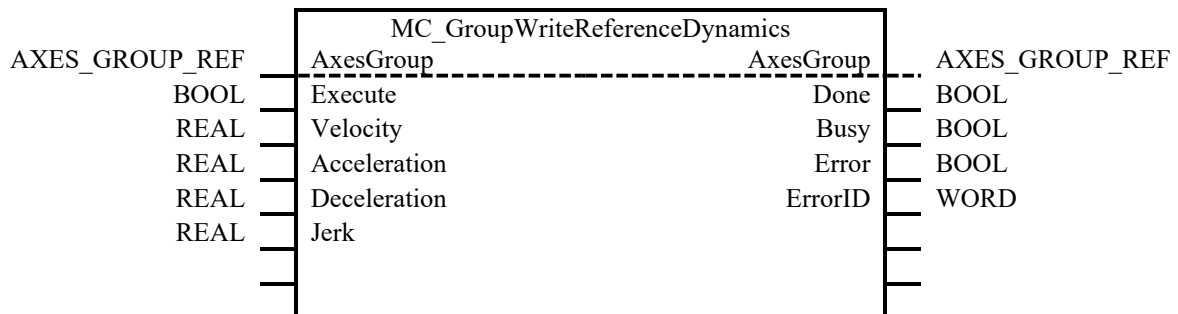
9.20 MC_GroupWriteParameter

FB-Name		MC_GroupWriteParameter	
This function block writes the value of an axes group parameter.			
VAR_IN_OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR_INPUT			
B	Execute	BOOL	Start of the command at the rising edge
B	ParameterNumber	MC_GROUP_PARAMETER	Number of the parameter. See 9.19.1 Table of Parameters
B	Value	REAL	Value of the parameter, expressed as a REAL
E	ExecutionMode	MC_EXECUTION_MODE	Defines the chronological sequence of the FB.
VAR_OUTPUT			
B	Done	BOOL	The command has been completed successfully.
E	Busy	BOOL	The FB is being processed
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	The command was aborted by another command.
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
Notes:			



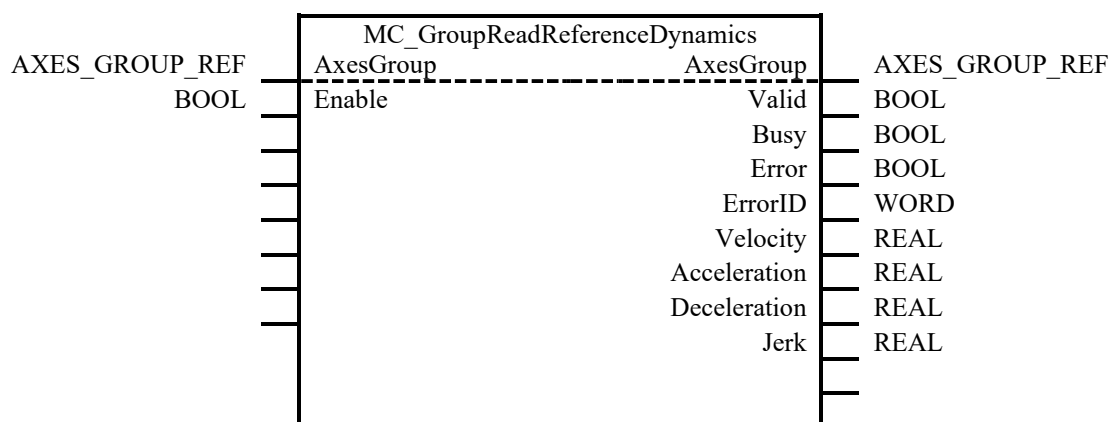
9.21 MC_GroupWriteReferenceDynamics

FB-Name		MC_GroupWriteReferenceDynamics	
This function block writes the reference values of the group dynamics for path movements. The reference can be set for velocity, acceleration, deceleration, and jerk. The reference dynamics are only relevant if the value of parameter mcDynamicsMode is mcPercentage. This is an administrative FB that does not cause any motion and does not influence the active motion. The new reference dynamics only affects movements that are commanded after this FB is Done.			
VAR_IN_OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR_INPUT			
B	Execute	BOOL	Writes the path reference dynamics at rising edge
B	Velocity	REAL	Path velocity (tangent) in u/s at 100% >= 0: change value according to input value
E	Acceleration	REAL	Path acceleration in [u/s²] at 100% >= 0: change value according to input value
E	Deceleration	REAL	Path deceleration in [u/s²] at 100% >= 0: change value according to input value.
E	Jerk	REAL	Path jerk in [u/s³] at 100% >= 0: change value according to input value.
VAR_OUTPUT			
B	Done	BOOL	The path reference dynamics has been set
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
Note: The behavior for input values equal to zero is vendor specific.			



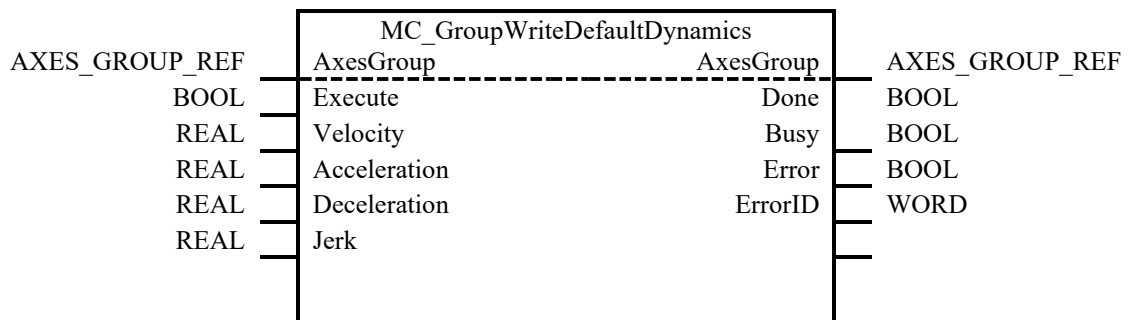
9.22 MC_GroupReadReferenceDynamics

FB-Name		MC_GroupReadReferenceDynamics		
This function block reads the reference values of the group dynamics for path movements and returns the references of velocity, acceleration, deceleration, and jerk.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL		Reads path reference dynamics
VAR OUTPUT				
B	Valid	BOOL		True if valid outputs are available
E	Busy	BOOL		The FB is not finished
B	Error	BOOL		Signals that an error has occurred
E	ErrorID	WORD		Error identification
B	Velocity	REAL		Path velocity (tangent) in [u/s] at 100%
E	Acceleration	REAL		Path acceleration in [u/s²] at 100%
E	Deceleration	REAL		Path deceleration in [u/s²] at 100%
E	Jerk	REAL		Path jerk in [u/s³] at 100%
Notes: -				



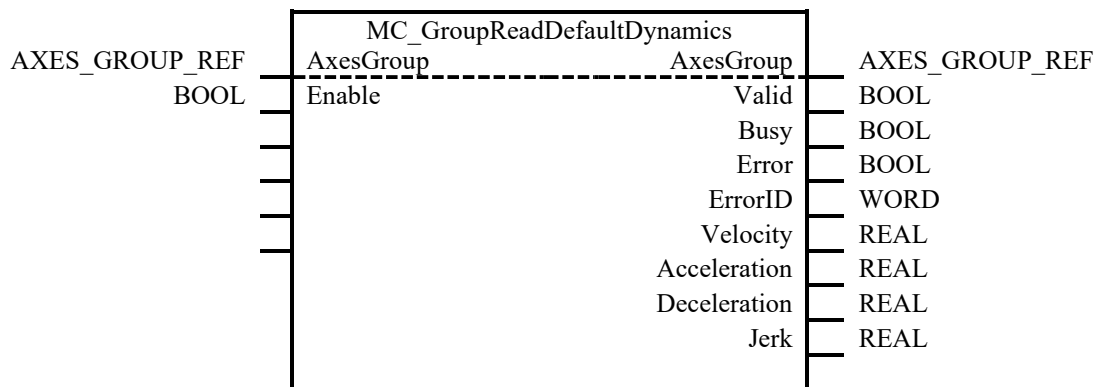
9.23 MC_GroupWriteDefaultDynamics

FB-Name		MC_GroupWriteDefaultDynamics	
This function block writes the default values of the group dynamics for path movement. These values are used for unset inputs to motion commands. The default values can be set for velocity, acceleration, deceleration, and jerk. The new default dynamics only affects movements that are commanded after this FB is Done.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR INPUT			
B	Execute	BOOL	Writes the path default dynamics at rising edge
B	Velocity	REAL	Default path velocity (tangent) >= 0: change value according to input value
E	Acceleration	REAL	Default path acceleration >= 0: change value according to input value
E	Deceleration	REAL	Default path deceleration >= 0: change value according to input value
E	Jerk	REAL	Default path jerk >= 0: change value according to input value
VAR OUTPUT			
B	Done	BOOL	The path reference dynamics have been set
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
Note:			
The behavior for input values equal to zero is vendor specific.			
If the mcDynamicsUnits parameter is changed, all values for velocity, acceleration, deceleration, and jerk should be provided.			
The technical units for inputs velocity, acceleration, deceleration and jerk are depending on the axes group parameter mcDynamicsUnits.			



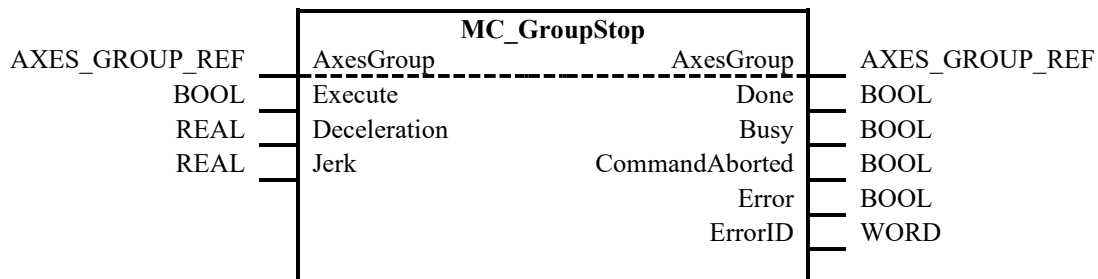
9.24 MC_GroupReadDefaultDynamics

FB-Name		MC_GroupReadDefaultDynamics	
This function block reads the default values (presetting's) of the group dynamics for path movements and returns the default values of velocity, acceleration, deceleration, and jerk.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR INPUT			
B	Enable	BOOL	Reads path reference dynamics
VAR OUTPUT			
B	Valid	BOOL	True if valid outputs are available
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
B	Velocity	REAL	Default path velocity (tangent)
E	Acceleration	REAL	Default path acceleration
E	Deceleration	REAL	Default path deceleration
E	Jerk	REAL	Default path jerk
Notes: -			



9.25 MC_GroupStop

FB-Name		MC_GroupStop	
This function block commands a controlled motion stop and transfers the axes group to the state “GroupStopping”. It aborts any ongoing FB execution. While the axes group is in state GroupStopping, no other FB can perform any motion on the same axes group. After the axes group has reached velocity zero, the Done output is set to TRUE immediately. The axes group remains in the state ”GroupStopping” as long as Execute is still TRUE or velocity zero is not yet reached. As soon as “Done” is SET and “Execute” is FALSE the axes group goes to state “GroupStandBy”. The command can only be aborted by MC_GroupDisable.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT			
B	Execute	BOOL	Start the action at rising edge
E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached
E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached
VAR OUTPUT			
B	Done	BOOL	Stop for all axes done.
E	Busy	BOOL	The FB is not finished
E	CommandAborted	BOOL	Command is aborted by disabling MC-Power of one or more of the axes in the group. The state changes to GroupDisabled.
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes:			
• The relevant axes stay on the path. • If Deceleration is set to zero the resulting action is vendor specific. • If issued during a MoveDirectXxx command, the velocity/acc-/deceleration/jerk values as properties of the AxisRef of each axis are used, and not specified within this FB, and not to be exceeded during the movement. • Any synchronization and tracking of the group to a master is cancelled by issuing MC_GroupStop			



A typical timing diagram for MC_GroupStop is shown on the next page, including the relevant states and state-transitions.

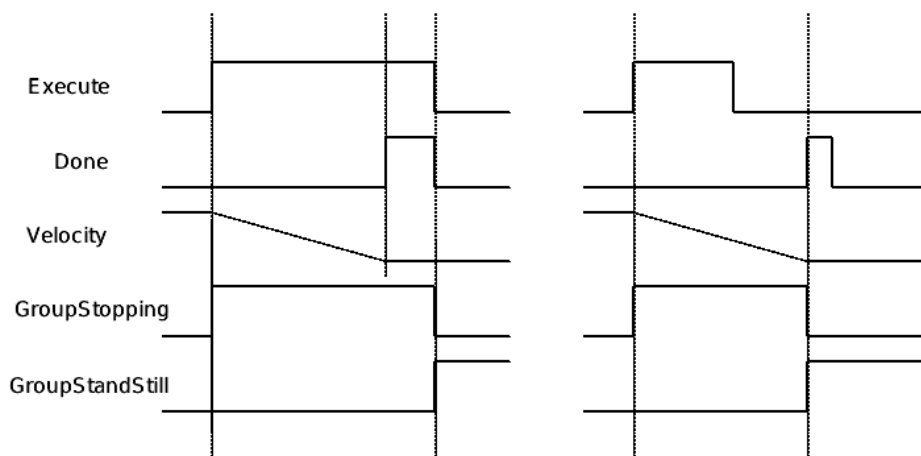


Figure 12: MC_GroupStop timing diagram

The example below shows the behavior in combination with a MC_MoveLinearRelative.

- An axes group in linear movement is ramped down with FB MC_GroupStop. The group stops on the original path.
- The axes group rejects motion commands as long as MC_GroupStop parameter “Execute” = TRUE. FB MC_MoveLinearRelative reports an error indicating the busy MC_GroupStop command. This error is an FB error, so the group is not moving to the state GroupErrorStop. At the 3rd “Exe1” rising edge, the group starts the next movement.

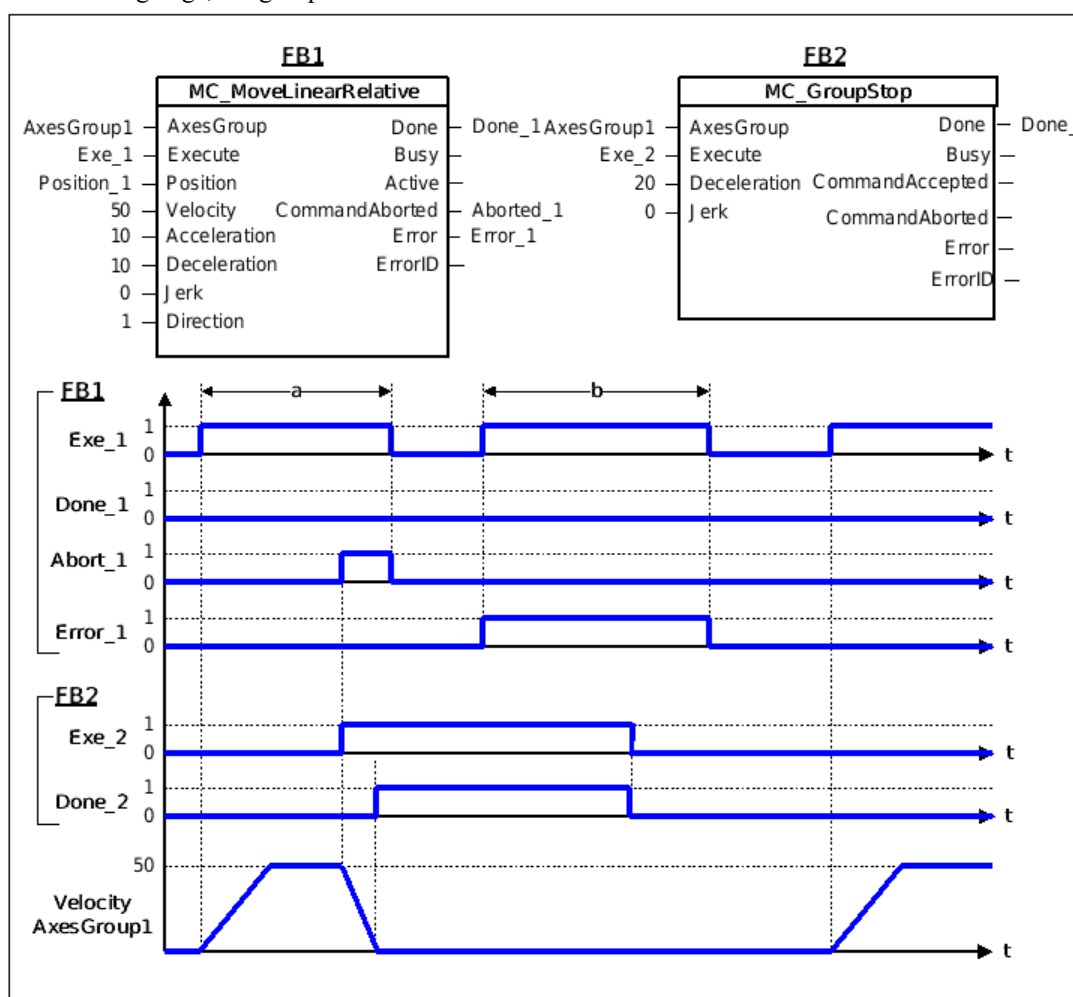


Figure 13: Behavior of MC_GroupStop in combination with MC_MoveLinearRelative

The following example demonstrates the behaviour of MC_GroupStop in combination with two MC_MoveLinearAbsolute which are blended with defined constant path velocity:

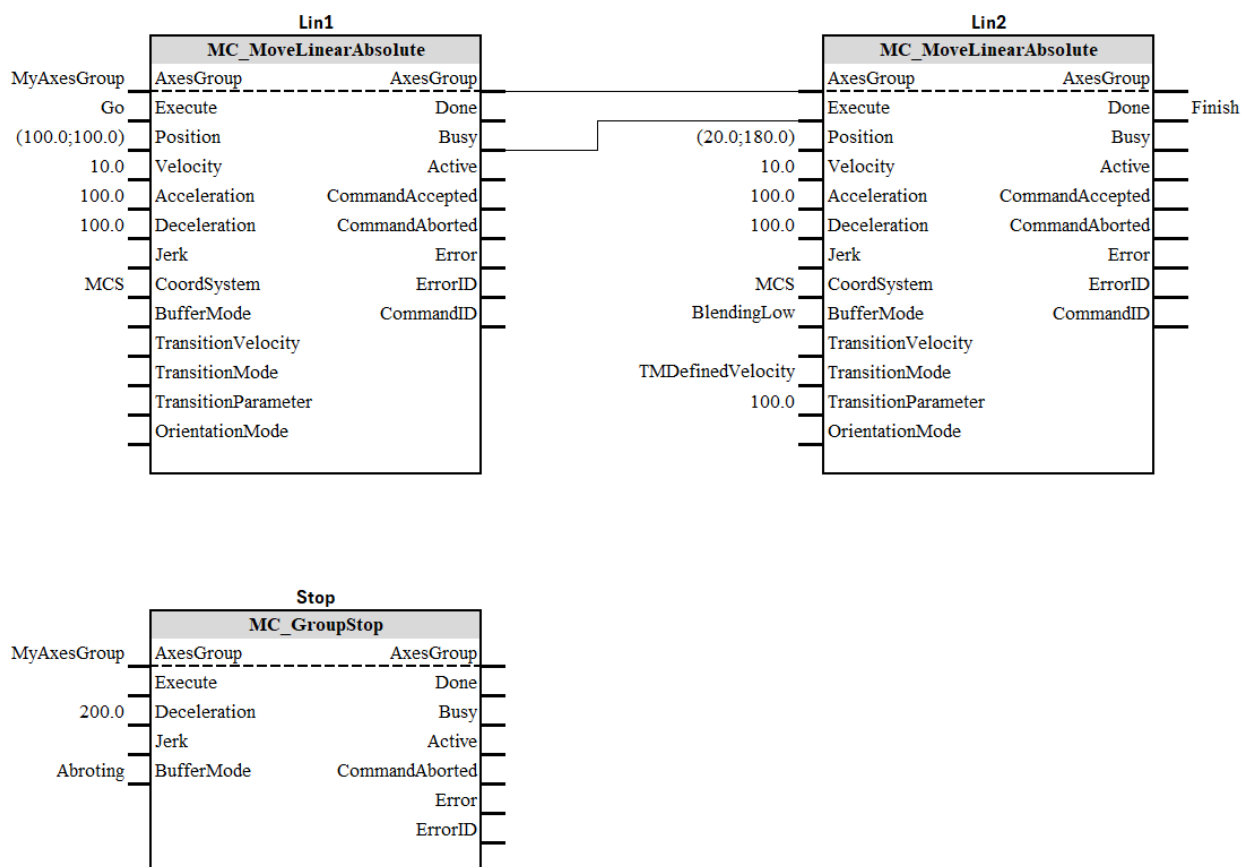
t₀) Two FBs MC_MoveLinearAbsolute are commanded on axes group MyAxesGroup. The first FB becomes active immediately and MyAxesGroup starts to move from its actual position (20.0; 20.0) towards the first target position.

t₁) Shortly after the TCP has started to move on the blending contour blending Lin1 into Lin2, a FB MC_GroupStop is issued in buffermode Aborting. The state of the axes group changes from GroupMoving to GroupStopping. MyAxesGroup decelerates, following the path which would have been executed without having issued MC_GroupStop.

Note: Though the path velocity of MyAxesGroup decreases strictly monotonic while stopping, single axes of the group might accelerate in between due to the given path and kinematic transformation of MyAxesGroup.

t₂) MyAxesGroup comes to standstill. The Done output of the FB MC_GroupStop is set. Since the input Execute of the FB Stop is still set the group stays in state GroupStopping.

t₃) The input Execute of the FB Stop is reset. All outputs of the FB Stop are reset. The group state changes to GroupStandby.



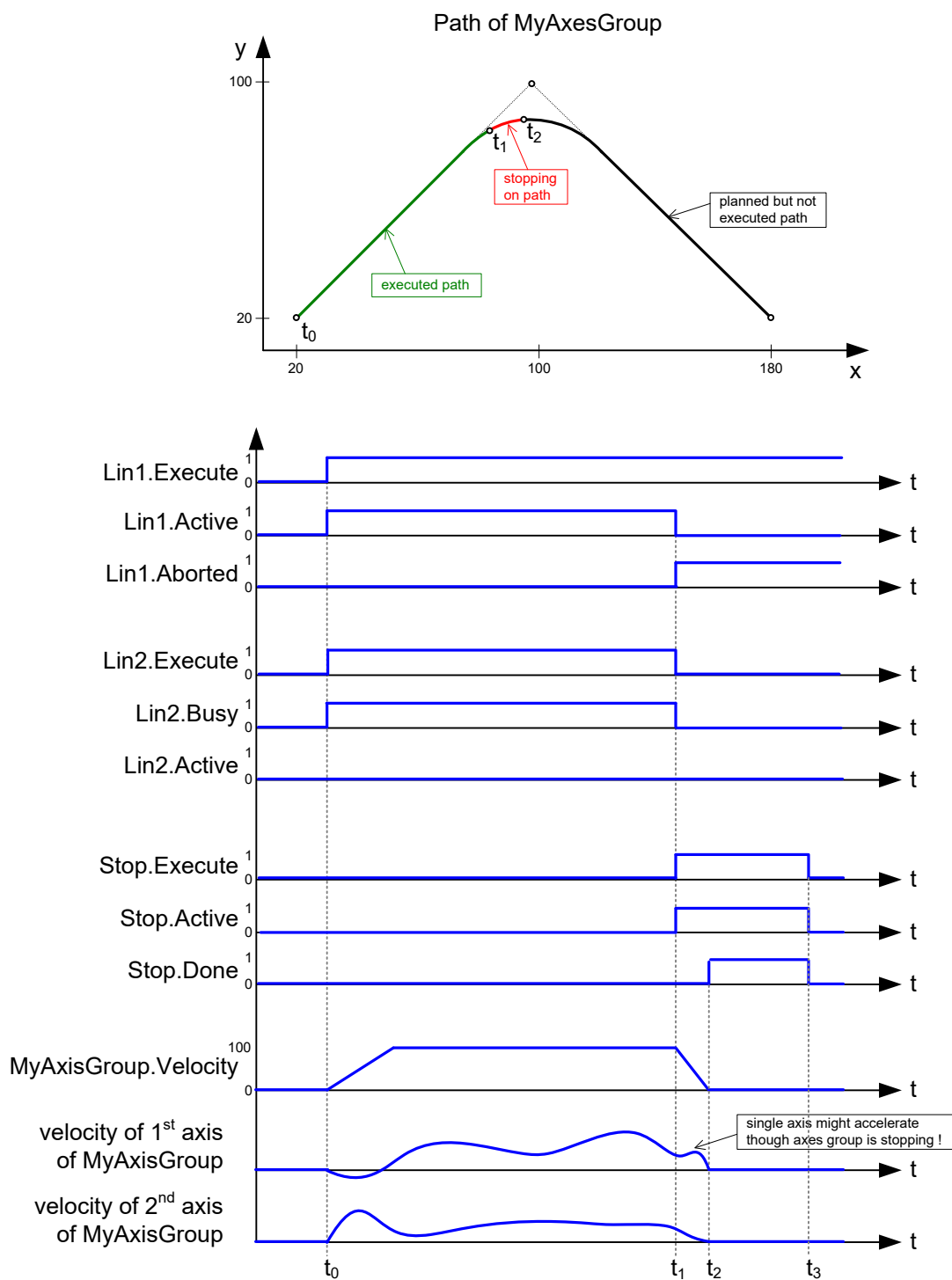
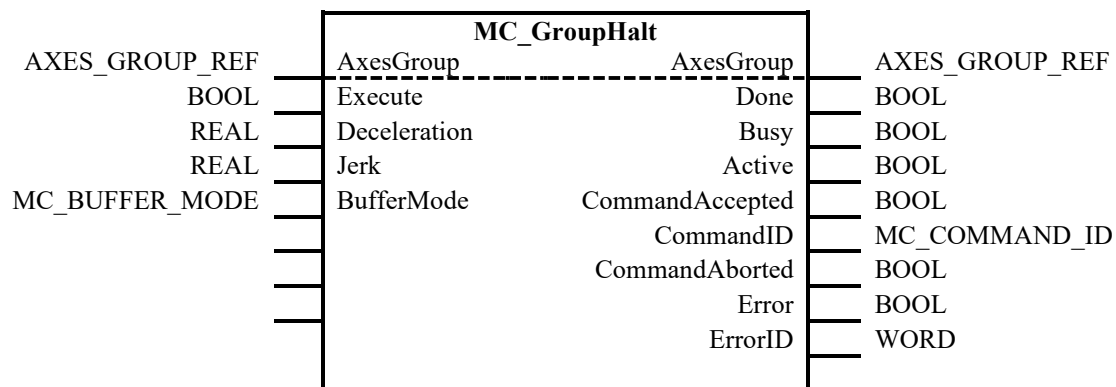


Figure 14: Example of MC_GroupStop in combination with two MC_MoveLinear_Absolute

9.26 MC_GroupHalt

FB-Name		MC_GroupHalt		
This function block commands a controlled motion stop. AxesGroup is moved to the state “GroupMoving“, until the velocity is zero. With the DONE output set, the state is transferred to GroupStandby, or GroupMoving if the coordinate system is a dynamic PCS.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the action at rising edge	
E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached	
E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached	
E	BufferMode	MC_BUFFER_MODE	Refer to Chapter 11.2 Buffer Modes	
VAR OUTPUT				
B	Done	BOOL	Zero velocity reached	
E	Busy	BOOL	The FB is not finished	
E	Active	BOOL	Indicates that the FB has control on the axes group	
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed	
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes:				
- MC_GroupHalt is used to stop the axes group under normal operation conditions. MC_GroupHalt can be aborted by another motion command. E.g. a driverless vehicle detects an obstacle and needs to stop. MC_GroupHalt is issued. Before the standstill is reached the obstacle is removed and the motion can be continued by setting another motion command, so the vehicle does not stop. - The relevant axes stay on the same path which would have been executed without having issued MC_GroupHalt.				



The following example shows the behaviour of MC_GroupHalt in combination with a MC_MoveCircularAbsolute:

- S) MyAxesGroup starts at Position (10.0; 10.0; 0.0). A FB MC_MoveCircularAbsolute is commanded with auxiliary position (30.0; 30.0; 0.0) and end position (50.0; 10.0; 0.0). This results in a 180° circular motion within the xy-plane of any coordinate system.
- H) The circular motion is aborted by FB MC_GroupHalt. MyAxesGroup stays on the path during halt.
- R) MC_MoveCircularAbsolute is executed again and aborts MC_GroupHalt. MC_GroupHalt allows this, in contrast to MC_GroupStop. AxesGroup can accelerate again without reaching standstill. (MC_MoveCircularAbsolute can be retriggered to continue the original circular motion as long as MyAxesGroup didn't pass the auxiliary position.)
- E) MyAxesGroup reaches end position (50.0; 10.0; 0.0)

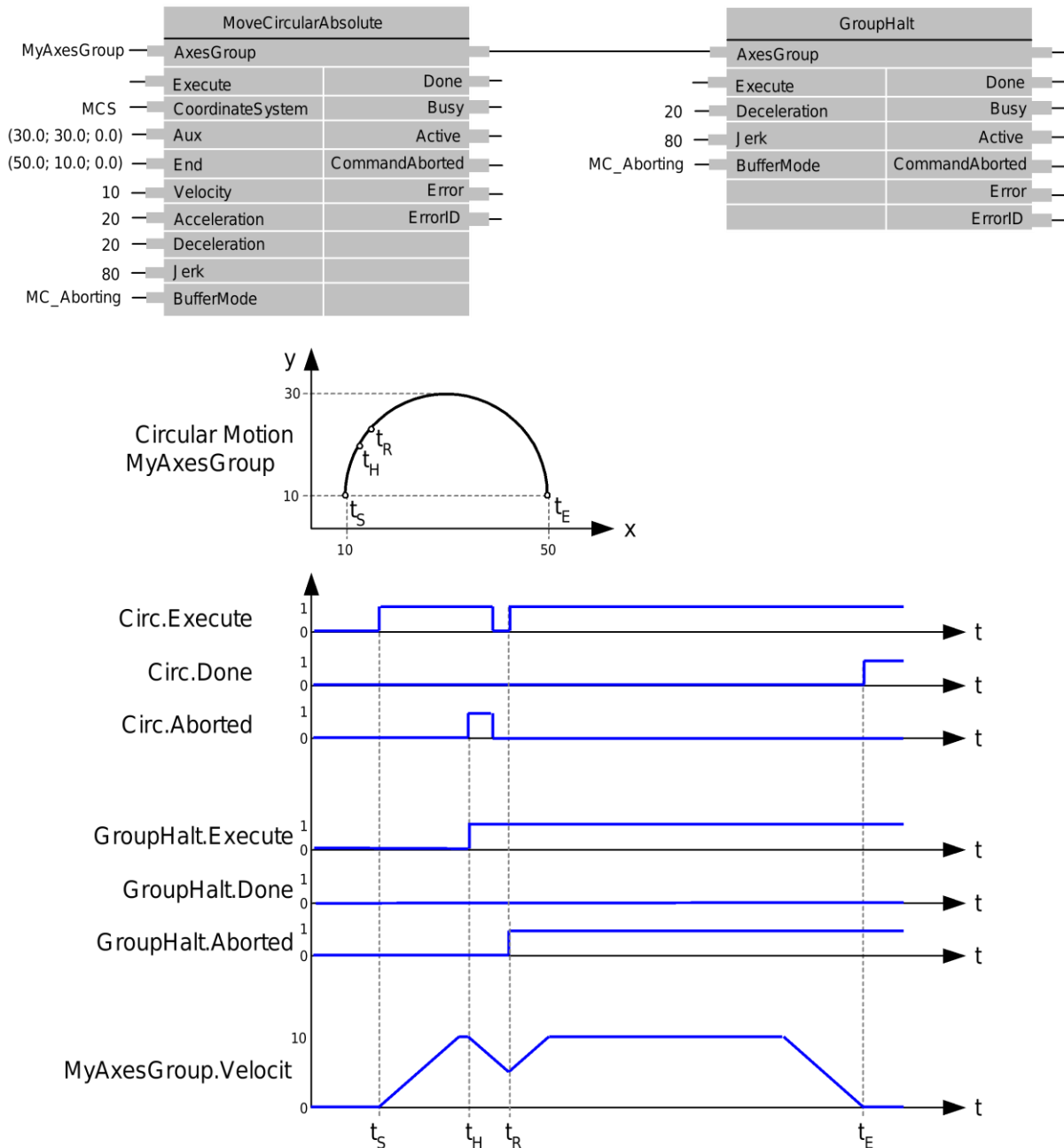
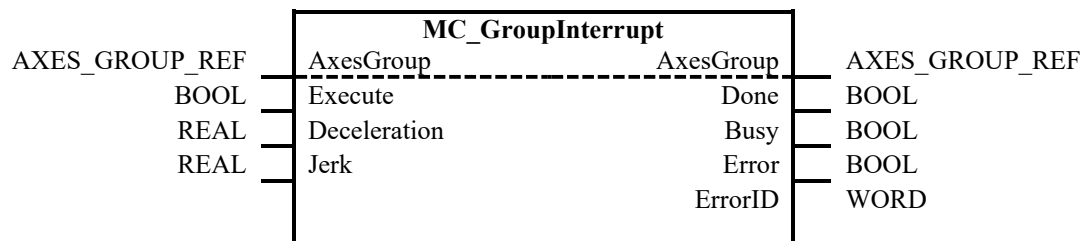


Figure 15: Behavior of MC_GroupHalt in combination with MC_MoveCircularAbsolute

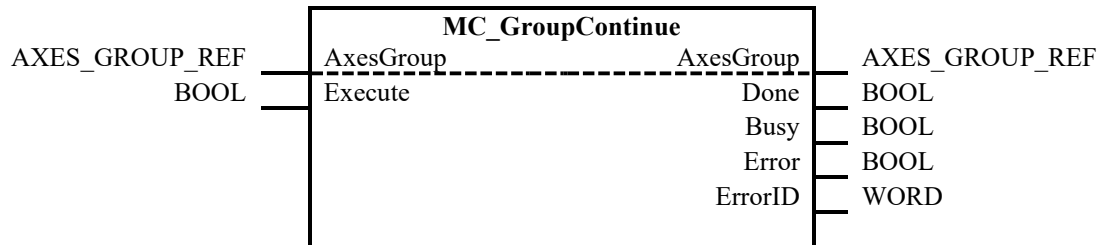
9.27 MC_GroupInterrupt

FB-Name		MC_GroupInterrupt	
This function block interrupts the on-going motion and stops the group from moving, however does not abort the interrupted motion (meaning that at the interrupted FB the output CommandAborted will not be Set, Busy is still high and Active is reset after the interrupt is done). It stores all relevant track or path information. The AxesGroup stays in the original state even if the velocity zero is reached and the DONE output set.			
VAR IN OUT			
B	AxesGroup	AXES GROUP REF	Reference to a group of axes
VAR INPUT			
B	Execute	BOOL	Start the action at rising edge
E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached
E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached
VAR OUTPUT			
B	Done	BOOL	Zero velocity reached
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes:			
• This FB is coupled to MC_GroupContinue. Issuing MC_GroupContinue transfers the program back to the situation at issuing MC_GroupInterrupt from the final position reached onwards. • Further motion commands may be accepted by the group. Also, if there is no tracking active, one manually can move the mechanics or do this via single axis FBs or Grouped related commands. • It will always lead to a velocity 0 in the coordinate system of the active motion command. • The behavior of multiple interruptions without matching continues is vendor specific.			



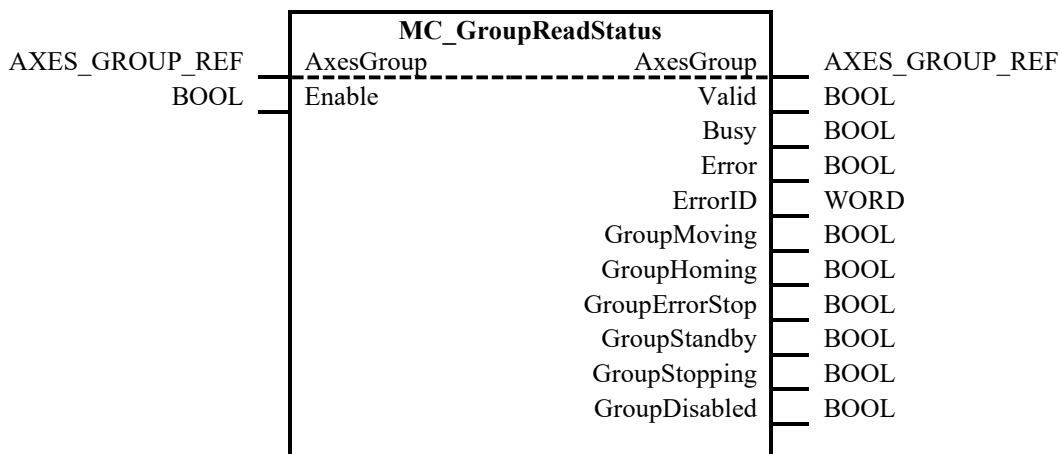
9.28 MC_GroupContinue

FB-Name		MC_GroupContinue	
This function block transfers the program back to the situation at issuing MC_GroupInterrupt. It uses internally the data set as stored at issuing MC_GroupInterrupt, and at the end (output DONE set) transfer the control on the group back to the original FB doing the movement on the axes group, meaning also that at the originally interrupted FB the output Busy is still high and Active is set again.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT			
B	Execute	BOOL	Start the action at rising edge
VAR OUTPUT			
B	Done	BOOL	Control transferred back to original FB
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes: -			



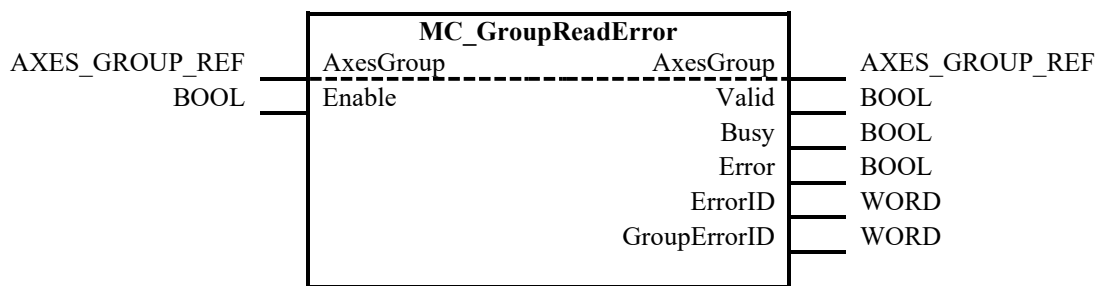
9.29 MC_GroupReadStatus

FB-Name		MC_GroupReadStatus		
This function block returns the status of an axes group according to the active Group-FB.				
VAR IN OUT				
B	AxesGroup	AXES GROUP REF	Reference to a group of axes	
VAR INPUT				
B	Enable	BOOL	Get the status of the axes group continuously while enabled	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
B	GroupMoving	BOOL	See group state diagram	
B	GroupHoming	BOOL	See group state diagram	
B	GroupErrorStop	BOOL	See group state diagram	
B	GroupStandby	BOOL	See group state diagram	
B	GroupStopping	BOOL	See group state diagram	
B	GroupDisabled	BOOL	See group state diagram	
Notes:				



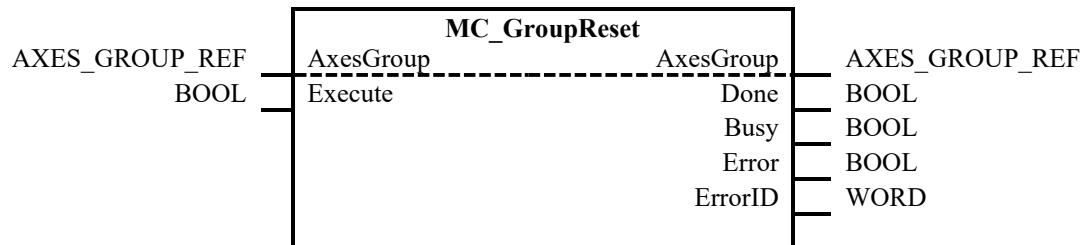
9.30 MC_GroupReadError

FB-Name		MC_GroupReadError		
This function block returns the axes group error.				
VAR_IN_OUT				
B	AxesGroup	AXES GROUP REF	Reference to a group of axes	
VAR_INPUT				
B	Enable	BOOL	Get the value of the GroupErrorID continuously while enabled	
VAR_OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
B	ErrorID	WORD	Error identification on an FB error	
E	GroupErrorID	WORD	The value of the axes group error. These values are vendor specific.	
Notes:				



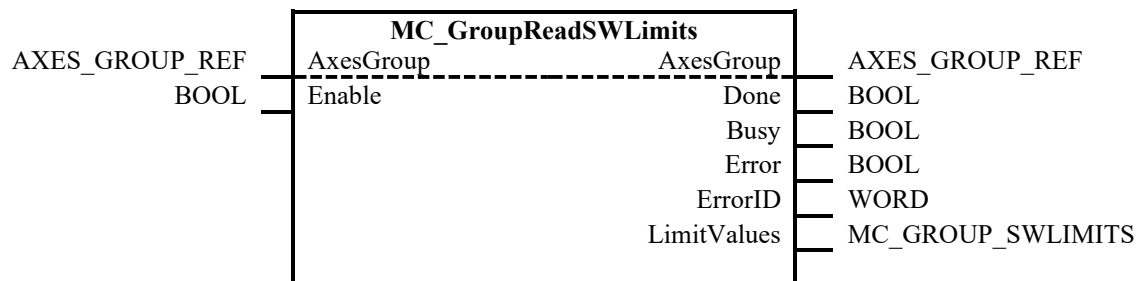
9.31 MC_GroupReset

FB-Name		MC_GroupReset		
This function block initiates the transition from the state GroupErrorStop to GroupStandby by resetting all internal group-related errors – it does not affect the output of the FB instances. This FB also resets all axes in this group like MC Reset.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Start the action at rising edge	
VAR OUTPUT				
B	Done	BOOL	Reset for axes group and all axes in this group done.	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes: -				



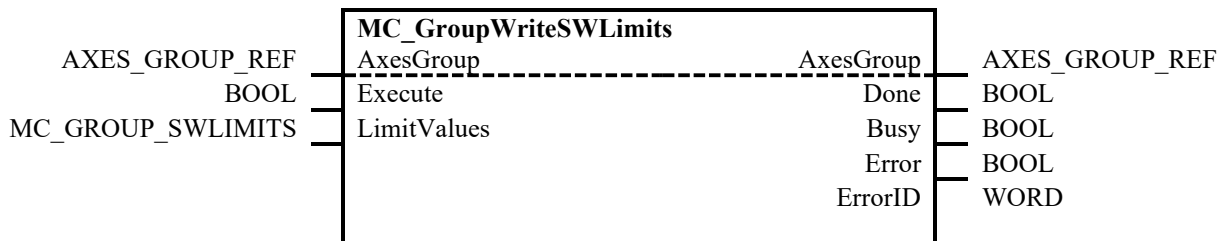
9.32 MC_GroupReadSWLimits

FB-Name		MC_GroupReadSWLimits	
The function block reads the positions of positive and negative software limit switches of the joints of the axes group. The specified limits are returned via the output "LimitValues" of the data type MC_GROUP_SWLIMITS			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR INPUT			
B	Enable	BOOL	Read the values of the axes group SW limits continuously while enabled
VAR OUTPUT			
B	Valid	BOOL	True if valid outputs are available
E	Busy	BOOL	The FB is being processed
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
B	LimitValues	MC_GROUP_SWLIMITS	SW limits for the axes group Vendor specific data type, example implementation see 11.7 MC_GROUP_SWLIMITS
Notes:			
• The SW limits of the robot joints are not necessarily identical to the SW limits of the connected ACS axes. E.g. this is the case, if an axes group has a differential coupling between the output shaft of at least two ACS axes and the corresponding joints of the axes group. • If an axes group has auxiliary axes, the fb directly reads the sw limits of the corresponding AXIS_REF. • It is vendor specific, if the fb is also capable of reading SW limits of passive joints, e.g. for parallel kinematic robots • For remote hosted systems with an external robot plc, this fb is required to read the SW limits to the external plc by one single command.			



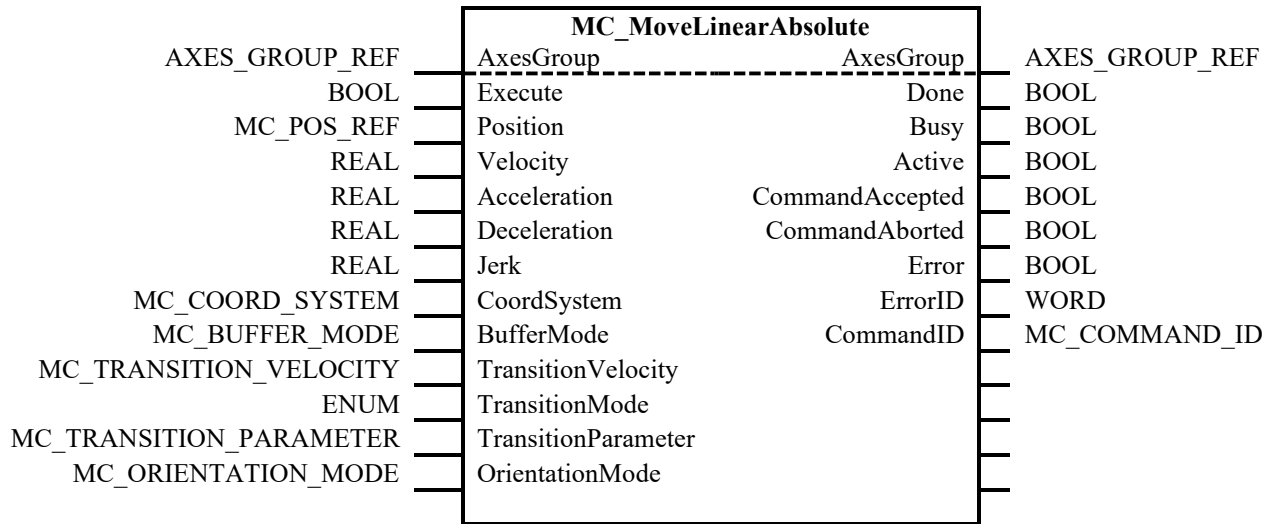
9.33 MC_GroupWriteSWLimits

FB-Name		MC_GroupWriteSWLimits	
The function block writes the positions of positive and negative software limit switches of the joints of the axes group. The specified limits are given via the input "LimitValues" of the data type MC_GROUP_SWLIMITS			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	
VAR INPUT			
B	Execute	BOOL	Start of the command at the rising edge
B	LimitValues	MC_GROUP_SWLIMITS	SW limits for the axes group Vendor specific data type, example implementation see 11.7 MC_GROUP_SWLIMITS
E	ExecutionMode	MC_EXECUTION_MODE	Defines the chronological sequence of the FB.
VAR OUTPUT			
B	Done	BOOL	SW limits are successfully written
E	Busy	BOOL	The FB is being processed
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
Notes:			
- It is vendor specific, if a change of the SW limit values can be performed in any state of the axes group or if restrictions are present. - The SW limits of the robot joints are not necessarily identical to the SW limits of the connected ACS axes. E.g. this is the case, if an axes group has a differential coupling between the output shaft of at least two ACS axes and the corresponding joints of the axes group. - If an axes group has auxiliary axes, the FB directly writes the sw limits of the corresponding AXIS_REF. - It is vendor specific, if the FB is also capable of writing SW limits of passive joints, e.g. for parallel kinematic robots - For remote hosted systems with an external robot plc, this FB is required to write the SW limits to the external plc in one single command.			



9.34 MC_MoveLinearAbsolute

FB-Name		MC_MoveLinearAbsolute		
This function block commands an interpolated linear movement of the axes group from the actual position of the TCP to an absolute cartesian position in the specified coordinate system.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT				
	B	Execute	BOOL	Commands a new movement at the rising edge
	B	Position	MC_POS_REF	See 4 Definition of Position
	E	Velocity	REAL	Maximum Velocity in [u/s] or [%] of nominal velocity for the path for the coordinate system in which the path is defined. Always positive. Not necessarily reached
	E	Acceleration	REAL	Maximum acceleration in [u/s ²] or [%] of nominal acceleration. Always positive. Not necessarily reached
	E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached
	E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached
	E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system.
	E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to Chapter 11.2 Buffer Modes
	E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see chapter 11.2.3 Transition Velocity behavior
	E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see chapter 11.3 TransitionMode
	E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode
	E	OrientationMode	MC_ORIENTATION_MODE	Linear, joined, fixed and path. See chapter 11.6 MC_ORIENTATION_MODE
VAR OUTPUT				
	B	Done	BOOL	Commanded end positions reached for all axes
	E	Busy	BOOL	The FB is not finished
	E	Active	BOOL	Indicates that the FB has control over the axes group
	E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
	E	CommandAborted	BOOL	Command is aborted by another command.
	B	Error	BOOL	Signals that an error has occurred within the FB
	E	ErrorID	WORD	Error identification
	E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
Note: -				



9.34.1 Example 1

The following example illustrates the blending of two "MoveLinearAbsolute" commands with MC_TRANSITION_REFERENCE mcStartPoint.. The corner distance 10.0 is parameterized by the input parameter "TransitionParameter" = 10.0 at FB2. The trajectory shown of the blending part is just an example and can vary with different vendors.

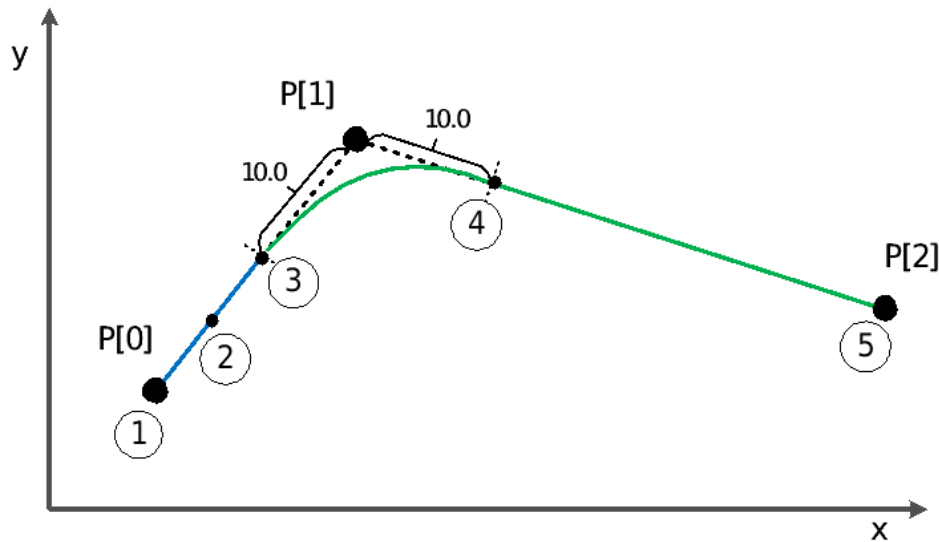
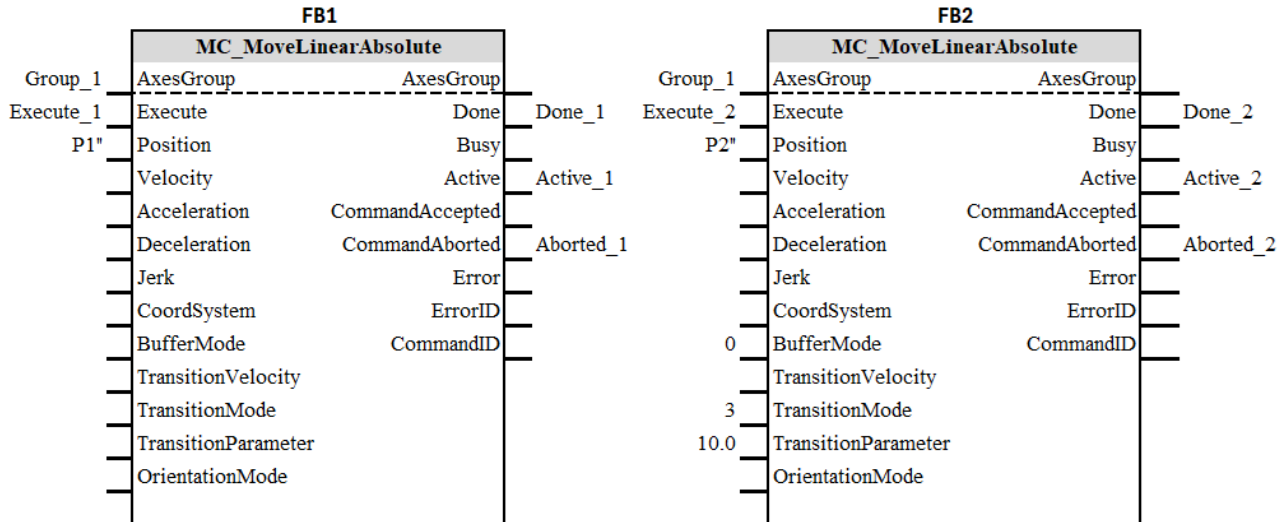


Figure 16: "MoveLinearAbsolute" example



(1)	The input parameter "Execute_1" starts the linear motion of the robot arm. The target of the motion is the absolute target coordinate P[1]. At the same time the output parameter "Active_1" changes to TRUE, indicating that the motion command is active.
(2)	The input parameter "Execute_2" starts a second "MoveLinearAbsolute" command. The target of this motion is the absolute target coordinate P[2].
(3)	First point where control might be switched from FB1 to FB2, depending on the vendor implementation and blending starts.
(4)	The movement has completed the blending motion and switches to linear motion. The blending motion showed in the figure is just an example and can vary, depending on the vendor. This is the last point where control might be switched from FB1 to FB2, depending on the vendor implementation .
(5)	The movement has reached the target position at the coordinate of P[2]. At FB 2 "Active_2" switches to FALSE and "Done_2" to TRUE.

9.34.2 Example 2

Timing diagram for example below (the dots on the red line are based on the same timing difference and represent the velocity)

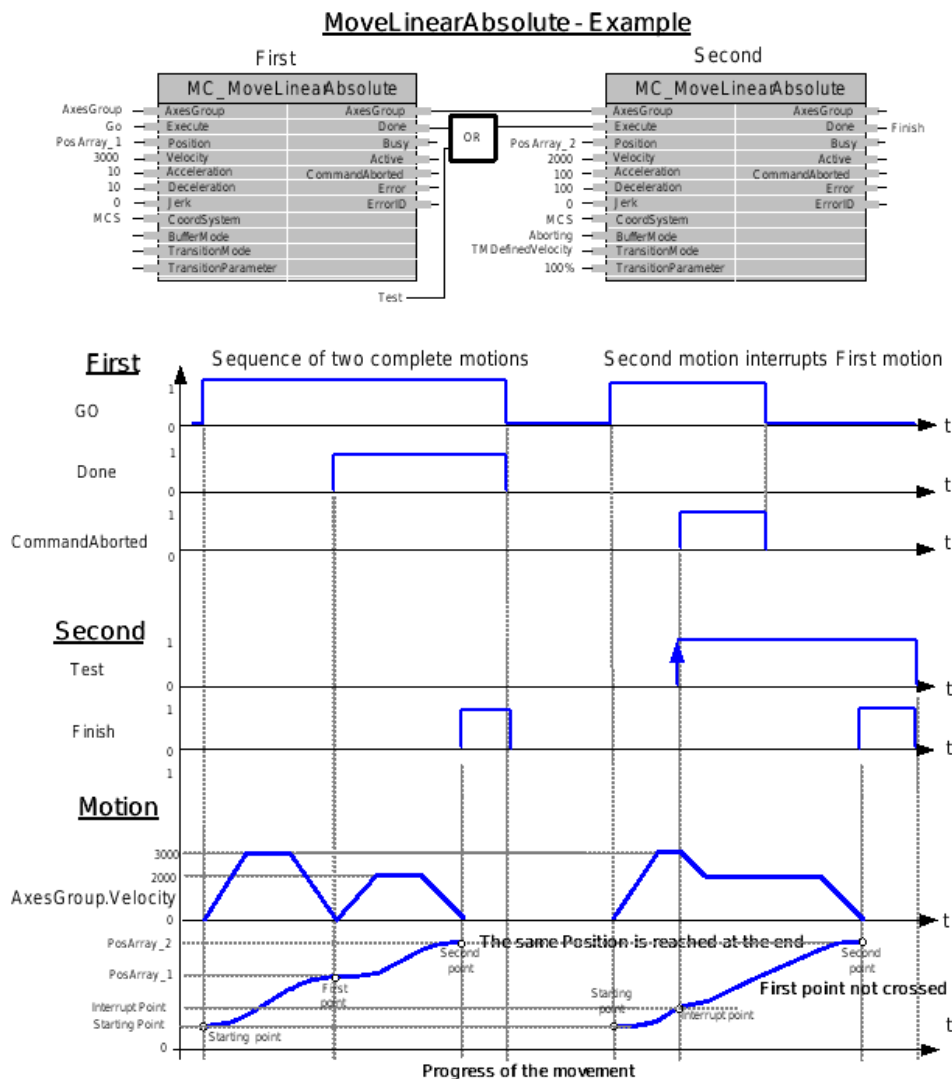
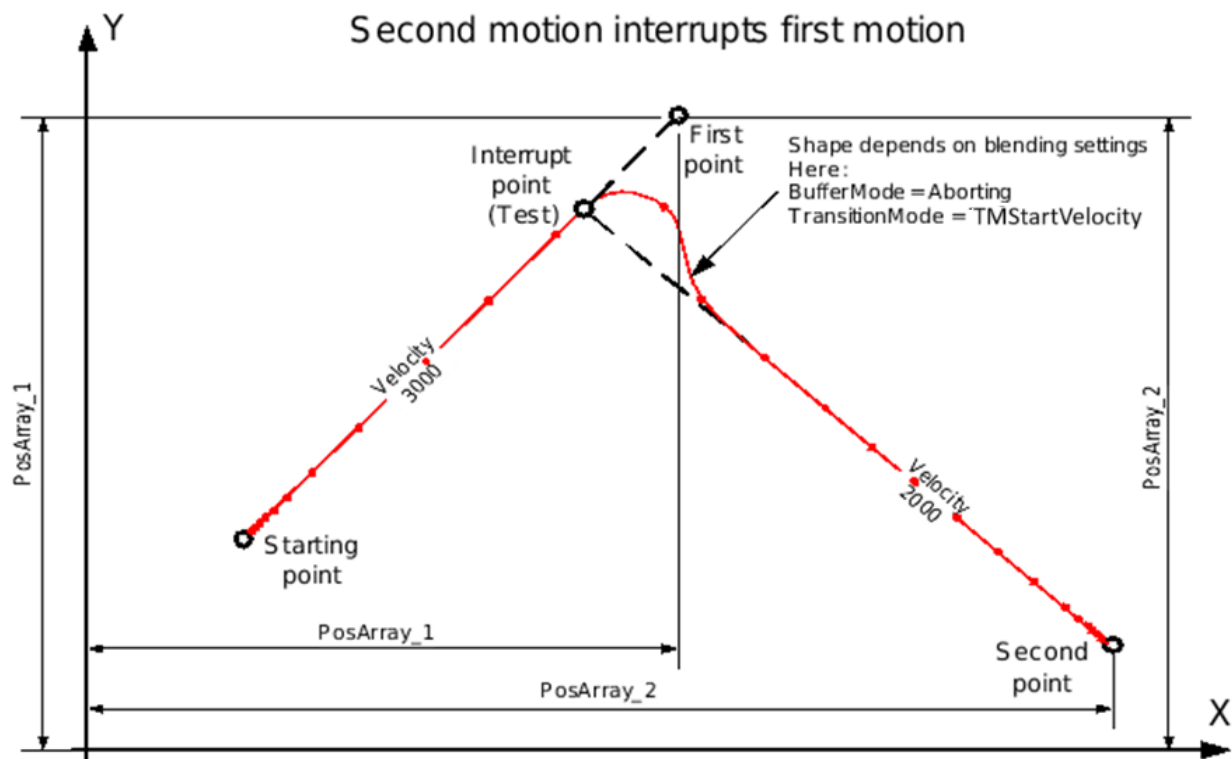
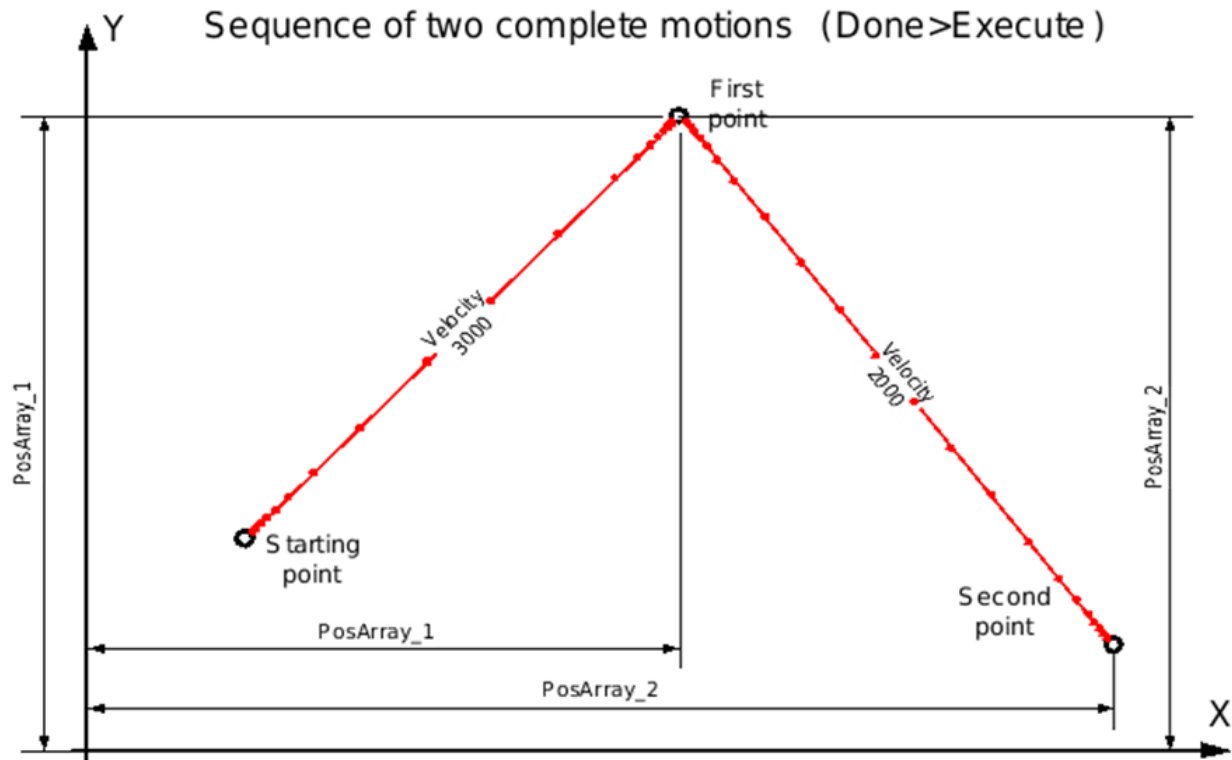


Figure 17: Example MC_MoveLinearAbsolute

The dots on the red line are based on the same timing difference and represent the velocity.

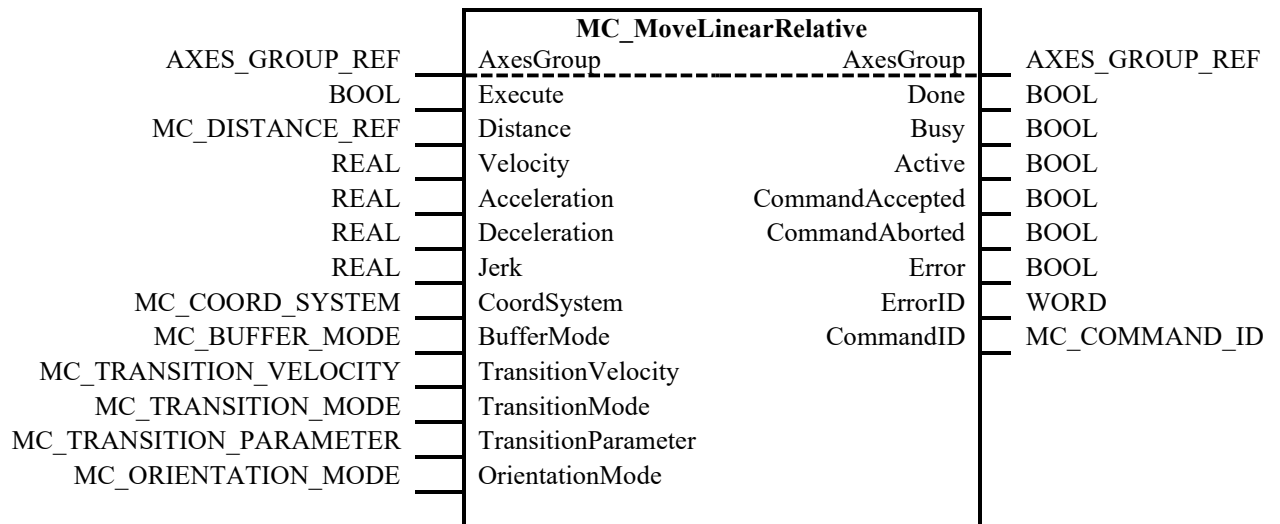
MC_MoveLinearAbsolute - Example



Note: TransitionVelocity = TransitionVelHigh

9.35 MC_MoveLinearRelative

FB-Name		MC_MoveLinearRelative		
This function block commands an interpolated linear movement on an axes group from the actual position of the TCP to a relative position in the specified coordinate system.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT				
	B	Execute	BOOL	Commands a new movement at the rising edge
	B	Distance	MC_POS_REF	Relative distances for each dimension in the specified coordinate system. See Ch 4 Definition of Position.
	E	Velocity	REAL	Maximum Velocity in [u/s] or [%] of nominal velocity for the path for the coordinate system in which the path is defined. Always positive. Not necessarily reached
	E	Acceleration	REAL	Maximum acceleration in [u/s ²] or [%] of nominal acceleration. Always positive. Not necessarily reached
	E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached
	E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached
	E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system.
	E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2 Buffer Modes
	E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see 11.2.3 Transition Velocity behavior
	E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see 11.3 TransitionMode
	E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode
	E	OrientationMode	MC_ORIENTATION_MODE	Linear, joined, fixed and path. See 11.6 MC_ORIENTATION_MODE
VAR OUTPUT				
	B	Done	BOOL	Commanded end positions reached for all axes
	E	Busy	BOOL	The FB is not finished
	E	Active	BOOL	Indicates that the FB has control on the axis
	E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
	E	CommandAborted	BOOL	The command is aborted by another command.
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID
Note: -				



9.35.1 Example 1

MC_MoveLinearRelative - Example

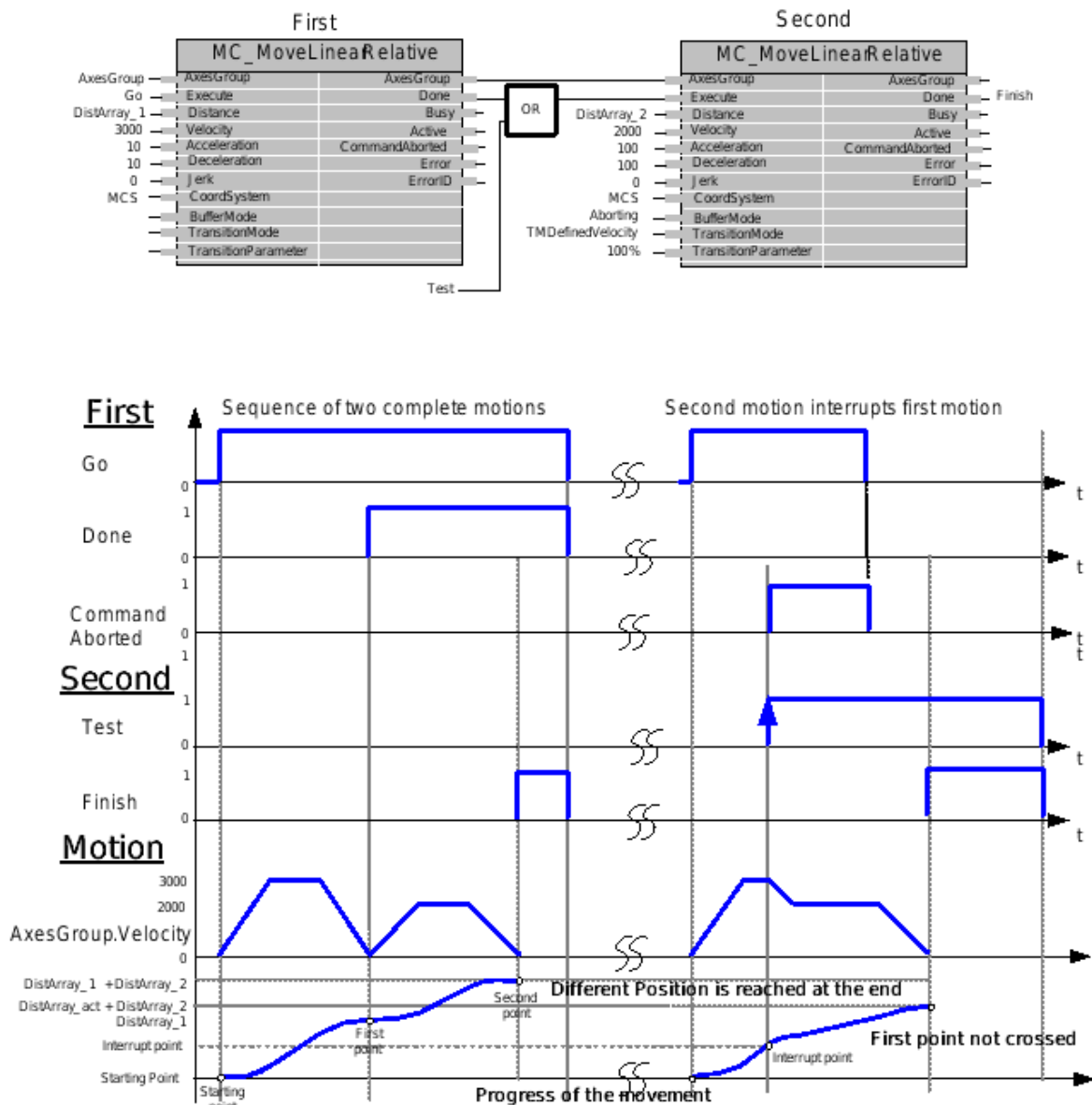
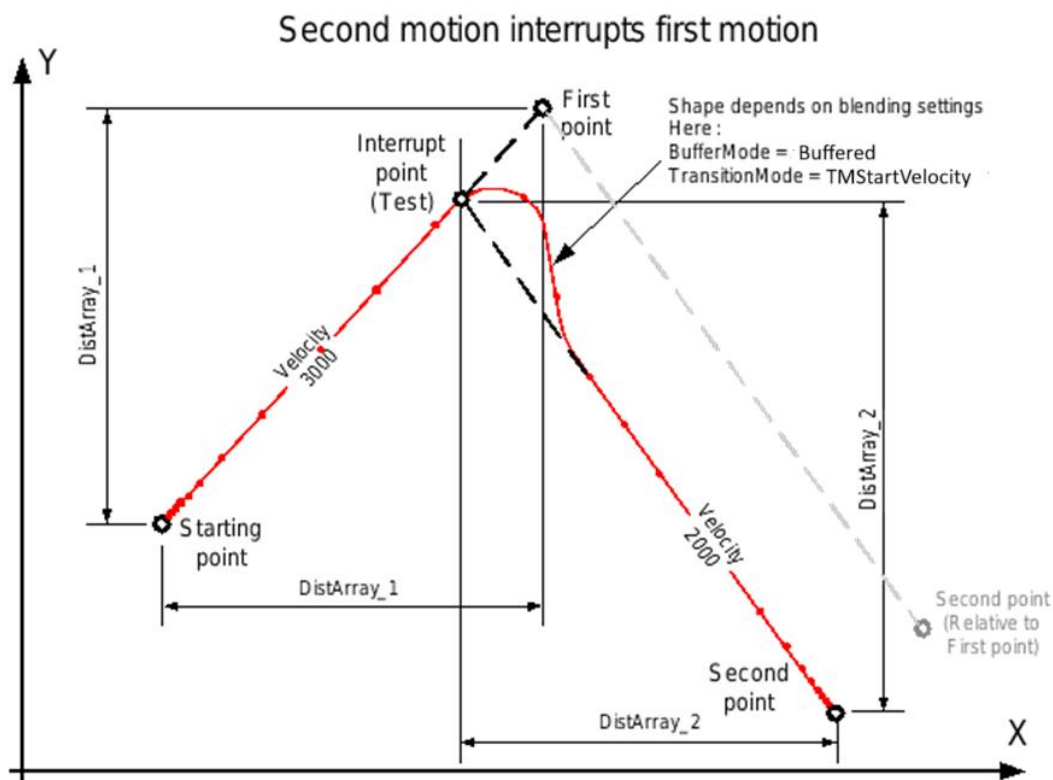
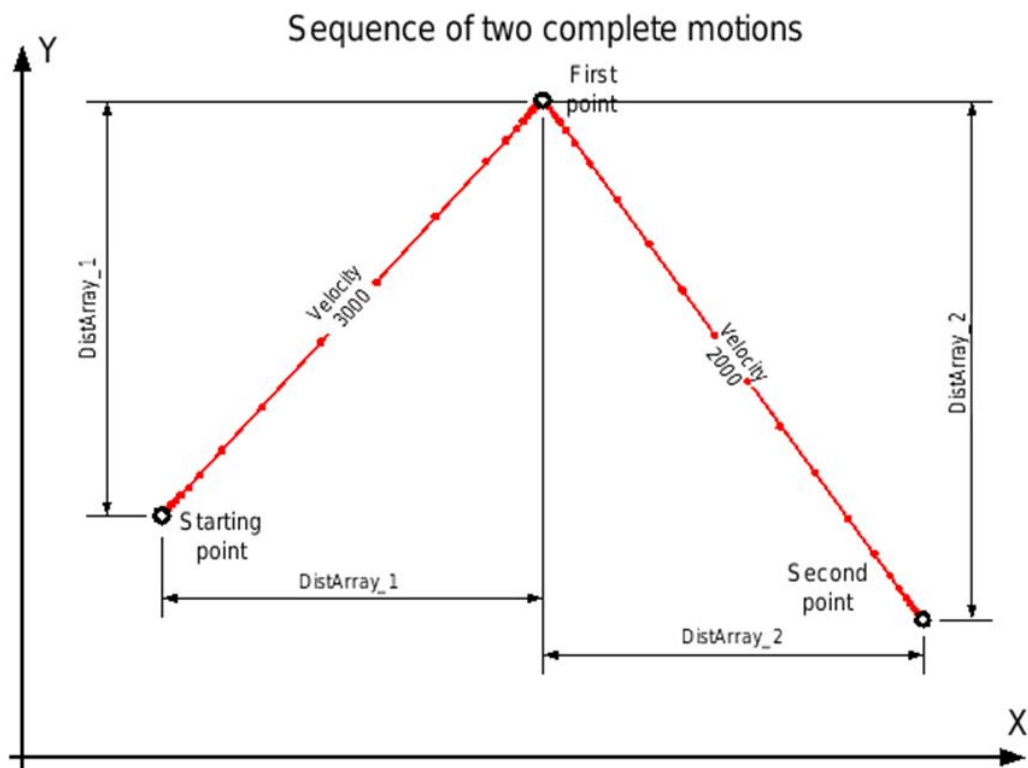


Figure18: Example MC_MoveLinearRelative

The dots on the red line are based on the same timing difference and represent the velocity.



Note: TransitionVelocity = TransitionVelHigh

9.35.2 Example 2

MC_MoveLinearRelative – Blending Example

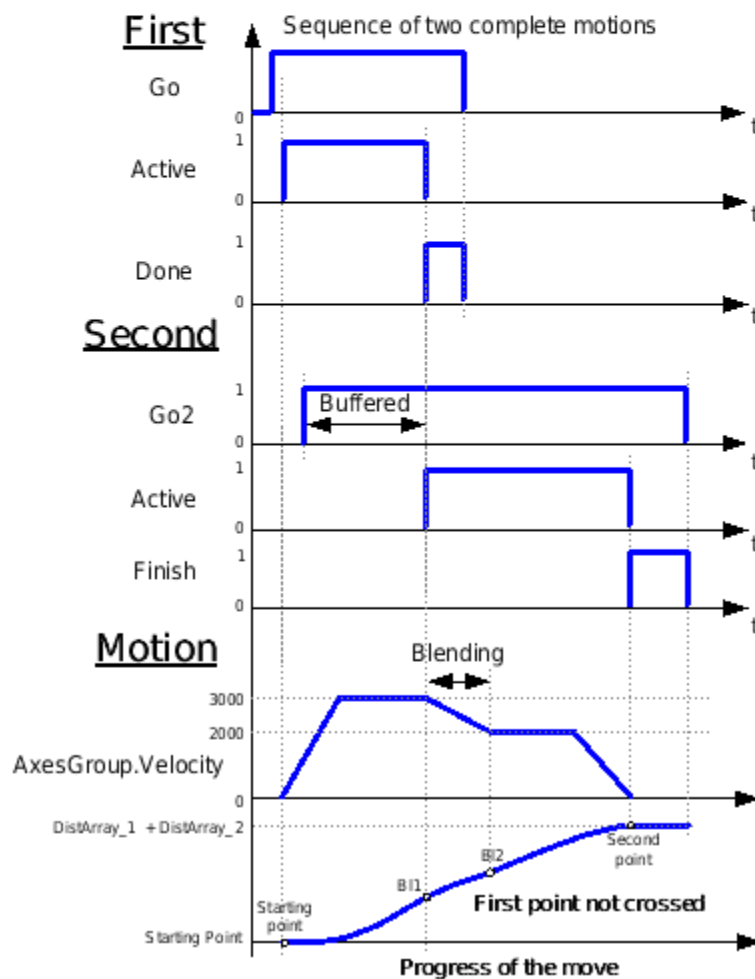
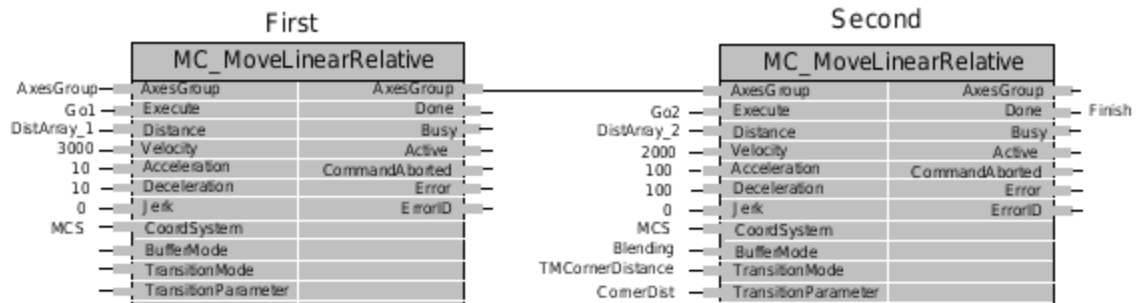
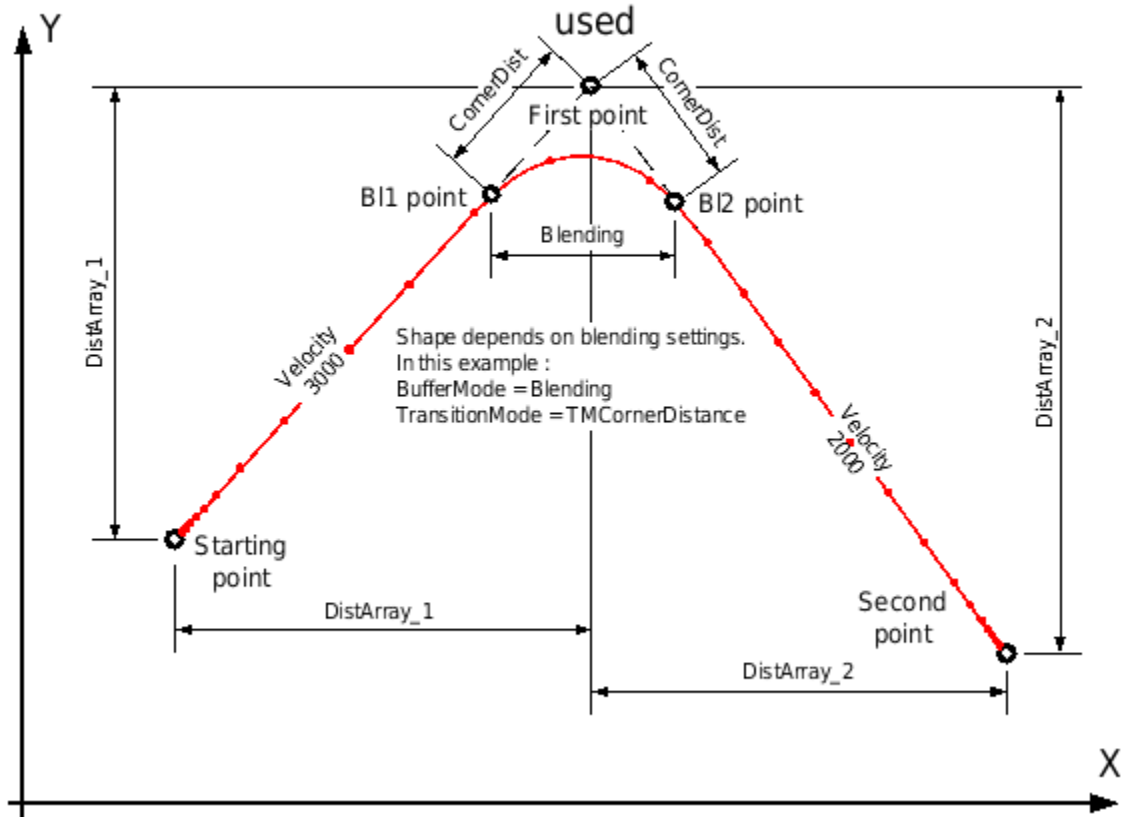


Figure19: Second example with MC_MoveLinearRelative and Blending

The dots on the red line are based on the same timing difference and represent the velocity.

MC_MoveLinearRelative – Blending Example

When not Aborting, always reference points are used



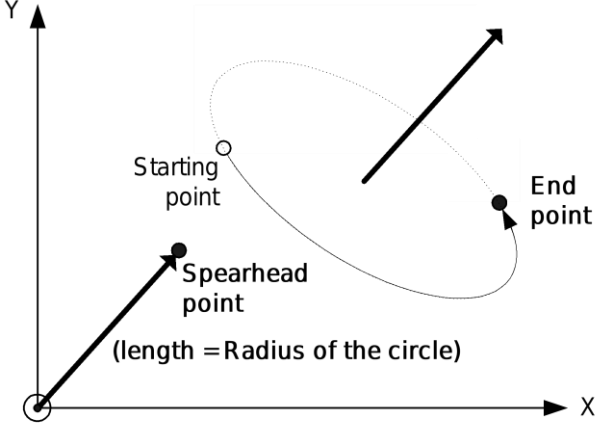
9.36 MC_MoveCircularAbsolute

FB-Name		MC_MoveCircularAbsolute		
This function block commands an interpolated circular movement on an axes group from the actual position of the TCP. The end point as well as the auxiliary point (meaning depending on applied mode, see below) are defined absolutely in the specified coordinate system.				
VAR IN OUT				
B	AxesGroup ²	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Commands a new movement at the rising edge	
B	CircMode	MC_CIRC_MODE	Specifies the meaning of the input 'AuxPoint' and 'PathChoice': mcBorder → 'AuxPoint' defines a point on the circle which is crossed on the path from the starting to the end point. mcCenter → 'AuxPoint' defines the center point of the circle. mcRadius → 'AuxPoint' defines the spearhead point of the perpendicular of the circle plane according to the rule of right thumb. The radius of the circle is the length of the vector. In the FB MC_MoveCircularAbsolute, the points are specified absolutely, i.e. the perpendicular vector begins in the origin and ends in the spearhead point specified at the input signal 'AuxPoint'.	
B	AuxPoint	MC_POS_REF	Vendor specific. See 4 Definition of Position.	
B	EndPoint	MC_POS_REF	See 4 Definition of Position.	
E	PathChoice	MC_CIRC_PATHCHOICE	Choice of path (clockwise or counterclockwise)	
E	Tolerance	REAL	Relates to CircMode =mcCenter: Permissible deviation of the distances from starting point to center point, from auxiliary point to center point and from end point to center point. These distances must be identical to travel a circular path. If the deviation is greater than the specified Tolerance, the command returns an error. See example below.	
B	Velocity	REAL	Maximum Velocity in [u/s] or [%] of nominal velocity for the path for the coordinate system in which the path is defined. Always positive. Not necessarily reached	
B	Acceleration	REAL	Maximum acceleration in [u/s ²] or [%] of nominal acceleration. Always positive. Not necessarily reached	
E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached	
E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.	
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. See 11.2 Buffer Modes	
E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see 11.3 TransitionMode	
E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see 11.2.3 Transition Velocity behavior	
E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode	
B	OrientationMode	MC_ORIENTATION_MODE	Linear, joined, fixed and path. See 11.6 MC_ORIENTATION_MODE	
VAR OUTPUT				
B	Done	BOOL	Commanded end positions reached for all axes	

E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axes group
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	Command is aborted by another command.
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID

Note: -

	CircMode = mcBorder The user defines the end point and a border point (= input 'AuxPoint') on the sector of the circle, which shall be cruised by the machine. Advantages of this mode: + The border point usually can be reached by the machine, i.e. it can be teachd. Inconvenience of this mode: – Restriction to angles $< 2\pi$ in one single command
	CircMode = mcCenter The user defines the end point and the center point (= input 'AuxPoint') of the circle. When using this mode, the input 'PathChoice' defines if the short or the long sector has to be cruised by the machine. Inconveniencies of this mode: – Restriction to angles $< 2\pi$ and $\neq \pi$ in one single command – Overdetermination of circle equation – The center point usually cannot be teachd in due to collisions with obstacles.

	CircMode = mcRadius The user defines the end point and the perpendicular vector of the circle plane according to the rule of right thumb (see figure below). The length of the vector corresponds to the radius of the circle. The spearhead point of the vector is the input signal 'AuxPoint' in absolute coordinates, i.e. referring to the origine of the coordinate system specified in 'CoordSystem'. If the diameter is larger than the distance between starting and end point, two different circles have to be considered. When using this mode, the input 'PathChoice' defines: if the circle with the short sector or the circle with the long sector to reach the end point must be cruised by the machine. With positive radius value the shortest possible circle is determined, and with negative radius value the largest possible circle.  Inconvenience of this mode: – Restriction to angles $< 2\pi$ in one single command – The perpendicular vector must be computed. – Overdetermination of circle equation Example: AuxPoint = (50,0,0) → Circle in plane parallel to y-z plane with radius 50 and rotation around axis parallel to x-axis according to the rule of right thumb (CoordSystem = MCS)
---	--

MC_MoveCircularAbsolute			
AXES_GROUP_REF	AxesGroup	AxesGroup	AXES_GROUP_REF
BOOL	Execute	Done	BOOL
MC_CIRC_MODE	CircMode	Busy	BOOL
MC_POS_REF	AuxPoint	Active	BOOL
MC_POS_REF	EndPoint	CommandAccepted	BOOL
MC_CIRC_PATHCHOICE	PathChoice	CommandAborted	BOOL
REAL	Tolerance	Error	BOOL
REAL	Velocity	ErrorID	WORD
REAL	Acceleration	CommandID	MC_COMMAND_ID
REAL	Deceleration		
REAL	Jerk		
MC_COORD_SYSTEM	CoordSystem		
MC_BUFFER_MODE	BufferMode		
MC_TRANSITION_MODE	TransitionMode		
MC_TRANSITION_PARAMETER	TransitionParameter		
MC_ORIENTATION_MODE	OrientationMode		

9.36.1 Example 1

The following example illustrates the movement of a "MoveCircularAbsolute" command which is defined with the "CircMode" = mcBorder. So, the circle is calculated by the starting position P[0], the "EndPoint" E1 and the "AuxPoint" A1 being on the circle's border.

The movement is defined with the input parameters "PathChoice" = mcClockwise so the movement is going from the start position P[0] clockwise to the "EndPoint" E1.

All positions are specified by absolute cartesian positions in the specified coordinate system.

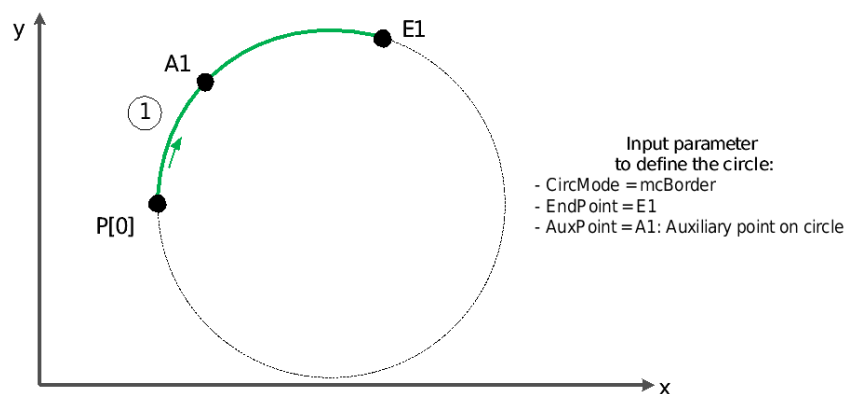
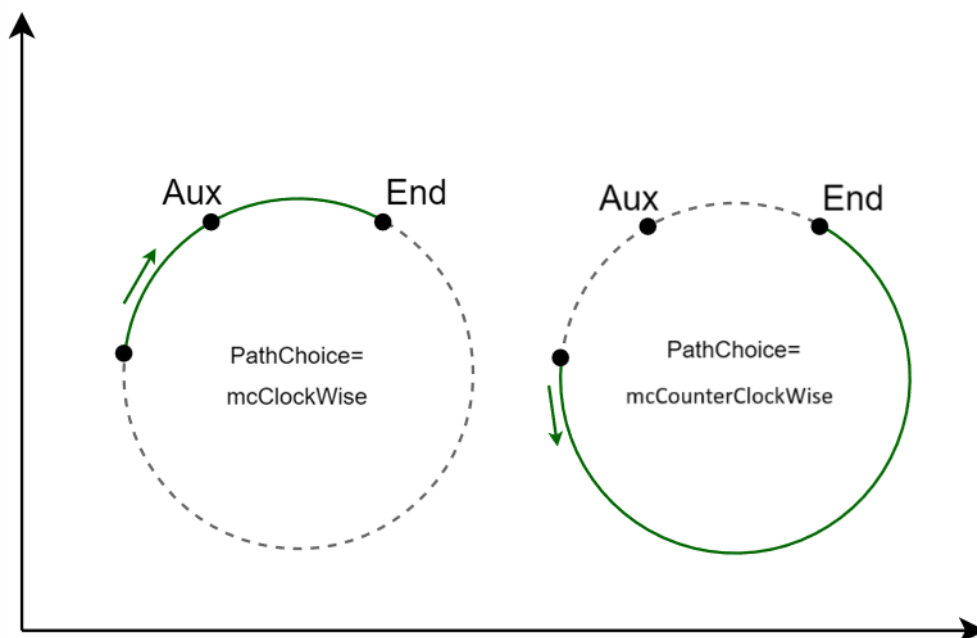


Figure 20: "MoveCircularAbsolute" Example 1 – "CircMode" = mcBorder

MC_MoveCircularAbsolute			
Group_1	AxesGroup	AxesGroup	
Execute_1	Execute	Done	Done_1
Center	CircMode	Busy	
A1	AuxPoint	Active	Active_1
E1	EndPoint	CommandAccepted	
0	PathChoice	CommandAborted	Aborted_1
	Tolerance	Error	
	Velocity	ErrorID	
	Acceleration	CommandID	
	Deceleration		
	Jerk		
	CoordSystem		
	BufferMode		
	TransitionMode		
	TransitionParameter		
	OrientationMode		

9.36.2 Example 2

This example shows the different path choices for mcBorder:



9.36.3 Example 3

The following example illustrates the movement of a "MoveCircularAbsolute" command which is defined with the "CircMode" = mcCenter. So, the circle is calculated by the starting position P[0], the "EndPoint" E1 and the "AuxPoint" A1 being in the circle's center.

The movement is defined with the input parameters "PathChoice" = mcClockwise, so the movement is going from the start position P[0] clockwise to the "EndPoint" E1.

All positions are specified by absolute cartesian positions in the specified coordinate system.

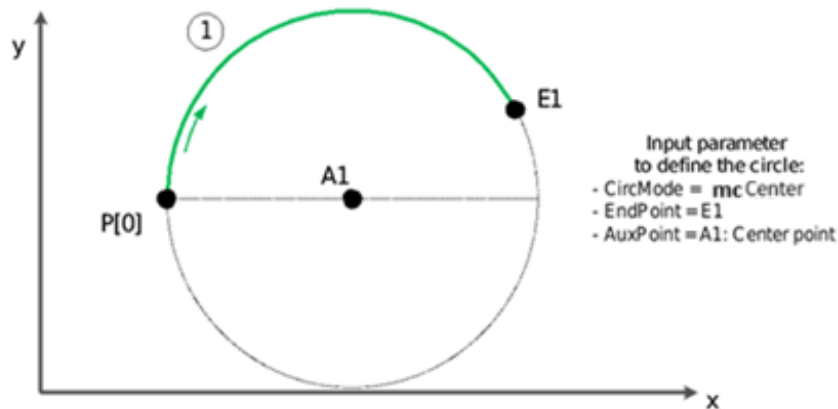
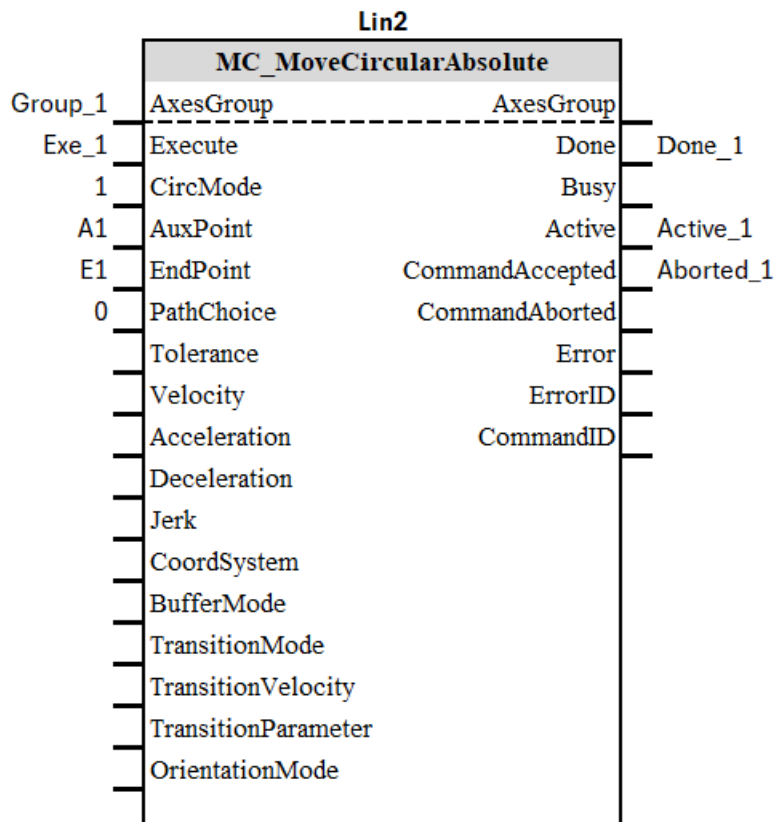


Figure 21: "MoveCircularAbsolute" Example – "CircMode" = mcCenter



When defining a circular path in "CircMode" = mcCenter, the input parameter "Tolerance" allows the axes group to relocate the center point, specified through the input parameter "AuxPoint" for calculation.

To calculate a circle from a starting point P[0], an end point E1 and a center point A1, the distances d_1 and d_2 (see **Fout! Verwijzingsbron niet gevonden.**) must be identical. If the difference between d_1 and d_2 is within the specified range given through the input parameter "Tolerance", the axes group may relocate the center point for calculation internally ($A1'$).

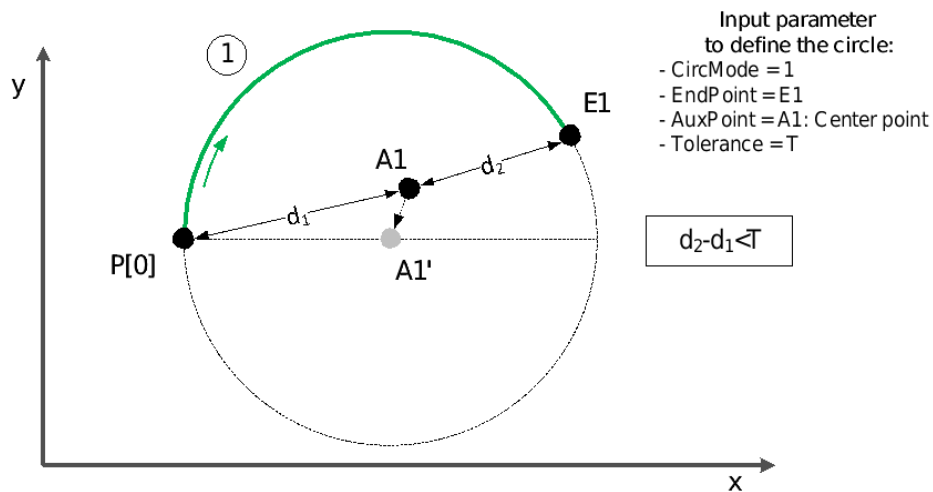


Figure 22: "MoveCircularAbsolute" Example – "CircMode" = mcCenter - "Tolerance"

MC_MoveCircularAbsolute – Sequenced Example

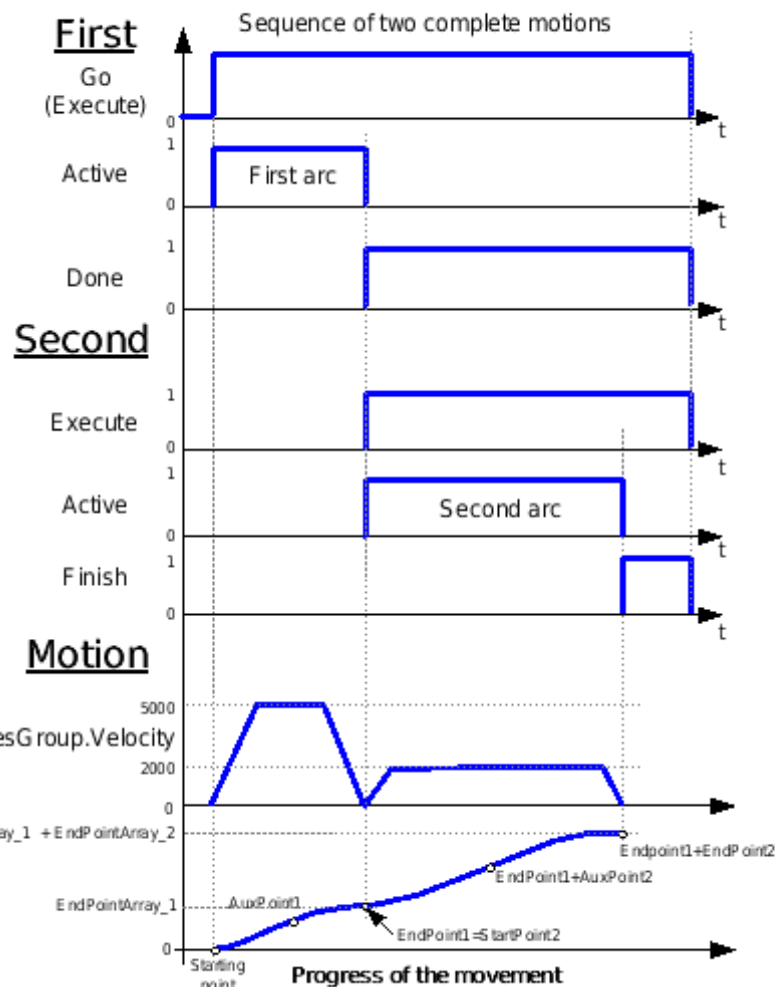
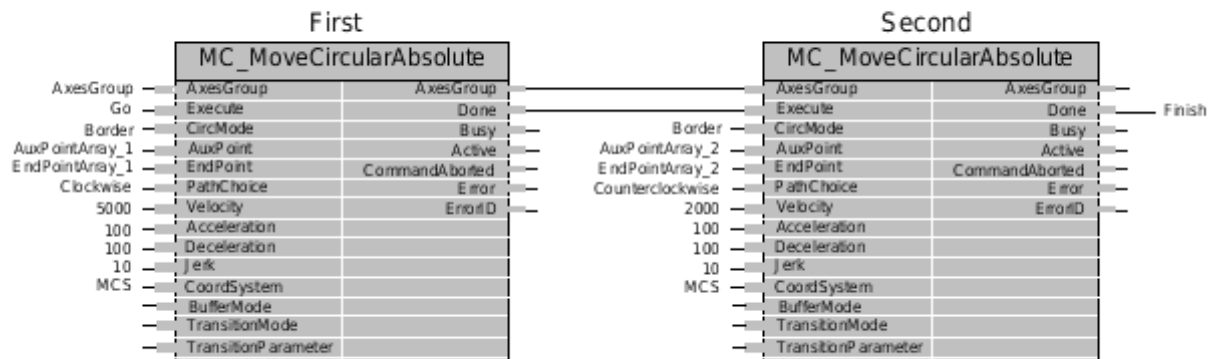


Figure 23: Example MC_MoveCircularAbsolute

MC_MoveCircularAbsolute - Example

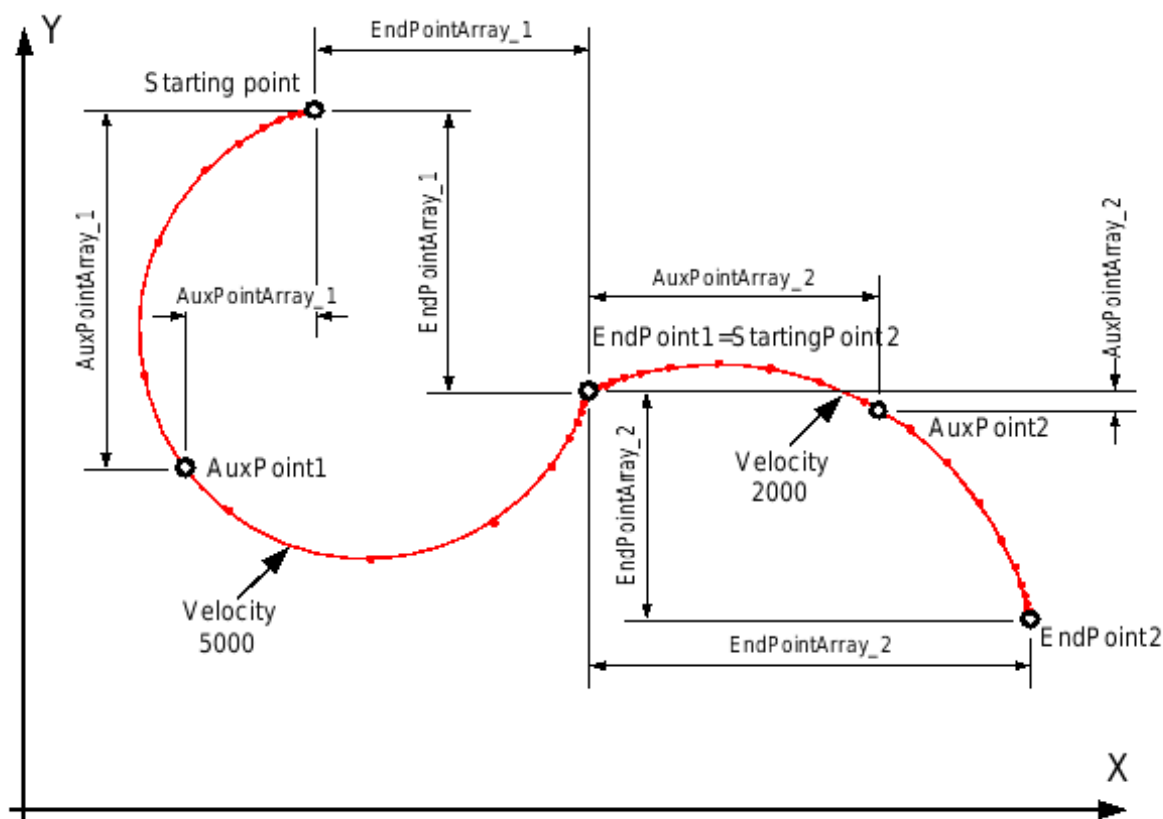


Figure 24: Visualisation of MC_MoveCircularAbsolute sequence

The dots on the red line are based on the same timing difference and represent the velocity.

9.37 MC_MoveCircularRelative

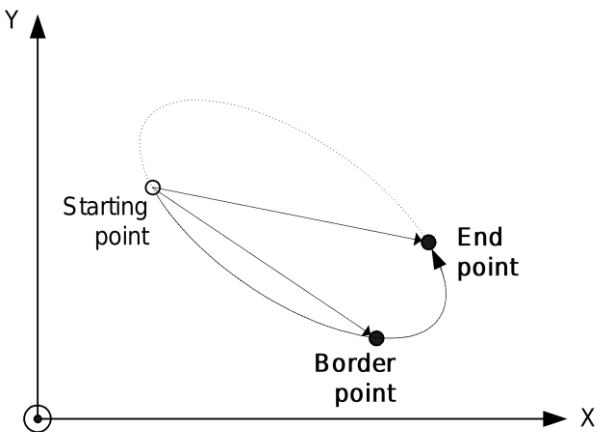
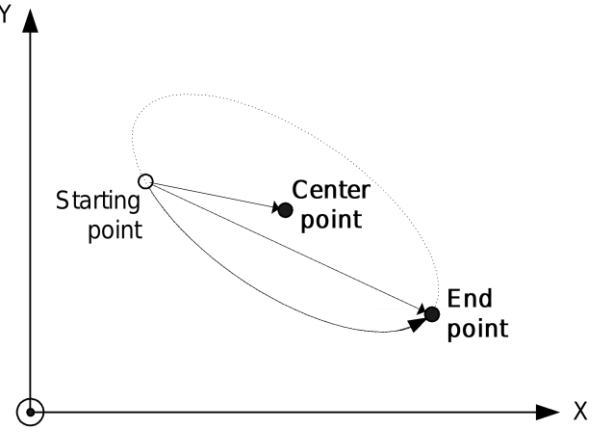
FB-Name		MC_MoveCircularRelative		
This function block commands an interpolated circular relative movement from the actual position of the TCP to a cartesian end point. The end point as well as the auxiliary point (meaning depending on applied mode, see below) are defined in the specified coordinate system relatively to the starting point There are different methods to define the circular motion, which are chosen with the input parameter "CircMode". It is possible to define the circle with an auxiliary point, a center point or a radius. The necessary positions are defined with the input parameter "AuxPoint".				
VAR IN OUT				
	B	AxesGroup ²	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT				
	B	Execute	BOOL	Commands a new movement at the rising edge
	B	CircMode	MC_CIRC_MODE	Specifies the meaning of the input signals 'AuxPoint' and 'PathChoice': mcBorder → 'AuxPoint' defines a point on the circle which is crossed on the path from the starting to the end point. It is specified relatively to the starting point. mcCenter → 'AuxPoint' defines the center point of the circle. It is specified relatively to the starting point. mcRadius → 'AuxPoint' defines the spearhead point of the perpendicular of the circle plane according to the rule of right thumb. The radius of the circle is the length of the vector. In the FB MC_MoveCircularRelative, the points are specified relatively, i.e. the perpendicular vector begins in the starting point and ends in the spearhead point specified at the input signal 'AuxPoint'.
	B	AuxPoint	MC_POS_REF	Describes the auxiliary position for specifying the circular movement. These positions are defined relatively to the positions of the starting point. See Ch 4 Definition of Position.
	B	EndPoint	MC_POS_REF	Describes the end point for the circular movement. These positions are defined relatively to the positions of the starting point. See Ch 4 Definition of Position.
	E	PathChoice	MC_CIRC_PATHCHOICE	Choice of the path: mcClockwise or mcCounterClockwise – see 11.5 MC_CIRC_PATHCHOICE
	E	Tolerance	REAL	Relates to CircMode = mcCenter: Permissible deviation of the distances from starting point to center point, from auxiliary point to center point and from end point to center point. These distances must be identical to travel a circular path. If the deviation is greater than the specified Tolerance, the command returns an error. See example below.
	B	Velocity	REAL	Maximum Velocity in [u/s] or [%] of nominal velocity for the path for the coordinate system in which the path is defined. Always positive. Not necessarily reached
	B	Acceleration	REAL	Maximum acceleration in [u/s ²] or [%] of nominal acceleration. Always positive. Not necessarily reached
	E	Deceleration	REAL	Maximum deceleration in [u/s ²] or [%] of nominal deceleration. Always positive. Not necessarily reached
	E	Jerk	REAL	Maximum jerk in [u/s ³] or [%] of nominal jerk. Always positive. Not necessarily reached
	E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.
	E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2

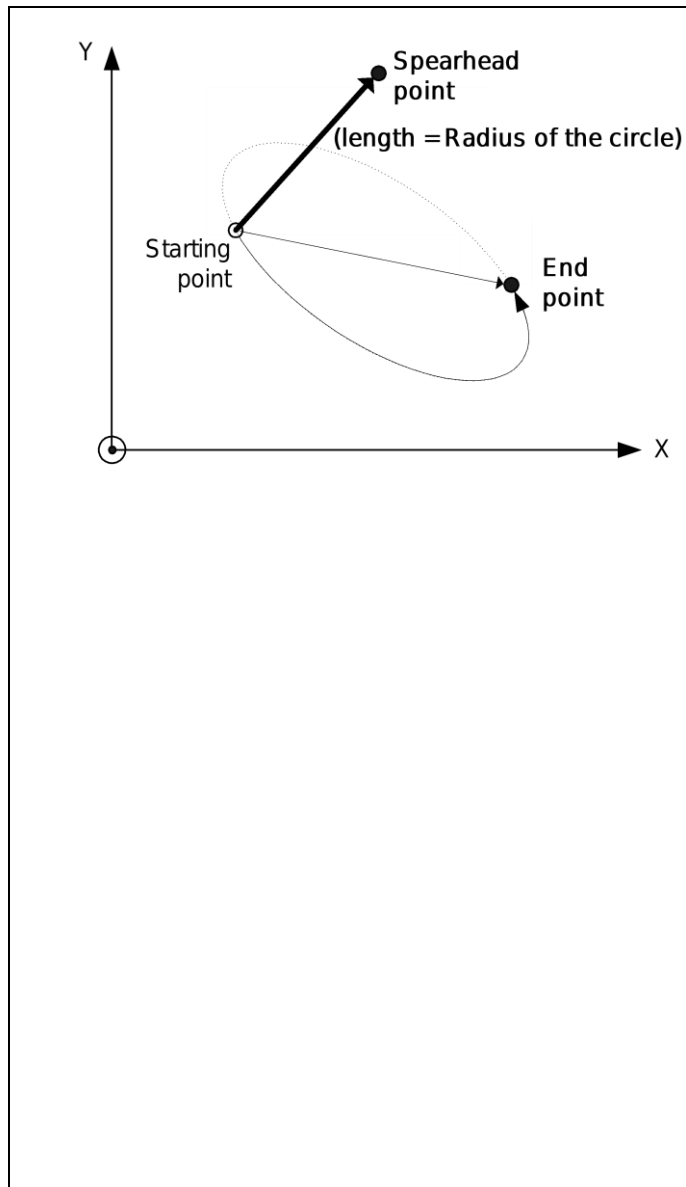
E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see chapter 11.3 TransitionMode.
E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see chapter 11.2.3 Transition Velocity behavior
E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode
B	OrientationMode	MC_ORIENTATION_MODE	Linear, joined, fixed and path. See chapter 11.6 MC_ORIENTATION_MODE

VAR OUTPUT

B	Done	BOOL	Commanded end positions reached for all axes
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axes group
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
E	CommandAborted	BOOL	The command is aborted by another command.
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID

Note: -

	CircMode = mcBorder The user defines the end point and a border point (= input 'AuxPoint') on the sector of the circle, which shall be cruised by the machine. Both points are defined relatively to the starting point. Advantages of this mode: + The border point usually can be reached by the machine, i.e. it can be teachd. Inconvenience of this mode: – Restriction to angles $< 2\pi$ in one single command
	CircMode = mcCenter The user defines the end point and the center point (= input 'AuxPoint') of the circle. Both points are defined relatively to the starting point When using this mode, the input 'PathChoice' defines: if the short or the long sector must be cruised by the machine. Inconveniencies of this mode: – Restriction to angles $< 2\pi$ and $\neq \pi$ in one single command – Overdetermination of circle equation – The center point usually cannot be teachd-in due to collisions with obstacles.
	CircMode = mcRadius The user defines the end point and the perpendicular vector of the circle plane according to the rule of right thumb (see figure below). The length of the vector corresponds to the radius of the



circle. The spearhead point of the vector is defined relatively to the starting point at the input signal 'AuxPoint'.

If the diameter is larger than the distance between starting and end point, two different circles have to be considered. When using this mode, the input 'PathChoice' defines: if the circle with the short sector or the circle with the long sector to reach the end point must be cruised by the machine. With positive radius value the shortest possible circle is determined, and with negative radius value the largest possible circle.



Inconvenience of this mode:

- Restriction to angles $< 2\pi$ in one single command
- The perpendicular vector has to be computed.
- Overdetermination of circle equation

Example:

AuxPoint = (starting_point[0], starting_point[1] - 30, starting_point[2]) → Circle in plane parallel to x-z plane with radius 30 and rotation around axis parallel to y-axis contrariwise the rule of right thumb (CoordSystem = MCS)

MC_MoveCircularRelative			
AXES_GROUP_REF	AxesGroup	AxesGroup	AXES_GROUP_REF
BOOL	Execute	Done	BOOL
MC_CIRC_MODE	CircMode	Busy	BOOL
MC_AUX_DISTANCE_REF	AuxPoint	Active	BOOL
MC_DISTANCE_REF	EndPoint	CommandAccepted	BOOL
MC_CIRC_PATHCHOICE	PathChoice	CommandAborted	BOOL
REAL	Tolerance	Error	BOOL
REAL	Velocity	ErrorID	WORD
REAL	Acceleration	CommandID	MC_COMMAND_ID
REAL	Deceleration		
REAL	Jerk		
REAL	CoordSystem		
MC_BUFFER_MODE	BufferMode		
MC_TRANSITION_MODE	TransitionMode		
MC_TRANSITION_VELOCITY	TransitionVelocity		
MC_TRANSITION_PARAMETER	TransitionParameter		
MC_ORIENTATION_MODE	OrientationMode		

9.37.1 Example 1

The following example illustrates the movement of a "MoveCircularRelative" command which is defined with the "CircMode" = mcBorder. So, the circle is calculated by the starting position P[0], the "EndPoint" E1 and the "AuxPoint" A1 being on the circle's border.

The movement is defined with the input parameters "PathChoice" = mcClockwise, so the movement is going from the start position P[0] clockwise to the "EndPoint" E1.

All points are specified by a relative distance from the start position of the TCP P[0].

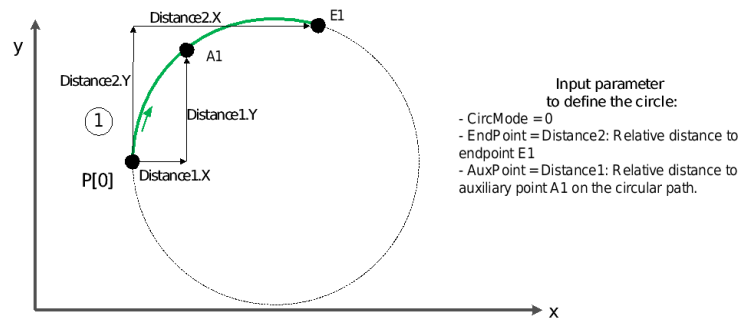
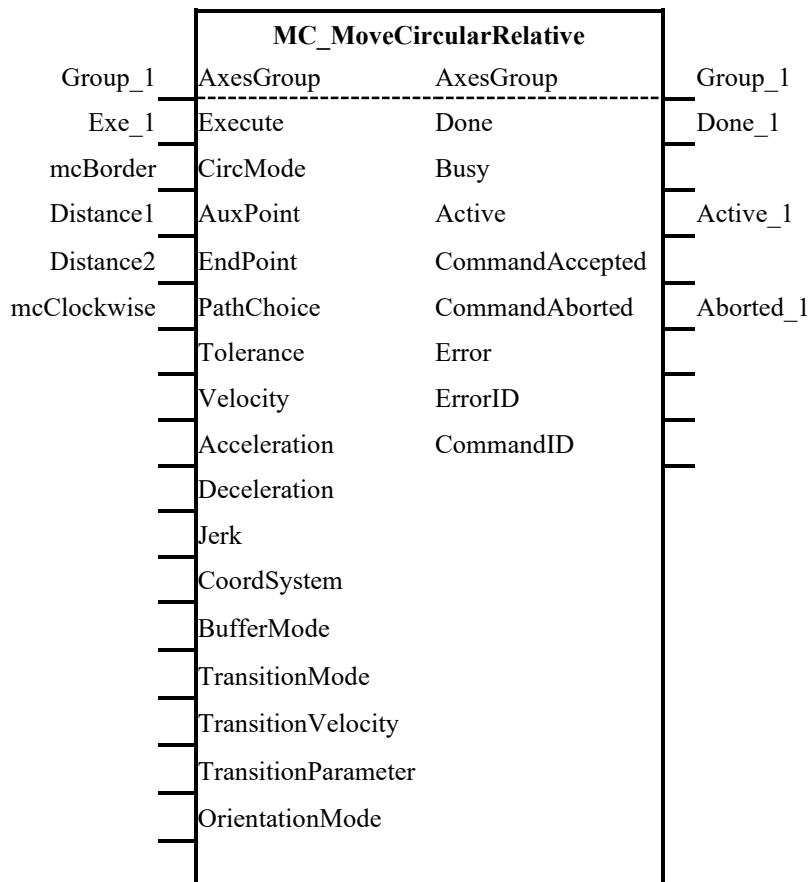


Figure 25: "MoveCircularRelative" Example – "CircMode" = mcBorder



9.37.2 Example 2

The following example illustrates the movement of a "MoveCircularRelative" commands which is defined with the "CircMode" = mcCenter. So, the circle is calculated by the starting position P[0], the "EndPoint" E1 and the "AuxPoint" A1 being in the circle's center.

All points are specified by a relative distance from the start position of the TCP P[0].

The movement is defined with the input parameters "PathChoice" = mcClockwise, so the movement is going from the start position P[0] clockwise to the "EndPoint" E1.

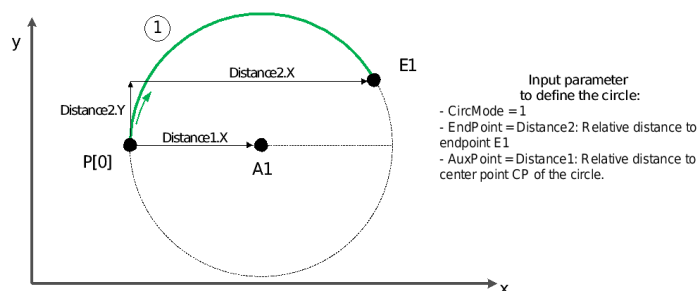


Figure 26: "MoveCircularRelative" Example – "CircMode" = 1

MC_MoveCircularRelative			
Group_1	AxesGroup	AxesGroup	Group_1
Exe_1	Execute	Done	Done_1
mcCenter	CircMode	Busy	
Distance1	AuxPoint	Active	Active_1
Distance2	EndPoint	CommandAccepted	
mcClockwise	PathChoice	CommandAborted	Aborted_1
	Tolerance	Error	
	Velocity	ErrorID	
	Acceleration	CommandID	
	Deceleration		
	Jerk		
	CoordSystem		
	BufferMode		
	TransitionMode		
	TransitionVelocity		
	TransitionParameter		
	OrientationMode		

When defining a circular path in "CircMode" = mcCenter, the input parameter "Tolerance" allows the axes group to relocate the center point, specified through the input parameter "AuxPoint" for calculation.

To calculate a circle from a starting point P[0], an end point E1 and a center point A1, the distances d_1 and d_2 (see following Figure 27: "MoveCircularRelative" Example – "CircMode" = mcCenter - "Tolerance" 115) must be identical. If the difference between d_1 and d_2 is within the specified range given through the input parameter "Tolerance", the axes group may relocate the center point for calculation internally (A1').

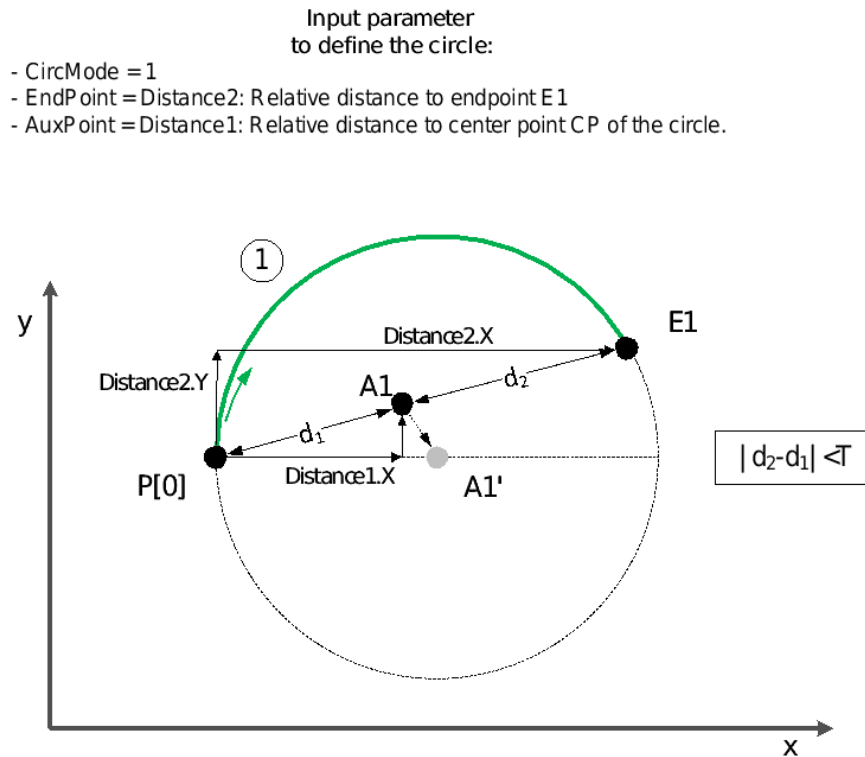
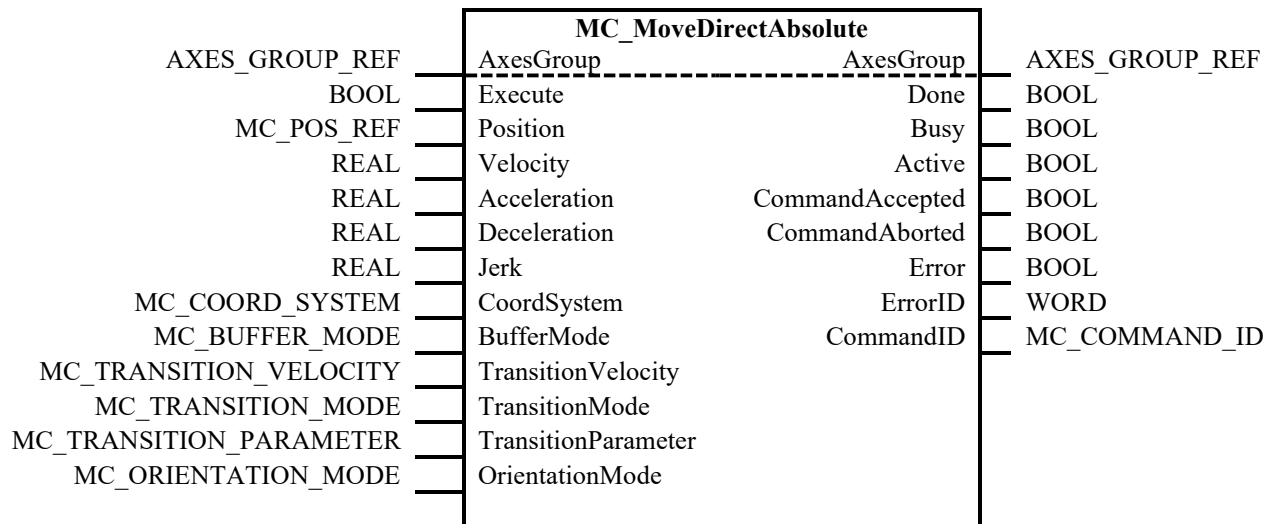


Figure 27: "MoveCircularRelative" Example – "CircMode" = mcCenter - "Tolerance"

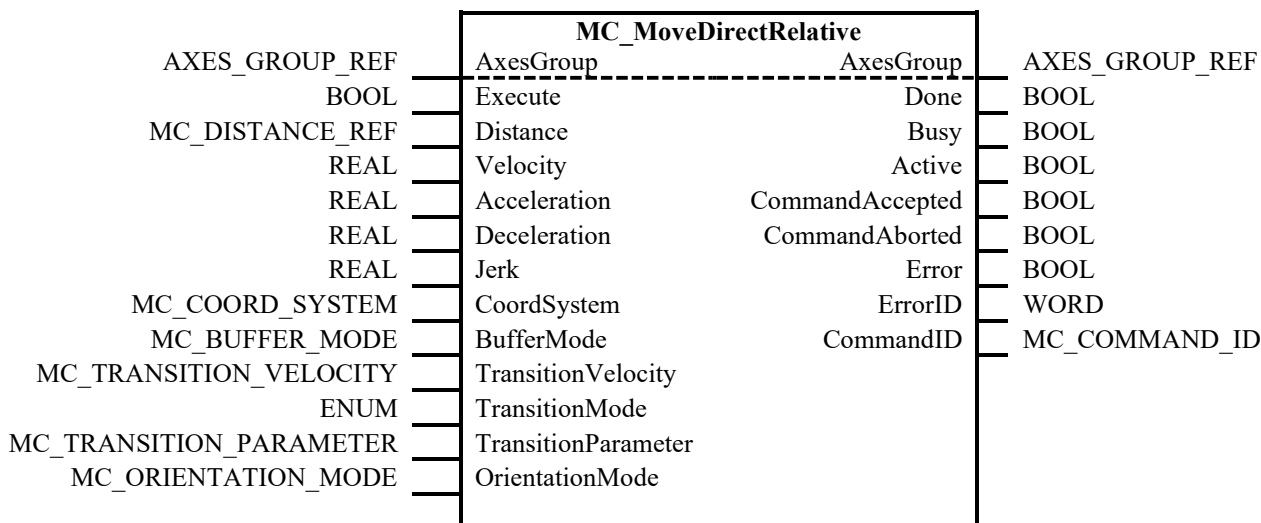
9.38 MC_MoveDirectAbsolute

FB-Name		MC_MoveDirectAbsolute		
This function block commands the movement of an axes group to the specified absolute position in the specified coordinate system without taking care of how (on which path) the target position is reached.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR INPUT				
	B	Execute	BOOL	Commands a new movement at the rising edge
	B	Position	MC_POS_REF	See Ch 4 Definition of Position
	E	Velocity	REAL	Maximum Velocity in [%] of nominal velocity of the axes. Always positive. Not necessarily reached
	E	Acceleration	REAL	Maximum acceleration in [%] of nominal axes acceleration. Always positive. Not necessarily reached
	E	Deceleration	REAL	Maximum deceleration in [%] of nominal axes deceleration. Always positive. Not necessarily reached
	E	Jerk	REAL	Maximum jerk in [%] of nominal axes jerk. Always positive. Not necessarily reached
	E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system.
	E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to Chapter 11.2 Buffer Modes
	E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see 11.2.3 Transition Velocity behavior
	E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see 11.3 TransitionMode
	E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode
VAR OUTPUT				
	B	Done	BOOL	Commanded end positions reached for all axes
	E	Busy	BOOL	The FB is not finished
	E	Active	BOOL	Indicates that the FB has control on the axes group.
	E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed
	E	CommandAborted	BOOL	The command is aborted by another command.
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID MC_COMMAND_ID
Note:				



9.39 MC_MoveDirectRelative

FB-Name		MC_MoveDirectRelative		
This function block commands a movement of an axes group to a relative position without taking care of how (on which path) the commanded position is reached. Start position is the actual position of the TCP.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
VAR INPUT				
B	Execute	BOOL	Commands a new movement at the rising edge	
B	Distance	MC_DISTANCE_REF	See Ch 4 Definition of Position	
E	Velocity	REAL	Maximum Velocity in [%] of nominal axes velocity. Always positive. Not necessarily reached	
E	Acceleration	REAL	Maximum acceleration in [%] of nominal axes acceleration. Always positive. Not necessarily reached	
E	Deceleration	REAL	Maximum deceleration in [%] of nominal deceleration. Always positive. Not necessarily reached	
E	Jerk	REAL	Maximum jerk in [u/s³] or [%] of nominal jerk. Always positive. Not necessarily reached	
E	CoordSystem	MC_COORD_SYSTEM	Reference to the coordinate system.	
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to Chapter 11.2 Buffer Modes	
E	TransitionVelocity	MC_TRANSITION_VELOCITY	For a more detailed description, see 11.2.3 Transition Velocity behavior	
E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see 11.3 TransitionMode	
E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode	
VAR OUTPUT				
B	Done	BOOL	Commanded end positions reached for all axes	
E	Busy	BOOL	The FB is not finished	
E	Active	BOOL	Indicates that the FB has control on the axis	
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed	
E	CommandAborted	BOOL	Command is aborted by another command.	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
Note:				



9.39.1 Example 1

Following example shows the behaviour of MC_MoveDirectRelative. All positions are related to MCS:

- Starting at position p_0 (10; 10) a MC_MoveLinearAbsolute to position p_1 (80; 35) is commanded.
- While the TCP is moving towards p_1 , the MC_MoveLinearAbsolute command is aborted by a MC_MoveDirectRelative command. The actual position of the TCP, when MC_MoveDirectRelative becomes active, is (44.5; 21.63).
- The TCP leaves the line p_0p_1 and moves to the new target position p_2 (54.5; 41.63). The resulting trajectory depends on the kinematic transformation of the axes group.

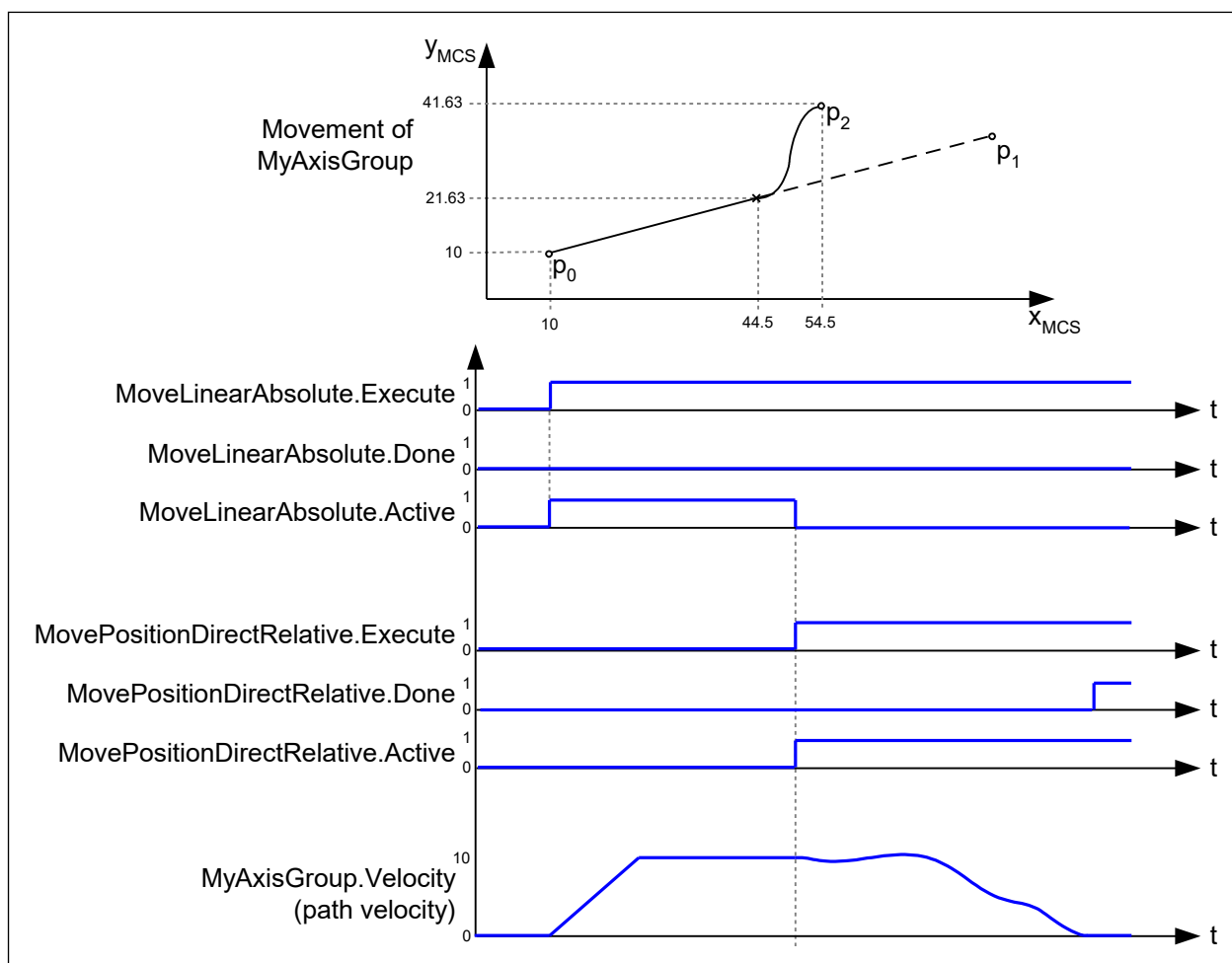
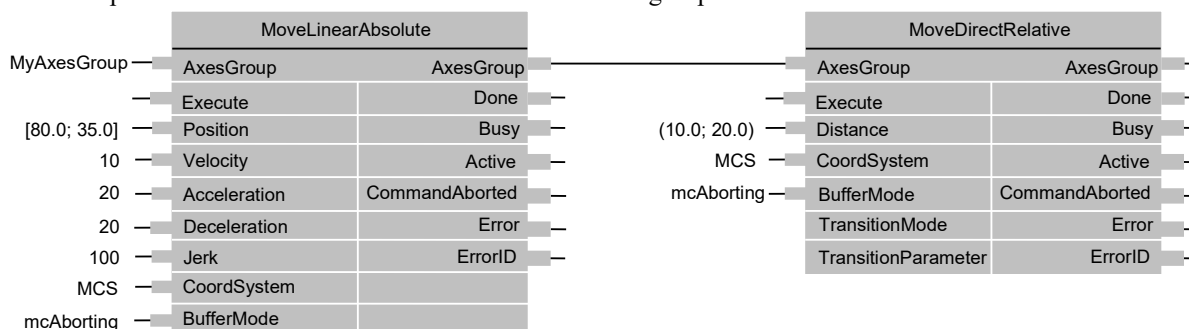
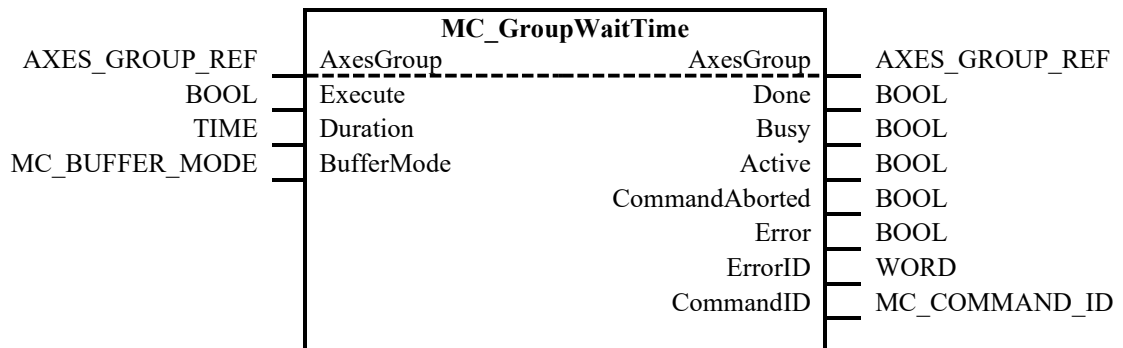


Figure 28: Example 1 MC_MoveDirectRelative

9.40 MC_GroupWaitTime

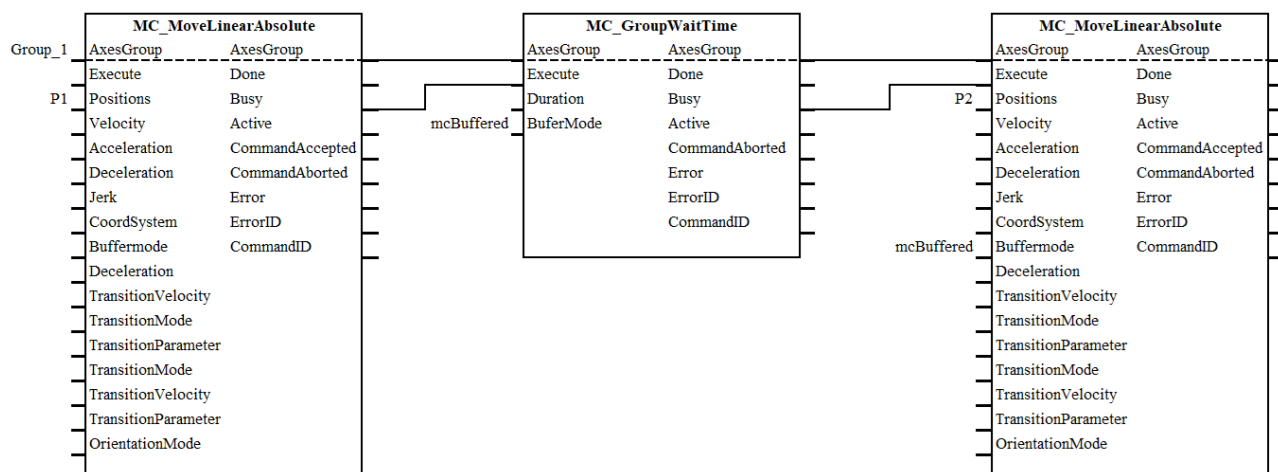
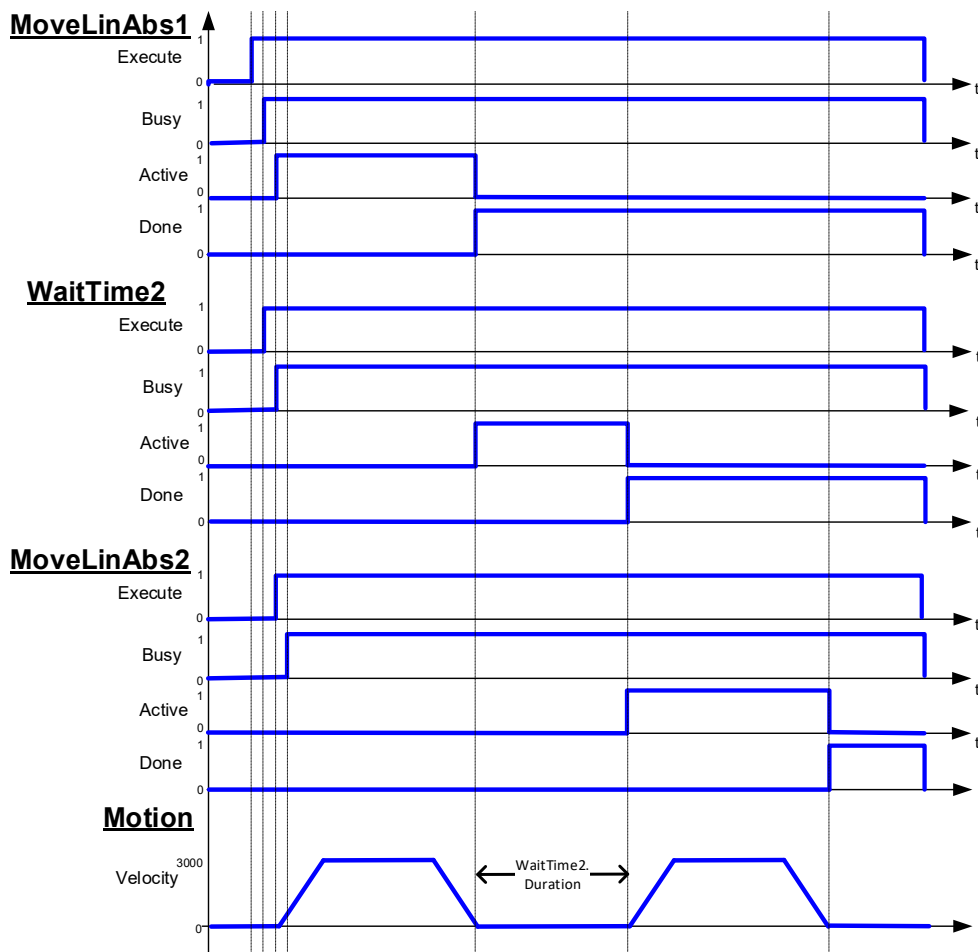
FB-Name		MC_GroupWaitTime		
Waits for a user defined time specified via the input "Duration" before starting the next movement. In case of aborting the duration starts when the axes group has reached velocity ‘0’.				
Blending into a wait command or blending out of a wait command is not supported.				
Note: MC_GroupReadCommandInfo can be used to read the remaining duration of a wait command.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	
VAR INPUT				
	B	Execute	BOOL	Start of the command at the rising edge
	B	Duration	TIME	Duration to wait. The duration must be greater than 0.
	B	BufferMode	MC_BUFFER_MODE	Refer to 11.2 Buffer Modes
VAR OUTPUT				
	B	Done	BOOL	The command has been completed successfully.
	E	Busy	BOOL	The FB is being processed
	E	Active	BOOL	The command takes control of the motion of the according axes group.
	E	CommandAborted	BOOL	The command was aborted by another command.
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID



See next page for an example of GroupWaitTime function.

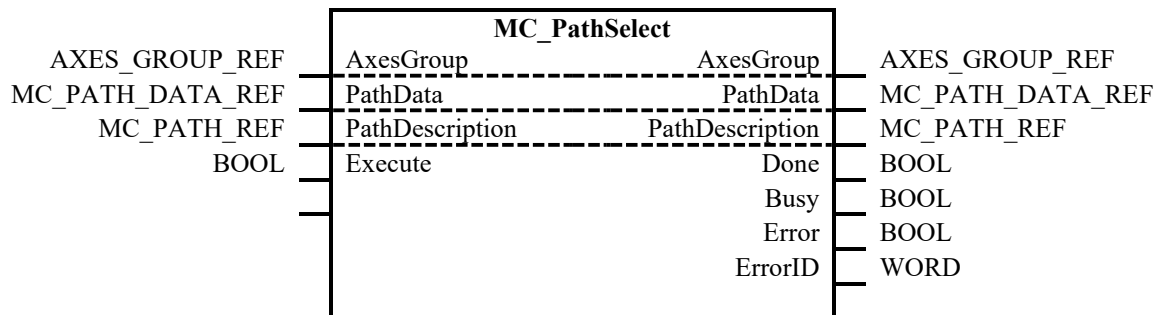
Example:

Below example shows the robot movement with the "GroupWaitTime" function at position P1. The GroupWaitTime command will be active between the two "MoveLinearAbsolute" commands.



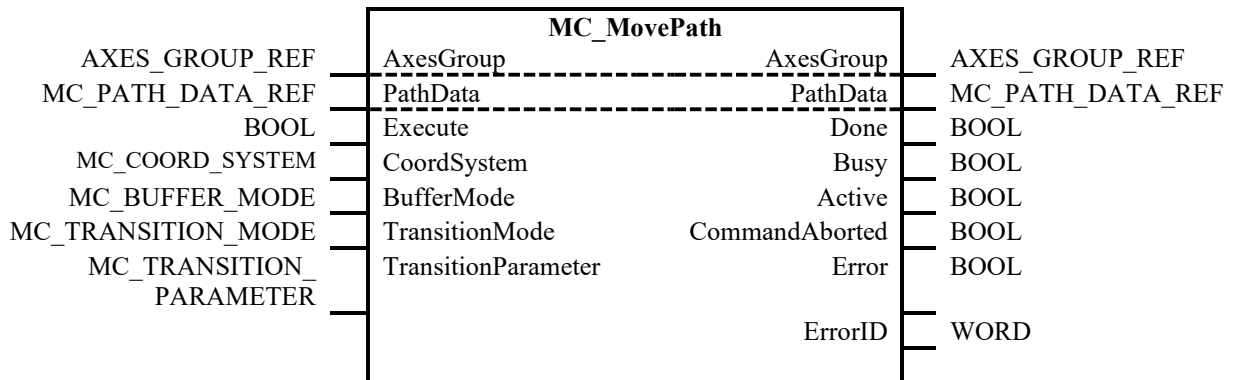
9.41 MC_PathSelect

FB-Name		MC_PathSelect	
This function block prepares the relevant path data and makes these available to the system as an output (PathData). Administrative FB.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to the group of axes
B	PathData	MC_PATH_DATA_REF	Reference to the resulting path data to be used in FBs requiring a path description.
B	PathDescription	MC_PATH_REF	Reference to the path description
VAR INPUT			
B	Execute	BOOL	Start the preparation at the rising edge
VAR OUTPUT			
B	Done	BOOL	The PathData is valid
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Note:			
- MC_PATH_DATA_REF is a vendor specific data type - MC_PATH_REF is a vendor specific data type - PathSelect makes data available. This can include: - Starting point of a download of a path profile, as represented in PathData and referenced by PathDescription - Start to generate a path profile			



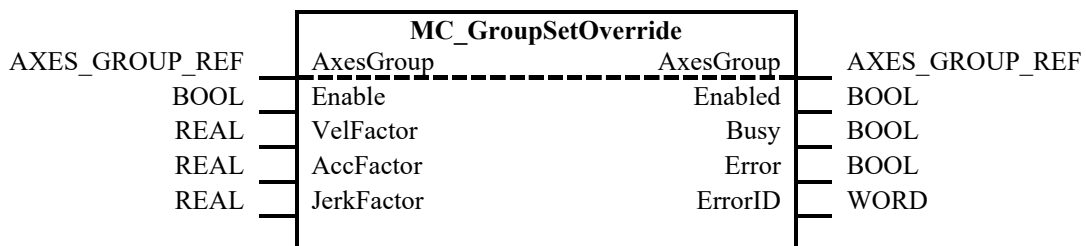
9.42 MC_MovePath

FB-Name		MC_MovePath	
This function block commands an AxesGroup to move according to the path specified in the PathData.			
VAR IN_OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to the AxesGroup
B	PathData	MC_PATH_DATA_REF	Reference to the path data, which can be prepared by MC_PathSelect
VAR INPUT			
B	Execute	BOOL	Commands a new movement at the rising edge
E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system. Interpretation is vendor specific.
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB relative to the previous block. Refer to 11.2 Buffer Modes
E	TransitionMode	MC_TRANSITION_MODE	For a more detailed description, see 11.3 TransitionMode
E	TransitionParameter	MC_TRANSITION_PARAMETER	Additional parameter for the transition mode
VAR OUTPUT			
B	Done	BOOL	Commanded end positions reached for all axes
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axes group
E	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Note: -			

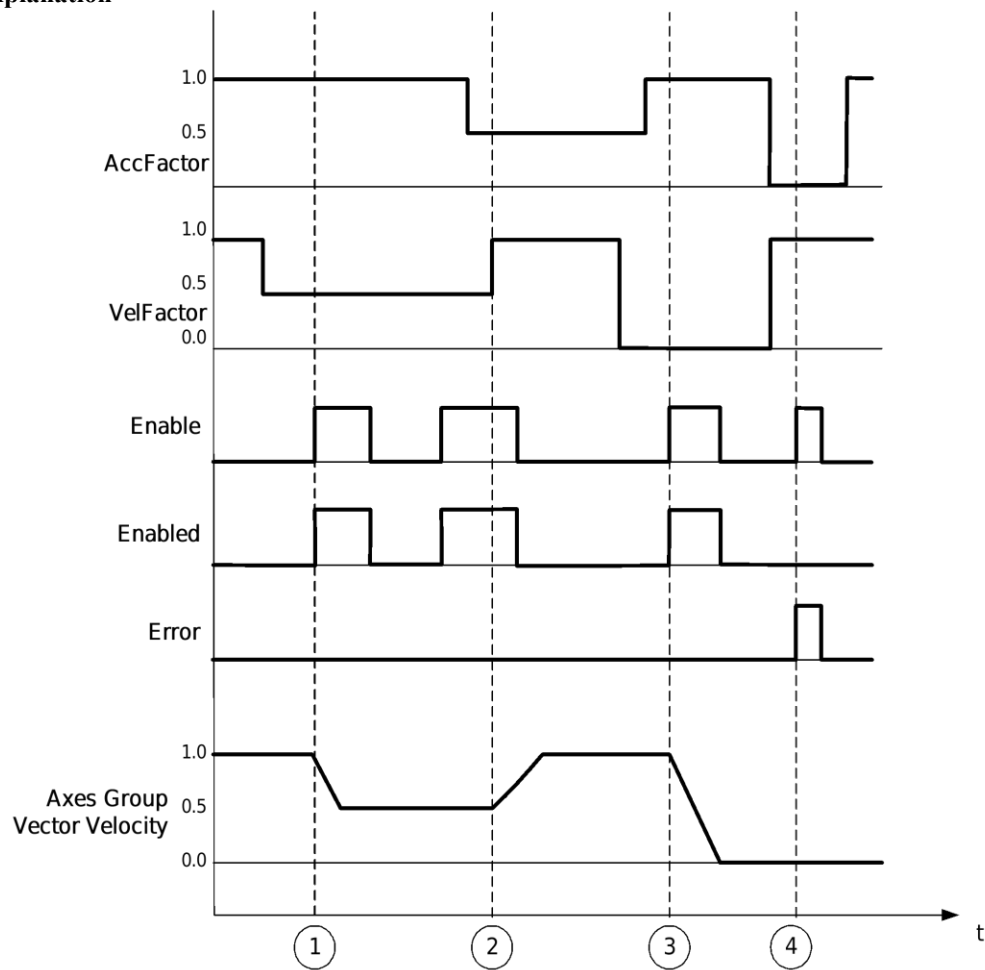


9.43 MC_GroupSetOverride

FB-Name		MC_GroupSetOverride	
This function block sets the values of override for the coordinated motion of several axes, and all functions that are working on that axes group. The override parameters act as a factor that is multiplied to the commanded vector velocity, acceleration, deceleration and jerk of all axes group FBs. Not applicable to master-slave group synchronized motion commands.			
VAR IN OUT			
B	AxesGroup	AXES GROUP REF	Reference to the group of axes
VAR INPUT			
B	Enable	BOOL	If SET, it writes the value of the override factor continuously. If RESET it should keep the last value.
B	VelFactor	REAL	Override factor for the vector velocity
E	AccFactor	REAL	Override factor for the vector acceleration/deceleration
E	JerkFactor	REAL	Override factor for the vector jerk
VAR OUTPUT			
B	Enabled	BOOL	Signals that the override factor(s) is (are) set successfully
E	Busy	BOOL	Is SET when the FB is active (not in idle mode).
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Notes:			
1. The Input AccFactor acts on positive and negative acceleration (deceleration).			
2. This FB sets the factor. The override factor is valid until a new override is set.			
3. The default values of the override factors are 1.0.			
4. The value of the overrides can be between 0.0 and 1.0. The behavior of values > 1.0 is vendor specific. Values < 0.0 are not allowed. The value 0.0 is not allowed for AccFactor and JerkFactor. and generates an error			
5. The value 0.0 set to the VelFactor stops the axes without bringing it to the state GroupStandby.			
6. Override does not act on slave axes groups. (Axes groups in the state Group Synchronized motion).			
7. The FB does not influence the state diagram of the axes group.			
8. VelFactor can be changed at any time and acts directly on the ongoing motion.			
9. Reducing the AccFactor and/or JerkFactor can lead to a position overshoot – a possible cause of damage.			



Graphical Explanation

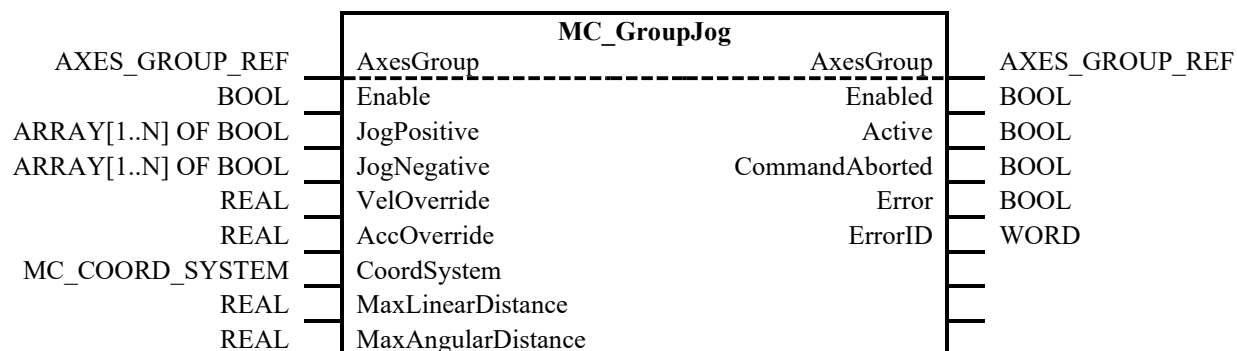


- ① Axes Group Vector Velocity changes to 50% with 100% of deceleration
- ② Axes Group Vector Velocity back to 100% with 50% acceleration
- ③ Axes Group Vector Velocity moves to 0% with 100% deceleration
- ④ No Change, because AccFactor 0.0 is not allowed; Error is set

Figure 29: Graphical explanation of MC_GroupSetOverride

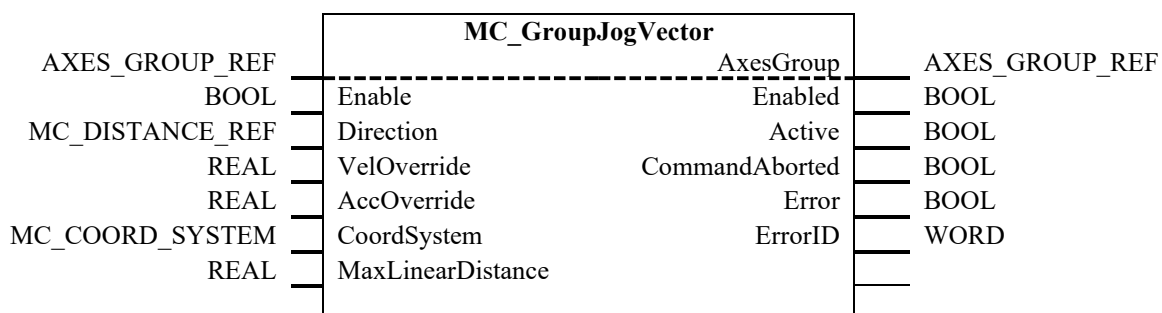
9.44 MC_GroupJog

FB-Name		MC_GroupJog	
This function block allows you to jog all axes of an Axes Group or all cartesian coordinates, depending on the selected coordinate system. The inputs are designed to be mapped on physical buttons like a teach pendant. Note that the dynamics for jogging (maximum velocity, acceleration, jerk) can be set using MC_GroupWriteJoggingDynamics. If no jogging dynamics are configured for the axes group, then the FB returns an error on enable. Whether the override set using MC_GroupSetOverride also affects jogging is vendor specific.			
VAR IN OUT			
B	AxesGroup	AXES GROUP REF	Reference to a group of axes
VAR INPUT			
B	Enable	BOOL	FB is enabled
B	JogPositive	ARRAY[1..N] OF BOOL	TRUE: Axis is moved in positive direction. No motion is executed if JogNegative is TRUE at the same time.
B	JogNegative	ARRAY[1..N] OF BOOL	TRUE: Axis is moved in negative direction. No motion is executed if JogPositive is TRUE at the same time
E	VelOverride	REAL	Override factor for the maximum velocity. This override applies to the maximum path velocity as well as to the maximum axis velocities.
E	AccOverride	REAL	Override factor for the maximum acceleration. This override applies to the maximum path acceleration as well as to the maximum axis accelerations.
B	CoordSystem	MC_COORD_ SYSTEM	Reference to the applicable coordinate system.
E	MaxLinearDistance	REAL	The maximum linear distance a coordinate is jogged while no new rising edge is detected on the Forward/Backward inputs. The distance must be positive, otherwise an error is reported. If the value is 0 (default value), then the linear jogging distance is not limited.
E	MaxAngularDistance	REAL	The maximum angular distance a coordinate is jogged while no new rising edge is detected on the Forward/Backward inputs. The distance must be positive, otherwise an error is reported. If the value is 0 (default value), then the angular jogging distance is not limited.
VAR OUTPUT			
B	Enabled	BOOL	Reflect the input Enable status without error nor command aborted
B	Active	BOOL	Indicates that the FB has control over at least one of the axes
B	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
B	ErrorID	WORD	Error identification
This is combined with a MC_GroupWriteJoggingDynamics (and Read) Also valid for Inching. Notes: when the MC_GroupJog is enabled but motors are not moving, the PLCopen state is in “GroupStandby”.			



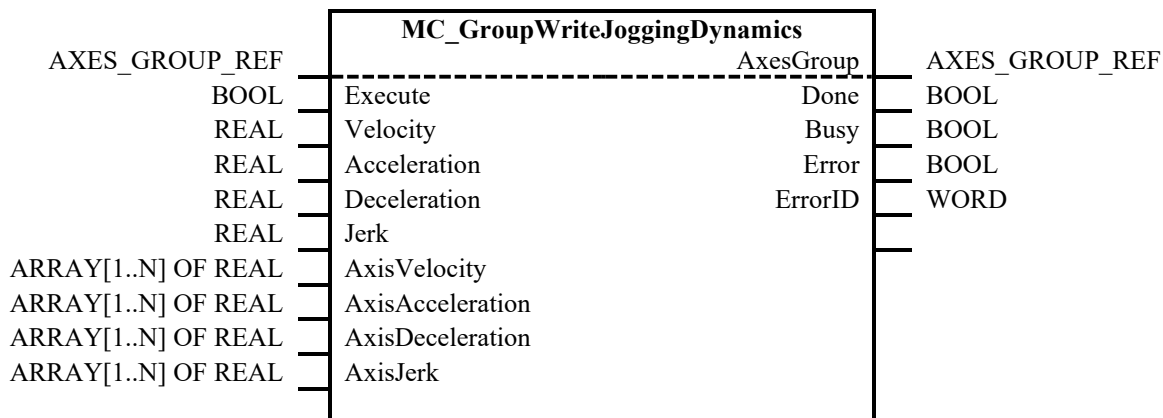
9.45 MC_GroupJogVector

FB-Name		MC_GroupJogVector	
This function block allows to jog an Axes Group in the direction of a programmed vector. The direction input contains an orientational part and a rotational part also. Note that the dynamics for jogging (maximum velocity, acceleration, jerk) can be set using MC_GroupWriteJoggingDynamics. If no jogging dynamics are configured for the axes group, then the FB returns an error on enable, Whether the override set using MC_GroupSetOverride also affects jogging is vendor specific.			
VAR_IN_OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
VAR_INPUT			
B	Enable	BOOL	Enables jogging
B	Direction	MC_DISTANCE_REF	Vector direction, that can be updated continuously. The cartesian part of this input defines the orientation of the vector in which the movement shall be done. The length of the vector in the range from 0 to 1 defines the velocity override. The length of the angular part in the range from 0 to 1 defines the angular velocity override. In both cases, 1 means 100% of the maximum jogging velocity, 0.5 means 50% of the maximum jogging velocity. If the length is greater than 1, the override is capped at 100%.
E	VelOverride	REAL	Override factor for the maximum velocity. This override applies to the maximum path velocity as well as to the maximum axis velocities.
E	AccOverride	REAL	Override factor of the maximum acceleration. This override applies to the maximum path acceleration as well as to the maximum axis accelerations.
B	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system. ACS is not applicable
VAR_OUTPUT			
B	Enabled	BOOL	Reflect the input Enable status without error nor command aborted
B	Active	BOOL	Indicates that the FB has control over at least one of the axes
B	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
B	ErrorID	WORD	Error identification
Note: -			



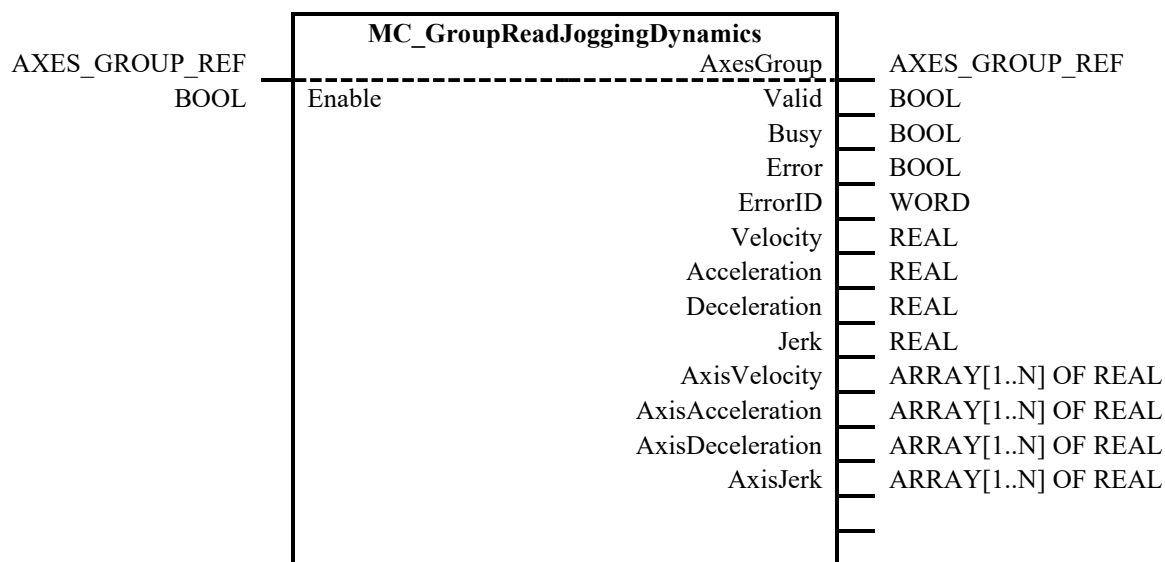
9.46 MC_GroupWriteJoggingDynamics

FB-Name		MC_GroupWriteJoggingDynamics	
This function block writes the dynamics for jogging. (See MC_GroupJog and MC_GroupJogVector). The maximum value for velocity, acceleration, deceleration, and jerk on the path and for each axis can be set. This is an administrative FB that does not cause any motion and does not influence the active motion. The updated jogging dynamics only affects jogging commands that are executed after this FB is Done.			
VAR_IN_OUT			
B	AxesGroup	AXES GROUP REF	
VAR_INPUT			
B	Execute	BOOL	Writes the path reference dynamics at rising edge
B	Velocity	REAL	Maximum path velocity (tangent) in u/s at 100% > 0: change value according to input value
E	Acceleration	REAL	Maximum path acceleration in [u/s²] at 100% > 0: change value according to input value
E	Deceleration	REAL	Maximum path deceleration in [u/s²] at 100% > 0: change value according to input value.
E	Jerk	REAL	Maximum path jerk in [u/s³] at 100% > 0: change value according to input value.
B	AxisVelocity	ARRAY[1..N] OF REAL	Maximum axis velocity for each axis of the group in axis specific units per second. [u/s] > 0: change value according to input value.
E	AxisAcceleration	ARRAY[1..N] OF REAL	Maximum axis acceleration for each axis of the group in axis specific units per second squared. > 0: change value according to input value.
E	AxisDeceleration	ARRAY[1..N] OF REAL	Maximum axis deceleration for each axis of the group in axis specific units per second squared. > 0: change value according to input value.
E	AxisJerk	ARRAY[1..N] OF REAL	Maximum axis jerk for each axis of the group in axis specific units per second cubed. > 0: change value according to input value.
VAR_OUTPUT			
B	Done	BOOL	The path reference dynamics has been set
E	Busy	BOOL	The FB is not finished
B	Error	BOOL	Signals that an error has occurred
E	ErrorID	WORD	Error identification
Note: In case a value should not be changed, a global constant like mcIgnore can be used.			



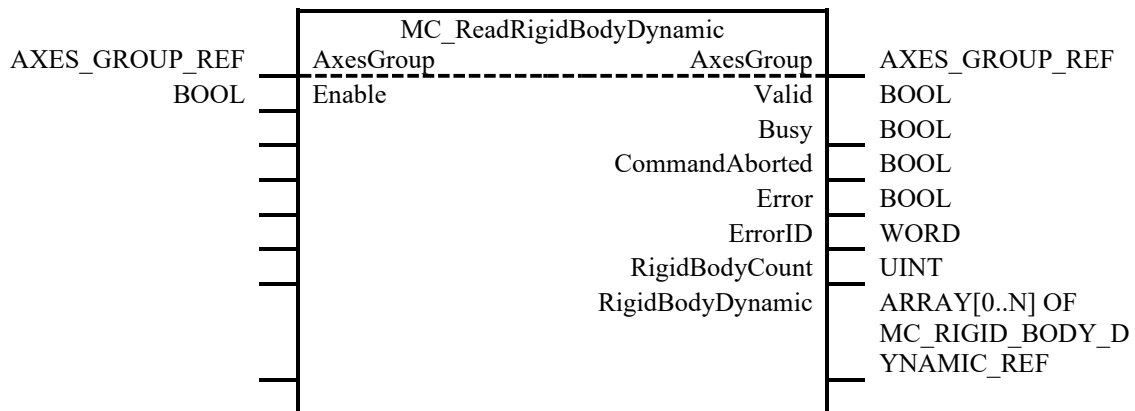
9.47 MC_GroupReadJoggingDynamics

FB-Name		MC_GroupReadJoggingDynamics		
This function block reads the values of the jogging dynamics. (See MC_GroupJog and MC_GroupJogVector.) It returns to the maximum velocity, acceleration, deceleration, and jerk. This is an administrative FB that does not cause any motion and does not influence the active motion.				
VAR IN_OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL	Reads the jogging dynamics	
VAR OUTPUT				
B	Valid	BOOL	True if valid outputs are available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
B	Velocity	REAL	Maximum path velocity (tangent) in u/s	
E	Acceleration	REAL	Maximum path acceleration in [u/s²]	
E	Deceleration	REAL	Maximum path deceleration in [u/s²]	
E	Jerk	REAL	Maximum path jerk in [u/s³]	
B	AxisVelocity	ARRAY[1..N] OF REAL	Maximum axis velocity for each axis of the group in axis specific units per second.	
E	AxisAcceleration	ARRAY[1..N] OF REAL	Maximum axis acceleration for each axis of the group in axis specific units per second squared.	
E	AxisDeceleration	ARRAY[1..N] OF REAL	Maximum axis deceleration for each axis of the group in axis specific units per second squared.	
E	AxisJerk	ARRAY[1..N] OF REAL	Maximum axis jerk for each axis of the group in axis specific units per second cubed.	
Comments: -				



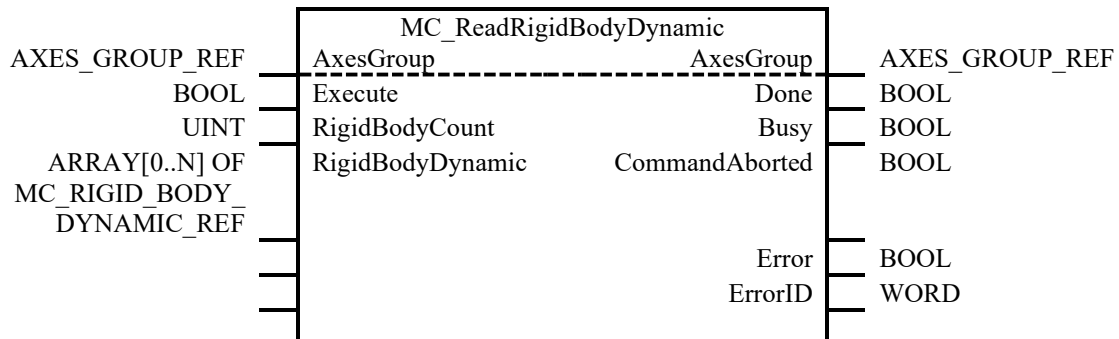
9.48 MC_ReadRigidBodyDynamic

FB-Name		MC_ReadRigidBodyDynamic		
This function block reads dynamic data of all rigid bodies of the axes group. Kinematic transformations can be read out via MC_ReadDHParameter.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL		Reads rigid body dynamics continuously if TRUE
VAR OUTPUT				
B	Valid	BOOL		A valid output is available at the FB
E	Busy	BOOL		The FB is not finished
E	CommandAborted	BOOL		Command is aborted by another command
B	Error	BOOL		Signals that an error has occurred
E	ErrorID	WORD		Error identification
B	RigidBodyCount	UINT		Number of rigid bodies that form the kinematic
B	RigidBodyDynamic	ARRAY[0..N] OF MC_RIGID_BODY_DYNAMIC_REF		Data for the rigid bodies. Element 0 contains the data for the base of the robot/axes group
Note: -				



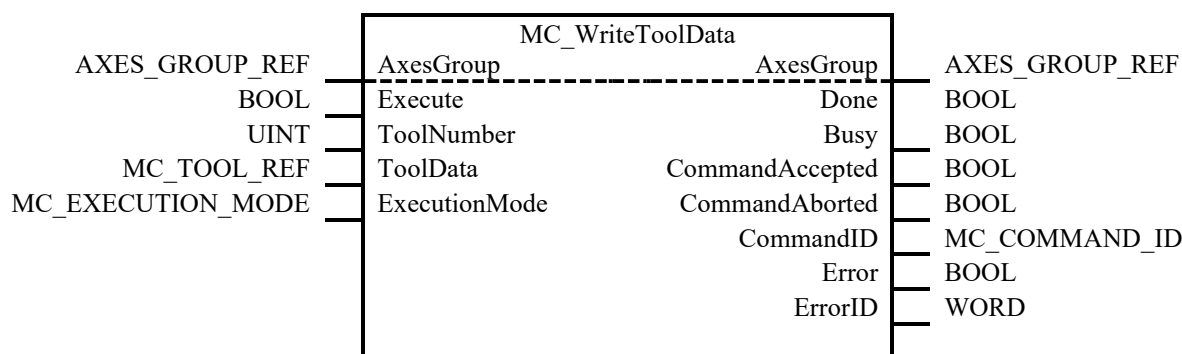
9.49 MC_WriteRigidBodyDynamic

FB-Name		MC_WriteRigidBodyDynamic		
This function block writes dynamic data of all rigid bodies of the axes group. Kinematic transformations can be written via MC_WriteDHPParameter.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Execute	BOOL	Start reading of the rigid body data at rising edge	
B	RigidBodyCount	UINT	Number of rigid bodies that form the kinematic	
B	RigidBodyDynamic	ARRAY[0..N] OF MC_RIGID_BODY_DYNAMIC_REF		Data for the rigid bodies. Element 0 contains the data for the base of the robot/axes group
VAR OUTPUT				
B	Done	BOOL	The rigid body dynamic definition has been read	
E	Busy	BOOL	The FB is not finished	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
Note: -				



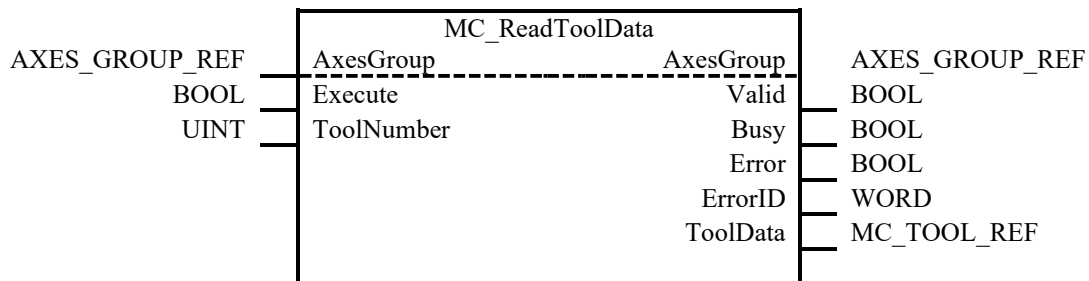
9.50 MC_WriteToolData

FB-Name		MC_WriteToolData		
This function block writes the parameters of a tool. "ToolData" contains the origin and orientation of the TCS relative to the flange. This is an administrative FB that does not cause motion and does not influence active motion. Whether it is allowed to change the definition of the active tool or tools that are in use by previously commanded movements that are not finished or aborted is vendor specific. If the tool is in use, the tool displacement is added to the kinematic transformation from the flange to the tool (FCS->TCS).				
VAR IN OUT				
B	AxesGroup	AXES GROUP REF		
VAR INPUT				
B	Execute	BOOL	Write the tool data at rising edge	
B	ToolNumber	UINT	Tool number 0 is reserved for the flange, which cannot be changed.	
B	ToolData	MC_TOOL_REF	The offset and orientation of the tool with respect to the flange.	
E	ExecutionMode	MC_EXECUTION_MODE	Describes when the command is executed and the new transformation becomes valid. (See 2.2.1 Input Execution Mode)	
VAR OUTPUT				
B	Done	BOOL	The tool definition has been set	
E	Busy	BOOL	The FB is not finished	
E	CommandAccepted	BOOL	Command is transferred to the motion engine and confirmed	
E	CommandAborted	BOOL	Command is aborted by another command	
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
Note: -				



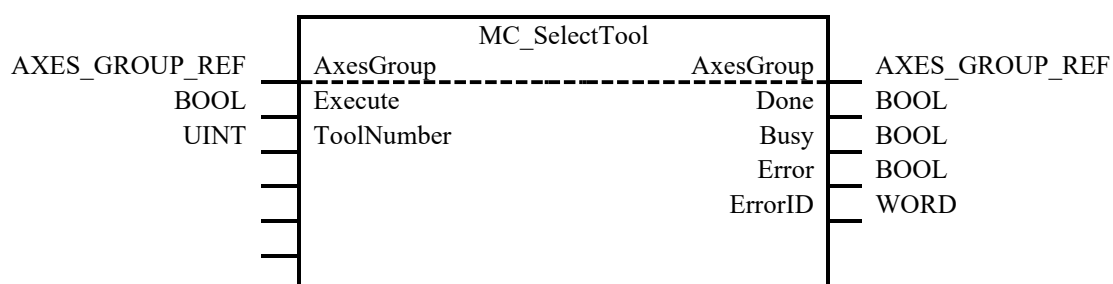
9.51 MC_ReadToolData

FB-Name		MC_ReadToolData		
This function block reads the current tool definition for the specified tool number. "ToolData" contains the origin and orientation of the coordinate system relative to the flange. This is an administrative FB that does not cause motion and does not influence active motion.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	
VAR INPUT				
	B	Enable	BOOL	Starts reading the tool at rising edge
	B	ToolNumber	UINT	The tool number 0 is reserved for the flange.
VAR OUTPUT				
	B	Valid	BOOL	The tool definition has been read
	E	Busy	BOOL	The FB is not finished
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	B	ToolData	MC_TOOL_REF	The offset and orientation of the tool with respect to the flange.
Note: queued changes to the tool data are not reflected until MC_WriteToolData is Done				



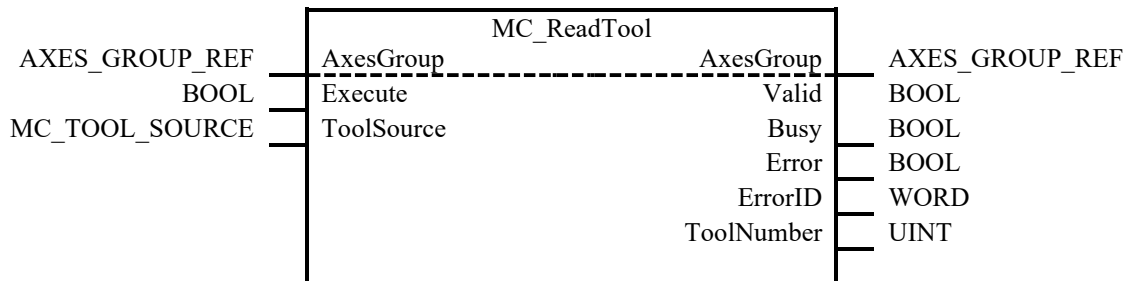
9.52 MC_SelectTool

FB-Name		MC_SelectTool		
This function block selects the tool for the next commanded movement until another tool is selected.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR_INPUT				
B	Execute	BOOL	Sets the tool at rising edge	
B	ToolNumber	UINT	The tool number 0 is reserved for the flange.	
VAR_OUTPUT				
B	Done	BOOL	The tool has been selected	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
Note: If the AxesGroup is in GroupStandby or GroupDisable, the active tool is the last selected tool.				



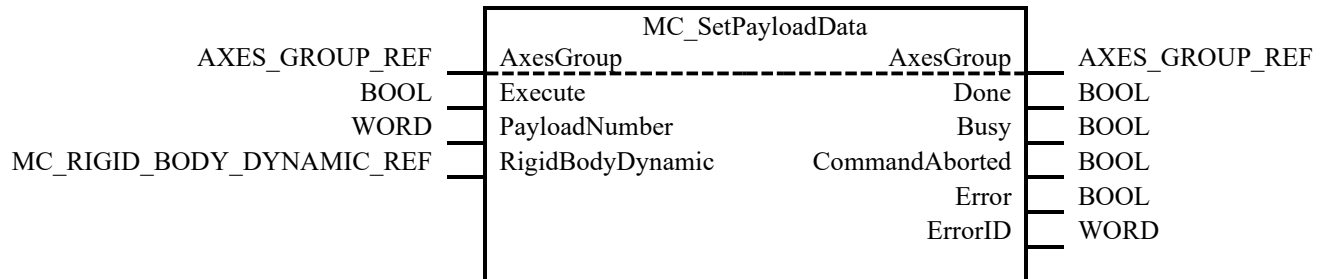
9.53 MC_ReadTool

FB-Name		MC_ReadTool		
This function block reads the tool number. This is an administrative FB that does not cause motion and does not influence active motion.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	
VAR INPUT				
	B	Enable	BOOL	Starts reading the tool at rising edge
	E	ToolSource	MC_TOOL_SOURCE	The tool number that should be returned: mcActive – the active tool is defined as the tool, which is used in the kinematic transformation of the axes group mcSelected - the tool that has been selected with the latest MC_SelectTool.
VAR OUTPUT				
	B	Valid	BOOL	The tool number has been read
	E	Busy	BOOL	The FB is not finished
	B	Error	BOOL	Signals that an error has occurred
	E	ErrorID	WORD	Error identification
	B	ToolNumber	UINT	Returned tool number
Note: -				



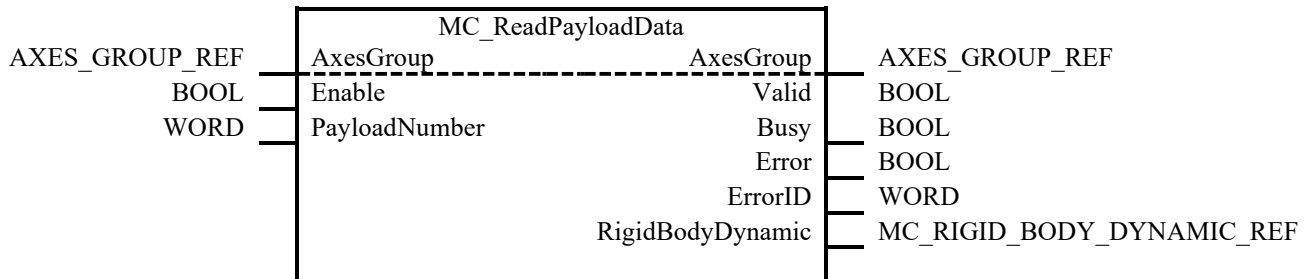
9.54 MC_WritePayloadData

FB-Name		MC_WritePayloadData		
This function block writes the parameters of a payload.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Execute	BOOL	Starts reading the tool at rising edge	
B	PayloadNumber	WORD	The load number. 0 is reserved for zero load.	
B	RigidBodyDynamic	MC_RIGID_BODY_DYNAMIC_REF	The payload relative to the TCP coordinate system.	
VAR OUTPUT				
B	Done	BOOL	The payload definition has been set	
E	Busy	BOOL	The FB is not finished	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
Note: -				



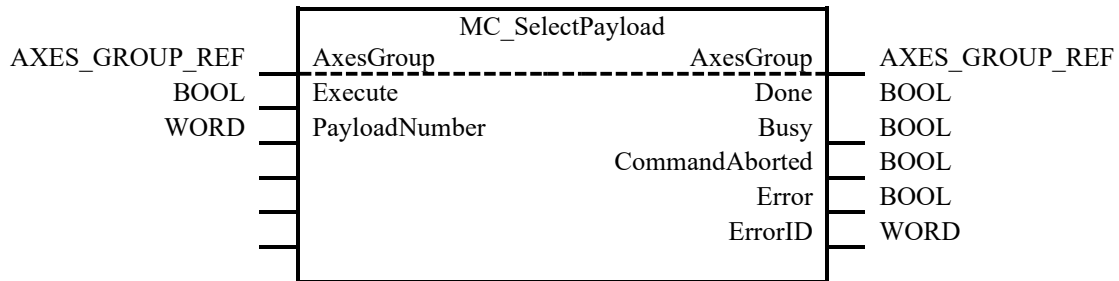
9.55 MC_ReadPayloadData

FB-Name		MC_ReadPayloadData		
This function block reads a payload that was set by MC_SetPayloadData.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR_INPUT				
B	Enable	BOOL	Read the payload definition while enabled	
B	PayloadNumber	WORD	The payload number.	
VAR_OUTPUT				
B	Valid	BOOL	True if valid output is available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
B	RigidBodyDynamic	MC_RIGID_BODY_DYNAMIC_REF	The payload relative to the TCP coordinate system	
Note: -				



9.56 MC_SelectPayload

FB-Name		MC_SelectPayload		
This function block activates a payload. The active payload only affects movements that are commanded after this FB is Done.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Execute	BOOL	Starts setting the payload at rising edge	
B	PayloadNumber	WORD	The payload number. 0 is reserved for zero load.	
VAR OUTPUT				
B	Done	BOOL	The tool definition has been read	
E	Busy	BOOL	The FB is not finished	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
Note: -				

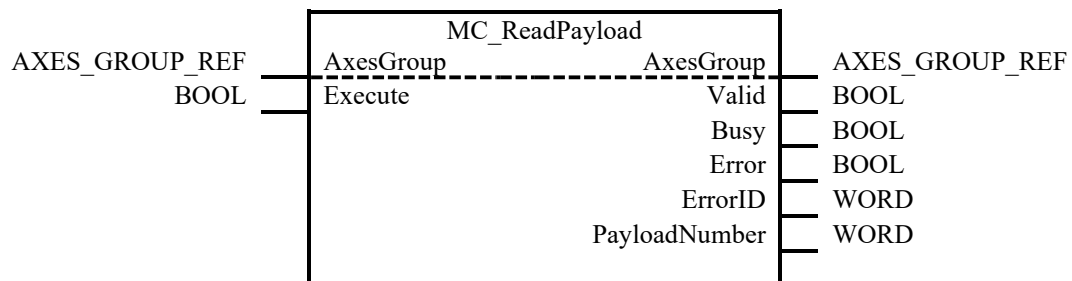


Note: how the payload is selected is vendor specific. Some use cases are:

- Associate the load with a tool
- Use FBs to select the payload independently from the tool
- Others might require FBs to add or remove payloads for a cumulative payload

9.57 MC_ReadPayload

FB-Name		MC_ReadPayload		
This function block reads the active payload that was set with MC_SetActivePayload.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR_INPUT				
B	Enable	BOOL	Starts reading the tool at rising edge	
E	PayloadSource	MC_TOOL_SOURCE	The payload number that should be returned: mcActive – the active tool is defined as the tool, which is used in the kinematic transformation of the axes group mcSelected - the tool that has been selected with the latest MC_SelectTool.	
VAR_OUTPUT				
B	Valid	BOOL	The tool definition has been read	
E	Busy	BOOL	The FB is not finished	
E	CommandAborted	BOOL	Command is aborted by another command	
B	Error	BOOL	Signals that an error has occurred	
E	ErrorID	WORD	Error identification	
B	PayloadNumber	WORD	The payload number of the active or selected payload.	



9.58 MC_TransformPosition

FB-Name		MC TransformPosition		
This function block transforms the input position from the specified input coordinate system to the specified output coordinate system.				
VAR IN OUT				
B	AxesGroup	AXES_GROUP_REF		
VAR INPUT				
B	Enable	BOOL	Calculates values ongoing	
E	InputPosition	MC_POS_REF	Array of position values that represents position relative to InputCoordSystem	
B	InputCoordSystem	MC COORD SYSTEM	Reference to the applicable coordinate system.	
B	OutputCoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system.	
VAR OUTPUT				
B	Valid	BOOL	True if valid output transformation is available	
E	Busy	BOOL	The FB is not finished	
B	Error	BOOL	Signals that error has occurred within unction block	
E	ErrorID	WORD	Error identification	
E	CommandID	MC_COMMAND_ID	See 11.4 MC_COMMAND_ID	
B	OutputPosition	MC_POS_REF	Array of position values that represents position relative to OutputCoordSystem	
E	SingularPosition	BOOL	Indicating a singularity (like cartesian position interlock in case the cartesian coordinate system as input & output; or kinematic singularity if one of the coordinate system is ACS. Note: there is no check if ACS is not involved.)	

Notes:

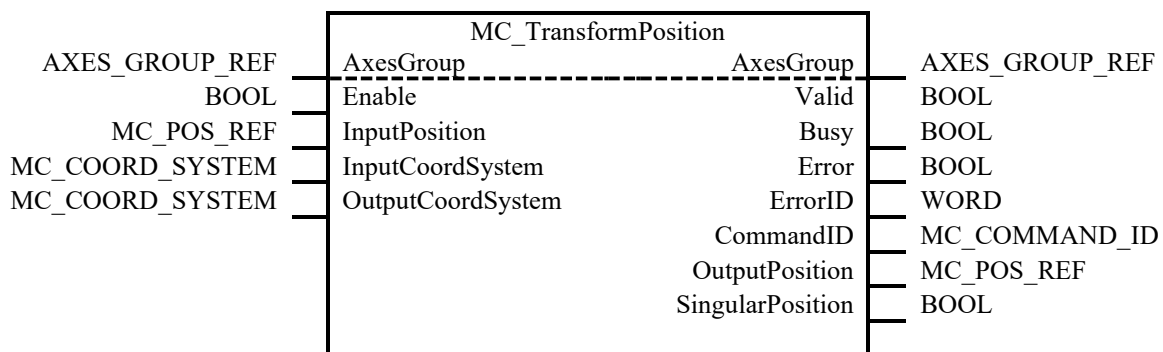
This FB will need additional input parameters to fully define a Cartesian position.

If converting between Cartesian (WCS, MCS or PCS) to ACS, it requires data for group configuration (shoulder, wrists, motor turns, external axes, etc)

If the group has external axes, they could be included in the indices after the joint or Cartesian values.

Example: CoordSystem = MCS, PCS, or WCS [20 20 20 11 22 33 **44 55 66**]. This would be [X Y Z Rx Ry Rz E1 E2 E3]

Example: CoordSystem = ACS (group with 6 joints) [10 20 30 40 50 60 70 80 90]. This would be [J1 J2 J3 J4 J5 J6 **E1 E2 E3**]



10 Axes group synchronized motion and tracking

The function blocks as defined in this chapter deal with a master/ slave relationship between a single or group of axes and a single or group of axes for coordination purposes.

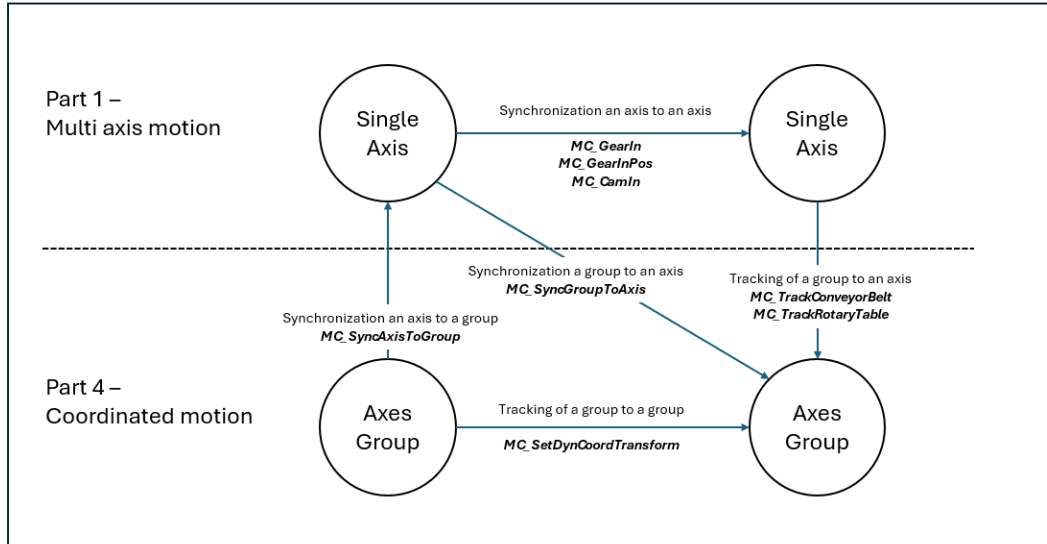


Figure 30: Graphical explanation of coordination

There are two kinds of coordinated motion that must be distinguished from a programming point of view and in the realization of motion control itself. These two modes are identified here through their names:

- **Synchronization**

Synchronization describes a coupling of the movement profiles between axis and/or axes groups.

This means that in case of an axes group as coupled slave, a FB commanding synchronization has no effect on the geometrical path of the group but defines the velocity, acceleration etc. along a predefined path.

If an axes group is part of the synchronization with a single axis, the path velocity of the group and the velocity of the single axis are coupled via a gear factor.

A synchronized slave is typically only controlled by the synchronization function block

- **Tracking**

Tracking describes the geometrical coupling of an axes group to another axis or axes group. With tracking, the path in space of the slave axes group is influenced by the movement of the master.

In most applications, a slave axes group is performing combined (superimposed) movements that result from the combination of move function blocks (e.g. MC_MoveLinearAbsolute) and tracking.

10.1 Synchronization

The relationship between single axis commands and synchronized motion is shown here.
There are 6 possibilities of associations:

1. Single-axis synchronized to an Axes-Group, Linear Synchronization
2. Single-axis synchronized to an Axes-Group, Non-Linear Synchronization (using MC_CamIn)
3. Axes-Group axis synchronized to a Single-axis, Linear Synchronization
4. Axes-Group axis synchronized to a Single-axis, Non-Linear Synchronization (using MC_CamIn)
5. Axes-Group axis synchronized to an Axes-Group, Linear Synchronization
6. Axes-Group axis synchronized to an Axes-Group, Non-Linear Synchronization (using MC_CamIn)

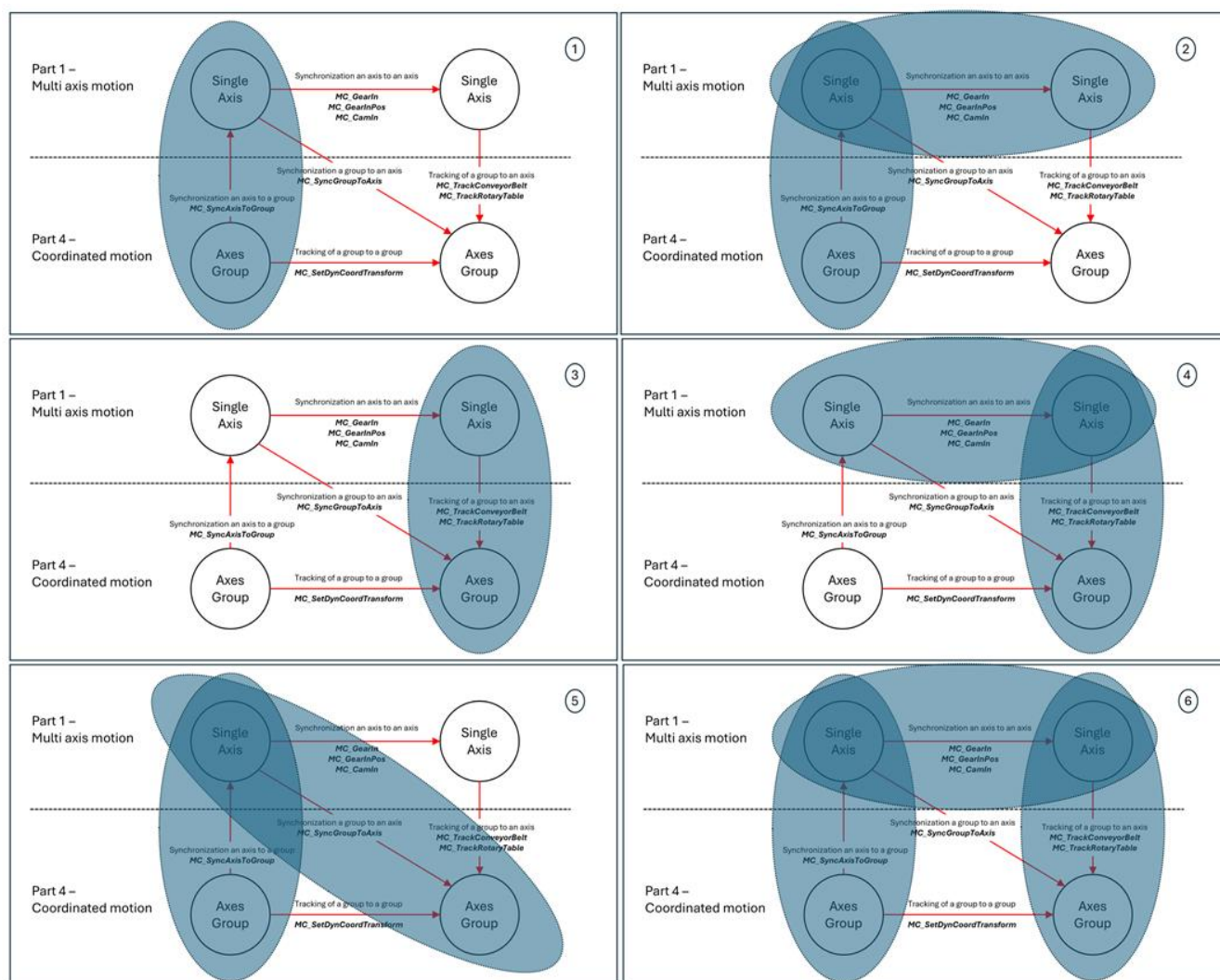


Figure 31: Graphical explanation of Synchronization

10.1.1 Synchronization of single axis to an axes group

This is an example of a single axis (as slave) synchronized to an axes group (master). The master follows its path and the slave is linked to the position, velocity, acceleration, or any other magnitude of the master. An example is glue dispensing, where the amount of glue to be dispensed is coupled to the velocity of the TCP of the robot.

The single axis slave motor movement of the glue dispenser is coupled to the trajectory of the TCP of the group over the surface of the object via MC_SyncAxisToGroup. Alternatively, if the position information is not critical, one can use MC_GroupReadActualVelocity, possibly combined with a gearing factor, and thus providing the input to the motor of the glue dispenser.

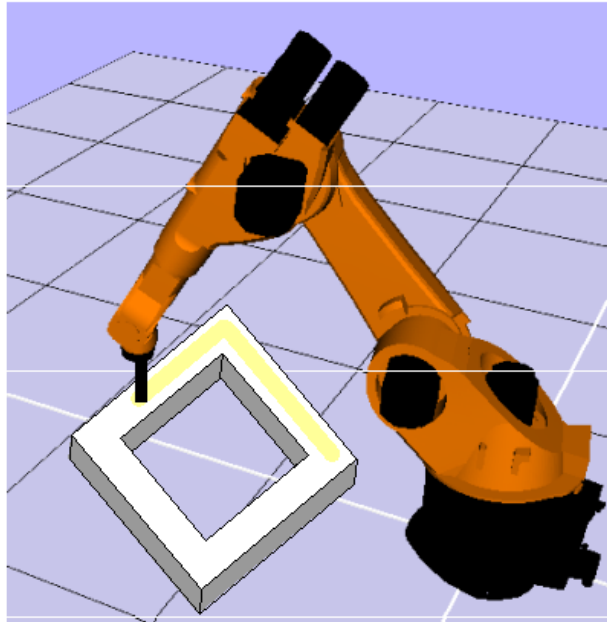


Figure 32: Synchronization of single axis to an axes group

10.1.2 Synchronization of an axes group to a single axis

This mode combines an axes group (as slave) with an axis as master in order that the slave executes its path with synchronization to the progress of the master, meaning linked to a 1-dimensional source for synchronization. Examples here include press synchronization. (Note: in case the slave is an axes group, a transformation to a virtual axis can be applicable to generate the 1-dimensional synchronization data. In case both the master and the slave are axes groups, a virtual master axis on the slave side is applicable (linked via cam profiles to the different axes) to use the 1-dimensional synchronization data of the master side).

As an example of synchronization between a master single axis and a group can be the robot which places material in a press machine: the robot has to synchronize to the opening phase of the press. A second robot also synchronizes to take the material out.

The master is the (single or virtual) axis controlling the press. The axes group for both robot 1 and robot 2 has to follow the press in a certain area of the master movement: opening for robot 2 to take the product out and closing for robot 1 to add the new product.

For this synchronization the FB MC_SyncGroupToAxis is defined.

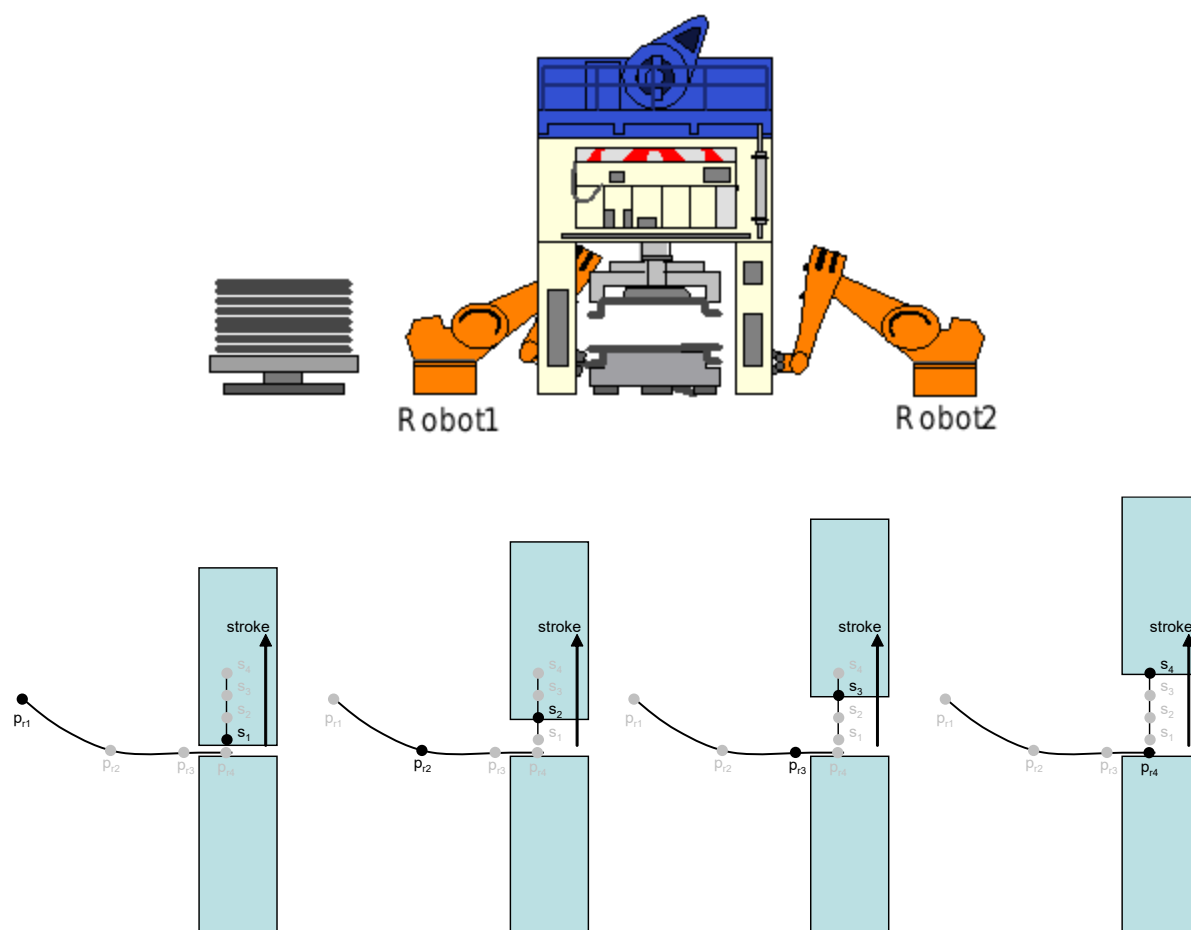


Figure 33: Synchronization of an axes group to a single axis

10.2 Tracking

Tracking is characterized by an axes group (A) that follows with its movement the movement of a single axis or another axes group (B). A is performing a movement/task relative to the movement of B. The tracking data is a multidimensional source incl. position and orientation. Solutions can include a moving coordinate system or a multi-dimensional gear functionality.

For tracking the following function blocks are defined here: MC_SetDynCoordTransform as a general one, while MC_TrackConveyorBelt and MC_TrackRotaryTable are for specific applications. (Note: The considerations on the limitations of the dynamics or mechanics are implementation specific.)

Tracking can be seen as a superposition of two movements, although these movements are independent. One, which is the movement of the product through single axis or axes group (B), resulting in a moving PCS of AxesGroup (A) and the second one, which describes the path of the TCP that would be executed if the product is standing still (Positions have to be defined in PCS_A). The Position of the PCS_A and therefore also the movement of the PCS_A relative to MCS_A (or WCS respectively) is described by the coordinate transformation chain WCS to MCS_B to TCS_B to PCS_A. For conveyor or rotary table tracking, the MCS_B is called conveyor origin or rotary table origin respectively.

The basic example for the tracking of an axes group and a single axis is conveyor tracking, where the robot picks or places parts on the moving conveyor or puts some cream on a cake moving on the belt. An example for the tracking of another axes group is having two robots, where robot B is holding a work piece, and robot A is performing some welding on the part at the same time B is moving the work piece (see picture).

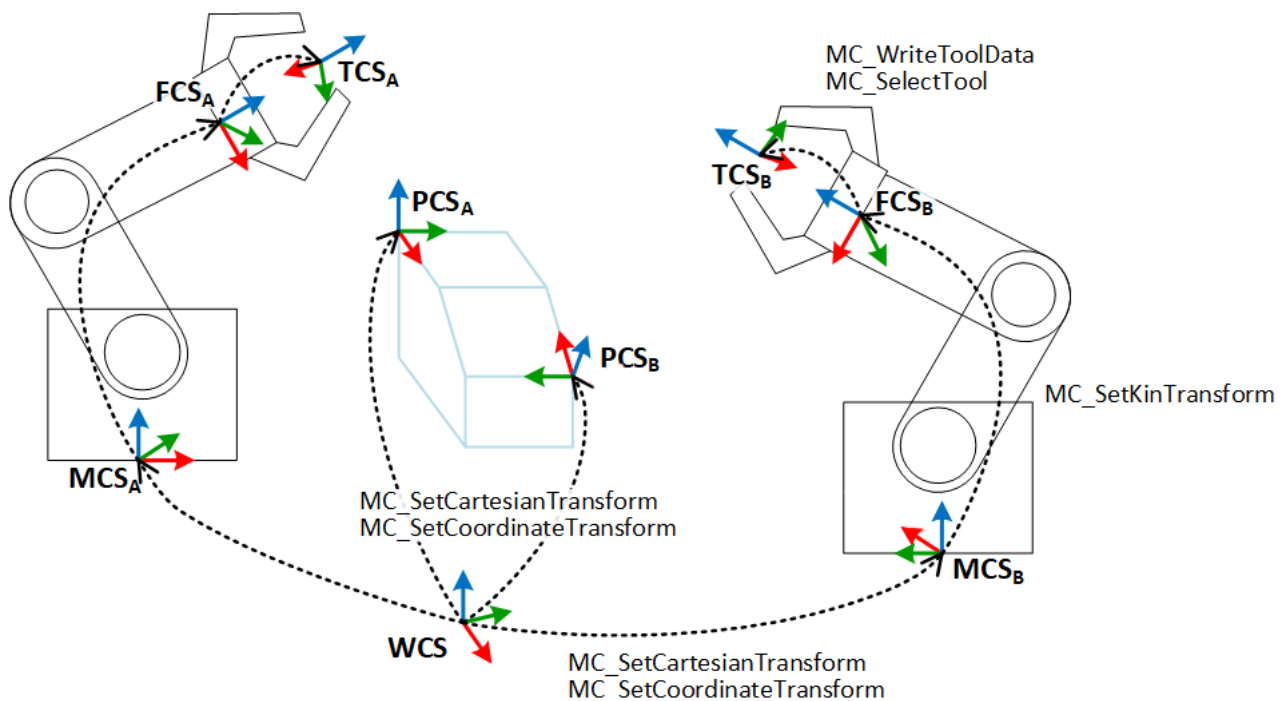
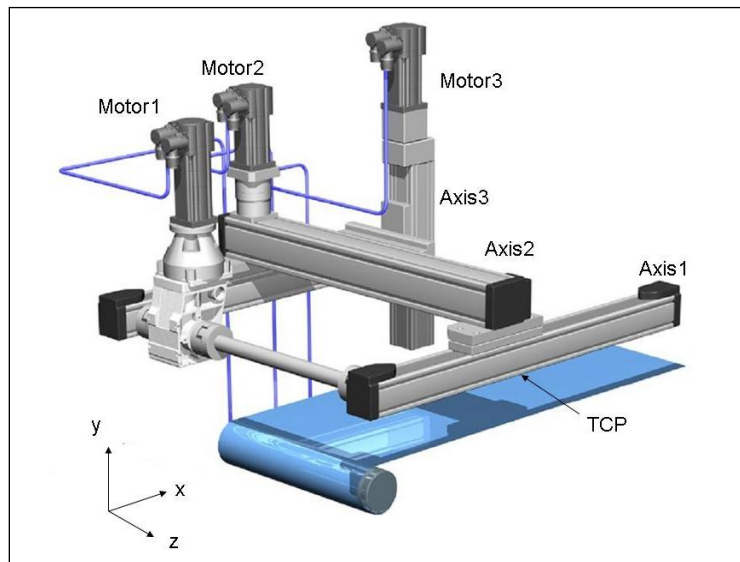


Figure 34: Tracking and Synchronization

Generally, there is no difference if A is tracking a single axis or an axes group, when thinking of a single axis as an axes group having only one axis but also a kinematic (even if it is very simple). Then concepts of motion planning as well as programming are the same.

A second tracking example deals with synchronization of a group and a transportation belt. The group synchronizes to the belt, which is the master.

We have a (simple) Cartesian robot, consisting of 3 motors moving 3 axes. Application examples are to pick something from the belt (with a correction in the Z-position), or to put some cream on a cookie that is lying on the belt.



All tracking FBs are in the group Synchronized/Tracking (see Ch. 1.3 – Overview of defined Function Blocks). This is because there is motion possible without any coordinated motion FB. For example, if a MoveLinearAbsolute in PCS is Done and no following movement is commanded. In this situation the Axesgroup is still moving synchronized to the PCS without another command (being synchronized to an external master axis or master axes group similar to the single axis MC_GearIn functionality in coordinated motion).

The usage of MC_SetDynCoordTransform, MC_TrackConveyorBelt and MC_TrackRotaryTable in combination with coordinated motion FBs lead to different situations. See **Fout! Verwijzingsbron niet gevonden.** for timing diagram example.

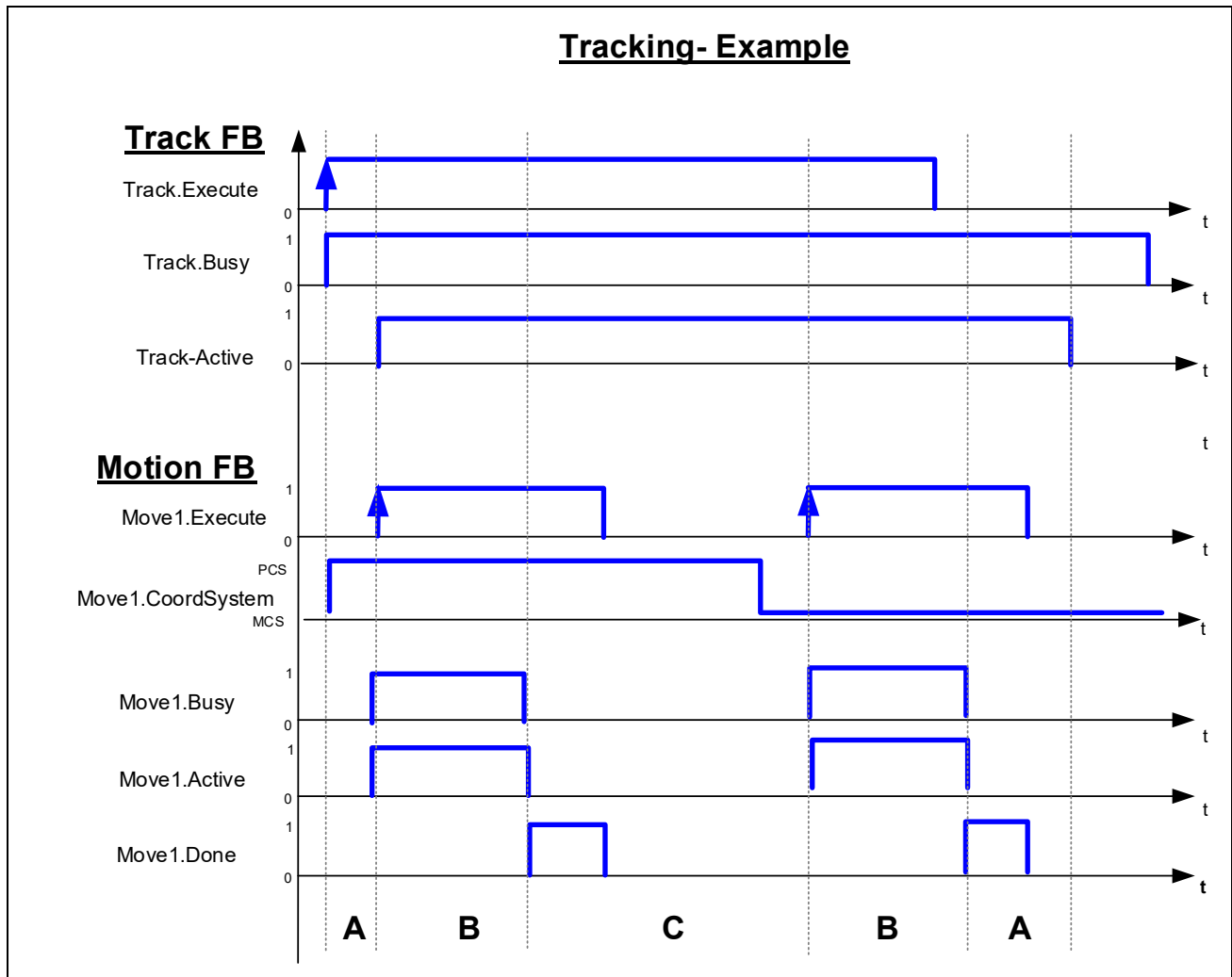


Figure 35: Example Timing Diagram Tracking

1st situation, period A:

Track FB is executed, and no Motion FB is active on the tracked PCS

=> The Track FB is Busy, but not Active.

During this period, the Track FB is only responsible for administrative calculations, e.g. updating the PCS location

2nd situation, period B:

Tracking and motion in tracked PCS are active.

=> The Track FB is Busy and Active

=> A Motion FB is also Busy and Active, calculating/interpolating the path.

Both FBs are responsible for the set value calculation, even if the calculation itself might be centralized within the axes group (implementation specific)

3rd situation, period C:

Tracking is still active, but Motion FB is done

=> The Track FB is Busy and Active

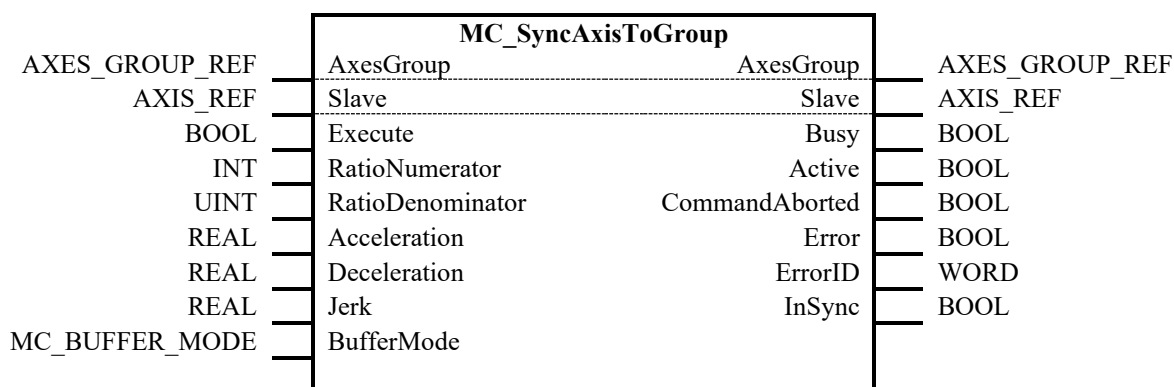
=> following Motion FBs aren't active yet

=> The AxesGroup is synchronized to the Master-Axis (CnvBelt or RotTable) / Master-AxesGroup.

During this period, only the Track FB is responsible for the set value calculation, even if the calculation itself might be centralized within the axes group (implementation specific)

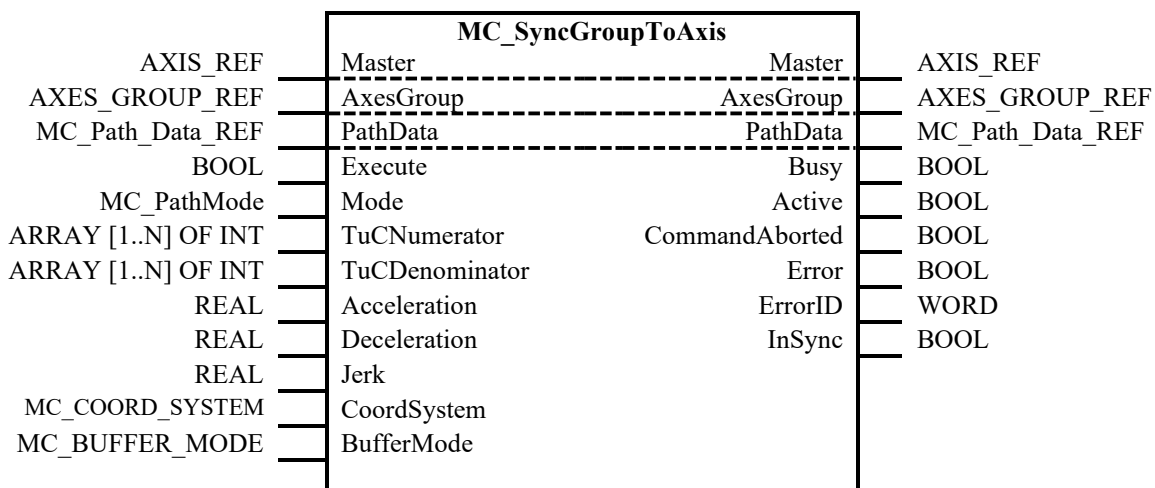
10.3 MC_SyncAxisToGroup

FB-Name		MC_SyncAxisToGroup	
This function block maps a single axis to a group. The single axis output represents the path length progression of the axes group. There is the ability to set a ratio between group and single axis.			
VAR IN OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to the group of axes
B	SlaveAxis	AXIS_REF	Reference to the axis (real or virtual)
VAR INPUT			
B	Execute	BOOL	Starts the synchronization process on the rising edge
E	RatioNumerator	INT	Gear Ratio Numerator.
E	RatioDenominator	UINT	Gear Ratio Denominator
E	Acceleration	REAL	Value of maximum acceleration
E	Deceleration	REAL	Value of maximum deceleration
E	Jerk	REAL	Value of maximum jerk
E	BufferMode	MC_BUFFER_MODE	Defines the behavior of the axis: modes are Aborting, Buffered, Blending. Refer to 11.2 Buffer Modes
VAR OUTPUT			
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axis
E	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
B	InSync	BOOL	The (virtual) slave generates valid values
Notes:			
- This FB equals the mileage counter (odometer) in a car of the group and shows this via the Slave axis. - The slave ramps up to the ratio of the path speed and locks in position when this is reached. - The gearing ratio can be changed while FB is running, using a consecutive call of the FB - InSync is set the first time the ratio is reached. - After being InSync, a position locking or just a speed locking is vendor specific.			
The FB is stopped by issuing a single axis FB.			



10.4 MC_SyncGroupToAxis

FB-Name		MC_SyncGroupToAxis	
This function block commands an interpolated path movement on an axes group in the applicable coordinate system. The multi axes motion is synchronized with the Master motion like in a cam function.			
VAR IN OUT			
B	Master	AXIS_REF	Reference to the master axis
B	AxesGroup	AXES_GROUP_REF	Reference to the group of axes
B	PathData	MC_Path_Data_REF	Reference to the path data, which can be prepared by the FB MC_PathSelect
VAR INPUT			
B	Execute	BOOL	Start the motion at rising edge
E	Mode	MC_PathMode	ENUM. Selects the mode of the FB like non_periodic, periodic.
E	TuCNumerator	ARRAY [1..N] OF INT	Numerator of the conversion factors of the technical units of each axis in the axes group to generate the applicable technical unit for the slave.
E	TuCDenominator	ARRAY [1..N] OF INT	Denominator of TuConversion
E	Acceleration	REAL	Maximum acceleration. Always positive. Not necessarily reached
E	Deceleration	REAL	Maximum deceleration. Always positive. Not necessarily reached
E	Jerk	REAL	Maximum jerk for the axes group during coupling. Not necessarily reached
E	CoordSystem	MC_COORD_SYSTEM	Reference to the applicable coordinate system. Interpretation is vendor specific.
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2 Buffer Modes
VAR OUTPUT			
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axis
E	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
B	InSync	BOOL	The axes group follows the master axis
Note: This synchronization of the axes group can be stopped via MC_GroupStop or any other motion command.			



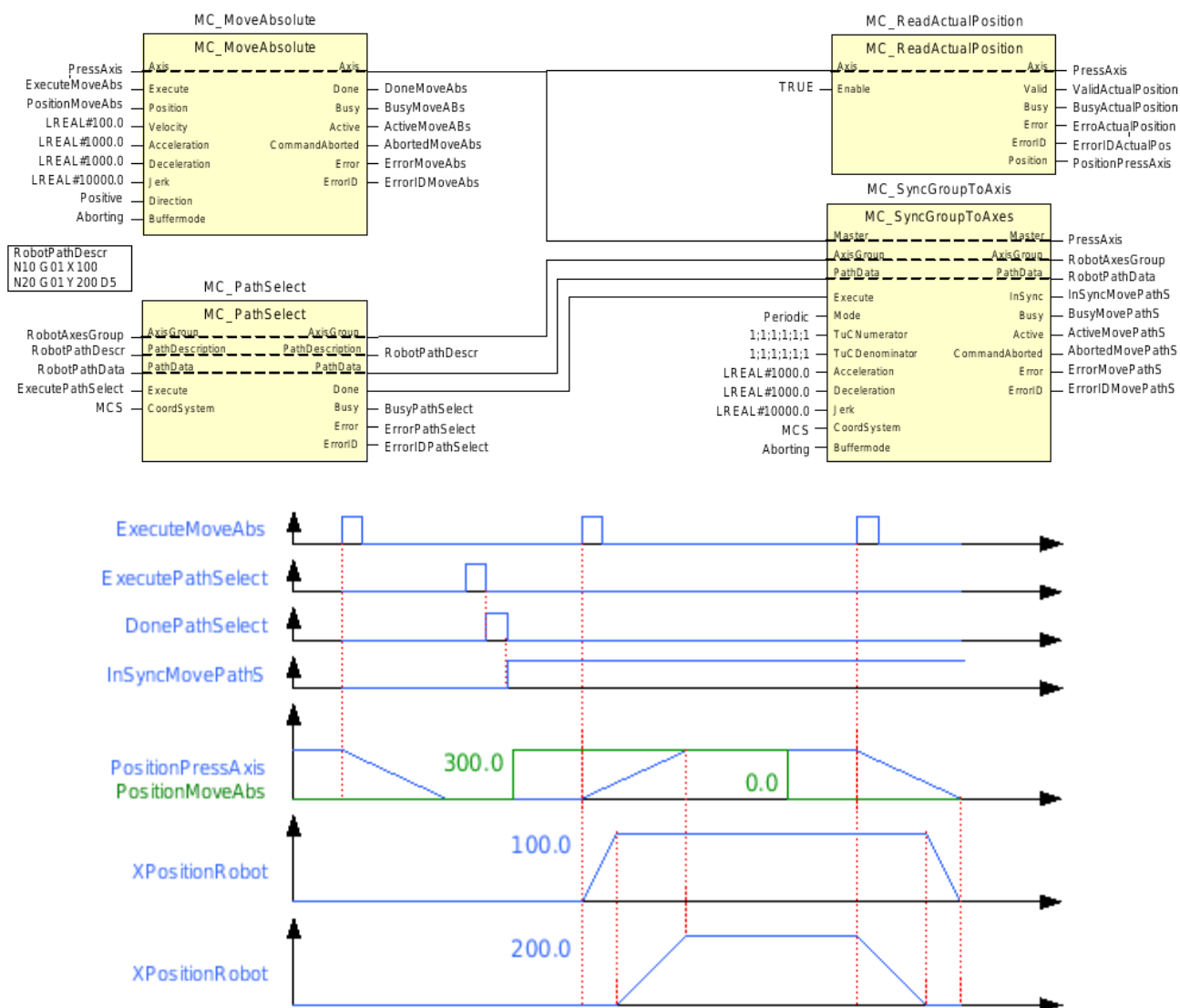
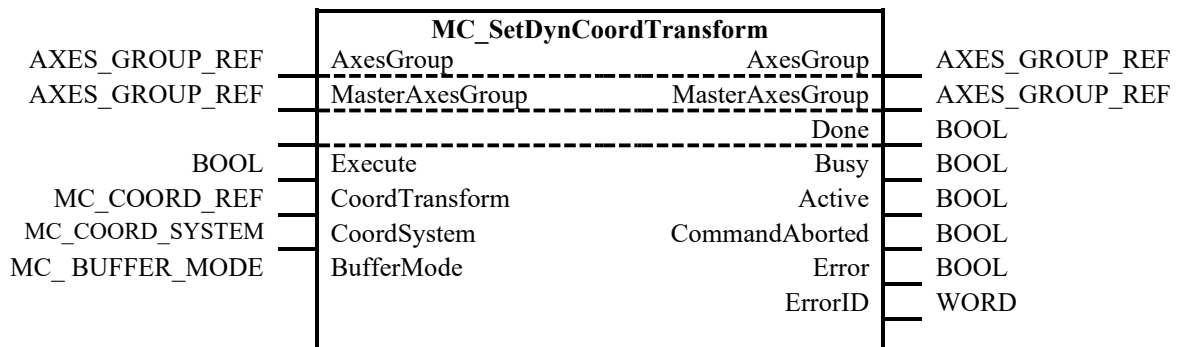


Figure 36: Example MC_SyncGroupToAxis

10.5 MC_SetDynCoordTransform

FB-Name		MC_SetDynCoordTransform	
This function block couples two Axes Groups via a dynamic coordinate transformation. The input for the coordinate transformation is the MasterAxesGroup. The result of the transformation is mapped to the AxesGroup, meaning that the coordinate system of AxesGroup will follow the MasterAxesGroup with the transformation as link			
VAR_IN_OUT			
B	AxesGroup	AXES_GROUP_REF	Reference to the group of axes
B	MasterAxesGroup	AXES_GROUP_REF	Reference to the master axes group
VAR_INPUT			
B	Execute	BOOL	Start the action at rising edge
B	CoordTransform	MC_COORD_REF	The relative position and orientation of the PCS with respect to the TCS of the MasterAxesGroup
E	CoordSystem	MC_COORD_SYSTEM	Selects the PCS of AxesGroup which should be used for the implicit automatic calculation. This input is only necessary if the axes group supports multiple PCS.
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2 Buffer Modes
VAR_OUTPUT			
B	Done	BOOL	The dynamic transformation is set successfully
E	Busy	BOOL	The FB is not finished
E	Active	BOOL	Indicates that the FB has control on the axes group
E	CommandAborted	BOOL	Command is aborted by another command
B	Error	BOOL	Signals that an error has occurred within the FB
E	ErrorID	WORD	Error identification
Note: The PCS specified by the CoordSystem input belongs to AxesGroup. It follows the movement of the MasterAxesGroup. The relation between both AxesGroups is specified by setting the individual MCS _A and MCS _B to the global WCS.			



MC_SetDynCoordTransform is in the group Synchronized/Tracking (see Ch. 1.6 – Overview of defined Function Blocks). This is because there is motion possible without any coordinated motion FB. For example, via MoveDirectAbsolute in PCS which is Done. In this case the Axesgroup is still moving synchronized to the PCS without another command (being synchronized to an external master axes group similar to the single axis MC_GearIn functionality in coordinated motion).

10.5.1 Behaviour of 'Busy' and 'Active'

The outputs 'Busy' and 'Active' of the FBs MC_SetDynCoordTransform, MC_TrackConveyorBelt and MC_TrackRotaryTable can show the state depending on different situations.

1st situation:

Track-FB is executed, and no Motion-FB is active on the tracked PCS
=> The Track-FB is Busy, but not Active.

2nd situation:

Tracking and motion in tracked PCS are active.
=> The Track-FB is Busy and Active
=> A Motion-FB is also Busy and Active, calculating/interpolating the path.

3rd situation:

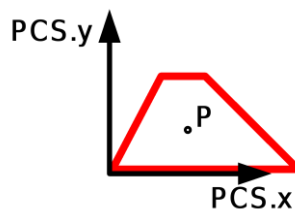
Tracking is still active, but Motion-FB is done
=> The Track-FB is Busy and Active
=> No Motion FB is active (no set values calculated anymore in this instance)
=> The AxesGroup is synchronized to the Master-Axis (CnvBelt or RotTable) / Master-AxesGroup.

10.6 MC_TrackConveyorBelt

FB-Name		MC_TrackConveyorBelt		
This function block offers an abstraction layer for a conveyor belt, assisting the user with tracking objects moving on a straight line in space. The FB activates a dynamic calculation of the coordinate system transformation from WCS to the selected coordinate system of the axes group. The pose of the conveyor belt relative to WCS is given by a dedicated input of the FB. A further input specifies the initial pose of an object lying on the conveyor belt. The actual position, velocity, etc. of the conveyor belt is given by a single axis.				
VAR IN OUT				
	B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes
	E	ConveyorBelt	AXIS_REF	Supplies the actual position, velocity, and etc. of the conveyor belt and thus the actual position of the object moved by the conveyor belt. The actual position is evaluated relative to the position of the conveyor belt when the FB had been started.
VAR INPUT				
	B	Execute	BOOL	Start the action at rising edge
	B	ConveyorBeltOrigin	MC_CART_REF	Specifies the pose of the conveyor belt relative to WCS. The conveyor belt might be shifted and/or rotated relative to WCS. This introduces a coordinate system of the conveyor belt in which the position of an object lying on the conveyor belt can be specified. The x-axis of the coordinate system of the conveyor belt has to point into the direction of movement of the conveyor belt.
	E	InitialObjectPosition	MC_CART_REF	Specifies the pose of an object lying on the conveyor belt relative to the conveyor belt. The object might be shifted and/or rotated relative to the coordinate system of the conveyor belt.
	E	CoordSystem	MC_COORD_SYSTEM	Selects the PCS which should be used for the implicit automatic calculation. This input is only necessary if the axes group supports multiple PCS.
	E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2 Buffer Modes
VAR OUTPUT				
	B	Done	BOOL	The transformation is set successfully
	E	Busy	BOOL	The FB is not finished
	E	Active	BOOL	Indicates that the implicit calculation of the transformation is going on.
	E	CommandAborted	BOOL	Command is aborted by another command which changes the transformation for the selected coordinate system. This might be another ConveyorBelt, a RotaryTable or MC_SetCartesianTransform or MC_Set-CoordinateTransform.
	B	Error	BOOL	Signals that an error has occurred within the FB
	E	ErrorID	WORD	Error identification
Note: This FB does not start any motion on itself. Motion is started by a motion command in PCS, and then the TCP is tracking the moving PCS, i.e. conveyor belt. This provides a coordinate system for the slave, following the master				

MC_TrackConveyorBelt			
AXES_GROUP_REF	AxesGroup	AxesGroup	AXES_GROUP_REF
AXIS_REF	ConveyorBelt	ConveyorBelt	AXIS_REF
BOOL	Execute	Done	BOOL
ARRAY [1..N] OF REAL	ConveyorBeltOrigin	Busy	BOOL
ARRAY [1..N] OF REAL	InitialObjectPosition	Active	BOOL
MC_COORD_SYSTEM	CoordSystem	CommandAborted	BOOL
MC_BUFFER_MODE	BufferMode	Error	BOOL
		ErrorID	WORD

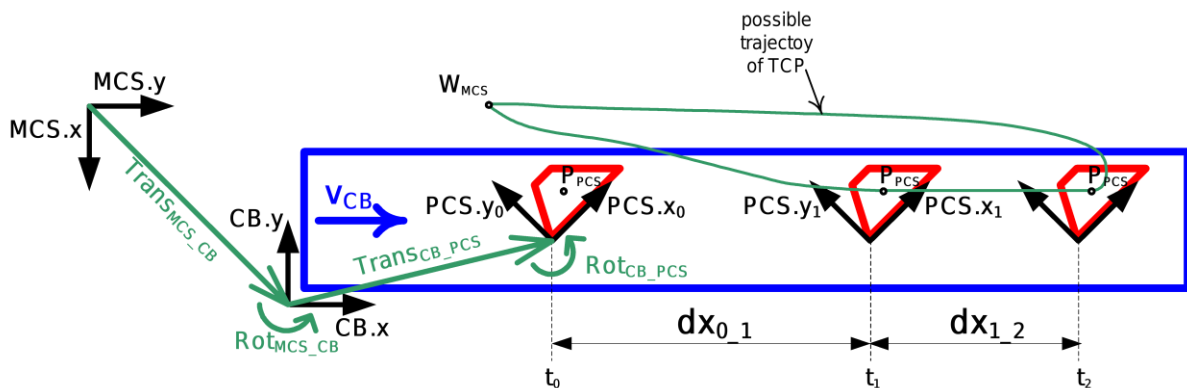
Following example should demonstrate the usage of the FB MC_TrackConveyorBelt.
The task is to manipulate a workpiece, starting from a certain position P_{PCS} (x_{PCS} ; y_{PCS} ; 0) on the workpiece.



The workpiece is lying on a conveyor belt (CB) which is moving with speed v_{CB} .
The workpiece is detected by a digital camera, which also provides the position (and orientation) of the workpiece relative to the conveyor belt when the workpiece is detected.

At point in time t_0 , when the workpiece is detected by the camera, the FB MyConveyorBelt is executed. This activates the automatic calculation of the coordinate transformation $MCS \leftrightarrow PCS$. Activating MyConveyorBelt simultaneously executes the FB GotoP. The axes group starts to move from its waiting position W_{MCS} towards P_{PCS} .

At point in time t_1 the axes group has reached P_{PCS} . In meantime the conveyor belt has moved the distance dx_{0_1} . After having reached P_{PCS} the process can be started. All positions within the process have to be related to PCS. At point in time t_2 the process is finished, and the axes group moves back to its waiting position W_{MCS} . A possible resulting trajectory of the TCP is drawn in the picture below.



The following example demonstrates the usage:

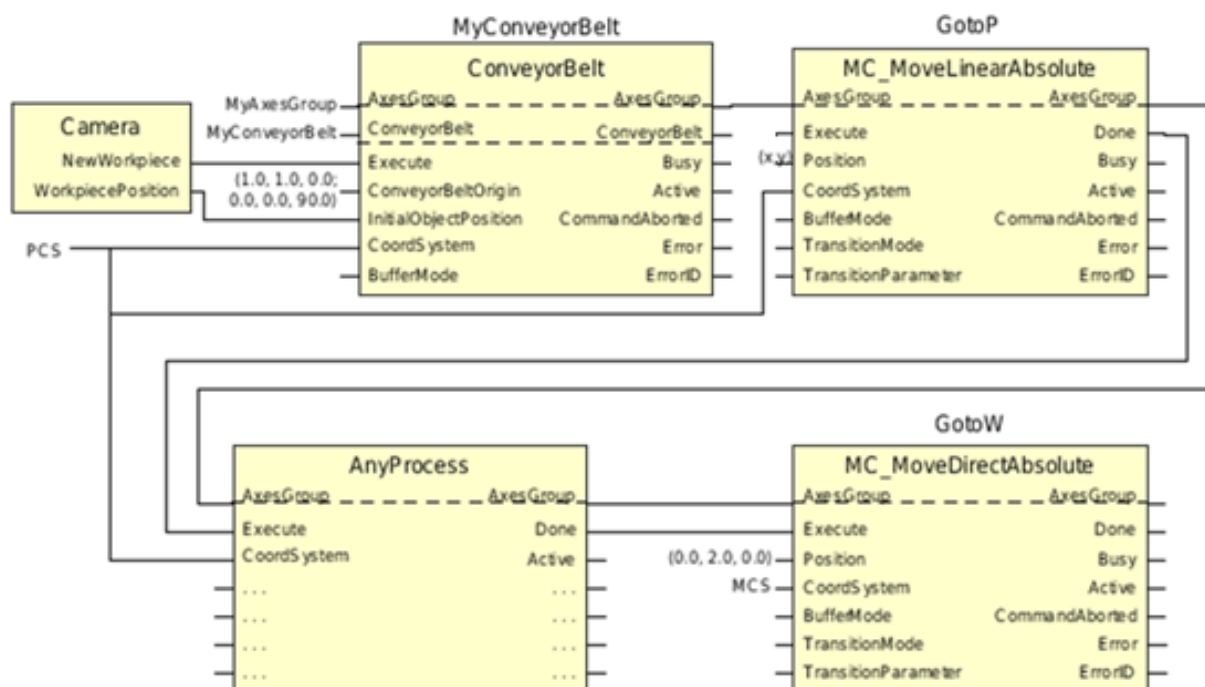


Figure 37: Example MC_TrackConveyorBelt

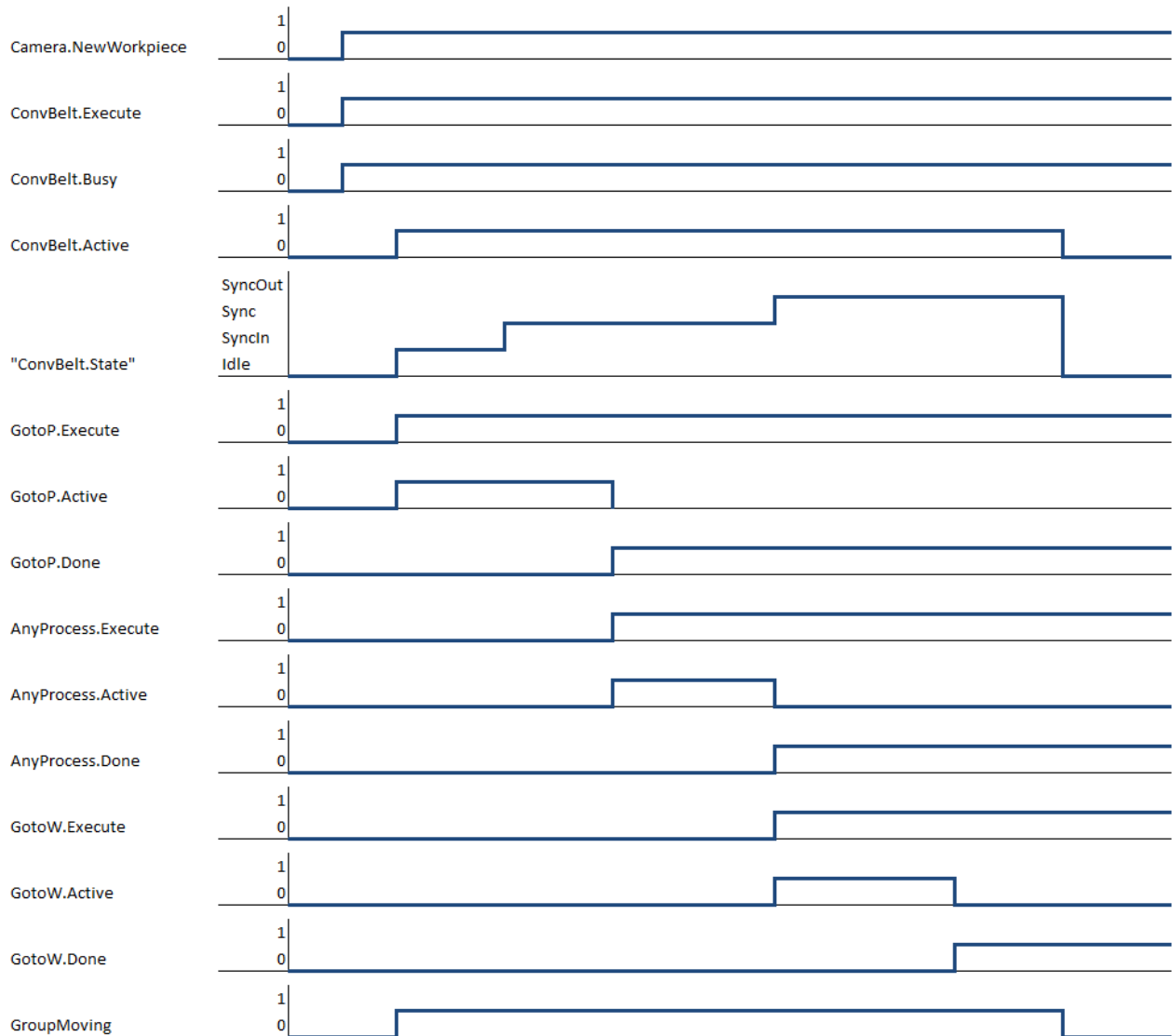
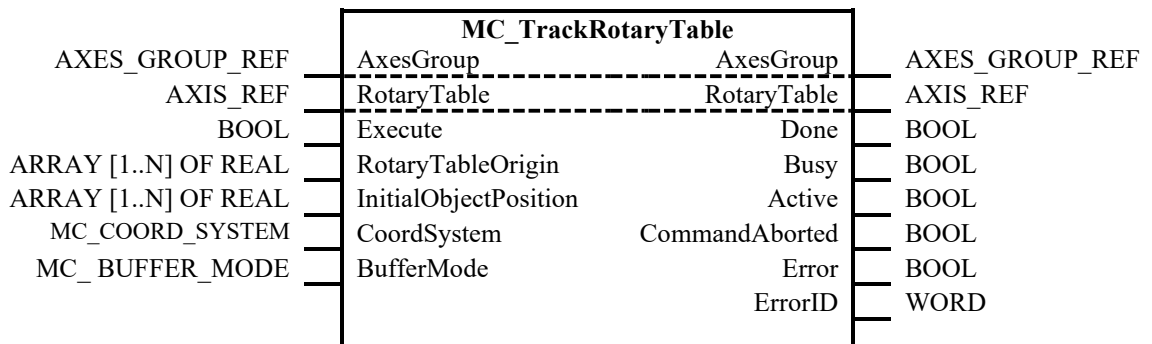


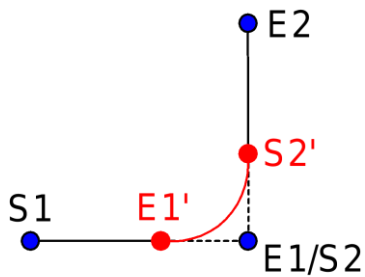
Figure 38: Timing diagram for example of MC_TrackConveyorBelt

10.7 MC_TrackRotaryTable

FB-Name		MC_TrackRotaryTable		
This function block offers an abstraction layer for a rotary table, assisting the user with tracking objects moving on a circle in space. The pose of the rotary table relative to WCS is given by a dedicated input of the FB. A further input specifies the initial pose of an object lying on the rotary table. The actual position, velocity, etc. of the rotary table is given by the position of a single axis. The FB activates a dynamic calculation of the coordinate system transformation from WCS to the selected coordinate system of the axes group.				
VAR_IN_OUT				
B	AxesGroup	AXES_GROUP_REF	Reference to a group of axes	
E	RotaryTable	AXIS_REF	Supplies the actual position, velocity, etc. of the rotary table and thus the actual position of the object moved by the rotary table. The actual position is evaluated relative to the position of the rotary table when the FB had been started.	
VAR_INPUT				
B	Execute	BOOL	Start the action at rising edge	
B	RotaryTableOrigin	MC_CART_REF	Specifies the pose of the rotary table relative to WCS. The rotary table might be shifted and/or rotated relative to WCS. This results in a coordinate system of the rotary table in which the position of the object can be specified. The z-axis of the coordinate system of the rotary table has to be perpendicular to the rotary table.	
E	InitialObjectPosition	MC_CART_REF	Specifies the pose of an object lying on the rotary table relative to the rotary table. The object might be shifted and/or rotated relative to the coordinate system of the rotary table.	
E	CoordSystem	MC_COORD_SYSTEM	Selects the PCS that should be used for the implicit automatic calculation. This input is only necessary if the axes group supports multiple PCS.	
E	BufferMode	MC_BUFFER_MODE	Defines the chronological sequence of the FB. Refer to 11.2 Buffer Modes	
VAR_OUTPUT				
E	Done	BOOL	The transformation is set successfully	
E	Busy	BOOL	The FB is not finished	
E	Active	BOOL	Indicates that the implicit calculation of the transformation is going on.	
E	CommandAborted	BOOL	Command is aborted by another command which changes the transformation for the selected coordinate system. This might be another RotaryTable, a ConveyorBelt or MC_SetCartesianTransform or MC_SetCoordinateTransform.	
B	Error	BOOL	Signals that an error has occurred within the FB	
E	ErrorID	WORD	Error identification	
Notes: -				



11 Transition, blending and buffering of movements - Terminological definitions

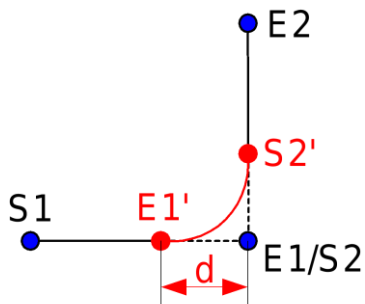


Contour curve: Inserted curve, which modifies the original path (E1' - S2').

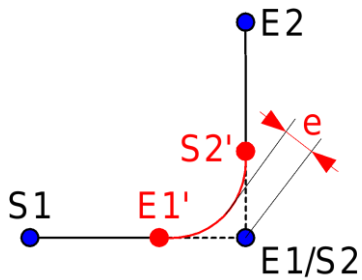
Pre-block: Motion block before the contour curve (S1 – E1)

Post-block: Motion block after the contour curve (S2 – E2)

Corner distance: Distance (d) of the start point of the contour curve (E1') to the programmed target point (E1), see following figure.



Corner deviation: The shortest distance between the programmed corner point (E1/S2) and the contour curve (see following figure).



11.1 Input parameter for blending

The input parameter “TransitionMode”, combined with the input “TransitionParameter”, define the shape and dynamics of the inserted contour to connect the current motion block with the following motion block. For this purpose, the programmed motion blocks are modified. This parameter does not define the chronological execution time; this is given by the group specific input “BufferMode”.

11.2 Buffer Modes

The buffer modes here are used differently from the ones in Part 1. Via the additional input “TransitionVelocity” the 4 blending modes are configured independently from the chronological sequence of the FB’s. The buffer modes mcBlendingLow, mcBlendingPrevious, mcBlendingNext and mcBlendingHigh are obsolete in new implementations. They’re only here for compatibility with Part 1. It is recommended to use the input TransitionVelocity instead. See below Table 6: Overview of Transition Velocity Modes for more detailed explanation.

MC_BUFFER_MODE	Description
mcAborting	Start FB immediately (default mode)
mcBuffered	Start FB after current motion is finished
mcBlendingLow	The velocity is blended with the lowest velocity of both FBs
mcBlendingPrevious	The velocity is blended with the velocity of the first FB
mcBlendingNext	The velocity is blended with the velocity of the second FB
mcBlendingHigh	The velocity is blended with the highest velocity of both FBs

Table 5: Overview of Buffer Modes

MC_TRANSITION_VELOCITY	Description
mcTVZero	Exact stop at the transition from the previous command to the next
mcTVLow	The velocity is blended with the lowest velocity of both FBs
mcTVPrevious	The velocity is blended with the velocity of the first FB
mcTVNext	The velocity is blended with the velocity of the second FB
mcTVHigh	The velocity is blended with the highest velocity of both FBs

Table 6: Overview of Transition Velocity Modes

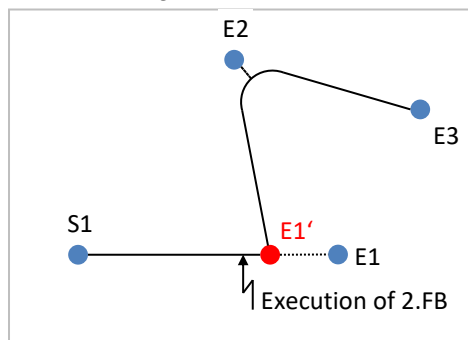
Note 1: Vendor specific extensions are allowed, but are not allowed to start with “mc”

Note 2: The transition velocity depends on MC_TRANSITION_REFERENCE

Note 3: it is vendor specific which acceleration-, deceleration- and jerk-limits are used during the transition mode blending.

Note 4: MC_TRANSITION_VELOCITY is only evaluated if BufferMode is equal to mcAborting or mcBuffered.

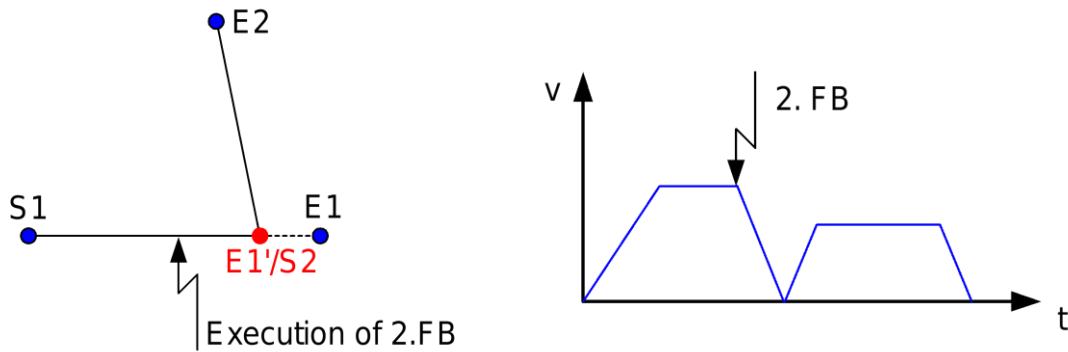
Example for a movement that cannot be programmed with traditional “BufferMode”: movement to E2 is aborting movement to E1 and is blended with movement to E3



In classical PLC applications, movements are executed sensor based, so blending is programmed with respect to the start position of this function block. In traditional CNC and robotic languages, it is common to define the blending at the end of a command line with respect to the target position. To enable both ways of programming “BufferMode” was adapted and support for MC_TRANSITION_REFERENCE has been added.

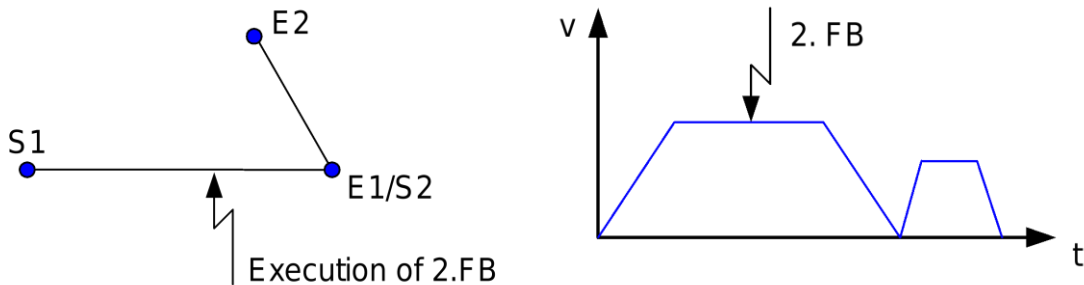
11.2.1 BufferMode “mcAborting”

A FB with buffer mode “mcAborting” aborts an ongoing motion and starts the new motion immediately. The following figure shows just an example for buffer mode “mcAborting” and TransitionMode “TMNone”. In this example the first motion is stopped and then the next motion is started. The available transition modes are described later.



11.2.2 BufferMode “mcBuffered”

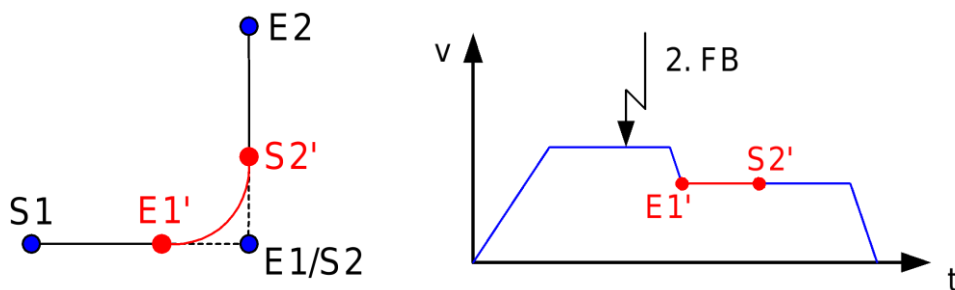
The next FB affects the axes group as soon as the previous motion is ”Done”. The transition velocity is depending on the value of MC_TRANSITION_VELOCITY.



11.2.3 Transition Velocity behavior

With the input Transition mode several options can be selected as defined in the structure MC_TRANSITION_VELOCITY. The current and the next motion FBs are blended, so the axes group will not stop between the motions except in mode TransitionVelZero. The velocity is blended according to the specified blending modes (VelLow, VelPrevious, VelNext, VelHigh).

The following figure shows just an example for buffer mode mcTVNext. The available transition modes are described later.



11.3 TransitionMode

Depending on the transition mode different vendor specific transition parameters can be given, which characterize the contour curve.

The basic transition modes are defined. Vendor specific modes can be added.

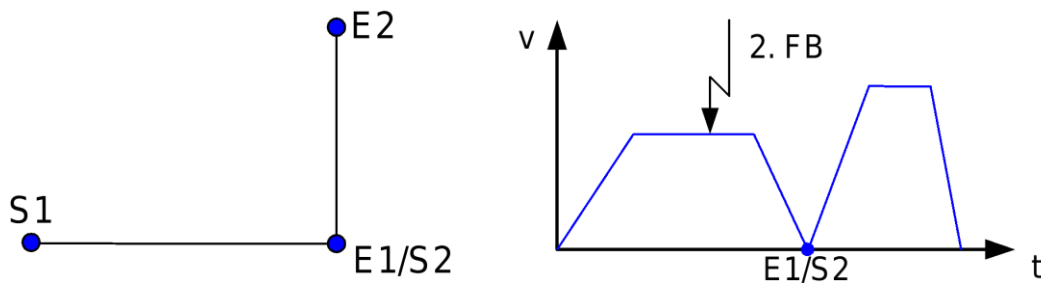
MC TRANSITION MODE	Description
mcTMNone	Insert no transition curve (default mode)
mcTMStartVelocity	Transition with given start velocity
mcTMConstantVelocity	Transition with given constant velocity
mcTMCornerDistance	Transition with given corner distance
mcTMMaxCornerDeviation	Transition with given maximum corner deviation

Table 7: Overview of available transition modes

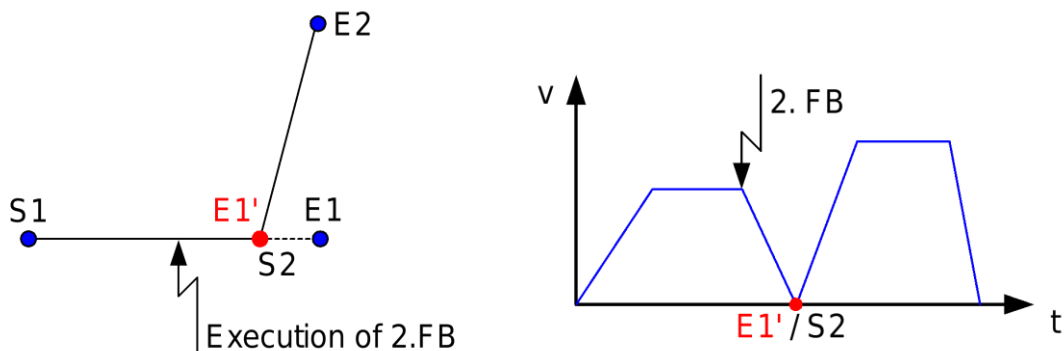
11.3.1 TransitionMode “TMNone” (insert no transition curve)

The motion blocks are not modified and no transition curve is inserted.

Issuing any "mcBlending" buffer mode results in a "mcBuffered" mode behavior, since there are no blending modes separately defined for this (or one issues a warning or error if a blending mode is issued) (see following figure).



The following figure shows the contour for BufferMode “mcAborting”.



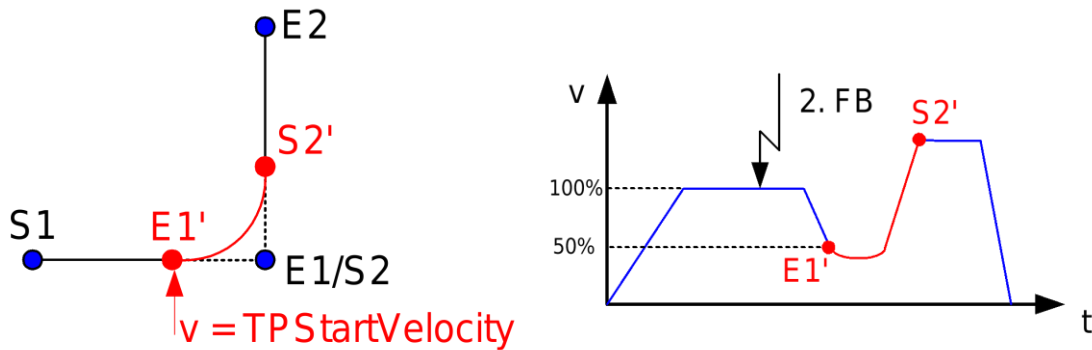
11.3.2 TransitionMode “TMStartVelocity” (Transition with given maximum velocity)

In this type of contouring, the corner distance of the contour curve is defined by a specified maximum percentage value “TPStartVelocity” of the programmed velocity of the pre-block in order to start the curve.

With mcBuffered” and mcTransitionVelZero all TransitionModes are ignored and TMNone is used.

The velocity on the contour curve does not have to be constant.

If e.g., TPStartVelocity = 50%, the contour curve is started after the velocity has reached 50% of the velocity programmed in the pre-block during deceleration.



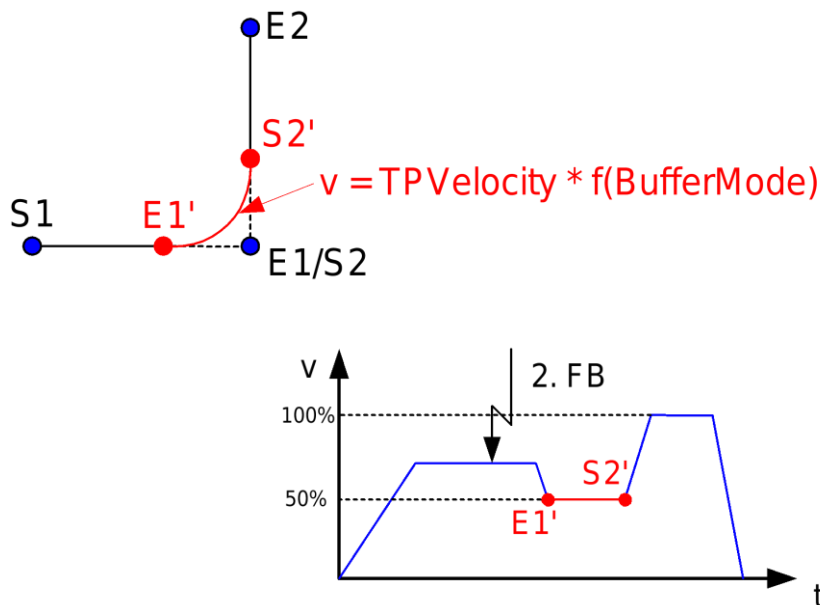
TransitionParameter	Description
TPStartVelocity <expr>	Maximum velocity in percent of pre-block

11.3.3 TransitionMode “TMConstantVelocity”(Transition with given constant velocity)

In this type of contouring, the contour curve is dimensioned in a manner, that the curve can be traversed with a constant specified percentage value “TPVelocity” of the transition velocity that results out of MC_BUFFER_MODE of the next motion block. If e.g. TPVelocity = 50%, the contour curve is traversed with half the velocity that results out of the buffer mode of the next motion block.

Because of the curvature of the contour and the given maximum acceleration of the axis nevertheless it is possible, that the desired velocity is not reached.

The following figure shows the contour a non-zero transition velocity. The value TPVelocity refers to an entry of the array TransitionParameter and defines velocity value in percentage.I

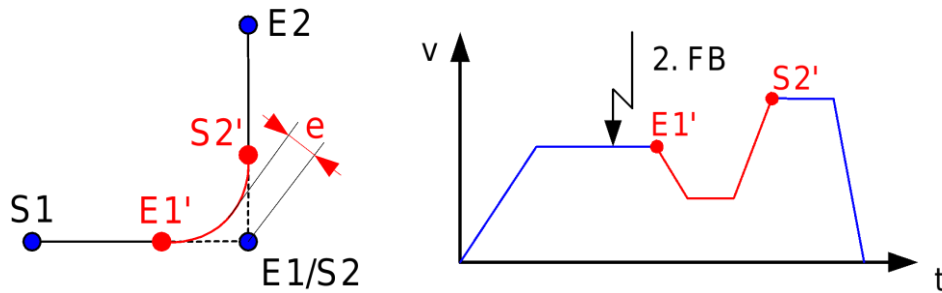


For BufferMode “Aborting” the following contour curve is generated. The motion of the first FB is aborted, and it is tried in a vendor specific way to get back to the commanded path.

11.3.5 TransitionMode “TMMaxCornerDeviation” (Transition with given maximum corner deviation)

The corner distances, by which the bordering motion blocks are shortened, are automatically determined upon geometrical considerations in such a manner that the given corner deviation is not exceeded.

Because of the curvature of the contour and the given maximum acceleration of the axis nevertheless it is possible, that the desired velocity is not reached.



TransitionParameter	Description
TPCornerDeviation <expr>	Corner deviation (e), the shortest distance between the programmed corner point and the contour curve.

11.4 MC_COMMAND_ID

MC_COMMAND_ID is a vendor specific data type, that uniquely identifies a buffered motion or queued administrative command.

It is used for the extended output CommandID of these buffered and queued commands.

The CommandID of the active motion command can be read by MC_GroupReadMotionState.

The CommandID can be used for the input CommandID of the function block MC_GroupReadCommandInfo, to get additional status information.

The CommandID = 0 on a buffered or queued function block means that the command hasn't been accepted yet.

Example implementations:

(a)

MC_COMMAND_ID: UINT;

MC_COMMAND_ID is increased with every new command, sent to the axes group.

MC_COMMAND_ID might be reset under certain conditions, e.g. GroupDisabled to prevent overflow

(b)

MC_COMMAND_ID: vendor specific interface

11.5 MC_CIRC_PATHCHOICE

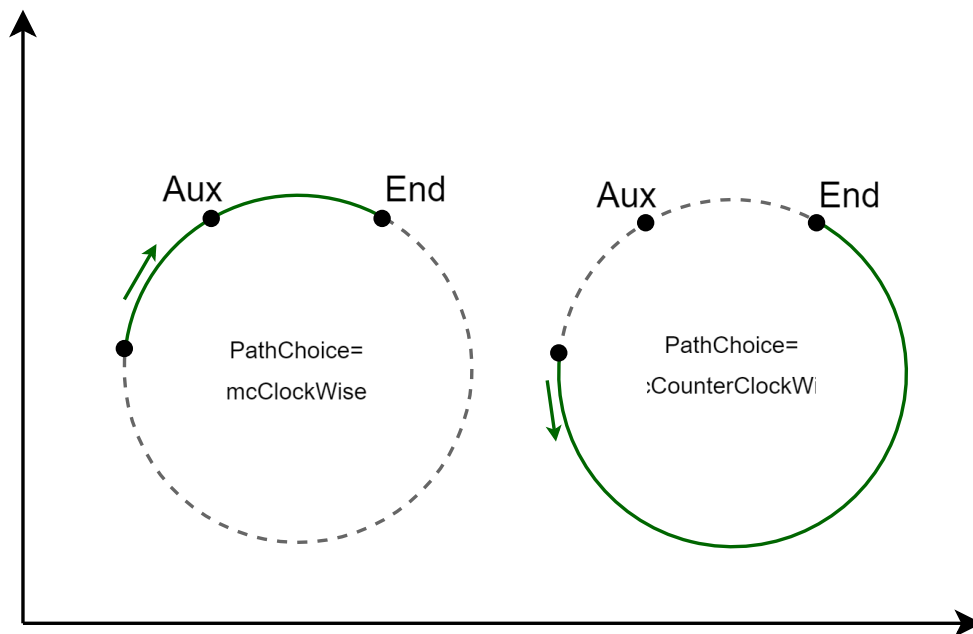
The enumeration MC_CIRC_PATHCHOICE defines the direction of a circular movement.

The direction of the arc is chosen according to the rule of right thumb: counterclockwise direction is along the fingers bending around the axis of the thumb which represents the normal vector.



No.	MC_TRANSITION_REFERENCE	Description
1	mcClockwise	An arc in clockwise direction is created
2	mcCounterClockwise	An arc in counterclockwise direction is created

The following example uses CircMode = mcBorder. The plane normal is pointing out of the sheet towards the reader.



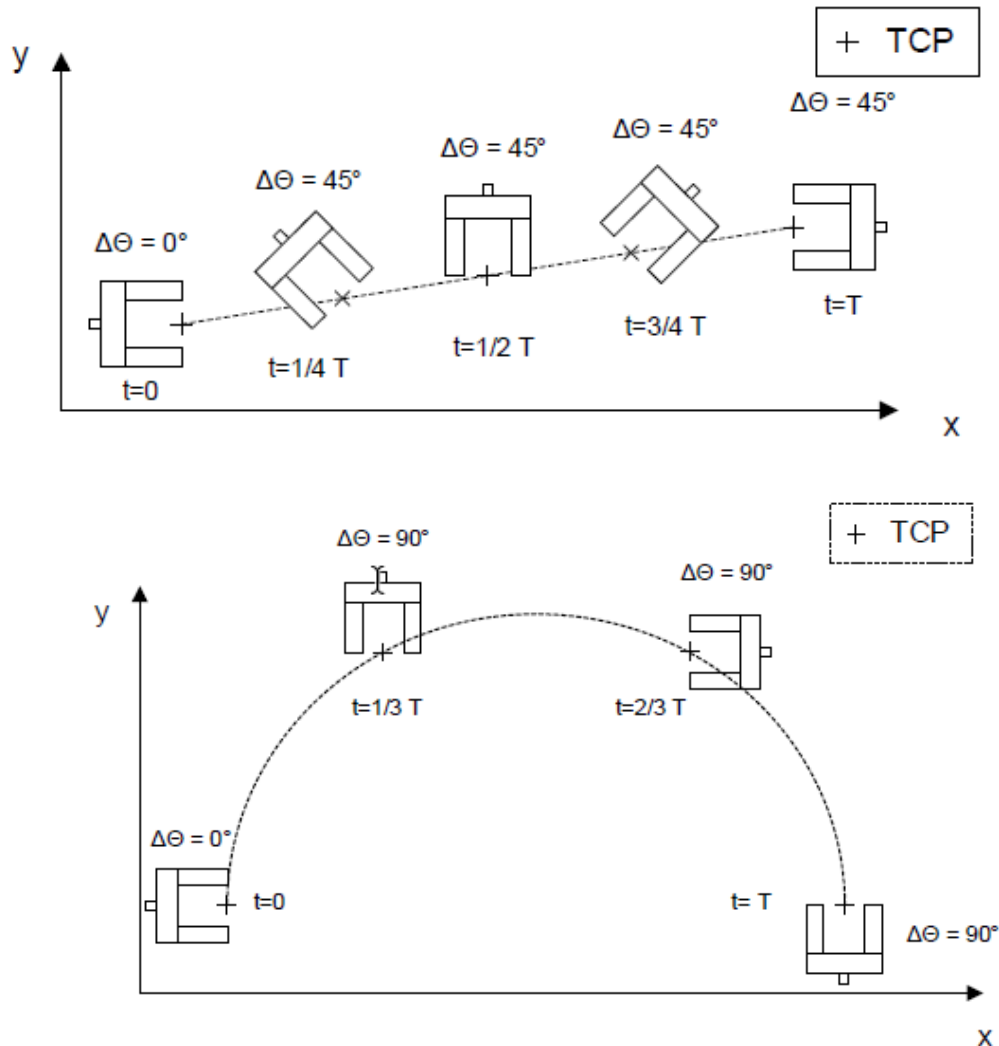
11.6 MC_ORIENTATION_MODE

MC_Orientation_Mode defines interpolation of orientation of a path movement.

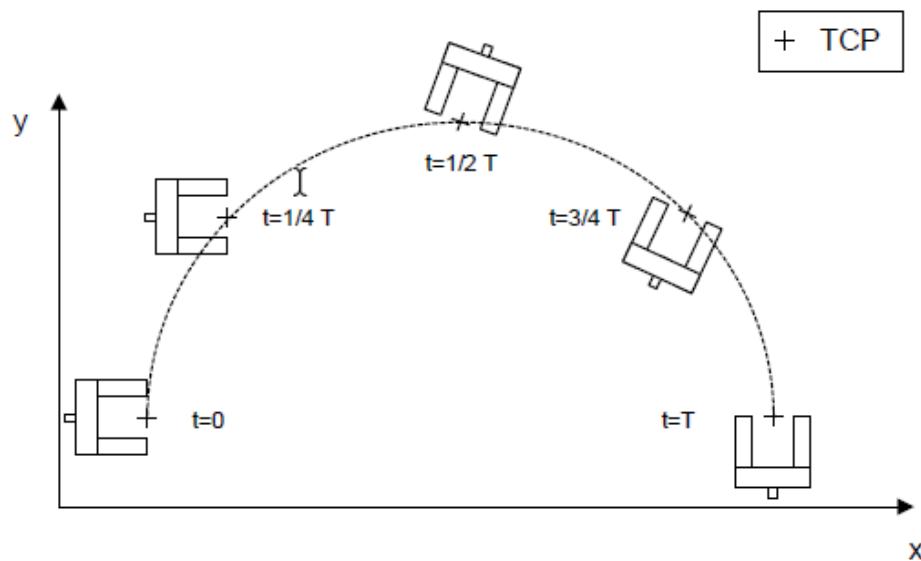
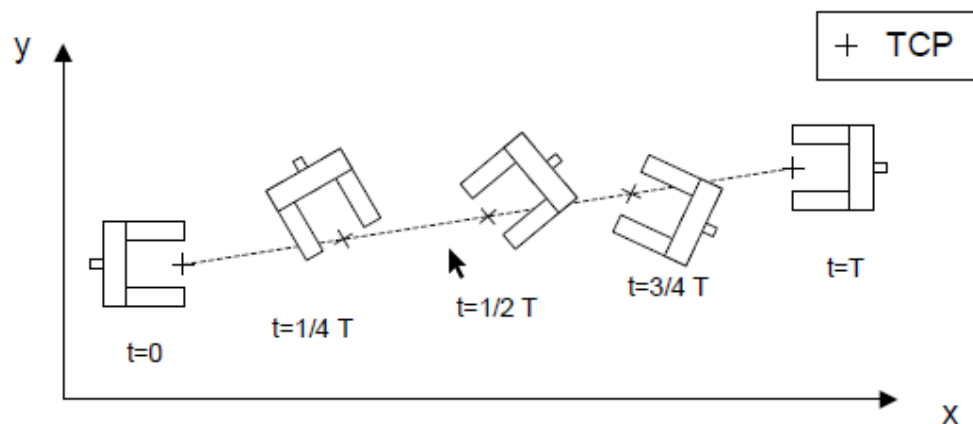
MC_ORIENTATION_MODE	Description
mcLinear	TCP changes orientation continuously linear to path
mcJointInterpolated	TCP orientation changes continuously from starting wrist to target wrist position. Therefore, the resulting orientation during movement is not predictable but reproducible.
mcFixed	TCP orientation remains constant in relation to cartesian coordinate system
mcPathBased	TCP orientation remains constant in relation to current path

Figure 39: Orientation Modes

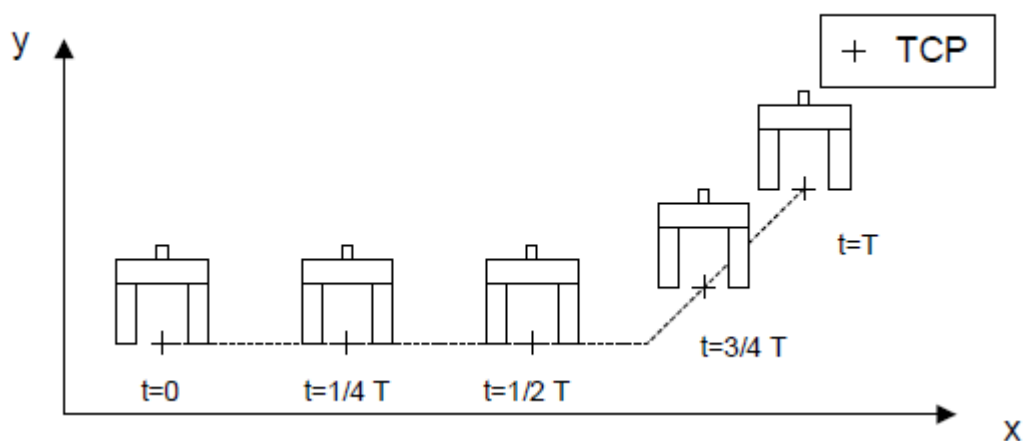
mcLinear

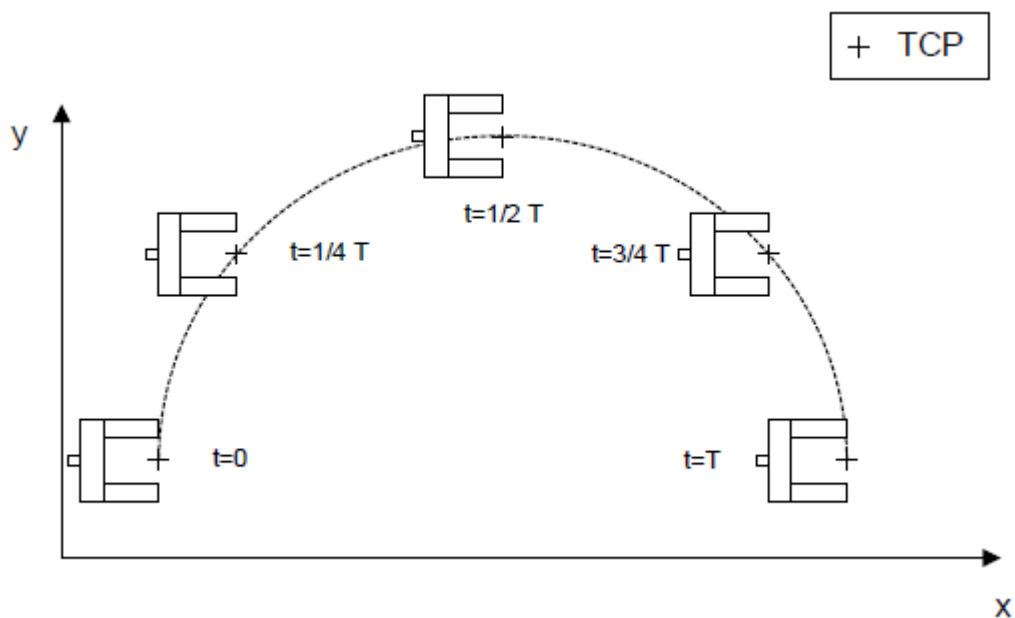


mcJointInterpolated

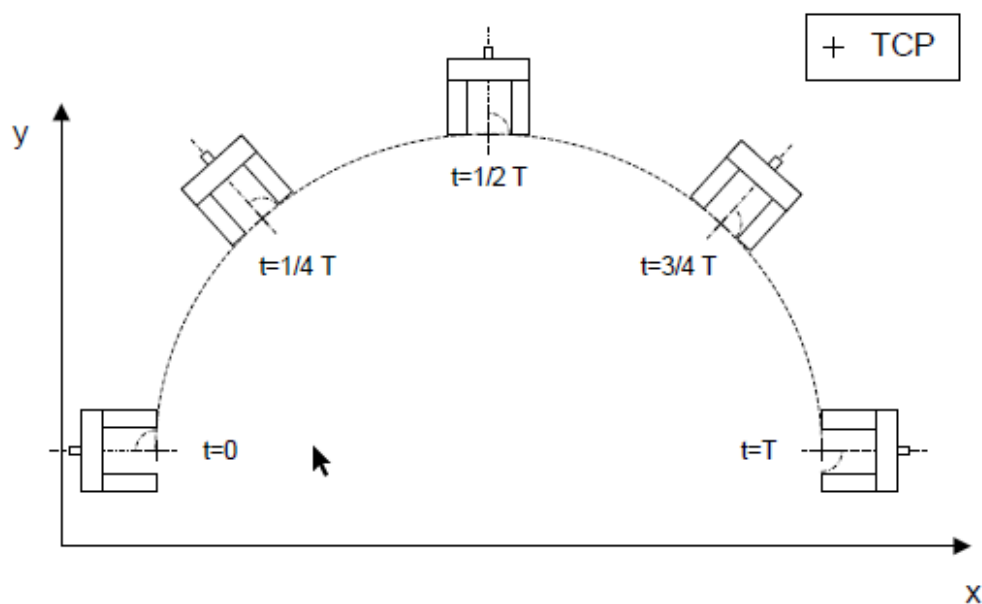
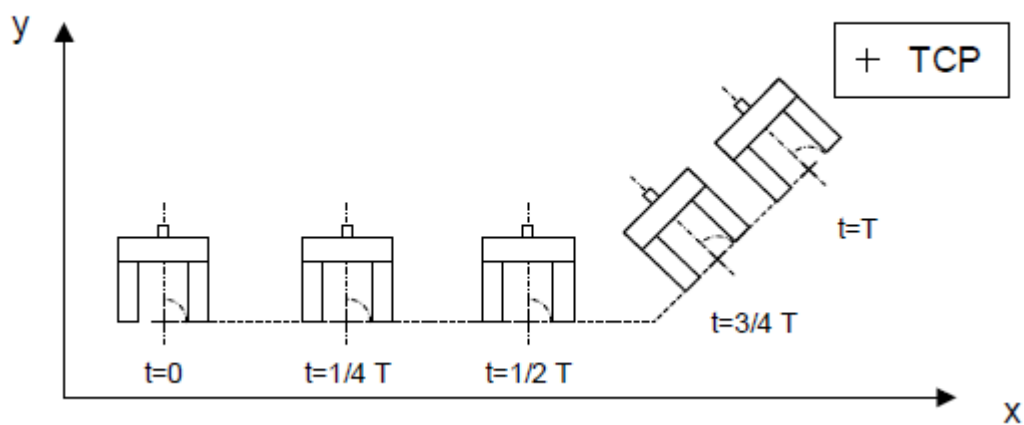


mcFixed





mcPathBased



11.7 MC_GROUP_SWLIMITS

The datatype MC_GROUP_SWLIMITS is vendor specific

The two following examples show possible implementations.

Example 1:

This example used a structure with SW limits for each joint of the axes group.

The structure is then combined in an ARRAY for reading and writing all joint SW limits of the group at the same time.

For each supported joint of the axes group, the negative and positive SW limits are combined in a structure.

MC_SWLIMIT	Datatype	R/W	Description
SWLimitPos	REAL	R/W	Positive Software limit switch position of a joint
SWLimitNeg	REAL	R/W	Negative Software limit switch position of a joint

MC_GROUP_SWLIMITS is defined as an ARRAY[1...N] OF MC_SWLIMIT.

Example 2:

This example shows a singular structure including the SW limits for six available joints J1...J6 and six auxiliary axes E1...E6. This would be applicable for classic six axes articulated robots.

MC_GROUP_SWLIMIT	Datatype	R/W	Description
J1SWLimitNeg	REAL	R/W	Negative software limit for Joint J1
J1SWLimitPos	REAL	R/W	Positive software limit for Joint J1
J2SWLimitNeg	REAL	R/W	Negative software limit for Joint J2
J2SWLimitPos	REAL	R/W	Positive software limit for Joint J2
Negative software limit for Joint J3J3SWLimitNeg	REAL	R/W	Negative software limit for Joint J3
J3SWLimitPos	REAL	R/W	Positive software limit for Joint J3
J4SWLimitNeg	REAL	R/W	Negative software limit for Joint J4
J4SWLimitPos	REAL	R/W	Positive software limit for Joint J4
J5SWLimitNeg	REAL	R/W	Negative software limit for Joint J5
J5SWLimitPos	REAL	R/W	Positive software limit for Joint J5
J6SWLimitNeg	REAL	R/W	Negative software limit for Joint J6
J6SWLimitPos	REAL	R/W	Positive software limit for Joint J6
E1SWLimitNeg	REAL	R/W	Negative software limit for axis E1
E1SWLimitPos	REAL	R/W	Positive software limit for axis E1
E2SWLimitNeg	REAL	R/W	Negative software limit for axis E2
E2SWLimitPos	REAL	R/W	Positive software limit for axis E2
E3SWLimitNeg	REAL	R/W	Negative software limit for axis E3
E3SWLimitPos	REAL	R/W	Positive software limit for axis E3
E4SWLimitNeg	REAL	R/W	Negative software limit for axis E4
E4SWLimitPos	REAL	R/W	Positive software limit for axis E4
E5SWLimitNeg	REAL	R/W	Negative software limit for axis E5
E5SWLimitPos	REAL	R/W	Positive software limit for axis E5
E6SWLimitNeg	REAL	R/W	Negative software limit for axis E6
E6SWLimitPos	REAL	R/W	Positive software limit for axis E6

11.8 Transition reference point

In classical PLC applications, movements are executed sensor based. This implies that an executed MC_MoveXXX function block has no information about the following function block but knows the already active movements.

In this case it is useful to define the blending between the new function block and the previously executed motion command in the new function block to the with respect start position of this function block.

However, in traditional CNC and robotic languages it is common to define the blending at the end of a command line with respect to the target position.

The enumeration MC_TRANSITION_REFERENCE defines the reference position for blending two movements.

No.	MC_TRANSITION_REFERENCE	Description
1	mcStartPoint	TransitionMode and TransitionVelocity refer to the start point of the move command to define the blending between the last commanded move FB and the new executed move FB.
2	mcEndPoint	TransitionMode and TransitionVelocity refer to the end point of the move command to define the blending between this commanded move FB and the next executed move FB. If no further movement is commanded, the behavior is vendor specific.

11.9 Tool Coordinate System (TCS) and buffered movements

If a movement is commanded with “BufferMode” = “mcBuffered” and “CoordSystem” = “TCS”, then the Tool Coordinate System at the end of the previously commanded movement is used to determine the resulting target position in the MCS. If there is no previous movement, then the current Tool Coordinate System of the robot is used.

If the target position of the previously commanded movement is in a dynamic coordinate system, or if there is no previous movement and the robot is currently tracking a dynamic coordinate system, then it is vendor specific at which time the Tool Coordinate System of the position is evaluated.

Note that also for blending movements (input “TransitionMode” $\neq$ “TMNone”), the original target position of the previous movement is used, although it may not even be reached due to the blending.

Appendix 1 Compliance Procedure and Compliance List

Listed in this Appendix are the requirements for the compliance statement from the vendor of the Motion Control Function Blocks. The compliance statement consists of two main groups: supported data types (see Appendix 1.1 Supported Data types) and supported Function Blocks, in combination with the applicable inputs and outputs (see Appendix 1.1 Supported Data types and its paragraphs). The vendor is required fill out the tables for the used data types and Function Blocks, according to their product, committing their support to the specification.

By submitting these tables to PLCopen, and after approval by PLCopen, the list will be published on the PLCopen website, www.plcopen.org , as well as a shortform overview, as specified in Appendix 1.4 Short overview of the Function Blocks.

In addition to this approval, the vendor is granted access and usage rights of the PLCopen Motion Control logo, as described in chapterAppendix 2 The PLCopen Motion Control logo and its usage.

Data types

The data type REAL listed in the Function Blocks and parameters (e.g. for velocity, acceleration, distance, etc.) may be exchanged to SINT, INT, DINT or REAL without to be seen as incompliant to this standard, if they are consistent for the whole set of Function Blocks and parameters.

Implementation allows the extension of data types if the basic data type is kept. For example: WORD may be changed to DWORD, but not to REAL.

Function Blocks and Inputs and Outputs

An implementation which claims compliance with this PLCopen specification shall offer a set of Function Blocks for motion control, meaning one or more Function Blocks, with at least the **basic** input and output variables, marked as “**B**” in the tables. These inputs and outputs must be supported to be compliant.

For higher-level systems and future extensions any subset of the **extended** input and output variables, marked as “**E**” in the tables can be implemented.

Vendor specific additions are marked with “**V**” and can be listed as such in the vendor documentation.

- | | |
|--|---|
| - Basic input/output variables are mandatory | Marked in the tables with the letter “ B ” |
| - Extended input /output variables are optional | Marked in the tables with the letter “ E ” |
| - Vendor Specific additions | Marked in the vendor’s compliance documentation with “ V ” |

All the vendor specific items will not be listed in the comparison table on the PLCopen website, but in the detailed vendor specific list, which also is published.

All vendor specific in- and outputs of all FBs must be listed in the certification list of the vendor. With this, the certification listing from a vendor describes all the I/O’s of the relevant FBs, including vendor-specific extensions, and thus showing the complete FBs as used by the vendor.

Appendix 1.1 Statement of Vendor

Vendor name	
Vendor address	
City	
Country	
Telephone	
Fax	
Email address	
Product Name	
Product version	
Release date	

I hereby state that the following tables as filled out and submitted do match our product as well as the accompanying user manual, as stated above.

Name of representation (person):

Date of signature (dd/mm/yyyy):

Signature:

Appendix 1.1. Supported Data types

Defined datatypes with MC library:	Supported	If not supported, which datatype used
BOOL		
INT		
WORD		
REAL		
ENUM		

Table 8: Supported datatypes

Within the specification the following derived datatypes are defined. Which structure is used in this system:

Derived datatypes:	Where used	Supported	Which structure
AXES_GROUP_REF	Nearly all FBs		
IDENT_IN_GROUP_REF	MC_AddAxisToGroup MC_RemoveAxisFromGroup MC_GroupReadConfiguration		
MC_BUFFER_MODE	In all buffered FBs		
MC_KIN_REF	MC_SetKinTransform MC_ReadKinTransform		
MC_EXECUTION_MODE	MC_SetKinTransform		
MC_COORD_REF	MC_SetCoordinateTransformation		
MC_TRANSITION_MODE	MC_MoveLinearAbsolute MC_MoveLinearRelative MC_MoveCircularAbsolute MC_MoveCircularRelative MC_MoveDirectAbsolute MC_MoveDirectRelative MC_MovePath	[	
MC_CIRC_PATHCHOICE	MC_MoveCircularAbsolute MC_MoveCircularRelative		
MC_PATH_DATA_REF MC_PATH_REF	MC_PathSelect MC_MovePath		

Table 9: Supported derived datatypes

Appendix 1.2. Supported Buffer Modes

MC_BUFFER_MODE	Supported
mcAborting	
mcBuffered	
mcBlendingLow	
mcBlendingPrevious	
mcBlendingNext	
mcBlendingHigh	

Table 10: Overview of buffer modes

Supported Transition Velocities

MC_TRANSITION_VELOCITY	Supported
mcTransitionVelZero	
mcTransitionVelLow	
mcTransitionVelPrevious	
mcTransitionVelNext	
mcTransitionVelHigh	

Appendix 1.3. Supported Transition Modes

MC_TRANSITION_MODE	Supported
mcTMNone	
mcTMStartVelocity	
mcTMConstantVelocity	
mcTMCornerDistance	
mcTMMaxCornerDeviation	
Reserved by PLCopen	
Vendor specific modes	

Table 11: Overview of available transition modes

Appendix 1.4. Short overview of the Function Blocks

Coordinated Function Blocks	Supported Yes / No	Comments (<= 48 char.)
MC_AddAxisToGroup		
MC_GroupContinue		
MC_GroupDisable		
MC_GroupEnable		
MC_GroupHalt		
MC_GroupHome		
MC_GroupInterrupt		
MC_GroupReadAcceleration		
MC_GroupReadPosition		
MC_GroupReadVelocity		
MC_GroupReadConfiguration		
MC_GroupReadError		
MC_GroupReadStatus		
MC_GroupReset		
MC_GroupSetOverride		
MC_GroupSetPosition		
MC_GroupStop		
MC_MoveCircularAbsolute		
MC_MoveCircularRelative		
MC_MoveDirectAbsolute		
MC_MoveDirectRelative		
MC_MoveLinearAbsolute		
MC_MoveLinearRelative		
MC_MovePath		
MC_PathSelect		
MC_ReadCartesianTransform		
MC_ReadCoordinateTransform		
MC_ReadKinTransform		
MC_RemoveAxisFromGroup		
MC_SetCartesianTransform		
MC_SetCoordinateTransform		
MC_SetDynCoordTransform		
MC_SetKinTransform		
MC_SyncAxisToGroup		
MC_SyncGroupToAxis		
MC_TrackConveyorbelt		
MC_TrackRotaryTable		
MC_UngroupAllAxes		
MC_GroupJog		
MC_GroupJogVector		

MC_GroupPower		
MC_GroupReadCommandInfo		
MC_GroupReadDefaultDynamics		
MC_GroupReadJoggingDynamic		
MC_GroupReadMotionState		
MC_GroupReadParameter		
MC_GroupReadReferenceDynamics		
MC_GroupReadSWLimits		
MC_GroupWaitTime		
MC_GroupWriteDefaultDynamics		
MC_GroupWriteJoggingDynamics		
MC_GroupWriteParameter		
MC_GroupWriteReferenceDynamics		
MC_GroupWriteSWLimits		
MC_ReadAxisGroupInfo		
MC_ReadDHParameters		
MC_ReadJointInfo		
MC_ReadPayload		
MC_ReadPayloadData		
MC_ReadRigidBodyDynamic		
MC_ReadTool		
MC_ReadToolData		
MC_SelectPayload		
MC_SelectTool		
MC_TransformPosition		
MC_WritePayloadData		
MC_WriteRigidBodyDynamic		
MC_WriteToolData		
Coordinated Function Blocks	Supported Yes / No	Comments (<= 48 char.)
MC_AddAxisToGroup		
MC_GroupContinue		
MC_GroupDisable		
MC_GroupEnable		
MC_GroupHalt		
MC_GroupHome		
MC_GroupInterrupt		
MC_GroupReadAcceleration		
MC_GroupReadPosition		
MC_GroupReadVelocity		
MC_GroupReadConfiguration		
MC_GroupReadError		
MC_GroupReadStatus		
MC_GroupReset		
MC_GroupSetOverride		
MC_GroupSetPosition		
MC_GroupStop		
MC_MoveCircularAbsolute		
MC_MoveCircularRelative		

MC MoveDirectAbsolute		
MC MoveDirectRelative		
MC MoveLinearAbsolute		
MC MoveLinearRelative		
MC MovePath		
MC PathSelect		
MC ReadCartesianTransform		
MC ReadCoordinateTransform		
MC ReadKinTransform		
MC RemoveAxisFromGroup		
MC SetCartesianTransform		
MC SetCoordinateTransform		
MC SetDynCoordTransform		
MC SetKinTransform		
MC SyncAxisToGroup		
MC SyncGroupToAxis		
MC TrackConveyorbelt		
MC TrackRotaryTable		
MC UngroupAllAxes		
MC GroupJog		
MC GroupJogVector		
MC GroupPower		
MC GroupReadCommandInfo		
MC GroupReadDefaultDynamics		
MC GroupReadJoggingDynamic		
MC GroupReadMotionState		
MC GroupReadParameter		
MC GroupReadReferenceDynamics		
MC GroupReadSWLimits		
MC GroupWaitTime		
MC GroupWriteDefaultDynamics		
MC GroupWriteJoggingDynamics		
MC GroupWriteParameter		
MC GroupWriteReferenceDynamics		
MC GroupWriteSWLimits		
MC ReadAxisGroupInfo		
MC ReadDHPParameters		
MC ReadJointInfo		
MC ReadPayload		
MC ReadPayloadData		
MC ReadRigidBodyDynamic		
MC ReadTool		
MC ReadToolData		
MC SelectPayload		
MC SelectTool		
MC TransformPosition		
MC WritePayloadData		
MC WriteRigidBodyDynamic		
MC WriteToolData		

Table 12: Short overview of the Function Blocks

Appendix A 5.1. MC_AddAxisToGroup

	MC_AddAxisToGroup	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
B	Axis		
VAR_INPUT			
B	Execute		
E	IdentInGroup		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.2. MC_RemoveAxisFromGroup

	MC_RemoveAxisFromGroup	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	IdentInGroup		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.3. MC_UngroupAllAxes

	MC_UngroupAllAxes	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.4. MC_GroupReadConfiguration

	MC_GroupReadConfiguration	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	IdentInGroup		
E	CoordSystem		
VAR_OUTPUT			
B	Axis		
B	Valid		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.5. MC_GroupEnable

	MC_GroupEnable	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.6. MC_GroupDisable

	MC_GroupDisable	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.7. MC_GroupHome

	MC_GroupHome	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Position		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.8. MC_SetKinTransform

	MC_SetKinTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	KinTransform		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.9. MC_SetCartesianTransform

	MC_SetCartesianTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	TransX		
B	TransY		
B	TransZ		
B	RotAngle1		
B	RotAngle2		
B	RotAngle3		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.10. MC_SetCoordinateTransform

	MC_SetCoordinateTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	CoordTransform		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.11. MC_ReadKinTransform

	MC_ReadKinTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	KinTransform		
B	Error		
E	ErrorID		

Appendix A 5.12. MC_ReadCartesianTransform

	MC_ReadCartesianTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	TransX		
B	TransY		
B	TransZ		
B	RotAngle1		
B	RotAngle2		
B	RotAngle3		
B	Error		
E	ErrorID		

Appendix A 5.13. MC_ReadCoordinateTransform

	MC_ReadCoordinateTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	CoordTransform		
B	Error		
E	ErrorID		

Appendix A 5.14. MC_GroupSetPosition

	MC_GroupSetPosition	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Position		
E	Relative		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.15. MC_GroupReadPosition

	MC_GroupReadPosition	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	CoordSystem		
E	Source		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Position		

Appendix A 5.16. MC_GroupReadVelocity

	MC_GroupReadVelocity	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	CoordSystem		
E	Source		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Velocity		
E	PathVelocity		

Appendix A 5.17. MC_GroupReadAcceleration

	MC_GroupReadAcceleration	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	CoordSystem		
E	Source		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Acceleration		
E	Path Acceleration		

Appendix A 5.18. MC_GroupStop

	MC_GroupStop	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	Deceleration		
E	Jerk		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.19. MC_GroupHalt

	MC_GroupHalt	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	Deceleration		
E	Jerk		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.20. MC_GroupInterrupt

	MC_GroupInterrupt	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
E	Deceleration		
E	Jerk		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Comman Aborted		
B	Error		
E	ErrorID		

Appendix A 5.21. MC_GroupContinue

	MC_GroupContinue	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Comman Aborted		
B	Error		
E	ErrorID		

Appendix A 5.22. MC_GroupReadStatus

	MC_GroupReadStatus	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	GroupMoving		
B	GroupHoming		
B	GroupErrorStop		
B	GroupStandby		
B	GroupStopping		
B	GroupDisabled		
E	ConstantVelocity		
E	Accelerating		
E	Decelerating		
E	InPosition		
B	Error		
E	ErrorID		

—

Appendix A5.23. MC_GroupReadError

	MC_GroupReadError	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	GroupErrorID		

Appendix A 5.24. MC_GroupReset

	MC_GroupReset	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.25. MC_MoveLinearAbsolute

	MC_MoveLinearAbsolute	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Position		
E	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.26. MC_MoveLinearRelative

	MC_MoveLinearRelative	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Distance		
E	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.27. MC_MoveCircularAbsolute

	MC_MoveCircularAbsolute	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	CircMode		
B	AuxPoint		
B	EndPoint		
E	PathChoice		
E	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.28. MC_MoveCircularRelative

	MC_MoveCircularRelative	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	CircMode		
B	AuxPoint		
B	EndPoint		
E	PathChoice		
E	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.29. MC_MoveDirectAbsolute

	MC_MoveDirectAbsolute	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Position		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.30. MC_MoveDirectRelative

	MC_MoveDirectRelative	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Distance		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.31. MC_PathSelect

	MC_PathSelect	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
B	PathData		
B	PathDescription		
VAR_INPUT			
B	Execute		
E	CoordSystem		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.32. MC_MovePath

	MC_MovePath	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
B	PathData		
VAR_INPUT			
B	Execute		
E	CoordSystem		
E	BufferMode		
E	TransitionMode		
E	TransitionParameter		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.33. MC_GroupSetOverride

	MC_GroupSetOverride	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	VelFactor		
E	AccFactor		
E	JerkFactor		
VAR_OUTPUT			
B	Enabled	BOOL	
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.34. MC_SyncAxisToGroup

	MC_SyncAxisToGroup	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
B	SlaveAxis		
VAR_INPUT			
B	Execute		
E	RatioNumerator		
E	RatioDenominator		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	InSync		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.35 MC_SyncGroupToAxis

	MC_SyncGroupToAxis	Sup.Y/N	Comments
VAR_IN_OUT			
B	Master		
B	AxesGroup		
B	PathData		
VAR_INPUT			
B	Execute		
E	Mode		
E	TuCNumerator		
E	TuCDenominator		
E	Acceleration		
E	Deceleration		
E	Jerk		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	InSync		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.36. MC_SetDynCoordTransform

	MC_SetDynCoordTransform	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
B	MasterAxesGroup		
B	CoordTransform		
VAR_INPUT			
B	Execute		
E	Mode		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.37. MC_TrackConveyorBelt

	MC_TrackConveyorBelt	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
E	ConveyorBelt		
VAR_INPUT			
B	Execute		
B	ConveyorBeltOrigin		
E	InitialObjectPosition		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.38. MC_TrackRotaryTable

	MC_TrackRotaryTable	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
E	RotaryTable		
VAR_INPUT			
B	Execute		
B	RotaryTableOrigin		
E	InitialObjectPosition		
E	CoordSystem		
E	BufferMode		
VAR_OUTPUT			
E	Done		
E	Busy		
E	Active		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.39. MC_GroupJog

	MC_GroupJog	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	JogPositive		
B	JogNegative		
E	VelOverride		
E	AccOverride		
B	CoordSystem		
E	MaxLinearDistance		
E	MaxAngularDistance		
VAR_OUTPUT			
B	Enabled		
B	Active		
B	CommandAborted		
B	Error		
B	ErrorID		

Appendix A 5.40. MC_GroupJogVector

	MC_GroupJogVector	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	Direction		
E	VelOverride		
E	AccOverride		
B	CoordSystem		
VAR_OUTPUT			
B	Enabled		
B	Active		
B	CommandAborted		
B	Error		
B	ErrorID		

Appendix A 5.41. MC_GroupWriteJoggingDynamics

	MC_GroupWriteJoggingDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
B	AxisVelocity		
E	AxisAcceleration		
E	AxisDeceleration		
E	AxisJerk		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.42. MC_GroupReadJoggingDynamics

	MC_GroupReadJoggingDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
B	AxisVelocity		
E	AxisAcceleration		
E	AxisDeceleration		
E	AxisJerk		

Appendix A 5.43. MC_GroupPower

	MC_GroupPower	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Status		
E	Valid		
B	Error		
E	ErrorID		

Appendix A 5.44. MC_GroupReadCommandInfo

	MC_GroupReadCommandInfo	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	CommandID		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	CommandState		
E	ElapsedDuration		
E	RemainingDuration		
E	RemainingDistance		
E	Progress		
E	InfoID		
E	WarningID		

Appendix A 5.45. MC_GroupReadDefaultDynamics

	MC_GroupReadDefaultDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		

Appendix A 5.46. MC_GroupReadMotionState

	MC_GroupReadMotionState	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Tracking		
B	InSync		
B	InPosition		
B	Standstill		
B	ConstantVelocity		
B	Accelerating		
B	Decelerating		
E	ActiveCommandID		

Appendix A 5.47. MC_GroupReadParameter

	MC_GroupReadParameter	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	ParameterNumber		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Value		

Appendix A 5.48. MC_GroupReadReferenceDynamics

	MC_GroupReadReferenceDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		

Appendix A 5.49. MC_GroupReadSWLimits

	MC_GroupReadSWLimits	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	LimitValues		

Appendix A 5.50. MC_GroupWaitTime

	MC_GroupWaitTime	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Duration		
B	BufferMode		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	Tracking		
B	InSync		
B	InPosition		
B	Standstill		
B	ConstantVelocity		
B	Accelerating		
B	Decelerating		

Appendix A 5.51. MC_GroupWriteDefaultDynamics

	MC_GroupWriteDefaultDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.52. MC_GroupWriteParameter

	MC_GroupWriteParameter	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	ParameterNumber		
B	Value		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	CommandAccepted		
E	CommandAborted		
E	CommandID		
B	Error		
E	ErrorID		

Appendix A 5.53. MC_GroupWriteReferenceDynamics

	MC_GroupWriteReferenceDynamics	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	Velocity		
E	Acceleration		
E	Deceleration		
E	Jerk		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.54. MC_GroupWriteSWLimits

	MC_GroupWriteSWLimits	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	LimitValues		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.55. MC_ReadAxisGroupInfo

	MC_ReadAxisGroupInfo	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
E	AxesGroup		
E	AxesGroupID		
B	IdentInGroup		

Appendix A 5.56. MC_ReadDHParameters

	MC_ReadDHParameters	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	DHParameters		

Appendix A 5.57. MC_ReadJointInfo

	MC_ReadJointInfo	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	JointInfo		

Appendix A 5.58. MC_ReadPayload

	MC_ReadPayload	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	PayloadSource		
VAR_OUTPUT			
B	Valid		
E	Busy		
E	CommandAborted		
B	Error		
E	ErrorID		
B	PayloadNumber		

Appendix A 5.59. MC_ReadPayloadData

	MC_ReadPayloadData	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	PayloadNumber		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	RigidBodyDynamic		

Appendix A 5.60. MC_ReadRigidBodyDynamic

	MC_ReadRigidBodyDynamic	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
VAR_OUTPUT			
B	Valid		
E	Busy		
E	CommandAborted		
B	Error		
E	ErrorID		
B	RigidBodyCount		
B	RigidBodyDynamic		

Appendix A 5.61. MC_ReadTool

	MC_ReadTool	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	ToolSource		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	ToolNumber		

Appendix A 5.62. MC_ReadToolData

	MC_ReadToolData	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
B	ToolNumber		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
B	ToolData		

Appendix A 5.63. MC_SelectPayload

	MC_SelectPayload	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	PayloadNumber		
VAR_OUTPUT			
B	Done		
E	Busy		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.64. MC_SelectTool

	MC_SelectTool	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	ToolNumber		
VAR_OUTPUT			
B	Done		
E	Busy		
B	Error		
E	ErrorID		

Appendix A 5.65. MC_TransformPosition

	MC_TransformPosition	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Enable		
E	InputPosition		
B	InputCoordSystem		
B	OutputCoordSystem		
VAR_OUTPUT			
B	Valid		
E	Busy		
B	Error		
E	ErrorID		
E	CommandID		
B	OutputPosition		
E	SingularPosition		

Appendix A 5.66. MC_WritePayloadData

	MC_WritePayloadData	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	PayloadNumber		
B	RigidBodyDynamic		
VAR_OUTPUT			
B	Done		
E	Busy		
E	CommandAborted		

B	Error		
E	ErrorID		

Appendix A 5.67. MC_WriteRigidBodyDynamic

	MC_WriteRigidBodyDynamic	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	RigidBodyCount		
B	RigidBodyDynamic		
VAR_OUTPUT			
B	Done		
E	Busy		
E	CommandAborted		
B	Error		
E	ErrorID		

Appendix A 5.68. MC_WriteToolData

	MC_WriteToolData	Sup.Y/N	Comments
VAR_IN_OUT			
B	AxesGroup		
VAR_INPUT			
B	Execute		
B	ToolNumber		
B	ToolData		
E	ExecutionMode		
VAR_OUTPUT			
B	Done		
E	Busy		
E	CommandAccepted		
E	CommandAborted		
E	CommandID		
B	Error		
E	ErrorID		

Appendix 2 The PLCopen Motion Control logo and its usage

For quick identification of compliant products, PLCopen has developed a logo for the motion control Function Blocks:



Figure 40: The PLCopen Motion Control Logo

This motion control logo is owned and trademarked by PLCopen.

In order to use this logo free-of-charge, the relevant company has to fulfill all the following requirements:

1. the company must be a voting member of PLCopen;
2. the company must comply with the existing specification as specified by the PLCopen Task Force Motion Control, and as published by PLCopen, and of which this statement is a part;
3. this compliance application is provided in written form by the company to PLCopen, clearly stating the applicable software package and the supporting elements of all the specified tables, as specified in the document itself;
4. in case of non-fulfillment, which has to be decided by PLCopen, the company will receive a written statement concerning this from PLCopen. The company will have a one-month period to either adopt their software package in such a way that it complies, represented by the issuing of a new compliance statement, or remove all reference to the specification, including the use of the logo, from all their specification, be it technical or promotional material;
5. the logo must be used as is - meaning the full logo. It may be altered in size providing the original scale and color setting is kept.
6. the logo must be used in the context of Motion Control.

Appendix 3 Relation of PLCopen Coordinated Motion to SRCI

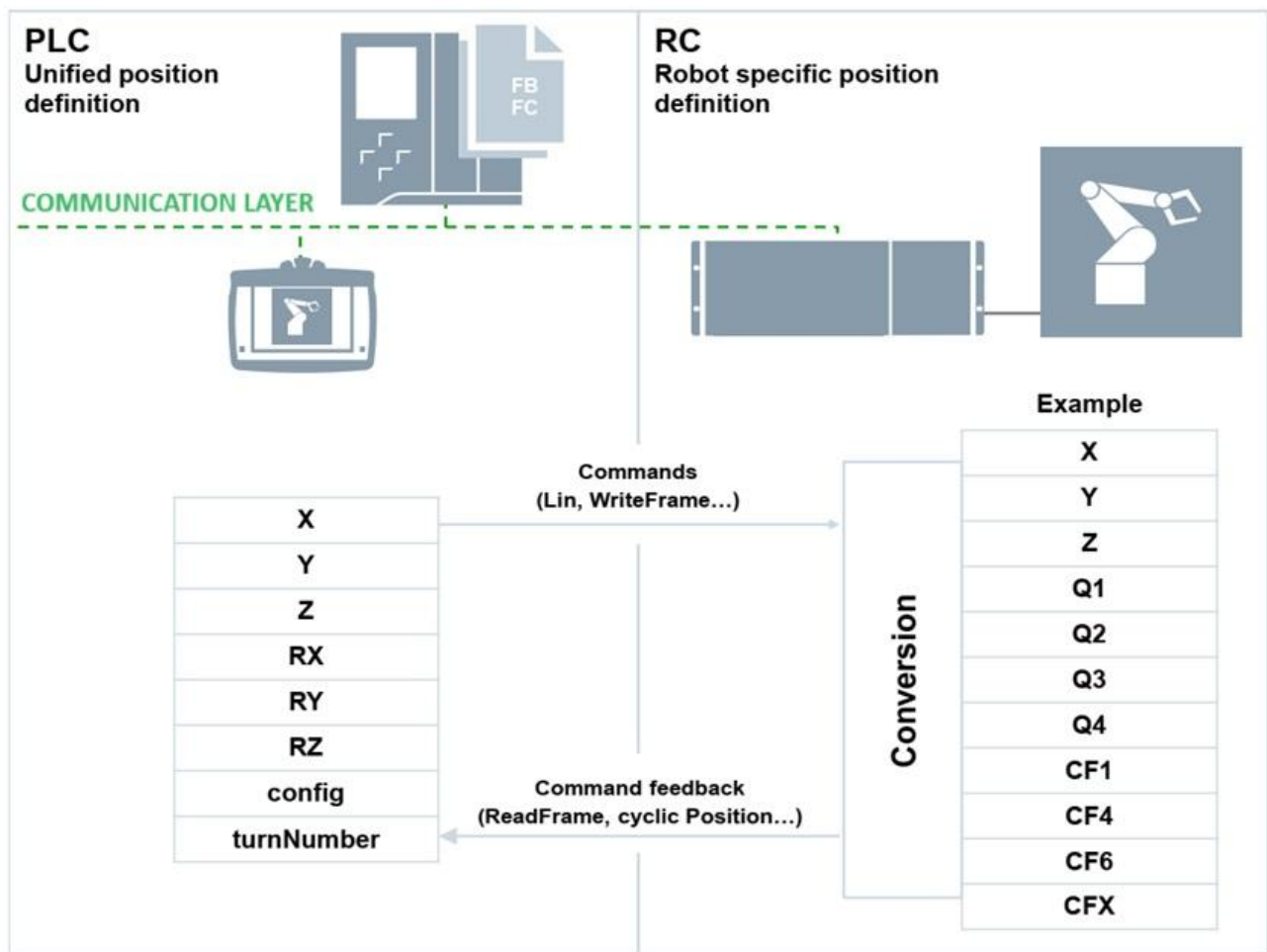
Link to layers in SRCI

The SRCI specification defines 2 levels: the application layer and the command management layer.

The application layer in robotics refers to the functional logic that governs the robot's behavior and task execution. As such it defines what the robot should do. It encompasses tasks like navigation, object manipulation, path planning, and interaction with the environment. It translates high-level goals into specific commands for the robot's actuators and sensors.

The command management layer focuses on the execution and coordination of low-level commands. It bridges the gap between the high-level application layer and the low-level control systems. It ensures that commands are executed in a coordinated manner, considering real-time constraints and system dynamics.

The PLCopen specification will focus on the application layer. Also, PLCopen has the flexibility to change the definition of the FBs if the mapping to the command level (e.g. the communication structure between the PLC and the RC) is kept equal to the defined SRCI version. This communication layer is outside of the scope of this document, and outside the scope of this PLCopen activity. For this the SRCI specification should be checked.



Added vision: standard format in the middle, coupled to interpreters / converters.